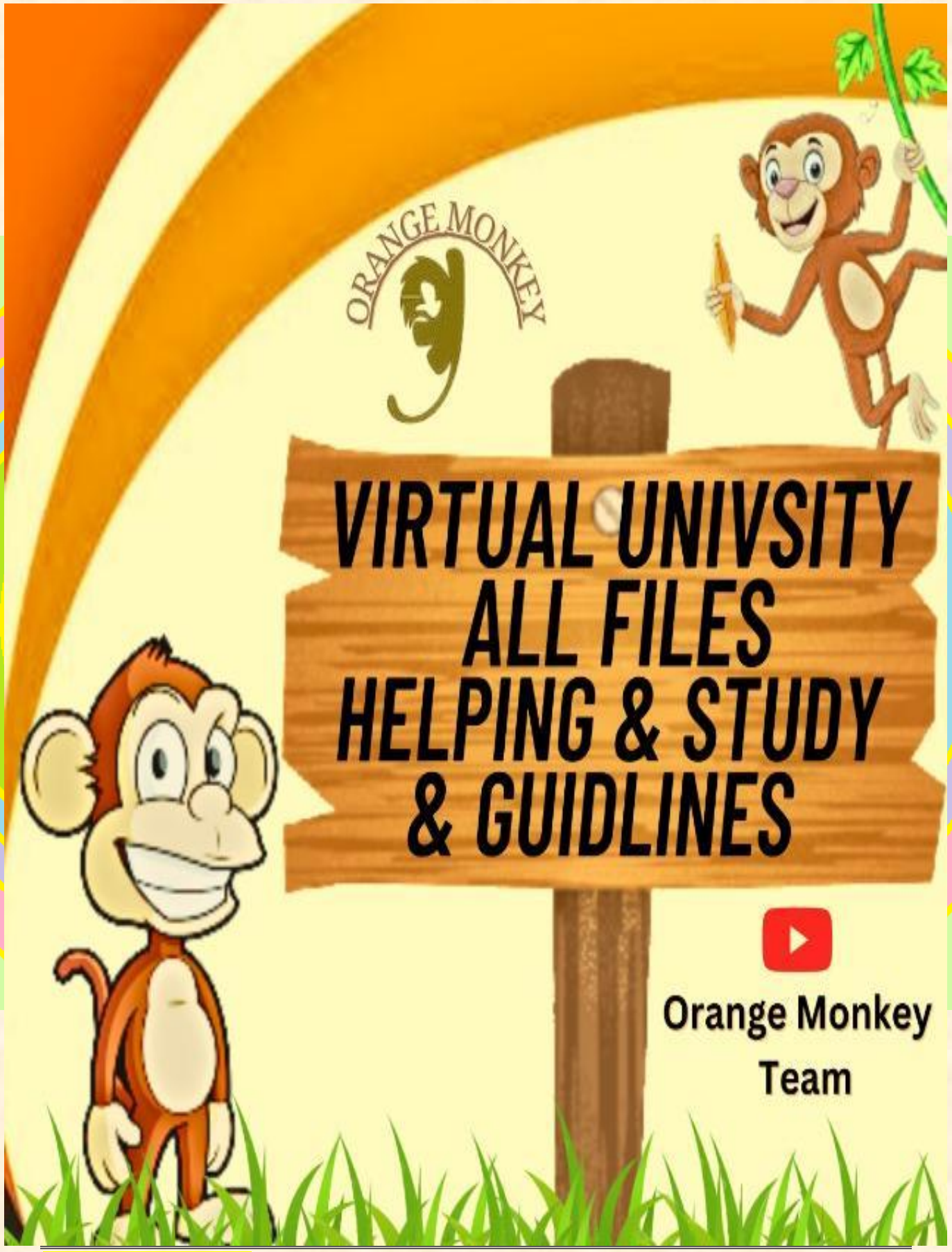

ORANGE MONKEY
VIRTUAL UNIVSITY
ALL FILES
HELPING & STUDY
& GUIDLINES
Orange Monkey
Team

CS506 (Midterm Mcqs)

1. **In Java, if a super class method is protected then overriding method _____**

May be either public or protected….confirm

In Java, if a superclass method is declared as "protected," then the overriding method in the subclass must be either "public" or "protected." The correct option is:

may be either public or protected

2. **Which command of the following commands is used to run JavaTest java file's program?**

Javac Javatest.java…..confirm

Which command of the following commands is used to compile "Java Test" java file's code?	javac JavaTest.java

3.

CS506-Web Designing & Developing
MID TERM MCQS
Prepared by: JUNAID MALIK

AL-JUNAID TECH INSTITUTE

www.vulmshelp.com



Language Courses Training Available

I'm providing paid courses in different languages within 3 Months,Certificate will be awarded after completion.

- HTML
- CSS
- JAVASCRIPT
- BOOTSTRAPS
- JQUERY
- PHP MYSQL
- NODES.JS
- REACT JS

LMS Handling Services

LMS Activities Paid Task

Assignments 95% Results

Quizes 95% Results

GDB 95% Results

For CS619 Project Feel Free To Contact With Me

Ph# 0304-1659294

Email: junaidfazal08@gmail.com

AL-JUNAID TECH INSTITUTE

1. Which of the following feature provides the facility for multiple inheritance?
    a. Adapter classes
    b. Wrapper classes
    c. Interfaces Page No 95
    d. Collections
2. setColor() is a method of_____ object.
    a. JPanel
    b. JFrame
    c. Graphics page No 175
    d. Image
3. If we want to animate something like ball, moving from one place to another. we constantly need to call paint () method.
    a. True
    b. False Page NO 176
4. If we want to call toString() method of parent class in JAVA. Which of the following statement will be used?
    a. this. toString();
    b. new. toString();
    c. sub. toString();
    d. super.toString(); p page NO 57
5. Which of the following syntax is used to create a client socket?
    a. Socket s = new Socket("serverName");
    b. Socket s = new Socket(serverport);
    c. Socket s = new Socket("serverName", serverPort); page No 195
    d. Socket s = new Socket(serverPort, "serverName");
6. In order to include graphics in your Java program, it is mandatory to extend from_____class.
    a. JLabel
    b. JFrame
    c. JPanel page 174
    d. JButton

7. What will happen if static modifier is removed from the signature of main method?
    a. The program does not compile
    b. The program compiles and runs successfully
    c. The program compiles but does not run Google
    d. None of given options

8. Timer class constructor requires time in_____.
    a. seconds
    b. milli seconds Page No 180.
    c. micro seconds
    d. nano seconds
9. Which of the following features is available in java?
    a. Pointers
    b. Operator overloading
    c. Global variables
    d. Primitive data types Page No 40.
10. ResultSet Meta Data will help you in answering the following questions except;
    a. How Many columns are in the ResultSet?
    b. What is the name of given column?
    c. How many rows are in the ResultSet? Page No 162.
    d. Are the column name is case sensitive?
11. _____returns the elements at specified position in the list.
    a. **Object get(int index) Page No 62.**
    b. Object set(int index)
    c. Object return(int index)
    d. Object remove(int index)
12. WindowListener interface contains_____methods.
    a. Four
    b. Six
    c. Seven page No.124
    d. Eight

AL-JUNAID TECH INSTITUTE

13. Which of the following driver type(s) has better performance than other?
    a. Type 1: JDBC - ODBC Bridge
    b. Type 2: Native - API/partly Java driver
    c. Type 3: Net - protocol/all-Java driver
    d. Type 4: Native - protocol/all-java driver page No 169
14. Which of the following syntax is used to create a Server Socket?
    a. ServerSocket ss = new ServerSocket("serverName");
    b. ServerSocket ss = new ServerSocket(serverPort); page No 197
    c. ServerSocket ss = new ServerSocket("serverName", serverPort);
    d. ServerSocket ss = new ServerSocket(serverPort, "serverName");
15. The keyword "this" is used to _____
    a. invoke constructor of the another class
    b. invoke alternate constructor of the same class page 55
    c. invoke constructor of super class from sub class
    d. invoke constructor of sub class from super class
16. An applet can be defined as:
    a. A small program written in Java and included in an HTML page page 182
    b. A small program written in JSP and included in an HTML page
    c. A small program written in Servlet and included in an HTML page
    d. A small program written in JSF and included in an HTML page
17. Choose the correct syntax to define an interface named as "Speaker" in java language
    a. public class Speaker{}
    b. public interface class Speaker{}
    c. public interface Speaker{} Page NO 95
    d. public Speaker class interface{}
18. Teacher java class wants to inherit from Employee.java class, _____ keyword should be used in blank space in given code to apply inheritance

```
class Teacher ---------- Employee{
public static void main (String args[]){
}}
```

a. inherit
b. implement
c. extends Page No 56
d. override

19. _____is correct syntax to create a general purpose container "myPanel" having flow layout.
a. JPanel myPanel = new JPanel (FlowLayout()};
b. JPanel myPanel = new JPanel (new FlowLayout};
c. JPanel myPanel = new JPanel (new FlowLayout()}; Page No 110.
d. JPanel myPanel = JPanel (FlowLayout()};

20. Java is a_____ language.
a. Complicated
b. Not portable
c. Static
d. Multi threaded Page No 22,24

21. Which one of the following blocks always executes irrespective of exception occurrence?
a. Try block
b. Catch block
c. Throws block
d. Finally block Page No 72

22. Based on functionality, the streams can be categorized as _____
a. Byte oriented stream and Node stream
b. Filter stream and Character oriented stream
c. Node stream and Filter stream Page No 83
d. Byte oriented stream and Character oriented stream

23. Bytecodes are created and stored on disk by_____.
a. editor
b. compiler Page no 30
c. class loader
d. bytecode verifier AL-JUNAID INSTITUTE OF GROUP

23. Static methods can only access ___
a. Instance variables
b. Instance methods
c. Static variables and methods page 50/Concept

c. Both static and instance members

24. Which of the following classes provide definitions for all the methods (empty bodies) of their corresponding Listener interface?
    a. private classes
    b. public classes
    c. adapter classes Page No 127
    d. inner classes

25. How many simultaneous connections can be opened using TCP and UDP ports?
    a. 64
    b. 128
    c. 64k
    d. 128k Page No 194

26. Strings in java are represented as _____
    a. Primitive data types
    b. Reference data types page 38/40(string is object in java and object is called by reference)
    c. Native data types
    d. Operators

27. In given lines of code Employee is super class, Teacher is child class, _____ is applying.
    Employee e = new Teacher():
    a. **upcasting Teacher to Employee type page No 60**
    b. upcasting Employee to Teacher type
    c. downcasting Teacher to Employee type
    d. downcasting Employee to Teacher type

28. The following is an example of_____.
    FileReaderfr = new FileReader("input txt");
    a. Byte oriented stream
    b. Node stream page No 83
    c. Filter stream
    d. File stream

29. In the context of Java Graphics, paintChildren( ) method tells any component contained by this component to paint itself.(themselves).
    a. True Page No 173
    b. False

30. Which of the following is called "pure abstract class"?
    a. Concrete class
    b. Wrapper class
    c. Interface page No 95 Ho
    d. Abstract class with no abstract method
31. Window, frame and dialog use_____as their default layout.
    a. Border layout
    b. Flow layout
    c. GridBag layout Concept
    d. Grid layout
32. Exception handling code is written in_____
    a. try block
    b. catch block page No 56 HO
    c. finally block
    d. throws clause
33. Suppose strings are stored in a collection "arraylist". Get one string element and store in element variable.
    Choose correct line of code for this requirement.
    a. String element = String arraylist.get(i);
    b. String element = (String)arraylist.get(i); page No 61 HO
    c. String element = element arraylist.get(i);
    d. String element = (element) arraylist.get(i);
34. Converting a bigger data type into smaller data type is called_____.
    a. Up-casting
    b. Down-casting Page No 60 HO
    c. Normal-casting
    d. High-casting
35. Which of the following method is used to convert a string into an int value?
    a. intValue(}
    b. parseInt() page No 42 Ho
    c. ParseInt()
    d. parseInteger()
36. Adapter classes have been defined for listener interfaces except _____ interface.
    a. MouseListener
    b. KeyListener
    c. WindowlListener

d. ActionListener Page No 128 HO

37. Which of the following is correct java code statement that is used to establish the connection?

a. **Connection con = DriverManager.getConnection(conURL) PG NO 140**

b. connection Con = driverManager.getConnection(conURL)

c. connection Con = driverManager.getConnection(conURL)

d. Connection con = driverManager.getConnection(conURL)

39. Painting is managed by the system, so calling paintComponent() directly is NOT recommended at all,

a. True page No 176

b. False

40. JPanel and Applet use_____as their default layout.

a. **Flow layout According to Concept**

b. Border layout

c. Grid layout

d. GridBag layout AL-JUNAID INSTITUTE OF GROUP

41. Which of the following is NOT a method of ResultSet object?

a. next()

b. absolute(int)

c. executeQuery(sql) page NO 150 HO

d. updateRow()

42. Which of the following methods is invoked by the AWT package to support paint and repaint operations?

a. **paint( ) for system trigger**

b. **repaint( ) for app trigger (portion newly exposed) page 170**

c. draw( )

d. redraw( )

43. Which of the following name is used by the system to recognize the underlying data source?

a. DNS

b. DSN page No 138 HO

c. JDBC

d. ODBC

44. A database that is created in MS Access 2003 will be saved with _____ extension.

a. .java
b. .class
c. .doc
d. .mdb page No 138 Ho

45. Which of the following is the correct syntax to make a PersonInfo class into a Serializable class?
a. Public class PersonInfo includes Serializabte { /* class body */}
b. Public class PersonInfo imports Serializabte { /* class body */}
c. Public class PersonInfo extends Serializabte { /* class body */}
d. Public class PersonInfo implements Serializabte { /* class body */}
Page 205

46. INSERT, UPDATE, DELETE are_____statements.
a. DDL
b. DML page NO 144
c. DCL
d. None of given

47. To compare the values of two strings._____must be used.
a. = Operator AL-JUNAID INSTITUTE OF GROUP
b. = = Operator
c. equals() method page No 38 HO
d. compare() method

48. AWT is also called_____.
a. Heavy weight components Page No 99 HO
b. Light weight components
c. Both Light weight and heavy weight components
d. Neither Light weight nor heavy weight

49. Which of the following methods takes the place of constructor in an Applet?
a. Run()
b. Main()
c. Init() page NO 185 HO
d. Start()

50. Which of the following methods is used to determine whether a database is read only or not?
a. isReadonty()
b. isReadOnly() page No 167 HO
c. isreadonly()

AL-JUNAID TECH INSTITUTE

51. The following is an example of_____.
BufferedReaderbr = new BufferedReader(fr);
    a. Byte oriented stream
    b. Node stream
    c. Filter stream page No 83 HO
    d. File stream

52. If we remove static from main then program_____
    a. Program cannot compile
    b. Program will compile and work properly
    c. Run time errors My Point of View
    d. Exception throws in main

53. If we remove String array from main then program_____
    a. Program cannot compile
    b. Program will compile and work properly
    c. Run time errors My Point of View
    d. Exception throws in main

54. Java developed in_____
    a. 1990
    b. 1995 (Page 22)
    c. 1997
    d. 2000 AL-JUNAID INSTITUTE OF GROUP

55. When defining a method you must include a_____to declare any exception that might be thrown but isnot caught in the method.
    a. try block
    b. finally block
    c. catch block
    d. throws-clause Page no 73 Ho

56. Aside from the Scanner class, which of the following classes can be used for text-file file input?
    a. ObjectInputStream
    b. Buffered Reader Page No 83 & 85
    c. StringTokenizer
    d. None of these

57. Which of the following statements is true regarding Vectors with no specified base type?

a. A base type is needed, because Java needs to know how to allocate memory.
b. No base type is needed, because Java will use the Object class as a base type. My Point of View
c. If a base type is not specified the code will not compile.
d. No base type is needed because Vectors default to storing String objects.

58. A serializable class must implement the method(s)_____
a. readObject and writeObject
b. Either readObject or writeObject, or both, depending upon the desired behavior
c. No need to implement any method page no 204
d. None of the given option

59. You can --------- to a network using sockets.
a. Read
b. Write
c. Read/Write (Page 207)
d. None of these

60. Socket is a ---------- communication channel between hosts.
a. Uni-directional
b. Bi-directional (Page 194)
c. Multi-directional
d. None of these

61. Which of the following are passed as an argument to the paint ( ) method?
a. A Canvas object
b. A Graphics object page 186/173
c. An Image object
d. A paint object

62. The code below draws a line. What color is the line?
g.setColor(Color.red. green. yellow.red);
g.drawLine(0, 0, 100,100);
a. Red
b. Green
c. Yellow AL-JUNAID INSTITUTE OF GROUP
d. Black My Point of view

63. To create DataBaseMetaData object we write following line of code
    a. **DatabaseMetaData db = con.getMeataData(); (Page 165)**
    b. ResultSetMetaDatarsmd = rs.getMetaData();
    c. ResultSetMetaDatarsmd = rs.setMetaDataQ);
    d. DataBaseMetaData db = con.setMeataData();
64. Event source can have --------- listeners registered on it.
    a. Single
    b. Double
    c. Triple
    d. Multiple (Page 118)
65. An abstract class ---------- instantiated.
    a. **Cannot be (Page 93)**
    b. Can be
    c. Must
    d. None of these
66. What if the way and method of communication between two parties the static modifier is removed from the signature of the main method?
    a. The program does not compile
    b. The program compiles but does not run
    c. The program compiles and runs properly
    d. The program throws an exception on compile time
    My Point of View
67. What if the main method is declared as private?
    a. The program does not compile...
    b. The program compiles but does not run
    c. The program compiles and runs properly
    d. The program throws an exception on compile time
    My Point of View
68. A member variable or method prefixed by the protected access modifier can be accessed:
    a. Within the same class
    b. Within the sub-class
    c. Within the same package
    d. All of the above page no 46 HO
69. A top level class without any modifier is accessible to

a. any class
b. any class within the same package page no 46 HO
c. any class within the same file
d. any subclass of this class

70. A top level class may have only the following access modifier.
a. Package
b. Private
c. Protected
d. Public page no 46 HO

71. In Java an abstract class cannot be sub-classed
a. True
b. False page 93

72. Given a one dimensional array arr, what is the correct way of getting the number of elements in arr
a. **arr.length google**
b. arr.length — 1
c. arr.size — 1
d. arr.lengthO AL-JUNAID INSTITUTE OF GROUP

73. Which of these are valid declarations for the main method?
a. public void main();
b. public static void main(String args[]); (Page 26)
c. static public void main(String);
d. public static int main(String args|]);

74. In Java, which of these classes implement the LayoutManager interface?
a. RowLayout
b. ColumnLayout
c. GridBagLayout My Point of View
d. Flow LayoutManager

75. Border Layout is the default layout manager for a JFrame's content pane
a. **True My Point of View**
b. False

76. You want a class to have access to members of another class in the same package. Which is the most restrictive access that accomplishes this objective?
a. public
b. private

c. protected page 46

d. transient

e. default access

77. In Java garbage collection is done by_____

a. **JVM (Java Virtual Machine) (Page 50)**

b. Programmer

c. Both JVM (Java Virtual Machine)and Programmer

d. OS (Operating System)

e. None of given

78. Which of following can be thrown using the throw statement?

a. Error

b. Throwable

c. Exception (Page 72)

d. RuntimeException

e. All of Given

79. A final class can't be extended.

a. **True My Point of View**

b. False

80. Java program code is compiled into form called_____

a. Machine code

b. native Code

c. Byte Code (Page 16)

d. Source Code

81. Overloading is_____whereas overriding is _____

a. Run time binding, compile time binding

b. Late binding, compile time binding

c. Compile time binding, run time binding My Point of View

d. Run time binding, late time binding

82. Which of the following is modifier is provided by default if no access is written explicitly?

a. public

b. Private AL-JUNAID INSTITUTE OF GROUP

c. Protected

d. Default page 46

83. Which of the following is used for inheritance in java?

a. implements
b. extends Page No 55
c. : (colon)
d. inherit

84. A collection can store.....
a. **Homogenous objects (any any kind of objects.) page no 61**
b. Heterogeneous objects
c. Objects as well as primitive values
d. At most 100 objects

85. Which of the following syntax is used to attach an input stream to console?
a. FileReaderfr = new FileReader(' input.txt ') ; (Page 83)
b. FileReaderfr = new FileReader(FileDescriptor.in);
c. FileReaderfr = new FileReader(FileDescriptor);
d. FileReaderfr = new FileReader(console);

86. Window frame and dialog use_____as their default layout.
a. Border layout My Point of View
b. Flow layout
c. GridBag layout
d. Grid layout

87. There are____types of exceptions in Java
a. 2 (Page 71)
b. 3
c. 4
d. 5

88. Which of the following belongs to a category of checked exception?
a. Null Pointer exception
b. IOException pg 72
c. Array index out of bounds
d. NumberFormatException

89. Which of the following exception belongs to a category of un-checked exception?
a. IOException
b. Null Pointer exception pg 71
c. AWT Exception
d. ClassNotFoundException

AL-JUNAID TECH INSTITUTE

90. All the exceptions and errors in java are inherited from____class.
    a. Exception
    b. Error
    c. Throwable page No 71
    d. IOException
91. Exceptions must be handled while handling files otherwise it may lead to___
    a. Logical error
    b. Syntax error AL-JUNAID INSTITUTE OF GROUP
    c. Run-time error (Page 70)
    d. No error
92. Which of the following package needs to be import while handling files?
    a. java.util
    b. java.io Page 60 HO
    c. java.awt
    d. javax.swing
93. The classes which contain the word_____are byte oriented streams.
    a. Reader
    b. Writer
    c. Stream (Page 84)
    d. Byte
94. Which of the following stream read/write data in the form of bytes?
    a. FileReader
    b. FileWriter
    c. PrintWriter
    d. FileInputStream (Page 84)
95. An instance of abstract class cannot be created.
    a. **TRUE**
    b. FALSE
96. The relationship between class and interface is called
    a. 'Is a' relationship
    b. ' Has a' relationship
    c. ' Responds to ' relationship (Page 98)
    d. None of the given options
    a. Abstract class with no abstract method
97. Which of the following is true about abstract class?

a. An abstract class must have all methods declared as abstract methods.
b. Aclass must have at least one abstract method to be an abstract class.
c. A class without any abstract method can be declared as abstract class pg 93
d. An instance of abstract class can be created.

98. Which of the following stream is a filter stream?
a. FileWriter
b. FileReader AL-JUNAID INSTITUTE OF GROUP`
c. BufferedReader (Page 83)
d. All of given options

99. Which of the following is true about AWT and SWING components?
a. **AWT components create a process whereas SWING component creates a thread. Page NO 99**
b. AWT components create a thread whereas SWING component creates a process.
c. Both AWT and SWING component creates a process.
d. Both AWT and SWING component creates a thread.

100. Which of the following is a general purpose container?
a. JFrame
b. Dialog
c. JPanel page 101
d. JApplet

101. Border layout divides the area into regions
a. 3
b. 4
c. 5 (Page 108)
d. Pm 6

102. Which of the following function will be used to register event handler with events generator(button)?
a. AddAction()
b. addActionListener() (Page 117)
c. addListener()
d. registerListener()

103. If a class needs to handle events generated by button then which of the following interface a classneeds to implement?

a. ComponentListener
b. KeyListener
c. MouseListener
d. Action Listener (Page 128)

104. Which of the following function is declared in MouseMotionListener interface?
a. public void mousePressed (MouseEvent me);
b. public void mouse Dragged (Mouse Event me); (Page 122)
c. public void mouseClicked (MouseEvent me);
d. public void mouseEntered (MouseEvent me);

105. Window Listener interface contains methods.
a. Four
b. Six AL-JUNAID INSTITUTE OF GROUP
c. Seven (Page 124)
d. Eight

106. Mouse events can be trapped for____GUI component.
a. JPanel
b. JFrame
c. JButton
d. Allofgiven (Page 122)

107. Which of the following package needs to import while interacting with relational database?
a. java.io
b. java.sql (Page 137)
c. javax.swing
d. java.awt

108. DSN stands for_____
a. Data System Name
b. Domain system Name
c. Data Source Name page No 138
d. Database System Name

109. Which of the following method is used to execute SELECT SQL statements?
a. ExecuteUpdate(sql);
b. ExecuteUpdate(sql);

c. Execute Query(sql); (Page 141)

d. ExecuteQuery(sql);

110. CREATE, ALTER, DROP are_____ SQL statements.

a. DML

b. DDL (Page 144)

c. DCL AL-JUNAID INSTITUTE OF GROUP

d. None of given

111. Which of the following method is used to execute INSERT, UPDATE and Delete SQL statements?

a. ExecuteQuery(sql);

b. ExecuteQuery(sql);

c. Execute Update(sql); (Page 141)

d. ExecuteUpdate(sql);

112. Which of the following statement object is used to execute stored procedures?

a. Statement

b. PreparedStatement

c. CallableStatement (Page 147)

d. None of given options

113. HashMap takes key as a/an ____

a. **Object (Page 63)**

b. Array

c. Primitive value

d. Function

114. To create DataBaseMetaData object we write following line of code

a. DataBaseMetaData db = con.getMeataData(); (Page 165)

b. ResultSetMetaDatarsmd = rs.getMetaData();

c. ResultSetMetaDatarsmd = rs.setMetaData();

d. DataBaseMetaData db = con.setMeataData();

115. Which of the following methods are invoked by the AWT to support paint and repaint operations?

a. **paint() page 176**

b. repaint( )

c. draw()

d. redraw()

116. If a super class method is protected then overriding method_____
    a. must be protected
    b. must be public
    c. must be private
    d. may be either protected or public
117. Aside from the Scanner class, which of the following class/es can be used for text file file input
    a. ObjectInputStream AL-JUNAID INSTITUTE OF GROUP
    b. BufferedReader page 83
    c. StringTokenizer
    d. None of these
118. Which of the following feature provide facility for multiple inheritances:
    a. Adapter Classes
    b. Wrapper Classes
    c. Interface page 96
    d. Collection
119. The collections API are defined in_____package?
    a. Java.io
    b. Java.util
    c. Java.sql (Page 137)
    d. Java.awt
120. A class that is defined in another class is called
    a. Private class
    b. Public class
    c. Inner class
    d. Derived class
121. Person DSN url
    a. **String conURL = "jdbc:odbc:personDSN; correct option**
122. Which database allows us to work having closed connection?
    a. Mysql
    b. PostgreSQL
    c. SQL
    d. SQLite
123. If a JPanel has a triangle drawn on it which of the following method will display it

a. PaintBorder()
b. Paint()
c. paintChildren()
d. **paintComponent()** **Page 170 HO**

124. Window Adaptor class implements Window Listener interface and provide the definition of all method inside.....
a. WindowListener interface **Page 118 HO**
b. Inner Class
c. Outer class
d. Window adaptor class

125. Event handling is_____step process.
a. 1
b. 2
c. 3
d. 4

126. Which of the following is not the method of ResultSetMetaData class.
a. **Previous()**
b. isReadOnly()
c. getColumnDisplay size()
d. getColumnCount()

127. The code generate following output
Student s1, s2, s3, s4
$1= new student();
$2= new student();
a. **1**
b. 2
c. 3
d. 4

128. An exception in java is represented as a/an
a. Operator
b. Object
c. Function
d. None

129. Window, frame and dialog use as_____their default layout.
a. Border Layout

b. Flow Layout

c. Grid Layout

130. A_____in HashMap is associated with each object that is stored.

a. None of the given options

b. Key (Page 42)

c. Value

d. Attribute.

131. Which of the following must be used to compare the values of two strings?

a. = operator

b. == operator

c. equals() method (Page:19)

d. compare() method

132. Port is a transport address to which processes can for connections request.

a. Read

b. Write

c. Listen (Page 196.

d. None of the given options

133. From following; which is the main reason that enters a thread into dead state:

a. It dies a natural death because thread completes its execution

b. It is killed because someone invoked its stop method.

c. Both a and b. (for a page 228 and b for google)

d. None of these.

134. From the following; which approach is used in java to create threads,

a. Interface

b. Inheritance

c. Both Interface and Inheritance (Page 212)

d. None of these.

135. Which of the following is a type of Java web application technologies?

a. JSTL

b. JSF

c. JSP

d. All of the given (Page 244)

136. In an applet class definition, the_____method takes the place of the constructor.

a. paint();
b. main();
AL-JUNAID INSTITUTE OF GROUP
c. init(); page 268
d. run();

137. From following classes; which one can't be instantiated?
a. Super class
b. Abstract Class page 82
c. Anonymous Class
d. Concrete Class

138. Java provide _____
a. Overloading
b. Overriding page 5
c. Pointers
d. Multiple Inheritance

139. int x = 7/2; value of x is:
a. 1
b. 2
c. 3.5
d. 3 decimal part is truncated.

140. From which object do you ask for DatabaseMetaData?
a. **Connection (Page 160)**
b. ResultSet
c. DriverManager
d. Driver

141. Which method(s) must a serializable class, implement?
a. **It must always implement both readObject and writeObject (page 203)**
b. It must implement either readObject or writeObject, or both, depending upon the desired behavior
c. No Need to implement any methods
d. None of the given

142. A final class can't be extended.
a. **True**
b. False

143. Which of the following interface needs to implement while creating threads?

a. Serializable
b. Runnable (Page 212)
c. ActionListener
d. WindowListener

144. Which of the following method needs to override while handling threads in java?
a. init()
b. start()
c. run() page 212
d. runnable().

145. Which of the following range of response code signify error by server?
a. 200-299
b. 300-399
c. 400-499
d. 500-599 (Page 234)

146. Which of the following method allows any other thread of same priority to execute?
a. sleep()
b. wait()
c. notify()
d. yield() (Page 216)

AL-JUNAID INSTITUTE OF GROUP

147. Which of the following problem occurs with lower priority threads?
a. Race condition
b. Starvation (page 218)
c. Average waiting time
d. Process death

148. Which of the following approach is used to create threads in java?
a. inheritance
b. interface
c. Both inheritance and interface
d. None of these

149. Which of the following range of response code indicates that request was successful?
a. 100-199
b. 200-299 (page 234)

c. 300-399
d. 400-499

150. Which of the following method of an Applet is called every time when page is minimized?
a. start()
b. init()
c. stop() (Page 185)
d. destroy()

151. Which of the following statement is true regarding paintComponent() method?
a. It tells any components contained by this component to paint themselves.
b. It tells the component to paint its border.
c. It first paints the background and then performs custom painting. Pg170
d. It first performs custom painting and then paints the background

152. Which of the following is NOT true for an Applet?
a. A small program written in Java and included in a HTML page.
b. It is independent of the operating system on which it runs.
c.They have no access to the client's file system. (Page 181) and Conceptual
d. An applet is a Panel that allows interaction with a Java program

153. Which of the following syntax is used to get ResultSetMetaData object?
a. ResultSetMetaData md = con.getMetaData(); // where con is Connection object
b. ResultSetMetaData md = rs.getMetaData() // where rs is ResultSet object (Page 156)
c. ResultSetMetaData md = st.getMetaDataQ); // where st is Statement object
d. ResultSet md = rs.getMetaData(); // where rs is ResultSetMetaDta object

154. getColumnCount() is a method of_____object.
a. Statement
b. ResultSet AL-JUNAID INSTITUTE OF GROUP
c. ResultSetMetaData (Page 157)
d. DataBaseMetaData

155. Which of the following method is used to move the cursor to given row number?

a. **absolute(int); (Page 146)**
b. absolute();
c. moveTolInsertRow(int);
d. insertRow();

156. Which of the following statement object is used to execute simple SQL statement?
a. **Statement (Page 133)**
b. PreparedStatement
c. CallableStatement
d. None of given

157. Java defines_____for the listener interfaces having more than one event handling methods.
a. Wrapper Classes
b. Abstract Classes
c. Concrete Classes
d. Adapter Classes Page 118 HO

158. Which of the following object is passed as an argument to paintComponent() method?
a. Image object
b. Jpanel object AL-JUNAID INSTITUTE OF GROUP
c. JFrame object
d. Graphic object page 171 HO

159. previous() is a method of_____object
a. Statement
b. ResultSet page 145
c. ResultSetMetaData
d. Connection

160. isReadOnly() is a method of_____ object.
a. ResultSet
b. ResultSetMetaData (Page 160)
c. DataBaseMetaData
d. Connection

161. getrequest() method gets information from a client and postrequest() method posts data to a client.
a. True

b. False **My Point of View**

162. Which of the following method of Graphics object is used to draw rectangle?

a. DrawRect(10,10,20,20);

b. drawRect(10,10,20,20); (Page 172)

c. drawReactangle(10,10,20,20);

d. DrawReactangle(10,10,20,20);

163. Which of the following is modifier is provided by default if no access is written explicitly?

a. Public

b. Private

c. Protected

d. Default (page 46)

164. FTP (File Transmission Protocol) works on_____port.

a. 21 Page 193 HO

b. 23

c. 80 AL-JUNAID INSTITUTE OF GROUP

d. 84

165. Which of the following is NOT true for java inner classes?

a. Can have constructors

b. Can have instance variables

c. Can have Methods

d. Generally used as public utility classes Page 121 HO

166. Which statement from the following statements is true about a java Anonymous Inner Class

a. It is difficult to understand

b. It has no name

c. It is much shorter

d. All of the given options Page 127 HO

167. While running following ExceptionExample.java class on command line without passing command line arguments gives_____exception.

```
public class ExceptionExample {
public static void main (String args[]) {
System.out.println(args[0]);
}}
```

a. Null pointer exception
b. Array index out of bound exception Page 58 HO
c. Divide by zero exception
d. IO Exception

168. Let suppose a column has index 5 in the ResultSet object. Which of the following statement is used to retrieve the column values?
a. String name = rs.getInt(4)
b. String name = rs.getInt(5)
c. String name = rs.getString(4)
d. String name = rs.getString(5)

169. Which of the following streams is a byte oriented stream?
a. FileReader
b. PrintWriter
c. BufferedReader
d. FileInputStream Page 71 HO

170. Which of the following statements is true about primitive data types and reference data types?
a. Primitive data types are created on heap whereas references are created on stack
b. Both primitive data type and references are created on stack
c. Primitive data types are created on stack whereas references are created on heap
d. Both primitive data types and references are created on heap

171. Which command of the following commands is used to compile "JavaTest" java file's code?
a. java JavaTest
b. javac JavaTest.java
c. javacJavaTest
d. java JavaTest.java

172. Interface imposes_____structure on any class that uses interface
a. code
b. design page 85
c. implementation
d. inheritance

173. Which of the following is NOT true about paintComponent( ) method?

a. It is a main method for painting
b. By default, it first paints the background
c. Tells the components border (if any) to paint. Page 170
d. It performs custom painting (drawing circle, rectangles etc.)

174. An object of Student java class is creating and setting its value in Test.java class._____error is there in given code.

```
public class Test{
public static void main (String args[]}{
        Student s2 = new Student();
        s2->setName("Aslam");
}
```


a. Class name is in wrong format
b. s2 is not valid name of object of Student class
c. setName method should not be accessed by arrow operator Page 32
d. no syntax error

175. The "MouseDragged" and "mouseMoved" methods are defined in _____interface.

a. MouseListener
b. ltemListener
c. MouseMotionListener Page 113 HO
d. ActionListener

176. For a programmer the event handling is a three step process in terms of code. Which of the following is NOT an event handling step?

a. Generate event
b. Handle event
c. Publish event Page 104 HO
d. Register handler with generator

177. Which feature of the following features does Java provide?

a. Operator Overloading
b. Pointers
c. Function Overriding page 5 HO
d. Multiple Inheritance

178. Bytecodes are put in memory by_____.

a. Editor

AL-JUNAID INSTITUTE OF GROUP

b. compiler
c. class loader Page 11 HO
d. bytecode verifier

179. The following code creates and starts a timer that fires an action event once per_

```
Timer t = new Timer (1000, this);
t.start();
```

a. **Second**
b. Minute
c. Hour
d. Day

180. CmdArgs java class is compiled. If the command "java CmdArgs CS506" run in cmd, then what will be output of given code.

```
Public class CmdArgs{
        public static void main(String[] args){
        System.out.println("First argument" + args[0]);
}
```

a. CS506
b. Syntax error
c. do not show any output
d. args[0]

181. Which of the following functions is declared in MouseMotionListener interface?

a. public void mousePressed (MouseEvent me);
b. public void mouseDragged(MouseEvent me); Page 113
c. public void mouseClicked (MouseEvent me);
d. public void mouseEntered (MouseEvent me);

182. To make sure array accesses touch only the memory allocated to them, JVM does_____checking at runtime.

a. **dynamic Page 5 HO**
b. static
c. consistent
d. not

183. A hash map object is to create, to store Teacher objects as values and their CNIC in the form of strings as keys.Which syntax is correct for this requirement?
    a. HashMap<String> h =new HashMap<String=();
    b. HashMap<String, Teacher> h =new HashMap<String, Teacher>(); page 279 HO
    c. HashMap<Teacher> h =new HashMap<String, Teacher>();
    d. HashMap<Teacher> h =new HashMap<Teacher>();
184. A scrollable and updatable ResultSet object has the following capabilities except
    a. insert new rows
    b. insert new columns Page 144 HO
    c. delete rows
    d. update rows
185. Which of the following classes is capable to instantiate?
    a. Abstract class
    b. Concrete class Page 83 HO
    c. Interface class
    d. None of the given
186. It is verified by_____that not all bytecodes violate java security restrictions.
    a. Editor
    b. Compiler
    c. class loader
    d. bytecode verifier Page 12 HO
187. An event in java is represented as
    a. Operator
    b. Function
    c. Object Page 103 HO
    d. Primitive data type
188. In given lines of code, is applying.
    double a = 9.5;
    int b = (int) a
    a. upcasting a to b type
    b. upcasting b to a type

c. downcasting a to b type page 44 HO

d. downcasting b to a type

189. The_____is directly connected with keyboard.

a. File stream

b. Character oriented stream

c. Node stream google

d. Filter stream

190. For each class contained in java source file, compiler generates _____ class file/s.

a. **One Page 8 HO**

b. Two

c. Three

d. Four

191. In given lines of code,_____is applying.

int a = 20:

double b = a;

a. **upcasting a to b type Page 43 HO**

b. upcasting b to a type

c. downcasting a to b type

d. downcasting b to a type

192. Which of the following methods is called only once during the life cycle of an Applet?

a. start( )

b. paint()

c. stop()

d. destroy() Page 185 HO

e. init() Page 184

HOHere is Two answer

193. If there is a class "Student", then the java file must be saved with_____ name.

a. Student_File java

b. student.java

c. Any name

d. Student.java

194. Which feature of the following features is not included in Java language?

a. Inheritance

b. Polymorphism
c. Pointer manipulation Page 2 HO
d. Data encapsulation

195. Which of the following is a valid variable name in Java?
a. 1xu
b. Vu.506
c. NumberOfStudent
d. Long

196. A port is a_____to which processes can listen for connections request.
a. **Local Address Page 194**
b. Server Address
c. Client Address
d. Transport Address

197. Which of the following statements is a correct statement for "mouse release" event in "MouseListener" interface?
a. public void mouseRelease(MouseEvent me)
b. Public void mouseRelease(MouseEvent me
c. public void mouseReleased(MouseEvent me) Page 114 HO
d. Public void mouseReleased(MouseEvent me)

198. Which of the given will clear all the previous painting?
a. super.paintComponent() page 171 HO
b. super paint()
c. super. repaint()
d. super.redraw()

199. _____is/are not part of java core language.
a. **ArrayList class page 04 HO**
b. Variables
c. Arrays
d. Objects

200. _____in a collection class creates a collection with no elements.
a. **Constructors page 45 HO**
b. Class
c. Function
d. Array

201. == operator compares_____of strings whereas equals() method compares_____of strings.
   a. Values, addresses
   b. Addresses, values Page 19 HO
   c. Size, addresses
   d. Addresses, size

202. Assume that we do not want to serialize phoneNum attribute of Personinfo class, this can be done by;
   a. server String phoneNum;
   b. client String phoneNum;
   c. transient String phoneNum; Page 208 HO
   d. serializable String phoneNum:

203. Java program's code is compiled into a form called_____.
   a. Machine code
   b. Byte code Page 8 HO
   c. Native code
   d. Source code

204. You must include a_____to declare an exception that might be thrown to another method.
   a. try block
   b. finally block
   c. catch block
   d. throws-clause Page 57 HO

205. In the context of Java Graphics, which of the following methods we should override while painting?
   a. **paint() Page 170 HO**
   b. paintComponet( )
   c. paintBorder( )
   d. paintChildern( )

206. Which of the following features is not available in Java?
   a. Data hiding
   b. Data encapsulation
   c. Single inheritance
   d. Multiple inheritance Page 27 HO

207. Which of the following is a correct registration method for Focus Listener interface?

a. **addFocusListener**
b. addFocusedListener
c. regFocusListener
d. regFocusedListener

208. Which of the given statement objects are used to execute using same query multiple times by only changing parameters?

a. Statement
b. CallableStatement
c. PreparedStatement
d. Both PreparedStatement and CallableStatement

209. ArrayList is an example of_____collection type.

a. **Sequence based** **AL-JUNAID INSTITUTE OF GROUP**
b. Set based
c. Hashmap based
d. None of the given

210. The output of given someTest function is_____.

```
public void someTest( ) {
        int x;
        System.out.printIn(x);
}
```

a. x
b. syntax error
c. compile time error Page 30 HO
d. run time error

211. HTTP works on port_____.

a. 21
b. 23
c. 80 Page 193 HO
d. 84

212. _____is correct syntax to get component area of"myFrame" top level container.

a. Container c = myFrame.getComponentPane();
b. Container c = myFrame.getContentArea();

c. Container c = myFrame.getContentPane(); page 92 HQ

d. Container c = myFrame->getComponentArea();

213. How many methods are there in the Serializable Interface?

a. **Zero** **AL-JUNAID INSTITUTE OF GROUP**

b. One

c. Two

d. Three

214. Which one of the following is not a method of MouseListener interface?

a. Pressed

b. Clicked

c. Moved Page 113 HO

d. Released

215. Which of the given methods is/are declared for"MouseListener" interface?

a. MouseMoved

b. MouseDragged

c. MousePressed Page 113 HO

d. MouseKeyPressed

216. In editor java programs are created while stored on/in_____.

a. **Disk page 11 HO**

b. primary memory

c. external file

d. usb

217. Following are the code statements used in java event handling. Identify the code statement that is used for handling an event.

a. button = new JButton("Reset") Page 105 HO

b. button.addActionListener(this)

c. container.setLayout(new FlowLayout());

d. public void actionPerformed(ActionEvent e)

218. In polymorphism_____are overridden.

a. Variables

b. Methods

c. Data types Page 8 HO

d. Classes

219. _____understands the byte code?

a. Operating system

b. Java compiler page 8 HO

c. Java Virtual Machine

d. Java Interpreter

220. Which of the following code statement is used to define a connection URL?

a. Class.forName("sun.jdbc.odbc.JdbcOdbcDriver)

b. Connection abc = DriverManager.getConnection(conURL) Page 113 HO

c. String asd = "jdbc:odbc:DSNname"

d. Statement xyz = abc.createStatement()

221. Which of the following is a top level container?

a. JPanel

b. ToolBar

c. ScrollPane AL-JUNAID INSTITUTE OF GROUP

d. Dialog Page 90 HO

222. A variable declared as static in a class is a/an _____

a. Instance variable

b. Class variable Google

c. Const variable

d. Global variable

223. JDBC-ODBC stands for_____.

a. Java DataBase Connectivity- Oracle DataBase Connectivity

b. Java DataBase Corporation- Open DataBase Corporation

c. Java DataBase Corporation- Oracle DataBase Corporation

d. Java DataBase Connectivity- Open DataBase Connectivity Google

224. _____is required in order to get an object from collection.

a. Up-casting

b. Down casting

c. Implicit casting My Point of View

d. None of the given options

225. Choose the correct syntax to declare an abstract java classnamed as "Person".

a. abstract public class Person{}

b. public class Person abstract{}

c. public class abstract Person{}

d. public abstract class Person{} Pge 83

226. Which of the following is a correct statement for "mouse click" event in "MouseListener"" interface?
   a. public void mouseclicked (MouseEvent me)
   b. public void MouseClicked (MouseEvent me)
   c. public void mouseClicked (MouseEvent me) Page 114
   d. public void Mouseclicked (MouseEvent me)

227. All java files in a directory are compiled using_____command in command prompt.
   a.**javac *.java Page 6**
   b. javac all.java
   c. javac all
   d. java *.java

228. In the context of Socket Programming, the client does NOT know about the hostname of the machine on which the server is running.
   a. True
   b. False Page 194

229. How many ports are available for TCP socket?
   a. 64
   b. 64k Page 193
   c. 128
   d. 128k AL-JUNAID INSTITUTE OF GROUP

230. Which of the following function will be used to register event handler with events generator, where event generator is button?
   a. AddAction()
   b. addActionListener() Page 107
   c. addListener()
   d. registerListener

231. Which of the following statement object is used to execute precompiled SQL statements?
   a. Statement
   b. PreparedStatement Page 140
   c. CallableStatement
   d. None of give Option

232. There are_____steps in order to communicate successfully with database.

a. Six
b. Seven
c. Eight Page 132 HO
d. Nine

233. In given lines of code_____is static variable and_____is static method.
System.out.printin("CS506");
JOptionPane.showMessageDialog(null,"CS506")
a. System, JOptionPane
b. PrintIn, showMessageDialog
c. PrintIn, JOptionPane
d. Out, showMessageDialog page 33

234. The class that handles mouse event needs to implement the corresponding interface and to provide the definition of:
a. One method in that interface
b. Two methods in that interface
c. All the methods in that interface Page 113
d. Do not need to provide definition of methods in that interface

235. ResultSet provides various getters to retrieve_____.
a. **data of the column page 144**
b. data of the row
c. data of the row and column
d. column headers

236. Following are the code statements used in java event handling. Identify the code statement that is used for registering event handler with event generator
a. b1 = new JButton("OK") page 105
b. b1.addActiobListener(this)
c. frame = new JFrame()
d. frame.setSize(150, 150)

237. If a class implements an interface then_____
a. an interface must provide implementation of all methods
b. a class may or may not provide implementation of all methods of interface
c. a class must provide implementation of all methods of interface page 85

d. class will compile successfully if class do not implements methods of interface

238. The method "Math.random()" returns positive double value greater than or equal to 1 or less than 10.

a. True

b. False page 187 HO

239. Which of the following exceptions can be caught in a throw clause?

a. **Null pointer exception Google**

b. Index out of bounds exception

c. Division by zero exception AL-JUNAID INSTITUTE OF GROUP

d. IOException

240. The toString() method returns a_____representation of_____.

a. int,string

b. string,int

c. string,object Google

d. object,string

241. What will happen if main method is declared as private?

a. The program will not compile

b. The program will throw an exception at run

c. The program will compile and run successfully

d. None of given options page 18 HO

242. Which of the following stream is a Node stream?

a. PrintWriter

b. FileWriter

c. BufferedReader Page 69 HO

d. All of given option

243. The server just waits, listening to the socket for a client to make a connection request.

a. **True Page 193 HO**

b. False

244. A default ResultSet object has a cursor that_____.

a. moves backward only

b. moves forward only page 143

c. moves backward and Forward

d. cannot move

245. Java core language is_____than C++.
   a. **Simpler page 5 HO**
   b. Difficult
   c. Complex
   d. Less secure

246. Java does not handle exception by_____as keyword.
   a. try
   b. catch
   c. exception page 56 HO
   d. finally

247. In Java, Serializable Interface can be found under_____package.
   a. **java.io Page 203 HO**
   b. java.net
   c. java.sql
   d. java.util

248. If ResultSet is TYPE_FORWARD_ONLY then it_____when you call to the previous () method.
   a. **Moves the cursor to previous row if cursor is on valid row page 145 HO**
   b. Returns false if cursor is not on valid row
   c. Throws an exception
   d. Executes normally

249. Which of the following Statements is true regarding paintChildren() method?
   a. It tells the component to paint its border
   b. It first paints the background and then performs custom painting
   c. If first performs custom painting and then paints the background
   d. It tells any components contained by this component to paint themselves page 170 HO

250. TELNET works on_____port.
   a. 21
   b. 23 Page 193
   c. 80
   d. 84

251. Which of the given Listener interface(s) have adopter class(s)?

a. ActionListener
b. MouseMotionListener Page 119 HO
c. AdjustmentListener
d. ltemListener

252. Java compiler is called_____on UNIX or windows.
a. **Javac Page 6 HO**
b. java c
c. cjava
d. c java

253. Java language has a little worse performance than C++ language, because
a. it is object oriented
b. it uses built in libraries page 4 HO
c. it does not use pointers
d. it checks all errors, malicious and viruses

254. Which of the given Listener Interfaces is needed to handle the "MouseEvent"?
a. **MouseListener page 105 HO**
b. MouseMotionListener
c. MouseActionListener
d. MouseFocusListener

255. In Java, Transient fields are returned as null on reading.
a. **True page 208 HO**
b. False

256. Which of the given Listener Interfaces is need to handle the "KeyEvent"?
a. MouseKeyListener
b. MotionKeyListener
c. KeyListener Page 105 HO
d. KeyActionListener

257. An applet is a JFrame that allows interaction with a Java program.
a. True
b. False Page 181 HO

258. Given code is showing the result of addition of two integer type numbers. Choose correct output of this code.
Int i = 7; int j = 8; int k = i+j;

System.out.printIn("Seven" + i + "plus Eight" + j + "is" + k);

a. **Seven 7 plus Eight 8 is 15**
b. Seven i plus Eight j is k
c. Seven i plus Eight j is +15
d. Seven plus Eight is k

259. An applet is typically embedded in a Web page and can be run from a browser.

a. **True Page 181 HO**
b. False

260. In Java, we can read/write to a network using sockets.

a. True
b. False page 206 HO

261. Which of the following statements attaches an output stream to console?

a. FileWriter fw = new FileWriter("output.txt"); Page 80 HO
b. FileWriter fw = new FileWriter(FileDescriptor.out);
c. FileWriter fw = new FileWriter(FileDescriptor.in);
d. FileWriter fw = new FileWriter(FileDescriptor);

262. In java, adapter classes have been defined for the following listener interfaces except_____interface

a. FocusListener
b. ComponentListener
c. MouseMotionListener
d. ItemListener Page 119 HO

263. In Java Graphics, while painting a swing component. which of the following functions is invoked first?

a. Paint()
b. paintComponent() Page 169 HO
c. paintBorder()
d. PaintChildern()

264. If the subclass overrides all the abstract methods of super class, then subclass is called_____.

a. Abstract class
b. Concrete class Page 83 HO
c. Base class
d. Parent class

265. Which of the following command is used to invoke an Applet from command line interpreter?
   a. Applet htmlfile
   b. appletviewer htmlfile page 183 HO
   c. appletviewer javafile
   d. viewer htmlfile

266. The classes which contain the word_____in their names are byte oriented streams.
   a. Reader
   b. Writer
   c. Stream Page 71 HO
   d. Byte

267. Which of the following syntax is used to get DataBaseMetaData object?
   a. DataBaseMetaData md = rs.getMetaData();
      // where rs is ResultSet object
   b. DataBaseMetaData md = st.getMetaData();
      // where st is Statement object
   c. DataBaseMetaData md = con.getMetaData();
      // where con is Connection object Page 160 HO
   d. DataBaseMetaData md = rsmd_.getMetaData();
      // where rsmd is ResultSetMetaData object

268. In the context of Java Graphics; most of time, painting is done automatically.
   a. **True Page 165 HO**
   b. False

269. How many ports are available for UDP socket?
   a. 64
   b. 128
   c. 64k page 193 HO
   d. 128k

270. _____is the correct syntax to set size of top-level container "myFrame" as 200 rows in pixel and 400 columns in pixel.
   a. **myFrame.setSize(200,400) ;**
   b. myFrame.setSize(400,200) ;
   c. myFrame->setSize(200,400) ;
   d. myFrame->setSize(400,200) ;

271. int i = (int)(Math.random() * 5); // will generate random numbers between _____.

a. 0 & 4 page 187 HO

b. 0 & 5

c. 1 & 4

d. 1 & 5

272. If you want the timer to fire only the first time and then stop, invoke _____.

a. **setRepeats(false) Concept**

b. setRepeats(true)

c. setOnce(false)

d. setOnce(true)

273. In ResultSetMetaData methods,_____method returns the suggested column label for printouts.

a. getColumnName()

b. getColumnCount ()

c. getColumnType (int)

d. getColumnLabel() Page 157

274. Which of the following method/s is a ResultSetMetaData method?

a. All of the given options Page 157

b. getColumnCount ()

c. getColumnName(int)

d. getColumnDisplaySize (int)

275. Which of the following method is used to release the JDBC and database resources in ResultSet?

a. next()

b. getters

c. previous()

d. close() Page 144

276. Which of the following is the correct way to access writeObject() function using “out” object of ObjectOutputStream class.

a. out->writeObject()

b. out<-writeObject()

c. out->(.writObject())

d. out.writObject() Page 204

277. In ResultSetMetaData methods,_____method returns the number of columns in the result set.

a. getColumnCount ( ) Page 157

b. getColumnDisplaySize (int)

c. getColumnLabel (int)

d. getColumnType (int)

278. By implementing this interface, a class declares that it is willing to be _____by automatic serialization machinery.

a. Checked

b. Decode

c. Read Page 203

d. Destroyed

279. The_____method returns true or false depending upon whether the next row is available (exist) or not in ResultSet.

a. getters

b. next() Page 134

c. close()

d. close()

280. In ResultSetMetaData methods,_____ method returns the SQL type for the column to compare against types in java.sql Types

a. getColumn Type (int) Page 157

b. getColumnCount()

c. getColumnName(int)

d. getColumnDisplay Size (int)

281. To compile "Test.java" class, command is used in java.

a.**javac Test.java HO**

b. javac Test

c. java Test.java

d. java Test

282. The given line of code will result in _____

System.out.println(5+9);

a. 59

b. 5+9

c. 14

e. Syntax error

283. Which of the following is a DDL statement?
   a. Select
   b. Delete
   c. Create Page 137 HO
   d. Update

284. The collection API are defined in_____package.
   a. java.io
   b. java.util My Point of View
   c. java.awt
   d. java.sql

285. In Java,_____types of statement objects are available for sending SQL queries to the databases.
   a. Two
   b. Three Page 140 HO
   c. Four
   d. Five

286. Which of the following command is used to compile Java code?
   a. java filename
   b. javac filename.java Page 12 HO
   c. javac filename
   d. java filename.java

287. Which of the given Listener Interface is need to handle the "ActionEvent"?
   a. MouseActionListener
   b. KeyAction Listener
   c. ActionListener Page 105 HO
   d. MotionListener

288. Timers are constructed by specifying both a delay parameter and a/an _____Listener.
   a. **Action Page 179 HO**
   b. Mouse
   c. Key
   d. Window

289. All AWT and swing components generate events.
   a. **True Page 103 HO**
   b. False

290. Which of the following operator is used to concatenate strings?
   a. +=
   b. +-
   c. ++
   d. + Page 19 HO

291. DDL stands for?
   a. **Data Definition Language**
   b. Data Driven Language
   c. Database Definition Language
   d. Database Driven Language

292. AWT is a GUI package used in java, AWT stands for_____.
   a. Apache Windows Toolkit
   b. Application Windowing Tools
   c. Application Windows Template
   d. Abstract Windowing Toolkit Page 88 HO

293. Array List can store objects of_____?
   a. **Same types only My Point of View**
   b. Different types only
   c. Both same and different types
   d. Can't store objects

294. Mouse motion event is generated when mouse is:
   a. Pressed
   b. Entered
   c. Clicked
   d. Dragged Page 113 HO

295. If there is no exception in the try nor then the catch block_____executes.
   a. Always
   b. Sometime
   c. Never Google
   d. None of the given

296. DML stands for?
   a. Data Manipulation Language Google
   b. Data Management Language
   c. Direct Manipulation Language
   d. Direct Management Language

297. Which of the following statement regarding Java is false?
   a. Java is a programming language
   b. A Java program compiles into machine language
   c. Java is case sensitive language
   d. Java uses an interpreter

298. When there is just a onetime use of a particular object of a java class, we use _____.
   a. named object
   b. anonymous object Page 127 HO
   c. either named object or anonymous object
   d. either named object nor anonymous object

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Web Design and Development

CS506

Lecture 1: Java Features 4
Lecture 2: Java Virtual Machine & Runtime Environment 8
Lecture 3: Learning Basics 19
Lecture 4: Object Oriented Programming 28
Lecture 5: Inheritance 38
Lecture 6: Collections 45
Lecture 7: Intro to Exceptions 54
Lecture 8: Streams 68
Lecture 9: Abstract Classes and Interfaces 82
Lecture 10: Graphical User Interfaces 88
Lecture 11: Event Handling 103
Lecture 12: More Examples of Handling Events 113
Lecture 13: Adapter Classes 118
Lecture 14: Java Database Connectivity 129
Lecture 15: More On JDBC 137
Lecture 16: Result Set 143
Lecture 17: Meta Data 156
Lecture 18: Java Graphics 165
Lecture 19: How to Animate? 174
Lecture 20: Applets 181
Lecture 21: Socket Programming 193
Lecture 22: 202
Lecture 23: Multithreading 209
Lecture 24: More on Multithreading 220
Lecture 25: Web Application Development 231

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 26: Java Servlets 245
Lecture 27: Creating a Simple Web Application in Tomcat 258
Lecture 28: Servlets Lifecycle 267
Lecture 29: More on Servlets 278
Lecture 30: Dispatching Requests 287
Lecture 31: Session Tracking 297
Lecture 32: Session Tracking 2 308
Lecture 33: Address Book Case Study Using Servlets 322
Lecture 34: JavaServer Pages 334
Lecture 35: JavaServer Pages 341
Lecture 36 349
Lecture 37: JSP Action Elements and Scope 361
Lecture 38: JSP Custom Tags 375
Lecture 39: MVC + Case Study 389
Lecture 40: Model 2 Architecture MVC 404
Lecture 41: Layers and Tiers 427
Lecture 42: Expression Language 443
Lecture 43: JavaServer Pages Standard Tag Library (JSTL) 461
Lecture 44: Client Side Validation & JavaServer Faces (JSF) 472
Lecture 45: JavaServer Faces 481

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 1: Java Features

This handout is a traditional introduction to any language features. You might not be able to comprehend some of the features fully at this stage but don't worry, you'll get to know about these as we move on with the course.

1.1 Design Goals of Java

The massive growth of the Internet and the World-Wide Web leads us to a completely new way of looking at development of software that can run on different platforms like Windows, Linux and Solaris etc.

1.1.1 Right Language, Right Time

- Java came on the scene in 1995 to immediate popularity.
- Before that, C and C++ dominated the software development
  - compiled, no robust memory model, no garbage collector causes memory leakages, not great support of built in libraries
- Java brings together a great set of "programmer efficient" features
  - Putting more work on the CPU to make things easier for the programmer.

1.1.2 Java - Buzzwords (Vocabulary)

- From the original Sun Java whitepaper: "Java is a simple, object-oriented, distributed, interpreted, robust, secure, architecture-neutral, portable, high-performance, multi-threaded, and dynamic language."
- Here are some original java buzzwords.

1.1.3 Java -- Language + Libraries

- Java has two parts.
  - The core language -- variables, arrays, objects
    - < The Java Virtual Machine (JVM) runs the core language
    - < The core language is simple enough to run on small devices phones, smart cards, PDAs.
  - The libraries
    - < Java includes a large collection of standard library classes to provide "off the shelf" code. (Useful built-in classes that comes with the language to perform basic tasks)
    - < Example of these classes is String, ArrayList, HashMap, StringTokenizer (to break string into substrings), Date ...
    - < Java programmers are more productive in part because they have access to a large set of standard, well documented library classes.

© Copyright Virtual University of Pakistan

1.1.4 Simple

- Very similar C/C++ syntax, operators, etc.
- The core language is simpler than C++ -- no operator overloading, no pointers, no multiple inheritance
- The way a java program deals with memory is much simpler than C or C++.

1.1.5 Object-Oriented

- Java is fundamentally based on the OOP notions of classes and objects.
- Java uses a formal OOP type system that must be obeyed at compile-time and run-time.
- This is helpful for larger projects, where the structure helps keep the various parts consistent. Contrast to Perl, which has a more anything-goes feel.

1.1.6 Distributed / Network Oriented

- Java is network friendly -- both in its portable, threaded nature, and because Common networking operations are built-in to the Java libraries.

1.1.7 Robust / Secure / Safe

- Java is very robust
  - Both, vs. unintentional errors and vs. malicious code such as viruses.
  - Java has slightly worse performance since it does all this checking. (Or put the other way, C can be faster since it doesn't check anything.)
- The JVM "verifier" checks the code when it is loaded to verify that it has the correct Structure -- that it does not use an uninitialized pointer, or mix int and pointer types. This is one-time "static" analysis -- checking that the code has the correct structure without running it.
- The JVM also does "dynamic" checking at runtime for certain operations, such as pointer and array access, to make sure they are touching only the memory they should. You will write code that runs into
- As a result, many common bugs and security problems (e.g. "buffer overflow") are not possible in java. The checks also make it easier to find many common bugs easy, since they are caught by the runtime checker.
- You will generally never write code that fails the verifier, since your compiler is smart enough to only generate correct code. You will write code that runs into the runtime checks all the time as you debug -- array out of bounds, null pointer.
- Java also has a runtime Security Manager can check which operations a particular piece of code is allowed to do. As a result, java can run untrusted code in a "sandbox" where, for example, it can draw to the screen but cannot access the local file system.

1.1.8 Portable

- "Write Once Run Anywhere", and for the most part this works.
- Not even a recompile is required -- a Java executable can work, without change, on any Java enabled platform.

1.1.9 Support for Web and Enterprise Web Applications

- Java provides an extensive support for the development of web and enterprise

applications

- Servlets, JSP, Applets, JDBC, RMI, EJBs and JSF etc. are some of the Java technologies that can be used for the above mentioned purposes.

1.1.10 High-performance

- The first versions of java were pretty slow.
- Java performance has gotten a lot better with aggressive just-in-time-compiler (JIT) techniques.
- Java performance is now similar to C -- a little slower in some cases, faster in a few cases. However memory use and startup time are both worse than C.
- Java performance gets better each year as the JVM gets smarter. This works, because making the JVM smarter does not require any great change to the java language, source code, etc.

1.1.11 Multi-Threaded

- Java has a notion of concurrency wired right in to the language itself.
- This works out more cleanly than languages where concurrency is bolted on after the fact.

1.1.12 Dynamic

- Class and type information is kept around at runtime. This enables runtime loading and inspection of code in a very flexible way.

1.1.13 Java Compiler Structure

- The source code for each class is in a .java file. Compile each class to produce ".class" file.
- Sometimes, multiple .class files are packaged together into a .zip or .jar "archive" file.
- On UNIX or windows, the java compiler is called "javac". To compile all the .java files in a directory use "javac *.java".

1.1.14 Java: Programmer Efficiency

- Faster Development
  - Building an application in Java takes about 50% less time than in C or C++. So, Faster time to market
  - Java is said to be "Programmer Efficient".
- OOP
  - Java is thoroughly OOP language with robust memory system
  - Memory errors largely disappear because of the safe pointers and garbage collector. The lack of memory errors accounts for much of the increased programmer productivity.
- Libraries
  - Code re-uses at last -- String, ArrayList, Date, available and documented in a standard way

© Copyright Virtual University of Pakistan

1.1.15 Microsoft vs. Java

- Microsoft hates Java, since a Java program (portable) is not tied to any particular operating system. If Java is popular, then programs written in Java might promote non-Microsoft operating systems. For basically the same reason, all the non-Microsoft vendors think Java is a great idea.
- Microsoft's C# is very similar to Java, but with some improvements, and some questionable features added in, and it is not portable in the way Java is. Generally it is considered that C# will be successful in the way that Visual Basic is: a nice tool to build Microsoft only software.
- Microsoft has used its power to try to derail Java somewhat, but Java remains very popular on its merits.

1.1.16 Java Is For Real

- Java has a lot of hype, but much of it is deserved. Java is very well matched for many modern problem
- Using more memory and CPU time but less programmer time is an increasingly appealing tradeoff.
- Robustness and portability can be very useful features
- A general belief is that Java is going to stay here for the next 10-20 years

1.1.17 References

- Majority of the material in this handout is taken from the first handout of course cs193j at Stanford.
- The Java™ Language Environment, White Paper, by James Gosling & Henry McGilton
    - Java's Sun site: http://java.sun.com
    - Java World : www.javaworld.com

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

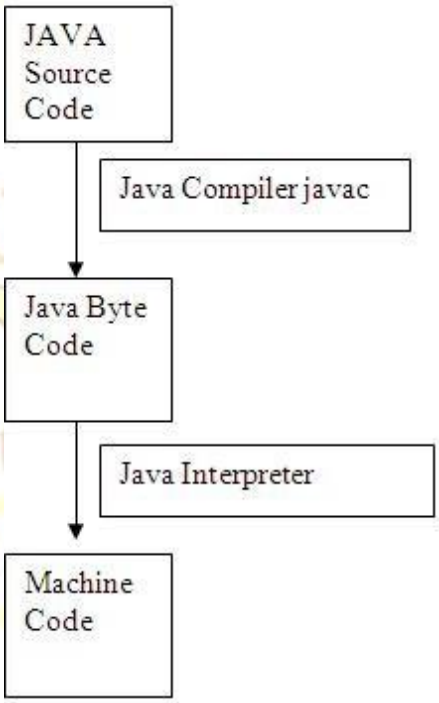
Lecture 2: Java Virtual Machine & Runtime Environment

2.1 Basic Concept

When you write a program in C++ it is known as source code. The C++ compiler converts this source code into the machine code of underlying system (e.g. Windows) If you want to run that code on Linux you need to recompile it with a Linux based compiler. Due to the difference in compilers, sometimes you need to modify your code.

Java has introduced the concept of WORA (write once run anywhere). When you write a java program it is known as the source code of java. The java compiler does not compile this source code for any underlying hardware system, rather it compiles it for a software system known as JVM (This compiled code is known as byte code). We have different JVMs for different systems (such as JVM for Windows, JVM for Linux etc). When we run our program the JVM interprets (translates) the compiled program into the language understood by the underlying system. So we write our code once and the JVM runs it everywhere according to the underlying system.

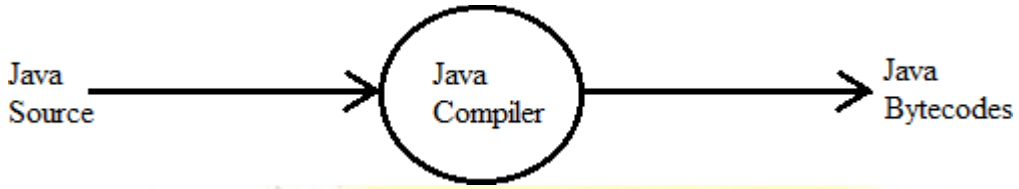
This concept is discussed in detail below



2.1.1 Byte code

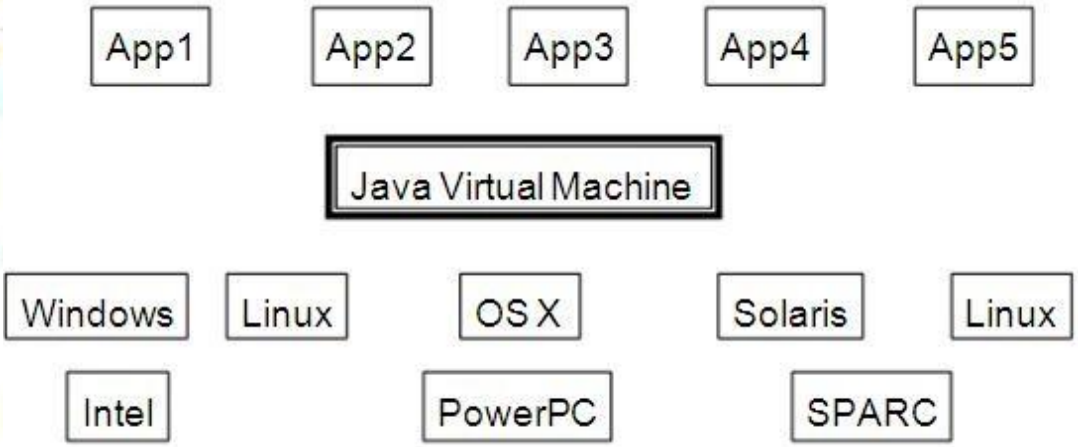
- Java programs (Source code) are compiled into a form called Java bytecodes.
- The Java compiler reads Java language source (.java) files, translates the source into Java bytecodes, and places the bytecodes into class (.class) files.
- The compiler generates one class file for each class contained in java source file.

© Copyright Virtual University of Pakistan



2.1.2 Java Virtual Machine (JVM)

- The central part of java platform is java virtual machine.
- Java bytecode executes by special software known as a "virtual machine".
- Most programming languages compile source code directly into machine code, suitable for execution
- The difference with Java is that it uses bytecode - a special type of machine code.
- The JVM executes Java bytecodes, so Java bytecodes can be thought of as the machine language of the JVM.

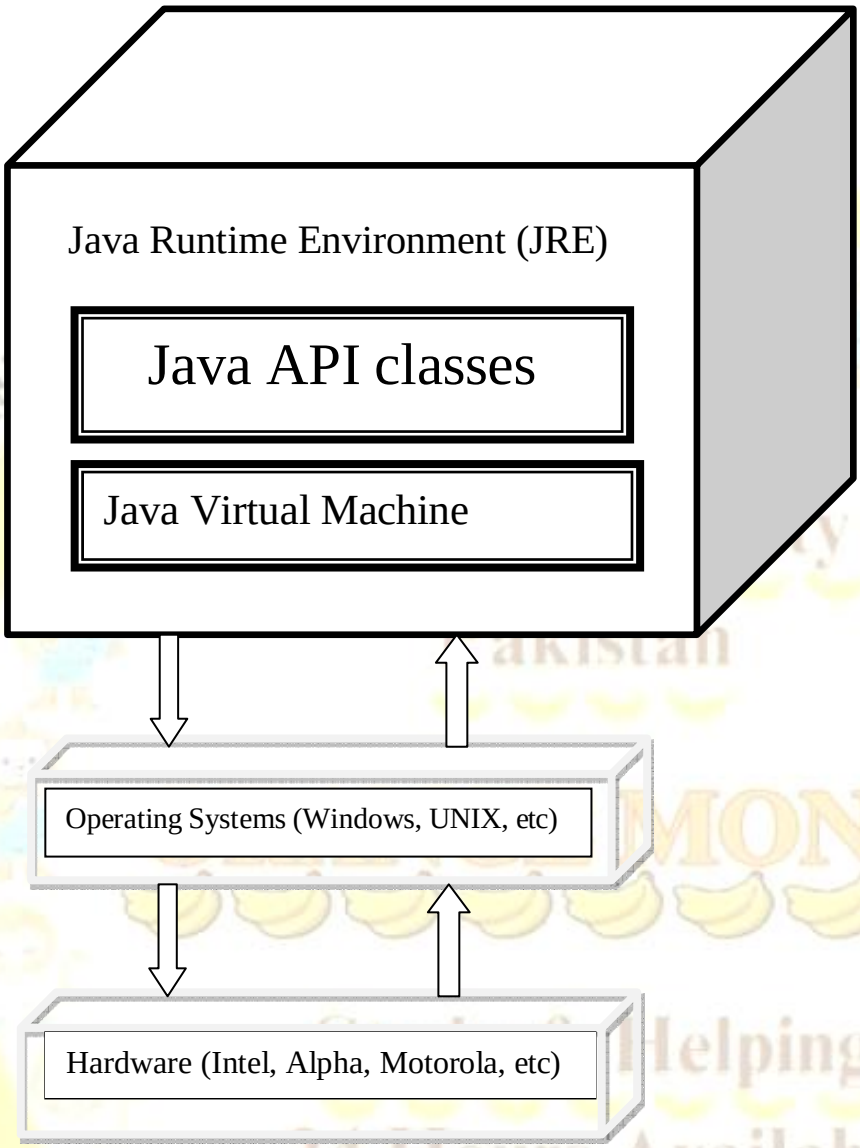


- JVM are available for almost all operating systems.
- Java byte code is executed by using any operating system's JVM. Thus achieve portability.

2.1.3 Java Runtime Environment (JRE)

- The Java Virtual Machine is a part of a large system i.e. Java Runtime Environment (JRE).
- Each operating system and CPU architecture requires different JRE.
- The JRE consists of set of built-in classes, as well as a JVM.
- Without an available JRE for a given environment, it is impossible to run Java software.

© Copyright Virtual University of Pakistan



2.1.4 References

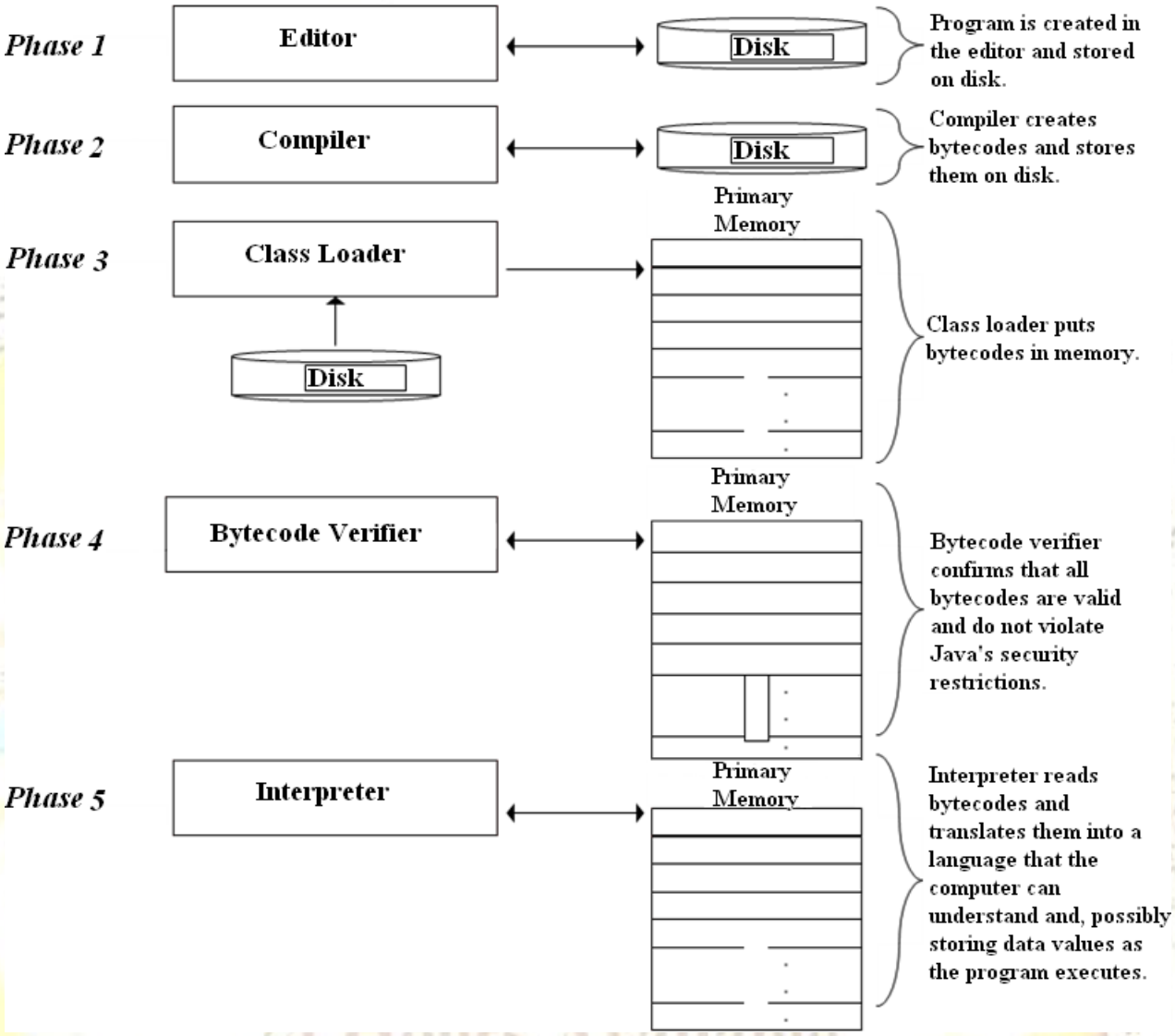
- Java World: http://www.javaworld.com
- Inside Java: http://www.javacoffeebreak.com/articles/inside_java

2.2 Java Program Development and Execution Steps

Java program normally go through five phases. These are

- Edit,
- Compile,
- Load,
- Verify and
- Execute

We look over all the above mentioned phases in a bit detail. First consider the following figure that summarizes the all phases of a java program.



2.2.1 Phase 1: Edit

Phase 1 consists of editing a file. This is accomplished with an editor program. The programmer types a java program using the editor like notepad, and make corrections if necessary.

When the programmer specifies that the file in the editor should be saved, the program is stored on a secondary storage device such as a disk. Java program file name ends with a .java extension.

On Windows platform, notepad is a simple and commonly used editor for the beginners. However java integrated development environments (IDEs) such as NetBeans, Borland JBuilder, JCreator and IBM's Eclipse Java built-in editors that are smoothly integrated into the programming environment.

© Copyright Virtual University of Pakistan

2.2.2 Phase 2: Compile

In Phase 2, the programmer gives the command javac to compile the program. The java compiler translates the java program into byte codes, which is the language understood by the java interpreter.

To compile a program called Welcome.java type **javac Welcome.java** at the command window of your system. If the program compiles correctly, a file called Welcome. Class is produced. This is the file containing the byte codes that will be interpreted during the execution phase.

2.2.3 Phase 3: Loading

In phase 3, the program must first be placed in memory before it can be executed. This is done by the *class loader*, which takes the .class file (or files) containing the byte codes and transfers it to memory. The .class file can be loaded from a disk on your system or over a network (such as your local university or company network or even the internet).

Applications (Programs) are loaded into memory and executed using the *java interpreter* via the command java. When executing a Java application called Welcome, the command

Java Welcome

Invokes the interpreter for the Welcome application and causes the class loader to load information used in the Welcome program.

2.2.4 Phase 4: Verify

Before the byte codes in an application are executed by the java interpreter, they are verified by the *byte code verifier* in Phase 4. This ensures that the byte codes for class that are loaded form the internet (referred to as *downloaded classes*) are valid and that they do not violate Java's security restrictions. Java enforces strong security because java programs arriving over the network should not be able to cause damage to your files and your system (as computer viruses might).

2.2.5 Phase 5: Execute

Finally in phase 5, the computer, under the control of its CPU, interprets the program one byte code at a time. Thus performing the actions specified by the program.Programs may not work on the first try. Each of the preceding phases can fail because of various errors. This would cause the java program to print an error message. The programmer would return to the edit phase, make the necessary corrections and proceed through the remaining phases again to determine if the corrections work properly.

2.2.6 References:

- Java™ How to Program 5th edition by Deitel & Deitel
- Sun Java online tutorial: http://java.sun.com/docs/books/tutorial/java/index.html

2.3 Installation and Environment Setting

2.3.1 Installation

- Download the latest version j2se5.0 (java 2 standard edition) from http://www.oracle.com/technetwork/java/javase/downloads/index.htmls or get it from any other source like CD.

- Note: j2se also called jdk (java development kit). You can also use the previous versions like jdk 1.4 or 1.3 etc. but it is recommended that you use either jdk1.4 or jdk5.0
- Install j2se5.0 on your system

Note: For the rest of this handout, assume that j2se is installed in C:\Program Files\Java\jdk1.5.0

2.3.2 Environment Setting

Once you successfully installed the j2se, the next step is environment or path setting. You can accomplish this in either of two ways.

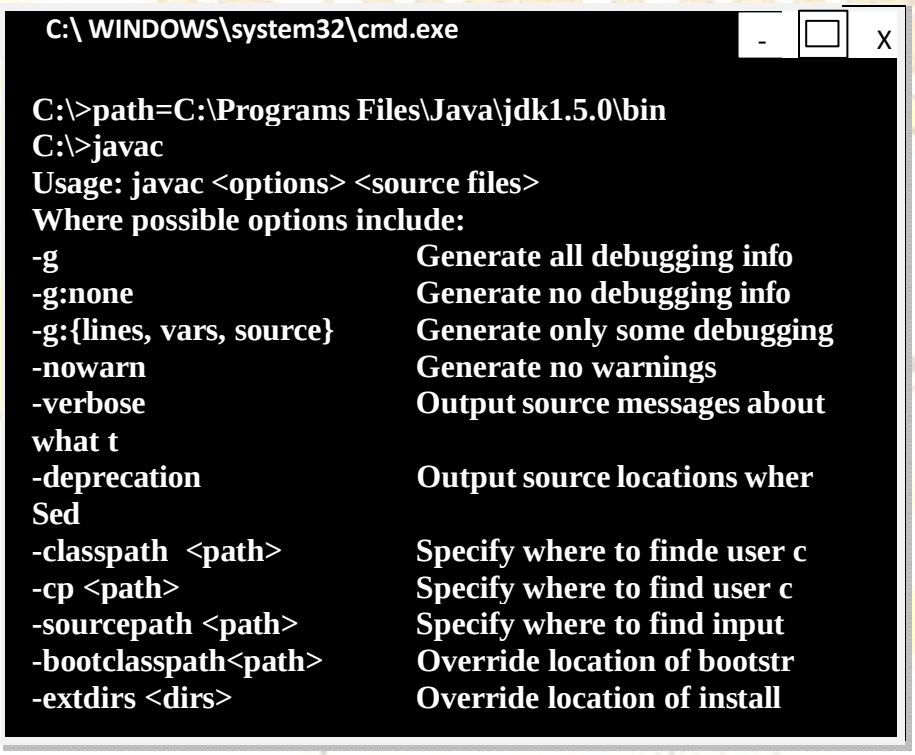
2.3.2.1 Temporary Path Setting

- Open the command prompt from Start -> Programs -> Accessories -> Command Prompt. The command prompt screen would be opened in front of you.
- Write the command on the command prompt according to the following format

  path = < java installation directory\bin >
- So, according to handout, the command will look like this

  path = C:\Program Files\Java\jdk1.5.0\bin
- To Test whether path has been set or not, write javac and press ENTER. If the list of options displayed as shown in the below figure means that you have successfully completed the steps of path setting.

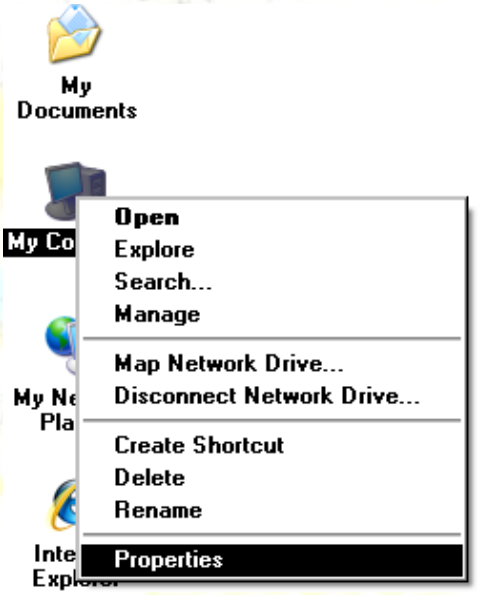
The above procedure is illustrates in the given below picture.



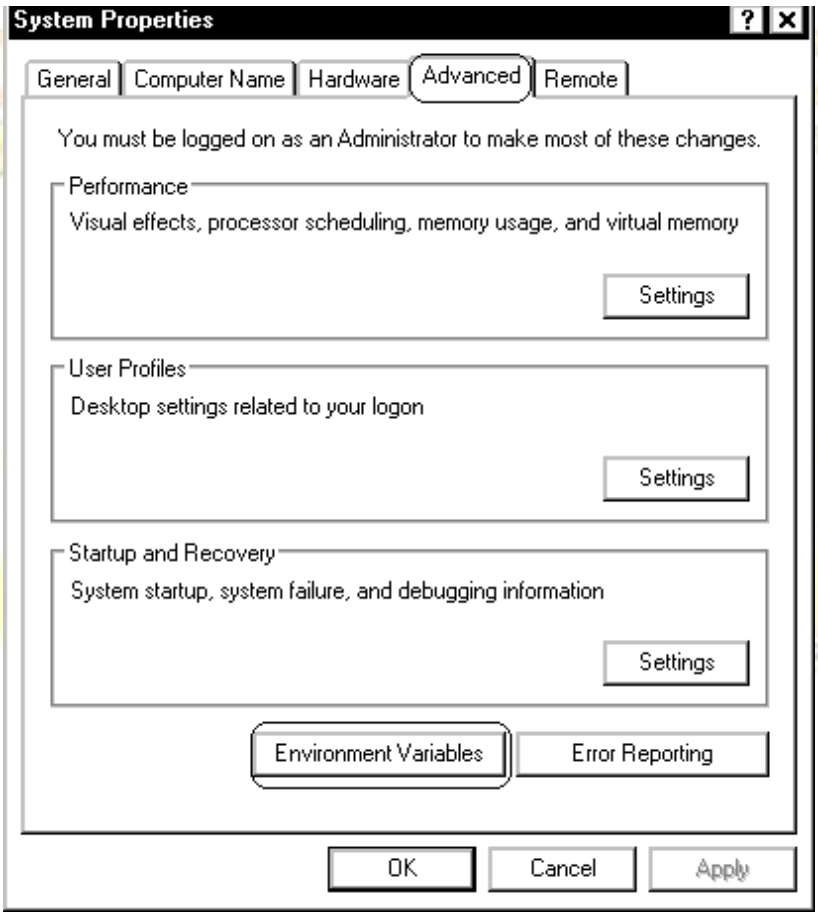
Note: The issue with the temporary path setting is you have to repeat the above explained procedure again and again each time you open a new command prompt window. To avoid this overhead, it is better to set your path permanently

2.3.2.2 Permanent Path Setting

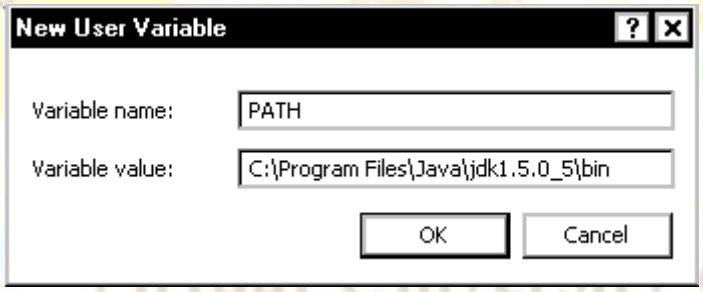
- In Windows NT (XP, 2000), you can set the permanent environment variable.
- Right click on my computer icon click on properties as shown below



A System Properties frame would appear as shown in the picture.



- Select the advanced tab followed by clicking the Environment Variable button. The Environment variables frame would be displayed in front of you
- Locate the Path variable in the System or user variables, if it is present there, select it by single click. Press Edit button. The following dialog box would be appeared.



- Write; **C:\Program Files\Java\jdk1.5.0\bin** at the end of the value field. Press OK button. Remember to write semicolon (;) before writing the path for java installation directory as illustrated in the above figure.
- If Path variable does not exist, click the New button. Write variable name "PATH", variable value **C:\Program Files\Java\jdk1.5.0\bin** and press OK button.

- Now open the command prompt and write **javac**, press enter button. You see the list of options would be displayed.
- After setting the path permanently, you have no need to set the path for each new opened command prompt.

2.3.3 References

Entire material for this handout is taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

2.4 First Program in Java

Like any other programming language, the java programming language is used to create applications. So, we start from building a classical "Hello World" application, which is generally used as the first program for learning any new language.

2.4.1 HelloWorldApp

- Open notepad editor from Start -> ProgarmFiles -> Accessories-> Notepad.
- Write the following code into it.

Note: Don't copy paste the given below code. Probably it gives errors and you can't able to remove them at the beginning stage.

```
1./*The HelloWorldApp class implements an application that
2.simply displays "Hello World!" to the standard output.
3.*/
4.  public class HelloWorldApp {
5.public static void main(String[] args) {
6.//Display the string. No global main
7.System.out.println("Hello World");
8. }
9. }
```

- To save your program, move to File menu and choose save as option.
- Save your program as "HelloWorldApp.java" in some directory. Make sure to add double quotes around class name while saving your program. For this example create a folder known as "examples" in D: drive

Note: Name of file must match the name of the public class in the file (at line 4). Moreover, it is case sensitive. For example, if your class name is MyClasS, than file name must be MyClasS. Otherwise the Java compiler will refuse to compile the program.

For the rest of this handout, we assume that program is saved in D:\examples directory.

2.4.2 HelloWorldApp Described

- **Lines 1-3**
  - Like in C++, You can add multiple line comments that are ignored by the compiler.
- **Lines 4**
  - Line 4 declares the class name as HelloWorldApp. In java, every line of code must reside inside class. This is also the name of our program (HelloWorldApp.java).The compiler creates the HelloWorldApp.class if this program successfully gets compiled.
- **Lines 5**
  - Line 5 is where the program execution starts. The java interpreter must find this defined exactly as given or it will refuse to run the program. (However you can change the name of parameter that is passed to main. i.e. you can write String[] argv or String[] someParam instead of String[] args)
  - Other programming languages, notably C++ also use the main()declaration as the starting point for execution. However the main function in C++ is global and resides outside of all classes where as in Java the main function must reside inside a class. In java there are no global variables or functions. The various parts of this main function declaration will be covered at the end of this handout.
- **Lines 6**
  - Again like C++, you can also add single line comment
- **Lines 7**
  - Line 7 illustrates the method call. The println() method is used to print something on the console. In this example println() method takes a string argument and writes it to the standard output i.e. console.

© Copyright Virtual University of Pakistan

- **Lines 8-9**
  - Line 8-9 of the program, the two braces, close the method main() and the classHelloWorldApprespectively.

2.4.3 Compiling and Running HelloWorldApp

- Open the command prompt from Start -> Program Files -> Accessories. OR alternatively you can write cmd in the run command window.
- Write cd.. to came out from any folder, and cd [folder name] to move inside the specified directory. To move from one drive to another, use [Drive Letter]: See figure given below
- After reaching to the folder or directory that contains your source code, in our case HelloWorldApp.java.
- Use "javac" on the command line to compile the source file (".java" file).
  - D:\examples> javac HelloWorld.java
- If program gets successfully compiled, it will create a new file in the same directory named HelloWorldApp.class that contains the byte-code.
- Use "java" on the command line to run the compiled .class file. Note ".class" would be added with the file name.
  - D:\examples> java HelloWorld
- You can see the Hello World would be printed on the console. Hurrah! You are successful in writing, compiling and executing your first program in java



2.4.4 Points to Remember

- Recompile the class after making any changes
- Save your program before compilation
- Only run that class using java command that contains the main method, because program executions always starts form main

2.5 An Idiom Explained

- You will see the following line of code often:
  - public static void main(String args[]) { …}
- About main()
  - "main" is the function from which your program starts
  - Why public?

- Since main method is called by the JVM that is why it is kept public so that it is accessible from outside. Remember private methods are only accessible from within the class

- Why static?
  - Every Java program starts when the JRE (Java Run Time Environment) calls the main method of that program. If main is not static then the JRE have to create an object of the class in which main method is present and call the main method on that object (In OOP based languages method are called using the name of object if they are not static). It is made static so that the JRE can call it without creating an object.
  - Also to ensure that there is only one copy of the main method per class
- Why void?
  - Indicates that main ( ) does not return anything.
- What is String args[] ?
  - Way of specifying input (often called command-line arguments) at startup of application. More on it latter

2.6 References

- Entire material for this handout is taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on "Downloads" section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 3: Learning Basics

3.1 Strings

A *string* is commonly considered to be a sequence of characters stored in memory and accessible as a unit. Strings in java are represented as objects.

3.1.1 String Concatenation

- "+" operator is used to concatenate strings
    - System.out.println("Hello" + "World") will print Hello World on console
- String concatenated with any other data type such as int will also convert that datatype to String and the result will be a concatenated String displayed on console. For example,
    - int i = 4;
    - int j = 5;
        - System.out.println ("Hello" + i);// will print Hello 4 on screen
    - However
        - System.out.println(i+j);//will print 9 on the console because both i and j are of type int.

3.1.2 Comparing Strings

For comparing Strings never use == operator, use *equals* method of String class.

- == operator compares addresses (shallow comparison) while equals compares values (deep comparison)
    - E.g. string1.equals(string2)

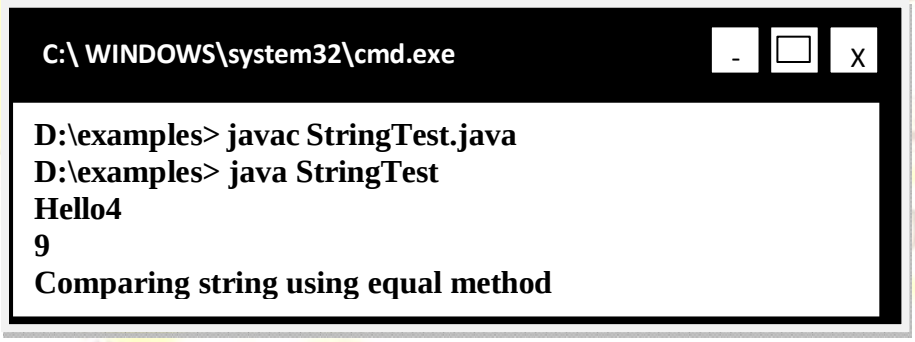
© Copyright Virtual University of Pakistan

Example Code: String concatenation and comparison

```
public class StringTest {
public static void main(String[] args) {
int i = 4;
int j = 5;
System.out.println("Hello" + i); // will print Hello4
System.out.println(i + j); // will print 9
String s1 = new String ("pakistan");


String s2 = "pakistan";
if (s1 == s2) {
System.out.println("comparing string using == operator");
}
if (s1.equals( s2) ) {
System.out.println("comparing string using equal method");
}
}
}
```

On execution of the above program, following output will produce



3.2 Taking in Command Line Arguments

In Java, the program can be written to accept *command-line-arguments.*

Example Code: command-line arguments

```
/* This Java application illustrates the use of Java command-line arguments. */
public class CmdLineArgsApp {
```

```
public static void main(String[] args){ //main method
System.out.println(”First argument ” + args[0]);
System.out.println(”Second argument ” +args[1]);
}//end main
}//End class.
```

To execute this program, we pass two arguments as shown below:

```
C:\java CmdLineArgsApp Hello World
First argument Hello
Second argument World
```

- These parameters should be separated by space.
- The parameters that we pass from the command line are stored as Strings inside the “args” array. You can see that the type of “args” array is String.

Example Code: Passing any number of arguments

In java, array knows their size by using the length property. By using, length property we can determine how many arguments were passed. The following code example can accept any number of arguments.

```
/* This Java application illustrates the use of Java command-line arguments. */
public class AnyArgsApp {
public static void main(String[] args){ //main method
for(int i=0; i < args.length; i++)
System.out.println(“Argument:” + i + “ value ” +args[i]);
}//end main
}//End class.
```

Output

```
C:\java AnyArgsApp
i can pass any number of arguments
Argument:0 value i
Argument:1 value can
Argument:2 value pass
Argument:3 value any
Argument:4 value number
Argument:5 value of
Argument:6 value arguments
```

3.3 Primitives vs. Objects

Everything in Java is an “Object”, as every class by default inherits from class “Object”,

except a few primitive data types, which are there for efficiency reasons.

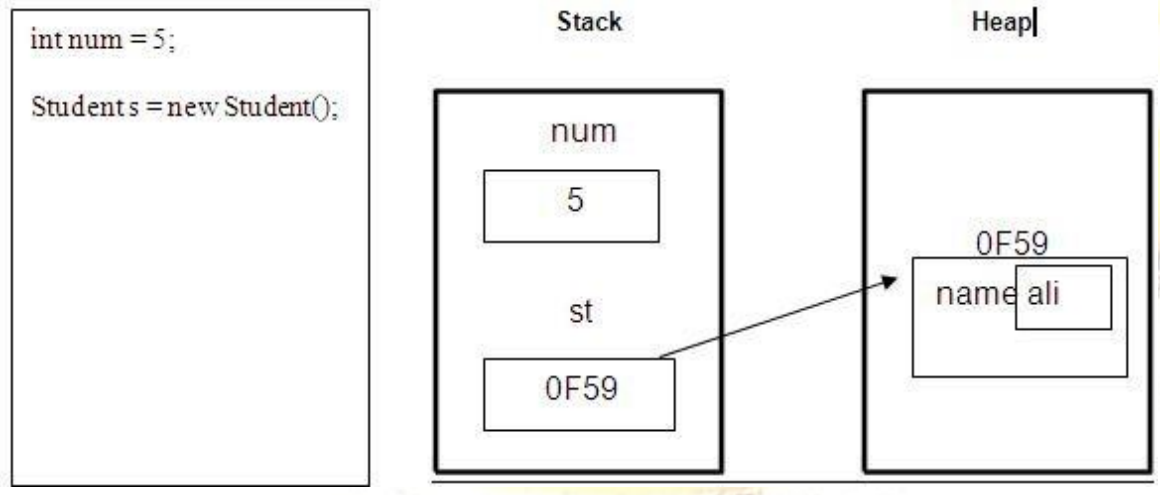
- Primitive Data Types
  - Primitive Data types of java

< boolean, byte	< 1 byte
< char, short	< 2 bytes
< int, float	< 4 bytes
< long, double	< 8 bytes

- Primitive data types are generally used for local variables, parameters and instance variables (properties of an object)
- Primitive data types are located on the stack and we can only access their value, while objects are located on heap and we have a reference to these objects
- Also primitive data types are always passed by value while objects are always passed by reference in java. There is no C++ like methods
  - void someMethod(int &a, int & b) // not available in java

3.4 Stack vs. Heap

Stack and heap are two important memory areas. Primitives are created on the stack while objects are created on heap. This will be further clarified by looking at the following diagram that is taken from Java Lab Course.



3.5 Wrapper Classes

Each primitive data type has a corresponding object (wrapper class). These wrapper classes provides additional functionality (conversion, size checking etc.), which a primitive data type cannot provide.

Primitive Data Type	Corresponding Object Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

3.5.1 Wrapper Use

You can create an object of Wrapper class using a String or a primitive data type

- Integer num = new Integer(4); or
- Integer num = new Integer("4");

Note: num is an object over here not a primitive data type

You can get a primitive data type from a Wrapper using the corresponding value function

- int primNum = num.intValue();

3.5.2 Converting Strings to Numeric Primitive Data Types

To convert a string containing digits to a primitive data type, wrapper classes can help. *parseXxx* method can be used to convert a String to the corresponding primitive data type.

- String value = "532";
  int d = Integer.parseInt(value);
- String value = "3.14e6";
  double d = Double.parseDouble(value);

The following table summarizes the parser methods available to a java programmer.

© Copyright Virtual University of Pakistan

Data Type	Convert String using either…
byte	Byte.parseByte(string) new Byte(string).byteValue()
short	Short.parseShort(string) new Short(string).shortValue()
int	Integer.parseInteger(string) new Integer(string).intValue()
long	Long.parseLong(String) new Long(string).longValue()
float	Float.parseFloat(string) new Float(string).floatValue()
double	Double.parseDouble(string) new Double(string).doubleValue()

Example Code: Taking Input / Output

So far, we learned how to print something on console. Now the time has come to learn how to print on the GUI. Taking input from console is not as straightforward as in C++. Initially we'll study how to take input through GUI (by using JOPtionPane class).

The following program will take input (a number) through GUI and prints its square on the console as well on GUI.

```
1. import javax.swing.*;
2. public class InputOutputTest {
3.     public static void main(String[] args) {
4. //takes input through GUI
5. String input = JOptionPane.showInputDialog("Enter number");
6. int number = Integer.parseInt(input);
7. int square = number * number;
8. //Display  square on console
9. System.out.println("square:" + square);
10. //Display square on GUI
11. JOptionPane.showMessageDialog(null, "square:"+ square);
12. System.exit(0);
13. }
14. }
```

On line 1, swing package was imported because it contains the *JOptionPane* class that will be used for taking input from GUI and displaying output to GUI. It is similar to header classes of C++.

On line 5, showInputDialog method is called of JOptionPane class by passing string argument that will be displayed on GUI (dialog box). This method always returns back a

String regardless of whatever you entered (int, float, double, char) in the input filed.

Our task is to print square of a number on console, so we first convert a string into a number by calling parseInt method of Integer wrapper class. This is what we done on line number 6.

Line 11 will display square on GUI (dialog box) by using showMessageDialog method of JOptionPane class. The first argument passed to this method is null and the second argument must be a String. Here we use string concatenation.

Line 12 is needed to return the control back to command prompt whenever we use JoptionPane class.

Compile & Execute

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available



3.6 Selection & Control Structure

The if-else and switch selection structures are exactly similar to we have in C++. All relational operators that we use in C++ to perform comparisons are also available in java with same behavior. Likewise for, while and do-while control structures are alike to C++.

3.7 Reference:

- Java tutorial: http://www.dickbaldwin.com/java
- Example code, their explanations and corresponding figures for this handout are taken from

the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on"**Downloads**" section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 4: Object Oriented Programming

Java is fundamentally object oriented. Every line of code you write in java must be inside a class (not counting import directives).OOP fundamental stones Encapsulation, Inheritance and Polymorphism etc. are all fully supported by java.

4.1 OOP Vocabulary Review

4.1.1 Classes

- Definition or a blueprint of a user defined data type
- Prototypes for objects
- Think of it as a map of the building on a paper

4.1.2 Objects

- Nouns, things in the world
- Anything we can put a thumb on
- Objects are instantiated or created from class

4.1.3 Constructor

- A special method that is implicitly invoked. Used to create an Object (that is, an Instance of the Class) and to initialize it.

4.1.4 Attributes

- Properties an object has

4.1.5 Methods

- Actions that an object can do

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

4.2 Defining a Class

```
class point {

private int xCord;

private int yCord;

public point()  { ….}

public void display() {  …..

      }

}// end of class
```

Instance variables and symbolic constants

Constructor: how to create and initialize objects

Methods: how to manipulate those objects (may or may not include its own "driver" i.e. main())

4.3 Comparison with C++

Some important points to consider when defining a class in java as you probably noticed from the above given skeleton are

- There are no global variables or functions. Everything resides inside a class. Remember we wrote our main method inside a class.(For example, in HelloWorldApp program)
- Specify access modifiers (public, private or protected) for each member method or data members at every line.
  - *public:* accessible anywhere by anyone
  - *private:* Only accessible within this class
  - *protect:* accessible only to the class itself and to its subclasses or other classes in the same package.
  - *default:* default access if no access modifier is provided. Accessible to all classes in the same package.
- There is no semicolon (;) at the end of class.
- All methods (functions) are written inline. There are no separate header and implementation files.
- Automatic initialization of class level data members if you do not initialize them
  - Primitives
    - < Numeric (int, float etc) with zero.
    - < Char with null
    - < Boolean with false
  - Object References
    - < With null

Note: Remember, the same rule is not applied to local variables (defined inside

method body). Using a local variable without initialization is a compile time error

```
public void someMethod( ) {
int x; //local variable
System.out.println(x); // compile time error
}
```

- Constructor
  - Same name as class name
  - Does not have a return type
  - No initialization list
  - JVM provides a *zero argument* (default) constructor only if a class doesn't define its own constructor
- Destructors
  - Are not required in java class because memory management is the responsibility of JVM.

4.4 Task - Defining a Student class

The following example will illustrate how to write a class. We want to write a "Student" class that

- Should be able to store the following characteristics of student
  - Roll No
  - Name
- Provide default, parameterized and copy constructors
  - Provide standard getters/setters (discuss shortly) for instance variables
    - < Make sure, roll no has never assigned a negative value i.e. ensuring the correct state of the object
    - < Provide print method capable of printing student object on console

4.5 Getters / Setters

The attributes of a class are generally taken as private or protected. So to access them outside of a class, a convention is followed knows as getters & setters. These are generally public methods. The words *set* and *get* are used prior to the name of an attribute. Another important purpose for writing getter & setters to control the values assigned to an attribute.

Student Class Code

```
// File Student.java
public class Student {
private String name;
```

```
private int rollNo;
// Standard Setters
public void setName (String name) {
this.name = name;
}
// Note the masking of class level variable rollNo
public void setRollNo (int rollNo) {
if (rollNo > 0) {
this.rollNo = rollNo; }else {
this.rollNo = 100;
}
}
// Standard Getters
public String getName ( ) {
return name;
}
public int getRollNo ( ) {
return rollNo;
}
// Default Constructor
public Student() {
name = “not set”;
rollNo = 100;
}
// parameterized Constructor for a new student
public Student(String name, int rollNo) {
setName(name);//call to setter of name
setRollNo(rollNo); //call to setter of rollNo
}
// Copy Constructor for a new student public Student(Student s) {
name = s.name;
rollNo = s.rollNo;
}
// method used to display method on console public void print () {
System.out.print("Student name: " +name);
System.out.println(", roll no: " +rollNo);    }
} // end of class
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

4.6 Using a Class

Objects of a class are always created on heap using the “new” operator followed by constructor

- Student s = new Student ( ); // no pointer operator “*” between Student and s
- Only String constant is an exception
    - String greet = “Hello” ; // No new operator

- However you can also use
- String greet2 = new String("Hello");

Members of a class (member variables and methods also known as instance variables/methods) are accessed using "." operator. There is no "–>" operator in java

- s.setName("Ali");
- s–>setName("Ali") is incorrect and will not compile in java

Note: Objects are always passed by reference and primitives are always passed by value in java.

4.6.1 Task - Using Student Class

- Create objects of student class by calling default parameterize and copy constructor
- Call student class various methods on these objects.

Student client code

```
// File Test.java
/* This class create Student class objects and demonstrates how to call various methods on objects
*/

public class Test{
public static void main (String args[]){
// Make two student obejcts

Student s1 = new Student("ali", 15);
Student s2 = new Student(); //call to default costructor
s1.print(); // display ali and 15
s2.print(); // display not set and 100
s2.setName("usman");
s2.setRollNo(20);

System.out.print("Student name:" + s2.getName());
System.out.println(" rollNo:" + s2.getRollNo());

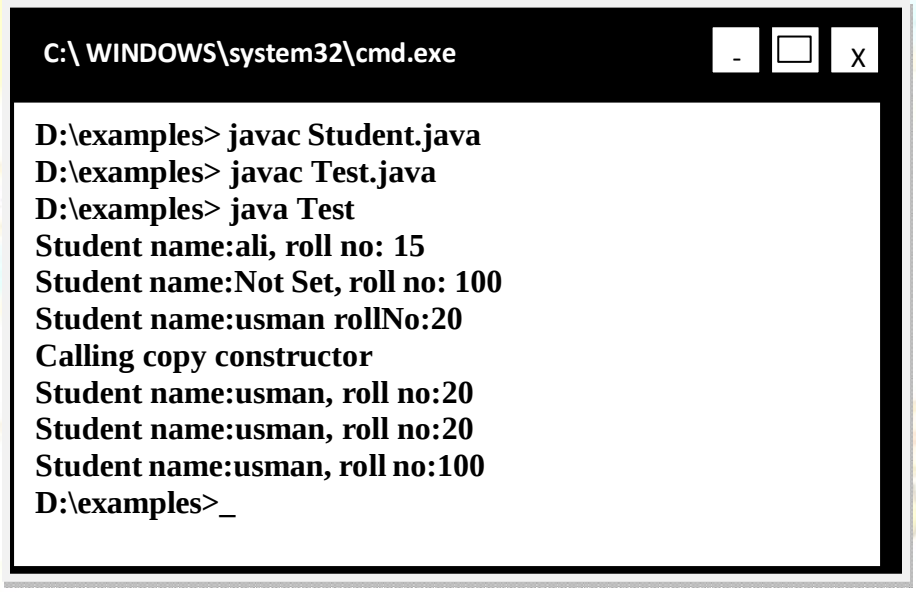
System.out.println("calling copy constructor");
Student s3 = new Student(s2); //call to copy constructor

s2.print();
s3.print();
s3.setRollNo(-10); //Roll No of s3 would be set to 100
s3.print();
/*NOTE: public vs. private
A statement like "b.rollNo = 10;" will not compile in a client of the Student class when rollNo is
```

```
declared
protected or private
*/
} //end of main
} //end of class
```

Compile & Execute

Compile both classes using **javac** command. Run Test class using **java** command.



4.7 More on Classes

4.7.1 Static

A class can have static variables and methods. Static variables and methods are associated with the class itself and are not tied to any particular object. Therefore statics can be accessed without instantiating an object. Static methods and variables are generally accessed by class name.

The most important aspect of statics is that they occur as a single copy in the class regardless of the number of objects. Statics are shared by all objects of a class. Non static methods and instance variables are not accessible inside a static method because no this reference is available inside a static method.

We have already used some static variables and methods. Examples are

- System.out.println("some text");---out is a static variable

- JOptionPane.**showMessageDialog**(null,"sometext");----- showMessageDialog is a static method

4.7.2 Garbage Collection & Finalize

Java performs garbage collection and eliminates the need to free objects explicitly. When an object has no references to it anywhere except in other objects that are also unreferenced, its space can be reclaimed. Before an object is destroyed, it might be necessary for the object to perform some action. For example: to close an opened file. In such a case, define a *finalize()* method with the actions to be performed before the object is destroyed.

4.7.2.1 Finalize

When a finalize method is defined in a class, Java run time calls *finalize()* whenever it is about to recycle an object of that class. It is noteworthy that a garbage collector reclaims objects in any order or never reclaims them. We cannot predict and assure when garbage collector will get back the memory of unreferenced objects.

The garbage collector can be *requested* to run by calling *System.gc()* method. It is not necessary that it accepts the request and run.

Example Code: using static & finalize ()

We want to count exact number of objects in memory of a Student class the one defined earlier. For this purpose, we'll modify Student class.

- Add a static variable *countStudents* that helps in maintaining the count of student objects.
- Write a getter for this static variable. (Remember, the getter also must be static one. Hoping so, you know the grounds).
- In all constructors, write a code that will increment the countStudents by one.
- Override *finalize()* method and decrement the countStudents variable by one.
- Override *toString()* method.

Class *Object* is a superclass (base or parent) class of all the classes in java by default. This class has already *finalize()* and *toString()* method (used to convert an object state into string). Therefore we are actually overriding these methods over here. (We'll talk more about these in the handout on inheritance).

By making all above modifications, student class will look like

```
// File Student.java
```

```
public class Student {
private String name;
private int rollNo;
private static int countStudents = 0;
// Standard Setters
public void setName (String name) {
this.name = name;
}

// Note the masking of class level variable rollNo
public void setRollNo (int rollNo) {
if (rollNo > 0) {
this.rollNo = rollNo; }else {
this.rollNo = 100;
}
}
// Standard Getters
public String getName ( ) {
return name;
}
public int getRollNo ( ) {
return rollNo;
}
// gettter of static countStudents variable
public static int getCountStudents(){
return countStudents;
}
// Default Constructor public Student() {
name = “not set”;
rollNo = 100;
countStudents += 1;
}
// method used to display method on console
public void print () {
System.out.print("Student name: " +name);
System.out.println(", roll no: " +rollNo);
}
// overriding toString method of java.lang.Object class
public String toString(){
return “name: ” + name + “ RollNo: ” + rollNo;
}
// overriding finalize method of Object class
public void finalize(){
countStudents -= 1;
}
} // end of class
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Next, we'll write driver class. After creating two objects of student class, we deliberately loose object's reference and requests the JVM to run garbage collector to reclaim the memory. By printing countStudents value, we can confirm that. Coming up code is of the Test class.

```
// File Test.java public class Test{
public static void main (String args[]){
int numObjs;
// printing current number of objects i.e 0
numObjs = Student.getCountStudents();
System.out.println("Students Objects" + numObjs);
// Creating first student object & printing its values
Student s1 = new Student("ali", 15);
System.out.println("Student: " + s1.toString());
// printing current number of objects i.e. 1
numObjs = Student.getCountStudents();
System.out.println("Students Objects" + numObjs);
// Creating second student object & printing its values
Student s2 = new Student("usman", 49);

// implicit call to toString() method
System.out.println("Student: " + s2);

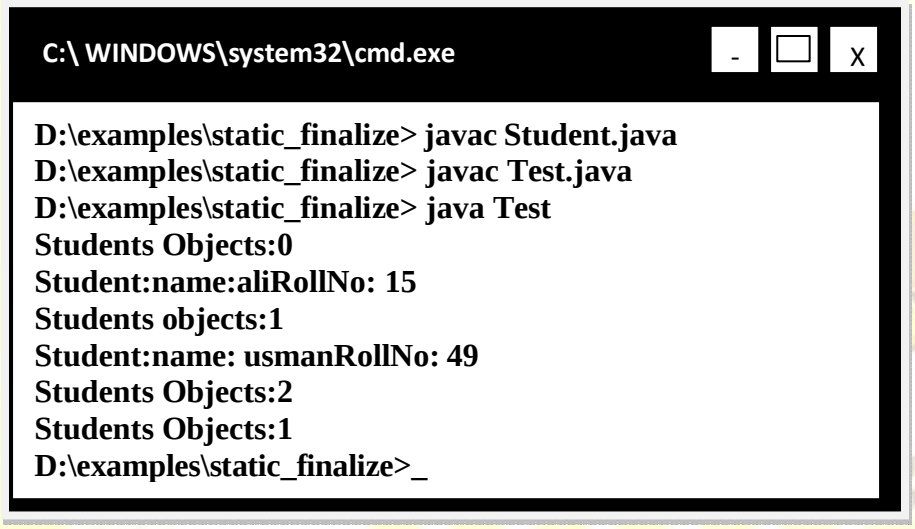
// printing current number of objects i.e. 2
numObjs = Student.getCountStudents();
System.out.println("Students Objects" + numObjs);

// loosing object reference s1 = null;
// requesting JVM to run Garbage collector but there is
// no guarantee that it will run
```

© Copyright Virtual University of Pakistan

```
System.gc();
// printing current number of objects i.e. unpredictable
numObjs = Student.getCountStudents();
System.out.println("Students Objects" + numObjs);
} //end of main
} //end of class
```

The compilation and execution of the above program is given below. Note that output may be different one given here because it all depends whether garbage collector reclaims the memory or not. Luckily, in my case it does.



4.8 Reference:

- Sun java tutorial: http://java.sun.com/docs/books/tutorial/java
- Thinking in java by Bruce Eckle
- Beginning Java2 by Ivor Hortan
- Example code, their explanations and corresponding execution figures for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Lecture 5: Inheritance

In general, inheritance is used to implement a "is-a" relationship. Inheritance saves code rewriting for a client thus promotes reusability.

In java parent or base class is referred as *super* class while child or derived class is known as *sub* class.

5.1 Comparison with C++

- Java only supports single inheritance. As a result a class can only inherit from one class at one time.
- Keyword *extends* is used instead of "**:**" for inheritance.
- All functions are virtual by default
- All java classes inherit from Object class (more on it later).
- To explicitly call the super class constructor, use *super* keyword. It's important to remember that call to super class constructor must be first line.
- Keyword super is also used to call overridden methods.

Example Code: using inheritance

We'll use three classes to get familiar you with inheritance. First one is Employee class. This will act as super class. Teacher class will inherit from Employee class and Test class is driver class that contains main method. Let's look at them one by one

```
class Employee{
protected int id;
protected String name;
//parameterized constructor
public Employee(int id, String name){
this.id = id;
this.name = name;
}
//default constructor
public Employee(){
// calling parameterized constructor of same (Employee)
// class by using keyword this
this (10, "not set");
}
//setters
public void setId (int id) {
```

```
this.id = id;
}
public void setName (String name) {
this.name = name;
}

//getters
public int getId () {
return id;
}
public String getName () {
return name;
}
// displaying employee object on console
public void display(){
System.out.println(“in employee display method”);
System.out.println("Employee id:" + id + " name:" + name);
}
//overriding object’s class toString method
public String toString() {
System.out.println(“in employee toString method”);
return "id:" + id + "name:" + name;
}
}//end class
```

The Teacher class extends from Employee class. Therefore Teacher class is a subclass of Employee. The teacher class has an additional attribute i.e. qualification.

```
class Teacher extends Employee{
private String qual;
//default constructor
public Teacher () {
//implicit call to superclass default construct qual = "";
}
//parameterized constructor
public Teacher(int i, String n, String q){
//call to superclass param const must be first line
super(i,n);
qual = q;
}
```

© Copyright Virtual University of Pakistan

```
//setter
public void setQual (String qual){
this.qual = qual;
}
//getter
public String getQual(){
return qual;
}
//overriding display method of Employee class
public void display(){
System.out.println("in teacher's display method");

super.display(); //call to superclass display method
System.out.println("Teacher qualification:" + qual);
}
//overriding toString method of Employee class
public String toString() {
System.out.println("in teacher's toString method");
String emp = super.toString();
return emp +" qualification:" + qual;
}
}//end class
```

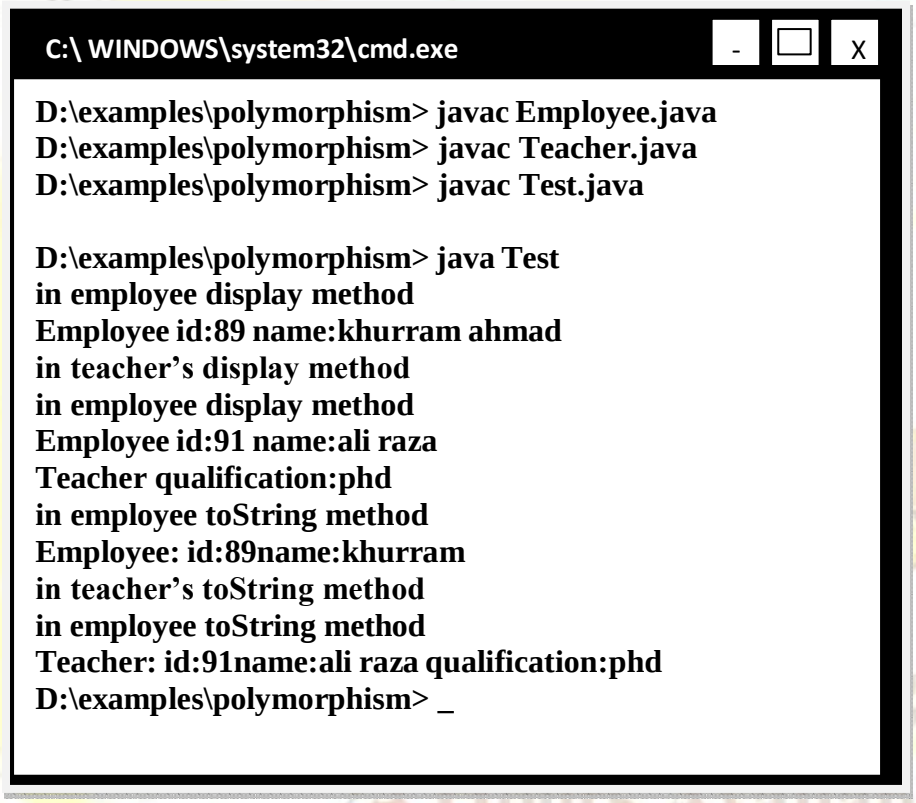
Objects of Employee & Teacher class are created inside main method in Test class. Later calls are made to display and toString method using these objects.

```
class Test{
public static void main (String args[]){
System.out.println("making object of employee");
Employee e = new Employee(89, "khurram ahmad");
System.out.println("making object of teacher");
Teacher t = new Teacher (91, "ali raza", "phd");
e.display(); //call to Employee class display method
t.display(); //call to Teacher class display method
```

© Copyright Virtual University of Pakistan

```
// calling employee class toString method explicitly
System.out.println("Employee: " +e.toString());
// calling teacher class toString implicitly
System.out.println("Teacher: " + t);
} //end of main
}//end class
```

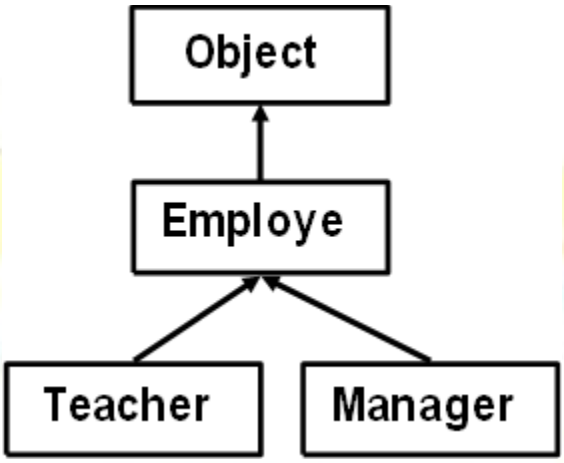
Output



5.2 Object - The Root Class

The Object class in Java is a superclass for all other classes defined in Java's class libraries, as well as for user-defined Java classes. For user defined classes, its not necessary to mention the Object class as a super class, java does it automatically for you.

The class Hierarchy of Employee class is shown below. Object is the super class of Employee class and Teacher is a subclass of Employee class. We can make another class Manager that can also extends from Employee class.



Virtual University of Pakistan

5.3 Polymorphism

"Polymorphic" literally means "of multiple shapes" and in the context of OOP, Polymorphic means "having multiple behaviors".

- A parent class reference can point to the subclass objects because of is-a relationship. For example a Employee reference can point to:
  - Employee Object
  - Teacher Object
  - Manager Object
- A polymorphic method results in different actions depending on the object being referenced
  - Also known as late binding or run-time binding

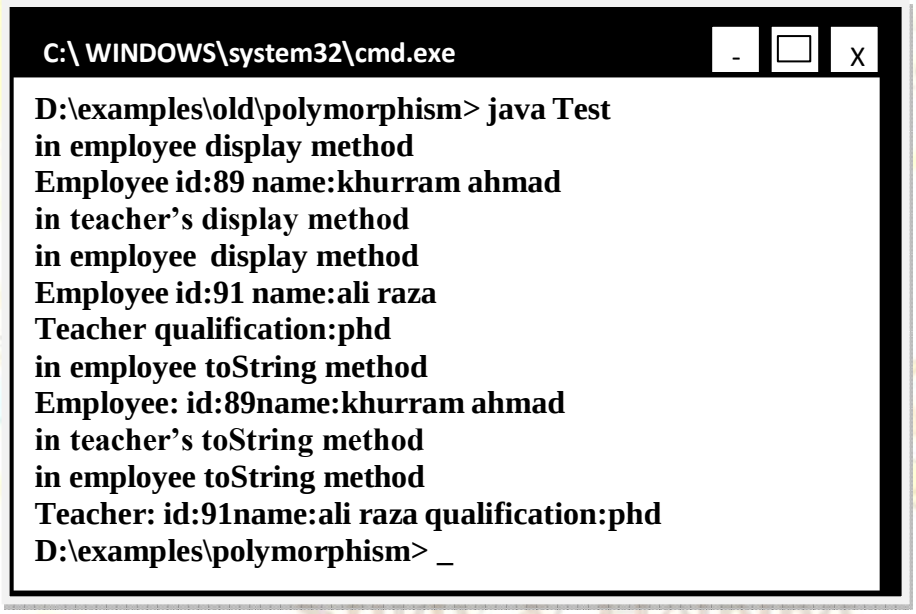
Example Code: using polymorphism

This Test class is the modification of last example code. Same Employee & Teacher classes are used. Objects of Employee & Teacher class are created inside main methods and calls are made to display and toString method using these objects.

```
class Test{
public static void main (String args[]){
// Make employee references
Employee ref1, ref2;
ref1 = new Employee(89, "khurram ahmad");
// assign employee object to first employee reference
ref2 = new Teacher (91, "ali raza", "phd");
```

```
// is-a relationship, polymorphism
//call to Employee class display method
ref1.display();
//call to Teacher class display method
ref2.display();
System.out.println("Employee: " +ref1.toString());// call to Employee class toString method
System.out.println("Teacher: " + ref2.toString());// call to Teacher class toString method
} //end of main
}//end class
```

Output



5.4 Type Casting

In computer science, **type conversion** or **typecasting** refers to changing an entity of one data type into another. Type casting can be categorized into two types

5.4.1 Up-casting

- converting a smaller data type into bigger one
- Implicit - we don't have to do something special
- No loss of information
- Examples of
  - Primitives

```
int a = 10;
double b = a;
```

  - Classes

```
Employee e = new Teacher( );
```

5.4.2 Down-casting

- converting a bigger data type into smaller one
- Explicit - need to mention
- Possible loss of information
- Examples of
  - Primitives

```
double a = 7.65;
int    b = (int) a;
```

  - Classes

```
Employee e = new Teacher( );// up-casting
Teacher t= (Teacher) e;// down-casting
```

5.5 References:

- Java tutorial: http://java.sun.com/docs/books/tutorial/java/javaOO/
- Stanford university
- Example code, their explanations and corresponding figures for handout 5-1,5-2 are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 6: Collections

A collection represents group of objects know as its elements. Java has a built-in support for collections. Collection classes are similar to STL in C++. An advantage of a collection over an array is that you don't need to know the eventual size of the collection in order to add objects to it. The java.utilpackage provides a set of collection classes that helps a programmer in number of ways.

6.1 Collections Design

All classes almost provides same methods like get (), size (), isEmpty() etc. These methods will return the object stored in it, number of objects stored and whether collection contains an object or not respectively.

Java collections are capable of storing any kind of objects. Collections store references to objects. This is similar to using a void* in C. therefore down casting is required to get the actual type. For example, if string in stored in a collection then to get it back, we write

```
String element = (String)arraylist.get(i);
```

6.2 Collection messages

Some basic messages (methods) are:

- Constructor
  - creates a collection with no elements.
- int size()
  - returns the number of elements in a collection.
- boolean add(Object)
  - adds a new element in the collection.
  - returns true if the element is added successfully false otherwise.
- boolean isEmpty()
  - returns true if this collection contains no element false otherwise.
- boolean contains(Object)
  - returns true if this collection contains the specified element by using iterative search.
- boolean remove(Object)
  - removes a single instance of the specified element from this collection, if it is present .

© Copyright Virtual University of Pakistan

6.3 Array List

It's like a resizable array. Array List actually comes as a replacement the old "Vector" collection. As we add or remove elements into or from it, it grows or shrinks over time.

6.3.1 Useful Methods

- add (Object)
  - With the help of this method, any object can be added into Array List because Object is the superclass of all classes.
  - Objects going to add will implicitly up cast.
- Object get(int index)
  - Returns the element at the specified position in the list
  - Index ranges from 0 to size()-1
  - Must cast to appropriate type
- remove (int index)
  - Removes the element at the specified position in this list.
  - Shifts any subsequent elements to the left (subtracts one from their indices).
- int size( )

Example Code: Using Array List class

We'll store Student objects in the Array List. We are using the same student class which we built in previous lectures/handouts.

We'll add three student objects and later prints all the student objects after retrieving them from Array List. Let's look at the code:

```
import java.util.*;
public class ArrayListTest {
public static void main(String[] args) {

// creating arrayList object by calling constructor
ArrayList<Student> al= new ArrayList<Student>();

// creating three Student objects
Student s1 = new Student ("ali" , 1);
Student s2 = new Student ("saad" , 2);
Student s3 = new Student ("raza" , 3);
// adding elements (Student objects) into arralylist
```

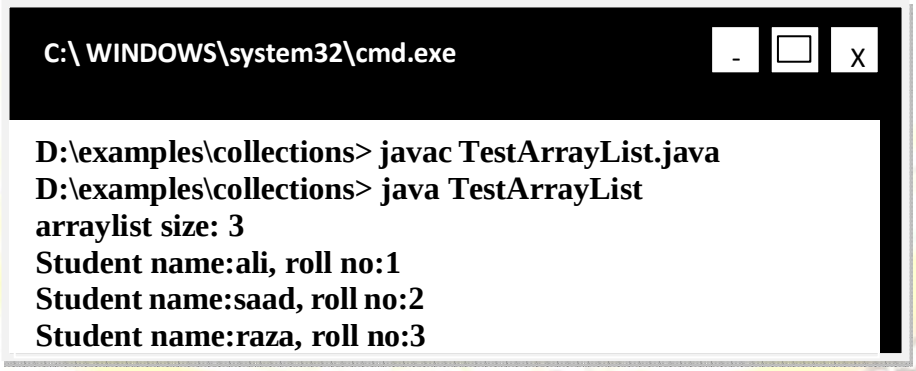
© Copyright Virtual University of Pakistan

```
 al.add(s1);
al.add(s2);
al.add(s3);

// checking whether arraylist is empty or not Boolean
b = al.isEmpty ();
if (b == true) {
System.out.println("arraylist is empty");
}
else {
int size = al.size();
System.out.println("arraylist size: " + size);
}

// using loop to iterate. Loops starts from 0 to one
// less than size
for (int i=0; i<al.size(); i++ ){
// retrieving object from arraylist
Student s = (Student) al.get(i);
// calling student class print method
s.print();
} // end for loop
} // end main
} // end class
```

Output



6.4 HashMap

Store elements in the form of *key- value* pair form. A key is associated with each object that is stored. This allows fast retrieval of that object. Keys are unique.

6.4.1 Useful Methods

- put(Object key, Object Value)
  - Keys & Values are stored in the form of objects (implicit up casting is performed).
  - Associates the specified value with the specified key in this map.
  - If the map previously contained a mapping for this key, the old value is replaced.
- Object get(Object key)
  - Returns the value to which the specified key is mapped in this identity hash map, or nullif the map contains no mapping for this key.
  - Must downcast to appropriate type when used
- int size( )

Example Code: using HashMap class

In this example code, we'll store Student objects as values and their rollnos in the form of strings as keys. Same Student class is used. The code is:

```
import java.util.*;
public class HashMapTest {
public static void main(String[] args) {
// creating HashMap object
HashMap <String, Student> h =new HashMap<String, Student>();
// creating Student objects
Student s1 = new Student ("ali" , 1);
Student s2 = new Student ("saad" , 2);
Student s3 = new Student ("raza" , 6);

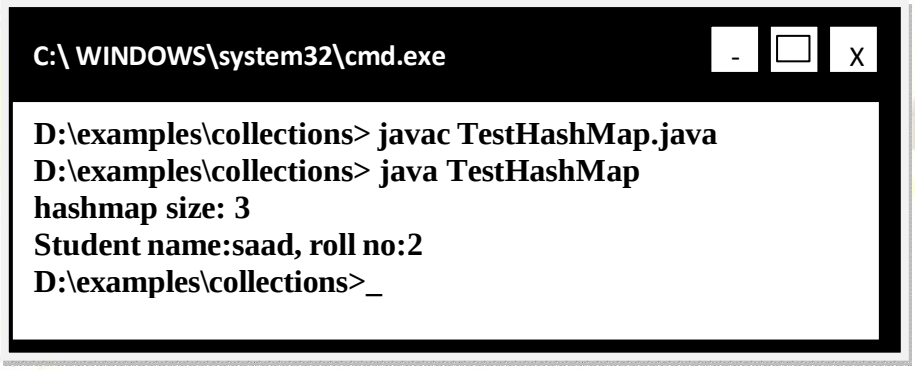
// adding elements (Student objects) where roll nos
// are stored as keys and student objects as values
h.put("one",s1);
h.put("two",S2);
h.put("three", S3);
// checking whether hashmap is empty or not
boolean b = h.isEmpty ();
if (b == true) {
System.out.println("hashmap is empty");
}
else {
int size = h.size();
System.out.println("hashmap size: " + size);
}
```

© Copyright Virtual University of Pakistan

```
// retrieving student object against rollno two and
// performing downcasting
Student s = (Student) h.get("two");
// calling student's class print method s.print();

} // end main
} // end class
```

Output



6.5 References:

- J2SE 5.0 new features: http://java.sun.com/j2se/1.5.0/docs/relnotes/features.html
- Technical Article: http://java.sun.com/developer/technicalArticles/releases/j2se15/
- Beginning Java2 by Ivor Horton
- Example code, their explainations and corresponding figures for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

6.6 Address Book

Warning: It is strongly advised that you have to type the code given in this example by yourself. Do not **copy/paste** it; most probably you will get unexpected errors that you have never seen. Some bugs are deliberately introduced as well to avoid copy-pasting. TAs will not cooperate with you in debugging such errors.

© Copyright Virtual University of Pakistan

6.6.1 Problem

- We want to build an address book that is capable of storing name, address & phone number of a person.
- Address book provides functionality in the form of a JOptionPane based menu. The feature list includes
    - Add - to add a new person record
    - Delete - to delete an existing person record by name
    - Search - to search a person record by name
    - Exit - to exit from application
- The Address book should also support persistence for person records

6.6.2 Approach for Solving Problem

Building a small address book generally involves 3 steps. Let us briefly discuss each step and write a solution code for each step

6.6.2.1 Step1 - Make PersonInfo class

- First of all you need to store your desired information for each person. For this you can create a user-defined data type (i.e. a class). Make a class PersonInfowith name, address and phone number as its attributes.
- Write a parameterized constructor for this class.
- Write print method in Person class that displays one person record on a message dialog box.

The code for **PersonInfo** class is given below.

```
import javax.swing.*;
class PersonInfo {

String name;
String address;
String phoneNum;
//parameterized constructor
public PersonInfo(String n, String a, String p) {
name = n; address = a; phoneNum = p;
}

//method for displaying person record on GUI
public void print( ) {
JOptionPane.showMessageDialog(null, “name: ” + name + “address:” +address + “phone no:” +
```

© Copyright Virtual University of Pakistan

```
phoneNum);
}
}
```

Note: Not declaring attributes as private is a bad approach but we have done it to keep things simple here.

6.6.2.2 Step2 - Make Address Book class

- Take the example of daily life; generally address book is used to store more than one person records and we don't know in advance how many records are going to be added into it.
- So, we need some data structure that can help us in storing more than one PersonInfoobjects without concerning about its size.
- Array Listcan be used to achieve the above functionality
- Create a class Address Book with an ArrayList as its attribute. This arraylist will be used to store the information of different persons in the form of PersonInfo Objects. This class will also provide *addPerson*, *deletePerson* & *searchPerson* methods. These methods are used for adding new person records, deleting an existing person record by name and searching among existing person records by name respectively.
- Input/Output will be performed through JOptionPane.

The code for **AddressBook**class is

```
import javax.swing.*;
import java.util.*;

class AddressBook {
ArrayList<PersonInfo> persons;
//constructor
public AddressBook ( ) {
persons = new ArrayList()<PersonInfo>();
}

//add new person record to arraylist after taking input
public void addPerson( ) {
String name = JOptionPane.showInputDialog("Enter name");
String add    = JOptionPane.showInputDialog("Enter address"); String pNum =
JOptionPane.showInputDialog("Enter phone no");
//construct new person object
PersonInfo p = new PersonInfo(name, add, pNum);
//add the above PersonInfo object to arraylist
persons.add(p);
}
//search person record by name by iterating over arraylist
public void searchPerson (String n) {
```

© Copyright Virtual University of Pakistan

```
for (int i=0; i< persons.size(); i++) {
PersonInfo p = (PersonInfo)persons.get(i);
if ( n.equals(p.name) ) {
p.print();
}
} // end for
} // end searchPerson
//delete person record by name by iterating over arraylist
public void deletePerson (String n) {
for (int i=0; i< persons.size(); i++) {
PersonInfo p = (PersonInfo)persons.get(i);
if ( n.equals(p.name))
{
persons.remove(i);
}
}
}
} // end class
```

The *addperson* method first takes input for name, address and phone number and than construct a PersonInfo object by using the recently taken input values. Then the newly constructed object is added to the arraylist - *persons*.

The *searchPerson* & *deletePerson* methods are using the same methodology i.e. first they search the required record by name and then prints his/her detail or delete the record permanently from the ArrayList.

Both the methods are taking string argument, by using this they can perform their search or delete operation. We used forloop for iterating the whole ArrayList.By using the size method of ArrayList, we can control our loop as ArrayList indexes range starts from 0 to one less than size.

Notice that, inside loop we retrieve each PersonInfo object by using down casting operation. After that we compare each PersonInfo object's name by the one passed to these methods using equal method since Strings are always being compared using equalmethod.

Inside ifblock of *searchPerson*, *print* method is called using PersonInfo object that will display person information on GUI. On the other hand, inside ifblock of *deletePerson* method, remove method of ArrayList class is called that is used to delete record from *persons* i.e. ArrayList.

6.6.2.3 Step3 - Make Test class (driver program)

- This class will contain a mainmethod and an object of AddressBook class.
- Build GUI based menu by using switch selection structure
- Call appropriate methods of AddressBookclass

The code for **Test**class is

```
import javax.swing.*;
class Test {
public static void main (String args[]) {
AddressBook ab = new AddressBook();
String input, s;
int ch;
while (true) {
input = JOptionPane.showInputDialog("Enter 1 to add " + "\n Enter 2 to Search \n Enter 3 to
Delete" +
"\n Enter 4 to Exit");
ch = Integer.parseInt(input);
switch (ch) {
case 1:
ab.addPerson();
break;
case 2:
s = JOptionPane.showInputDialog( "Enter name to search ");
ab.searchPerson(s);
break;
case 3:
s = JOptionPane.showInputDialog( "Enter name to delete ");
ab.deletePerson(s);
break;
case 4: System.exit(0);
}
}//end while
}//end main
}
```

Note that we use *infinite* while loop that would never end or stop given that our program should only exit when user enters 4 i.e. exit option.

Compile & Execute

Compile all three classes and run Test class. Bravo, you successfully completed the all basic three steps. Enjoy!

6.7 Reference

Entire content for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 7: Intro to Exceptions

7.1 Types of Errors

Generally, you can come across three types of errors while developing software. These are Syntax, Logic & Runtime errors.

7.1.1 Syntax Errors

- Arise because the rules of the language are not followed.

7.1.2 Logic Errors

- Indicates that logic used for coding doesn't produce expected output.

7.1.3 Runtime Errors

- Occur because the program tries to perform an operation that is impossible to complete.
- Cause exceptions and may be handled at runtime (while you are running the program)
- For example divide by zero

7.2 What is an Exception?

- An exception is an event that usually signals an erroneous situation at run time
- Exceptions are wrapped up as objects
- A program can deal with an exception in one of three ways:
    - ignore it
    - handle it where it occurs
    - handle it an another place in the program

7.3 Why handle Exceptions?

- Helps to separate error handling code from main logic (the normal code you write) of the program.
- As different sort/type of exceptions can arise, by handling exceptions we can

© Copyright Virtual University of Pakistan

distinguish between them and write appropriate handling code for each type for example we can differently handle exceptions that occur due to division by Zero and exceptions that occur due to non-availability of a file.

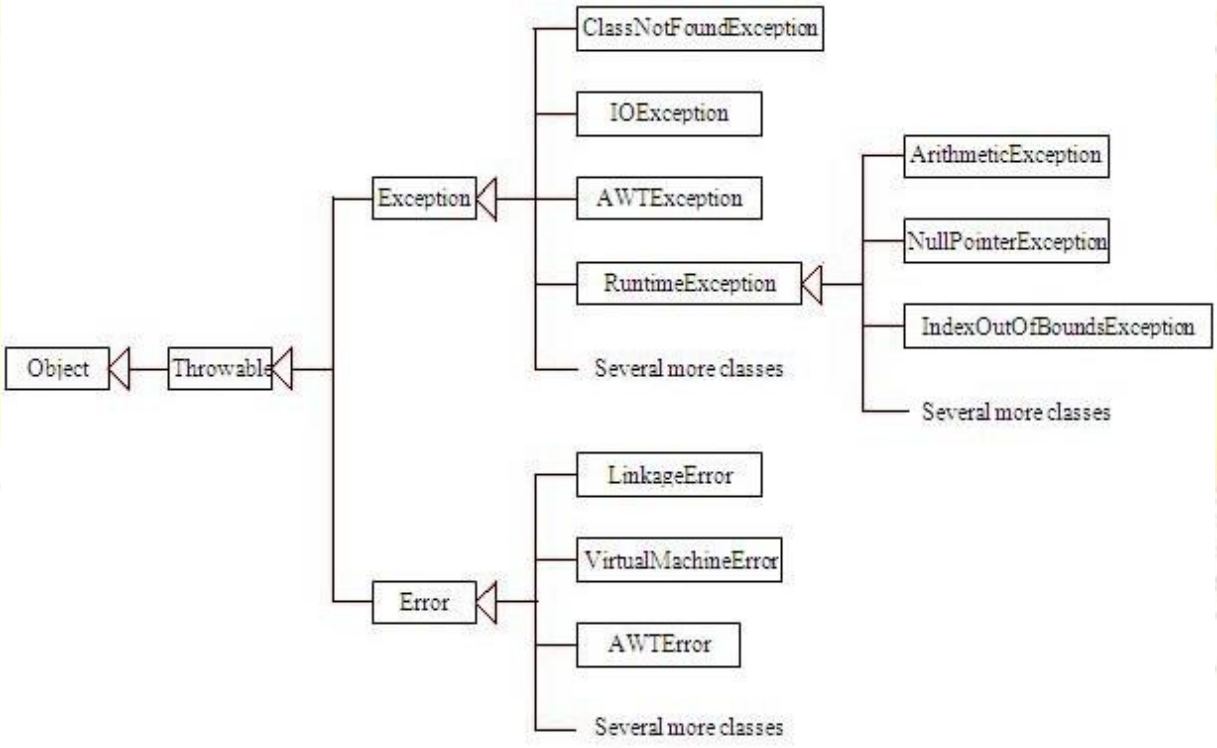
- If not handled properly, program might terminate.

7.4 Exceptions in Java

- An exception in java is represented as an object that's created when an abnormal situation arises in the program. Note that an error is also represented as an object in Java, but usually represents an unrecoverable situation and should not be caught
- The exception object stores information about the nature of the problem. For example, due to network problem or class not found etc.
- All exceptions in java are inherited from a class know as *Throwable*.

7.5 Exception Hierarchy

Following diagram is an abridged version of Exception class hierarchy



7.6 Types of Exceptions

Exceptions can be broadly categorized into two types, Unc*hecked* & *Checked Exceptions.*

7.6.1 Unchecked Exceptions

- Subclasses of Runtime Exception and Error.
- Does not require explicit handling
- Run-time errors are internal to your program, so you can get rid of them by debugging your code
- For example, null pointer exception; index out of bounds exception; division by zero exception.

7.6.2 Checked Exceptions

- Must be caught or declared in a throws clause
- Compile will issue an error if not handled appropriately
- Subclasses of Exception other than subclasses of Runtime Exception.
- Other arrives from external factors, and cannot be solved by debugging
- Communication from an external resource - e.g. a file server or database

7.7 How Java handles Exceptions

Java handles exceptions via 5 keywords. **Try**, **catch**, **finally**, **throw** & **throws.**

7.7.1 try block

- Write code inside this block which could generate errors

7.7.2 Catch block

- Code inside this block is used for exception handling
- When the exception is raised from try block, only than catch block would execute.

7.7.3 finally block

- This block always executes whether exception occurs or not.
- Write clean up code here, like resources (connection with file or database) that are opened may need to be closed.

The basic structure of using try - catch - finally block is shown in the picture below:

© Copyright Virtual University of Pakistan

```
try                                                    //try block
{
        // write code that could generate exceptions
} catch (<exception to be caught>)                     //catch block
{
        //write code for exception handling
}
 ……

catch (<exception to be caught>)                       //catch block
{
        //code for exception handling
} finally                                              //finally block
{
        //any clean-up code, release the acquired resources
}
```

7.7.4 throw

- To manually throw an exception, keyword throwis used. **Note:** we are not covering throw clause in this handout

7.7.5 throws

- If method is not interested in handling the exception than it can throw back the exception to the caller method using throwskeyword.
- Any exception that is thrown out of a method must be specified as such by a **throws** clause.

7.8 References:

- Java tutorial by Sun: http://java.sun.com/docs/books/turorial
- Beginning Java2 by Ivor Hortan
- Thinking in Java by Bruce Eckle
- CS193j Stanford University

7.9 Code Examples of Exception Handling

7.9.1 Unchecked Exceptions

Example Code: UcException.java

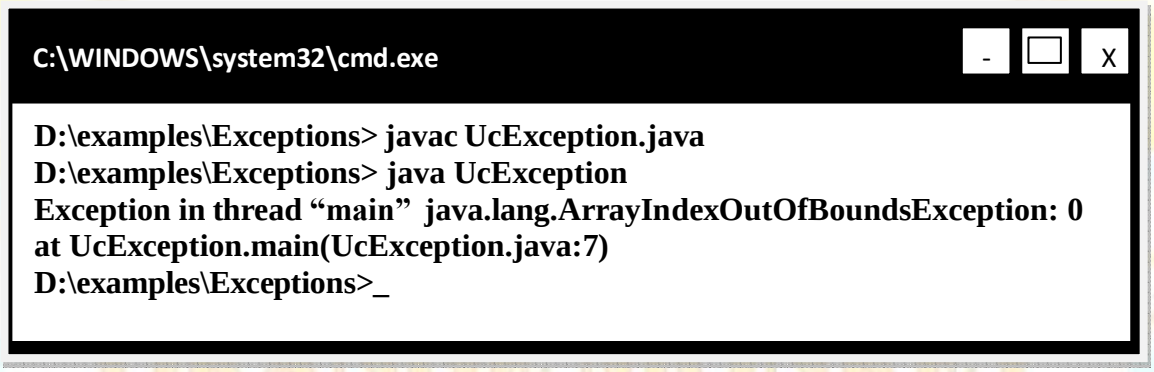
© Copyright Virtual University of Pakistan

The following program takes one command line argument and prints it on the console

```
// File UcException.java public class UcException {
public static void main (String args[ ]) {
System.out.println(args[0]);
}
}
```

Compile & Execute

If we compile & execute the above program without passing any command line argument, an ArrayIndexOutOfBoundsException would be thrown. This is shown in the following picture



7.9.2 Why?

Since we have passed no argument, therefore the size of String args[]is zero, and we have tried to access the first element (first element has index zero) of this array.

From the output window, you can find out, which code line causes the exception to be raised. In the above example, it is

System.out.println (args[0]);

7.9.3 Modify UcException.java

Though it is not mandatory to handle unchecked exceptions we can still handle Unchecked Exceptions if we want to. These modifications are shown in bold.

```
// File UcException.java
public class UcException {
public static void main (String args[ ]) {
```

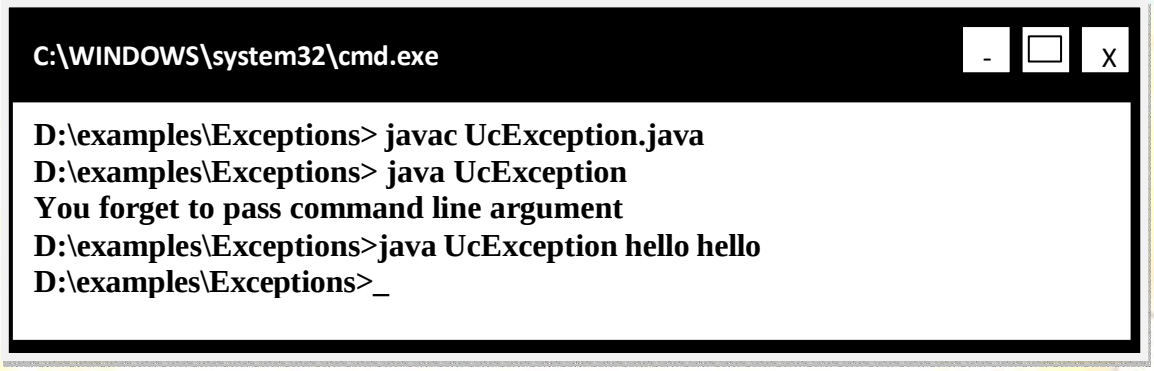
```
try {
System.out.println(args[0]);
catch (IndexOutOfBoundsException ex) {
System.out.println("You forget to pass command line argument");
}
}
```

The possible exception that can be thrown is IndexOutOfBoundsException, so we handle it in the catch block.

When an exception occurs, such as IndexOutOfBoundsException in this case, then an object of type IndexOutOfBoundesException is created and it is passed to the corresponding catch block (i.e. the catch block which is capable of handling this exception). The catch block receives the exception object inside a variable which is *ex* in this case. It can be any name; it is similar to the parameter declared in the method signature. It receives the object of exception type (IndexOutOfBoundsExceptoin) it is declared.

Compile & Execute

If we execute the modified program by passing command line argument, the program would display on console the provided argument. After that if we execute this program again without passing command line argument, this time information message would be displayed which is written inside catch block.



7.10 Checked Exceptions

Example Code: CException.java

The following program reads a line *(hello world)* from a file and prints it on the console. The File reading code is probably new for you. We'll explain it in the coming handouts (Streams). For now, assumed that the code written inside the main read one line from a file and prints that to console.

```
// File CException.java
```

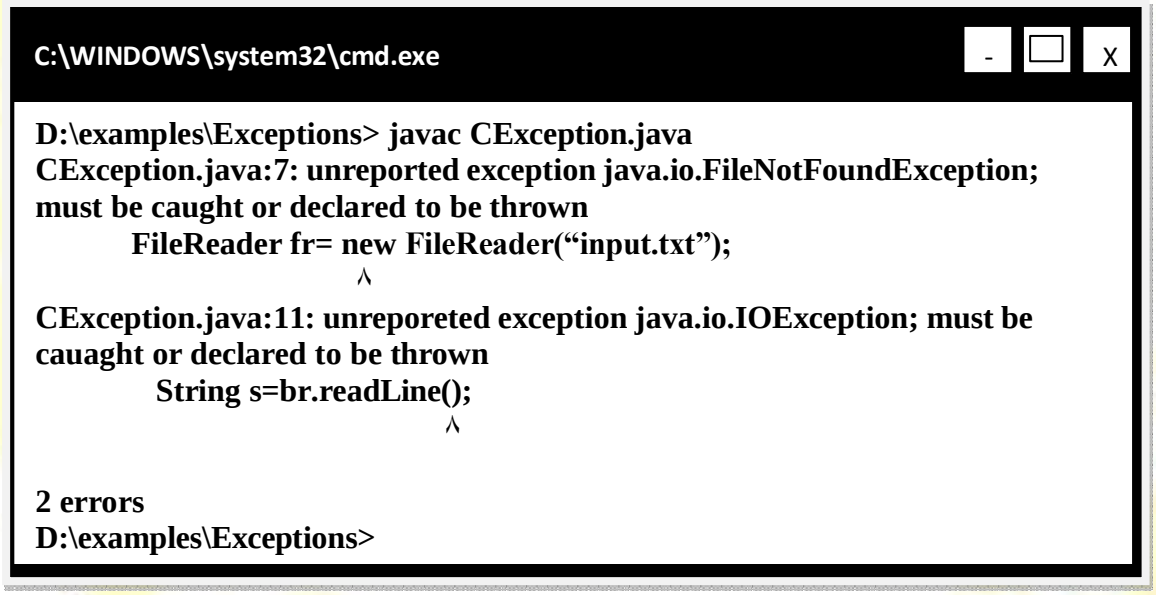
```
 import java.io.* ;
public class CException {
public static void main (String args[ ]) { FileReader fr = new FileReader (“input.txt”);
BufferedReader br = new BufferedReader (fr);

//read the line form file
String line = br.readLine(); System.out.println(line);        }
}
```

Compile & Execute

If you try to compile this program, the program will not compile successfully and displays the message of unreported exception. This happens when there is code that can generate a checked exception but you have not handled that exception. Remember checked exceptions are detected by compiler. As we early discussed, without handling Checked exception, out program won't compile.

Virtual University of Pakistan



Modify CException.java

As we have discussed earlier, it is mandatory to handle checked exceptions. In order to compile the code above, we modify the above program so that file reading code is placed inside a try block. The expected exception (IOException) that can be raised is caught in catch block.

```
// File CException.java import java.io.* ;
```

```
public class CException {
public static void main (String args[ ]) {
try{
FileReader fr = new FileReader ("input.txt");
BufferedReader br = new BufferedReader (fr);

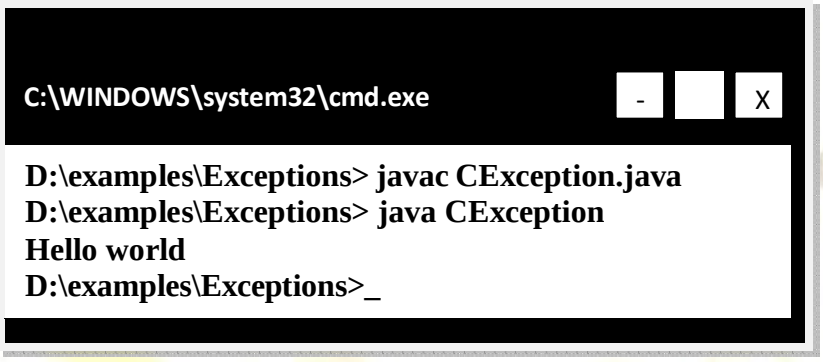
//read the line form file
String line = br.readLine();
System.out.println(line);
}
catch( IOException ex) {
System.out.println(ex);
}
}
}
```

The code line written inside the catch block will print the exception name on the console if exception occurs

Compile & Execute

After making changes to your program, it would compile successfully. On executing this program, *hello world* would be displayed on the console

Note: Before executing, make sure that a text file named *input.txt* must be placed in the same directory where the program is saved. Also write *hello world* in that file before saving it.



7.11 The finally block

The finally block always executes regardless of exception is raised or not while as you remembered the catch block only executes when an exception is raised.

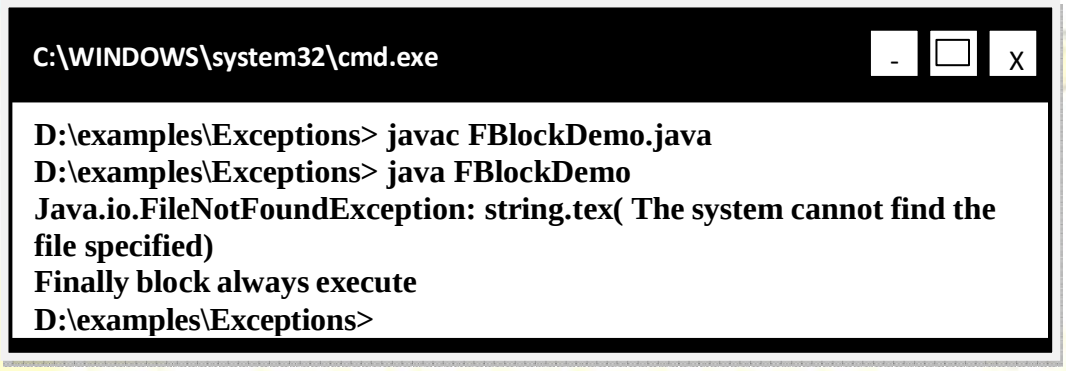
Example Code: FBlockDemo.java

```
// File FBlockDemo.java
```

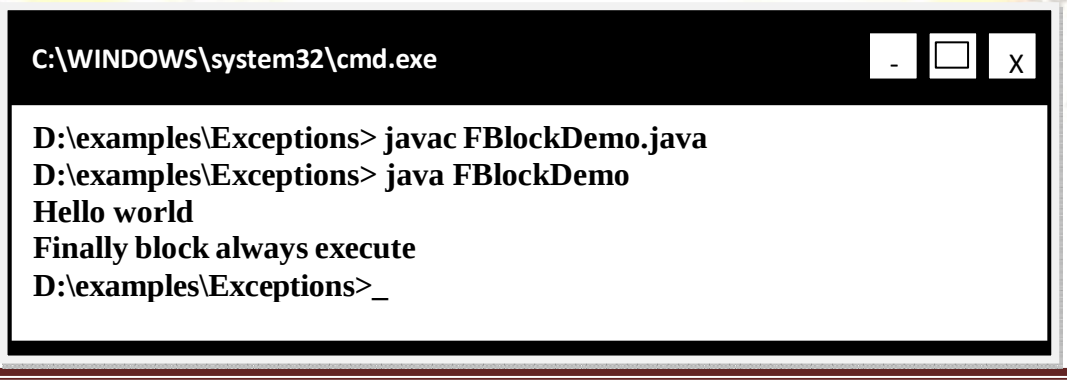
```
import java.io.* ;
public class FBlockDemo {
public static void main (String args[ ])
{
try{
FileReader fr = new FileReader (“strings.txt”);
BufferedReader br = new BufferedReader (fr);
//read the line form file
String line = br.readLine();
System.out.println(line);
}catch( IOException ex) {
System.out.println(ex);
}
finally {
System.out.println(“finally block always execute”);
}
}
}
```

Compile & Execute

The program above, will read one line from *string.txt* file. If *string.tx* is not present in the same directory the FileNotFoundException would be raised and catch block would execute as well as the finally block.



If *string.txt* is present there, no such exception would be raised but still finally block executes. This is shown in the following output diagram



© Copyright Virtual University of Pakistan

7.12 Multiple catch blocks

- Possible to have multiple catch clauses for a single try statement
  - Essentially checking for different types of exceptions that may happen
- Evaluated in the order of the code
  - Bear in mind the Exception hierarchy when writing multiple catch clauses!
  - If you catch Exception first and then IOException, the IOException will never be caught!

Example code: MCatchDemo.java

The following program would read a number form a file *numbers.txt* and than prints its square on the console

```
// File MCatchDemo.java
 import java.io.* ;
public class MCatchDemo {
public static void main (String args[ ]) {
try{
// can throw FileNotFound or IOException
FileReader fr = new FileReader ("numbers.txt");
BufferedReader br = new BufferedReader (fr);

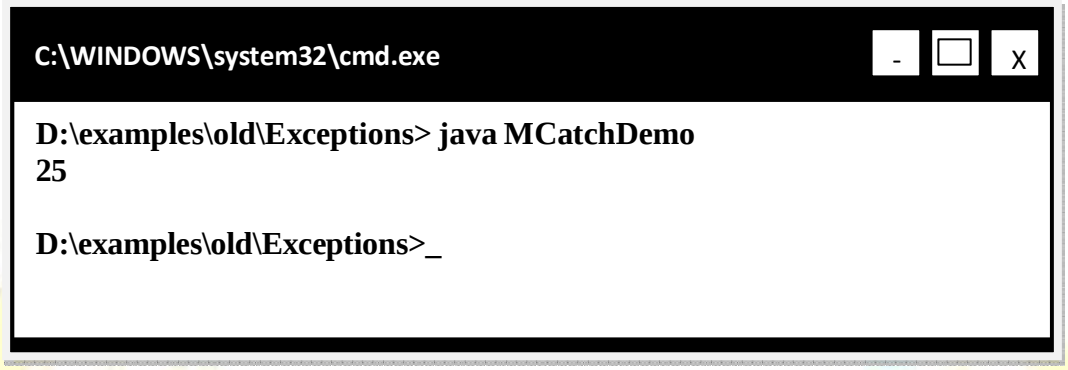
//read the number form file
String s = br.readLine();
//may throws NumberFormatException, if s is not a no.
int number = Integer.parseInt(s);
System.out.println(number * number);
}
catch( NumberFormatException nfEx) {
System.out.println(nfEx);
}
catch( FileNotFoundException fnfEx) {
System.out.println(fnfEx);
}
catch( IOException ioEx) {
System.out.println(ioEx);
}
}
}
```

We read everything from a file (numbers, floating values or text) as a String. That's why we first convert it to number and then print its square on console.

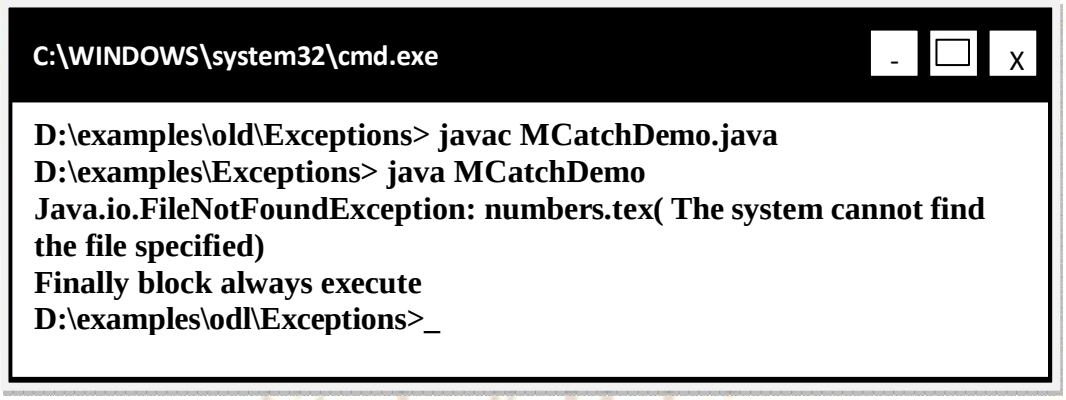
Compile & Execute

If file *numbers.txt* is not present in the same directory, the FileNotFoundException would be

thrown during execution.



If *numbers.txt* present in the same directory and contains a number, than hopefully no exception would be thrown.



7.13 The throws clause

The following code examples will introduce you with writing & using throws clause.

Example Code: ThrowsDemo.java

The ThrowsDemo.javacontains two methods namely method1 & method2and one main method. The mainmethod will make call to method1and than method1 will call method2. The method2 contains the file reading code. The program looks like one given below

```
// File ThrowsDemo.java
import java.io.* ;
public class ThrowsDemo {

// contains file reading code
public static void method2( ) {
```

```
try{
FileReader fr = new FileReader ("strings.txt");
BufferedReader br = new BufferedReader (fr);

//read the line form file
String s = br.readLine();
System.out.println(s);
}
catch( IOException ioEx) {
ioEx.printStackTrace(); }
}// end method 2
//only calling method2
public static void method1( )
{
method2();
}
public static void main (String args[ ]) {
ThrowsDemo.method1();
}
}
```

7.14 printStackTrace method

- Defined in the Throwableclass - superclass of Exception & Error classes
- Shows you the full method calling history with line numbers.
- extremely useful in debugging

Modify: ThrowsDemo.java

- Let method2 doesn't want to handle exception by itself, so it throws the exception to the caller of method2i.e. method1
- So method1 either have to handle the incoming exception or it can re-throw it to its caller i.e. main.
- Let method1 is handling the exception, so method1& method2 would be modified as:

```
// File ThrowsDemo.
java import java.io.* ;
public class ThrowsDemo {
// contains file reading code
public static void method2( ) throws IOException{
FileReader fr = new FileReader ("strings.txt");
BufferedReader br = new BufferedReader (fr);

//read the line form file
```

© Copyright Virtual University of Pakistan

```
String s = br.readLine();
System.out.println(s);
}// end method 2

// calling method2 & handling incoming exception
public static void method1( ) {
try {
method2();
}
catch (IOException ioEx) {
ioEx.printStackTrace(); }
}

public static void main (String args[ ]) {
ThrowsDemo.method1();
}
}
```

Compile & Execute

If file *strings.txt* is not present in the same directory, method2will throw an exception that would be caught by method1and the printStackTracemethod will print the full calling history on console. The above scenario is shown in the output below:



If file *strings.txt* exist there, than hopefully line would be displayed on the console.

© Copyright Virtual University of Pakistan

7.15 Reference

- Example code, their explanations and corresponding figures for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 8: Streams

I/O libraries often use the abstraction of a *stream*, which represents any data source or sink as an object capable of producing or receiving pieces of data.

The Java library classes for I/O are divided by input and output. You need to import java.io package to use streams. There is no need to learn all the streams just do it on the need basis.

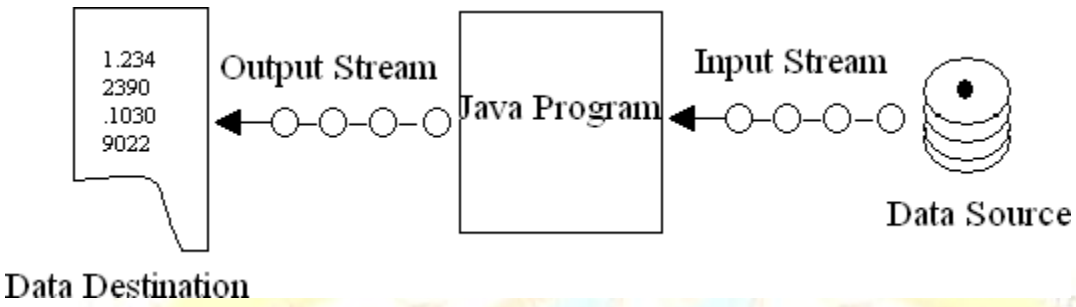
8.1 The concept of "streams"

- It is an abstraction of a data source/sink
- We need abstraction because there are lots of different devices (files, consoles, network, memory, etc.). We need to talk to the devices in different ways (sequential, random access, by lines, etc.) Streams make the task easy by acting in the same way for every device. Though inside handling of devices may be quite different, yet on the surface everything is similar. You might read from a file, the keyboard, memory or network connection, different devices may require specialization of the basic stream, but you can treat them all as just "streams". When you read from a network, you do nothing different than when you read from a local file or from users typing

```
//Reading from console
BufferedReader stdin =       new BufferedReader(new InputStreamReader(
System.in ));
-          ( your console)
// Reading from file
BufferedReader br=new BufferedReader(new
FileReader("input.txt"));

//Reading from network
BufferedReader br = new BufferedReader(new InputStreamReader
(s.getInputStream()));
---- "s" is the socket
```

- So you can consider stream as a data path. Data can flow through this path in one direction between specified terminal points (your program and file, console, Socket etc.)



8.2 Stream classification based on Functionality

Based on functionality streams can be categorized as Node Stream and Filter Stream. Node Streams are those which connect directly with the data source/sick and provide basic functionality to read/write data from that source/sink

```
FileReader fr = new FileReader("input.txt");
```

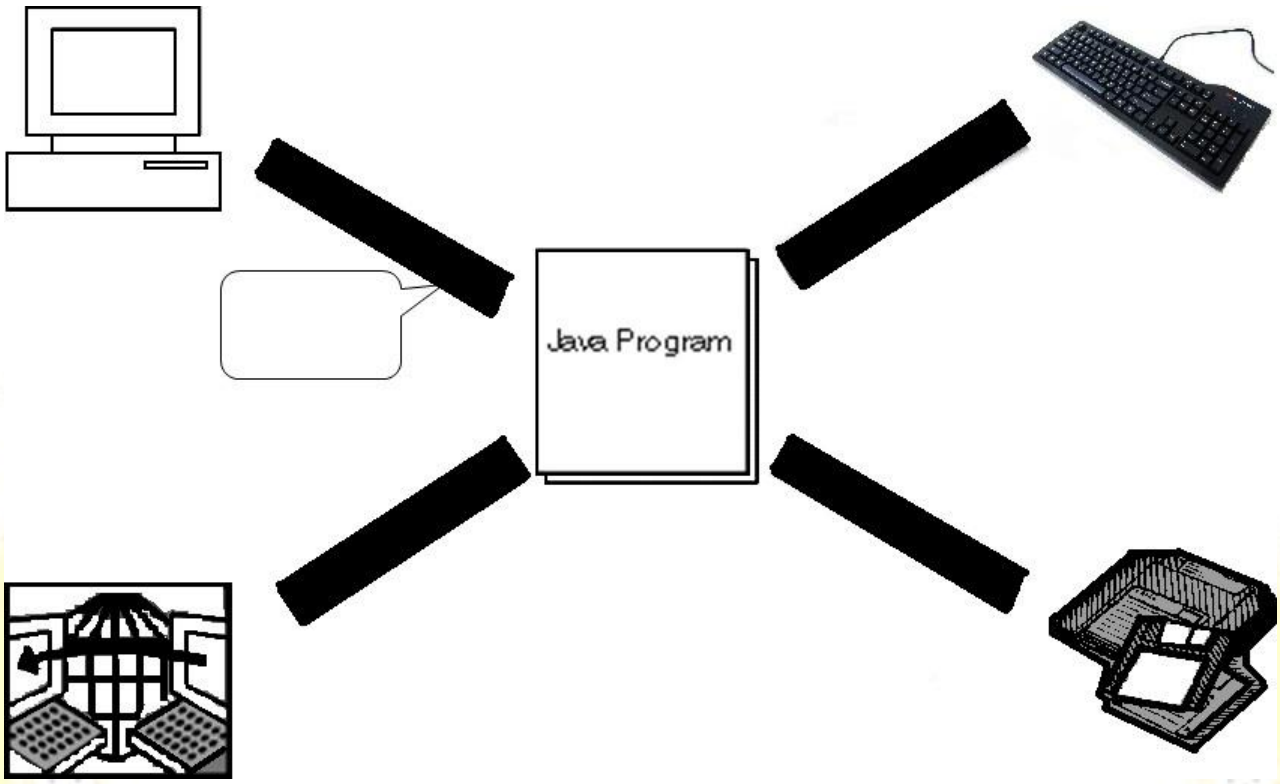
You can see that FileReader is taking a data/source "input.txt" as its argument and hence it is a node stream.

FilterStreams sit on top of a node stream or chain with other filter stream and provide some additional functionality e.g. compression, security etc. FilterStreams take other stream as their input.

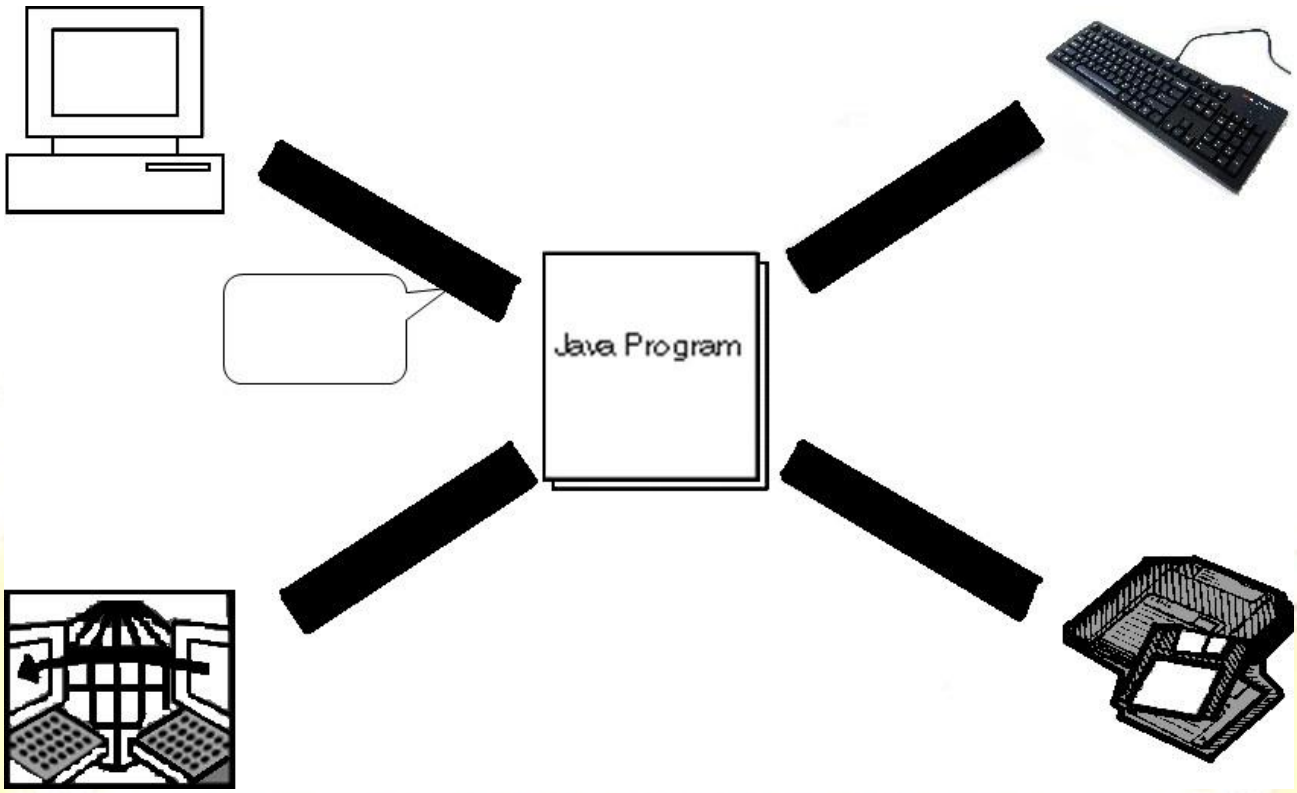
```
BufferedReader bt = new BufferedReader(fr);
```

BufferedReader makes the IO efficient (enhances the functionality) by buffering the input before delivering. And as you can see that BufferedReader is sitting on top of a node stream which is FileReader.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available



ORANGE MONKEY
Study & Helping
24 Hours Available



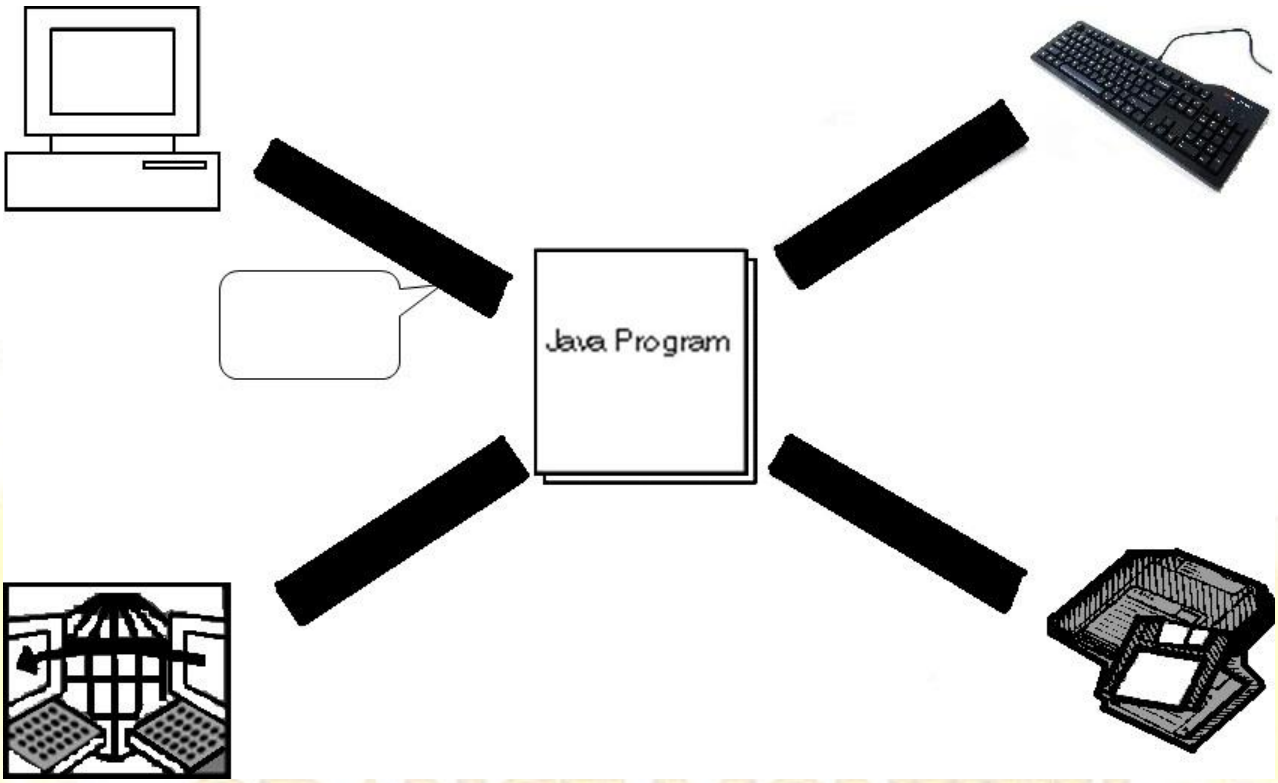
8.3 Stream classification based on data

Two types of classes exist:

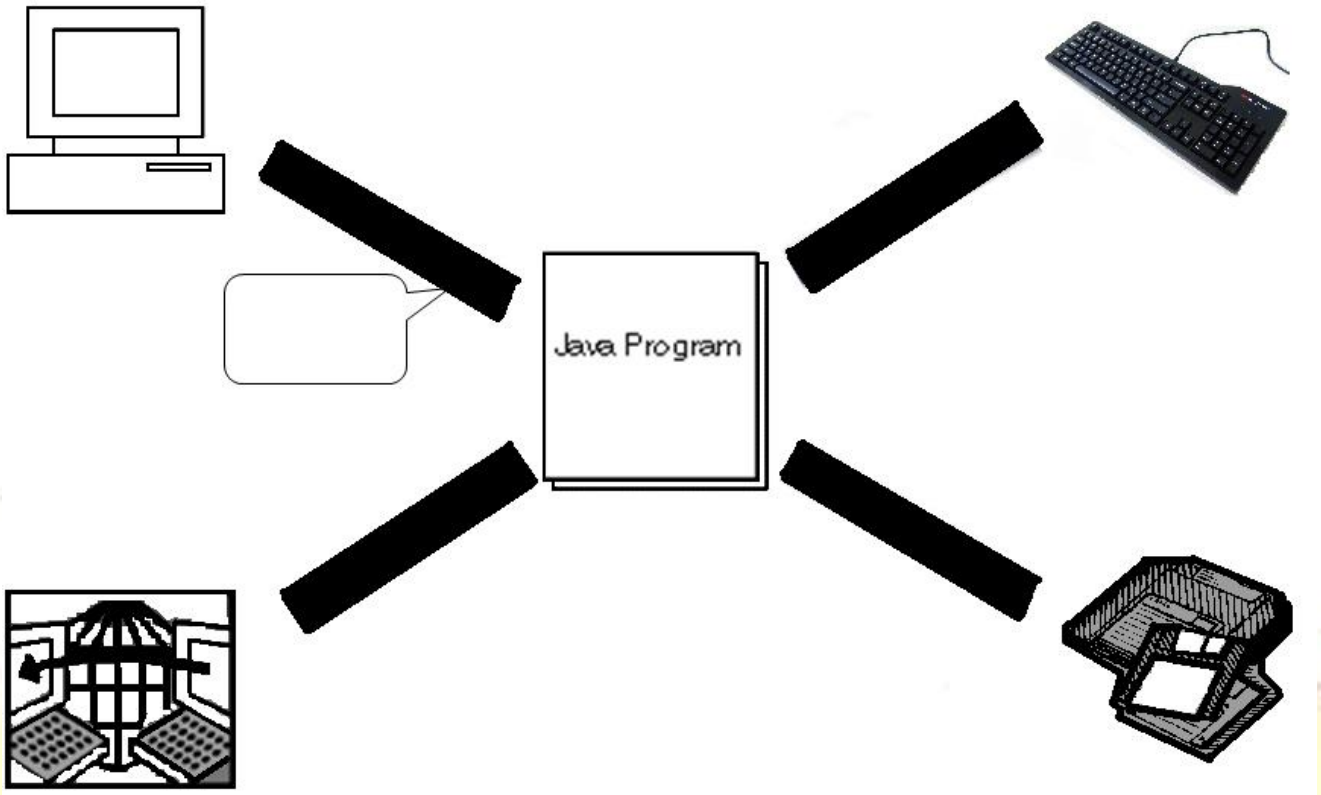
Classes which contain the word stream in their name are byte oriented and are here since JDK1.0. These streams can be used to read/write data in the form of bytes. Hence classes with the word stream in their name are byte-oriented in nature. Examples of byte oriented streams are FileInputStream, ObjectOutputStream etc.

Classes which contain the word Reader/Writer are character oriented and read and write data in the form of characters. Readers and Writers came with JDK1.1. Examples of Reader/Writers are FileReader, PrintWriter etc

ORANGE MONKEY
Study & Helping
24 hours Available



ORANGE MONKEY
Study & Helping
24 Hours Available



ORANGE MONKEY
Study & Helping
24 Hours Available

Example Code 8.1: Reading from File

The ReadFileEx.javareads text file line by line and prints them on console. Before we move on to the code, first create a text file (*input.txt*) using notepad and write following text lines inside it.

Text File: input.txt

```
Hello World
Pakistan is our homeland
Web Design and Development
```

```
// File ReadFileEx.
java import java.io.*;
public class ReadFileEx {
public static void main (String args[ ]) {
FileReader fr = null;
BufferedReader br = null;
try {
// attaching node stream with data source
fr = new FileReader("input.txt");

// attatching filter stream over node stream
br = new BufferedReader(fr);

// reading first line from file
String line = br.readLine();

// printing and reading remaining lines
while (line != null){
System.out.println(line);
line = br.readLine();
}
// closing streams
br.close();
fr.close();

}catch(IOException ioex){
System.out.println(ioex);
}
} // end main
} // end class
```

Virtual University of Pakistan ORANGE MONKEY Study & Helping 24 Hours Available

Example Code 8.2: Writing to File

The WriteFileEx.javawrites the strings into the text file named "output.txt". If "output.txt" file does not exist, the java will create it for you.

```
// File WriteFileEx.java

 import java.io.*;
public class WriteFileEx {
public static void main (String args[ ]) {
FileWriter fw = null;
PrintWriter pw = null;

try {
// attaching node stream with data source
// if file does not exist, it automatically creates it
fw = new FileWriter ("output.txt");

// attatching filter stream over node stream
pw = new PrintWriter(fw);

String s1 = "Hello World";
String s2 = "Web Design and Development";
// writing first string to file
pw.println(s1);
// writing second string to file
pw.println(s2);
// flushing stream
pw.flush();
// closing streams
pw.close();
fw.close();

}catch(IOException ioex){
System.out.println(ioex);
}
} // end main
} // end class
```

After executing the program, check the output.txt file. Two lines will be written there.

8.4 Reference

- Example code, their explanations and corresponding figures for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

8.5 Modification of Address Book Code

8.5.1 Adding Persistence Functionality

Hopefully, your address book you built previously is giving you the required results except one i.e. persistence. You might have noticed that after adding some person records in the address book; if you exit from the program next time on re-executing address book all the previous records are no more available.

To overcome the above problem, we will modify our program so that on exiting/starting of address book, all the previously added records are available each time. To achieve this, we have to provide the persistence functionality. Currently, we will accomplish this task by saving person records in some text file.

Supporting simple persistence by any application requires handling of two scenarios. These are

- On start up of application - data (person records) must be read from file
- On end/finish up of application - data (person records) must be saved in file

To support persistence, we have to handle the above mentioned scenarios

8.5.1.1 Scenario 1 - Start Up

- Establish a data channel with a file by using streams
- Start reading data (person records) from file line by line
- Construct PersonInfoobjects from each line you have read
- Add those PersonInfoobjects in arraylist persons.
- Close the stream with the file
- Perform these steps while application is loading up

We will read records from a text file named persons.txt. The person records will be present in the file in the following format.

```
Ali, defence, 9201211
Usman, gulberg, 5173940
Salman, LUMS, 5272670
```

persons.txt

As you have seen, each person record is on a separate line. Person's name, address & phone number is separated using comma (,).

We will modify our AddressBook.javaby adding a new method loadPersons into it. This method will provide the implementation of all the steps. The method is shown below:

```
public void loadPersons ( ){

String tokens[] = null;
String name, add, ph;

try {

FileReader fr = new FileReader("persons.txt");
BufferedReader br = new BufferedReader(fr);

String line = br.readLine();
while ( line != null ) {
tokens = line.split(",");
name = tokens[0];
add = tokens[1];
ph = tokens[2];
PersonInfo p =       new PersonInfo(name, add, ph);
persons.add(p);
line = br.readLine();
}

br.close();
fr.close();

}catch(IOException ioEx){
System.out.println(ioEx);
}
}
```

- First, we have to connect with the text file in order to read line by line person records from it. This task is accomplished with the following lines of code:

  FileReader fr = new FileReader("persons.txt");

  BufferedReader br = new BufferedReader(fr);

FileReaderis a character based (node) stream that helps us in reading data in the form of characters. As we are using streams, so we have to import the java.iopackage in the AddressBookclass.

© Copyright Virtual University of Pakistan

- We passed the file name persons.txtto the constructor of the FileReader.

Next we add BufferedReader (filter stream) on top of the FileReader because BufferedReader facilitates reading data line by line. (As you can recall from the lecture that filter streams are attached on top of node streams). That's why the constructor of BufferedReader is receiving the fr - the FileReader object.

- The next line of code will read line from file by using readLine() method of BufferedReaderand save it in a string variable called line.

```
String line = br.readLine();
```

- After that while loop starts. The condition of while loop is used to check whether the file is reached to end (returns null) or not. This loop is used to read whole file till the end. When end comes (null), this loop will finish.

```
while (line != null)
```

- Inside loop, the first step we performed is tokenizing the string. For this purpose, we have used split method of String class. This method returns substrings (tokens) according to the regular expression or delimiter passed to it.

```
tokens = line.split(",");
```

The return type of this method is array of strings that's why we have declared tokens as a String array in the beginning of this method as

```
String tokens[];
```

For example, the line contains the following string

```
Ali,defence,9201211
```

Now by calling split(",") method on this string, this method will return back three substrings *ali defence* and *9201211* because the delimiter we have passed to it is comma. The delimiter itself is not included in the substrings or tokens.

- The next three lines of code are simple assignments statements. The tokens[0]contains the *name* of the person because the name is always in the beginning of the line, tokens[1] contains *address* of the person and tokens[2] contains the phone number of the person.

```
name = tokens[0]; add=
tokens[1]; ph= tokens[2];
```

The name, add and ph are of type String and are declared in the beginning of this method.

- After that we have constructed the object of PersonInfo class by using parameterized constructor and passed all these strings to it.

```
PersonInfo p = new PersonInfo(name, add, ph);
```

- Afterward the PersonInfo object's p is added to the arraylist i.e. persons.

```
persons.add(p);
```

- The last step we have done inside loop is that we have again read a line from the file by using the readLine()method.
- By summarizing the task of while loop we can conclude that it reads the line from a file,tokenize that line into three substrings followed by constructing the PersonInfo object by using these tokens. And adding these objects to the arraylist. This process continues till the file reaches its end.
  The last step for reading information from the file is ordinary one - closing the streams, because files are external resources, so it's better to close them as soon as possible. Also observe that we used try/catch block because using streams can result in raising exceptions that falls under the checked exceptions category - that needs mandatory handling.
- The last important step you have to perform is to call this method while loading up.
- The most appropriate place to call this method is from inside the constructor of AddressBook.java. So the constructor will now look like similar to the one given below:

```
……
……
public AddressBook () { Persons = new
ArrayList(); loadPersons();
}
……
……
```

AddressBook.java

8.5.1.2 Scenario 2 - End/Finish Up

- Establish a datachanel(stream) with a file by using streams
- Take out PersonInfoobjects from ArrayList (persons)
- Build a string for each PersonInfo object by inserting commas (,) between name & address and address & phone number.
- Write the constructed string to the file
- Close the connection with file
- Perform these steps while exiting from address book.

Add another method savePersons into AddressBook.java. This method will provide the implementation of all the above mentioned steps. The method is shown below:

```
public void savePersons ( ){
try {
PersonInfo p;
```

```
String line;
FileWriter fw = new FileWriter("persons.txt");
PrintWriter pw = new PrintWriter(fw);
for(int i=0; i<persons.size(); i++)
{
p = (PersonInfo)persons.get(i);
line = p.name +","+ p.address +","+ p.phoneNum;

// writes line to file (persons.txt)
pw.println(line);
} pw.flush();
pw.close();
fw.close();
}
catch(IOException ioEx){
System.out.println(ioEx);
}
}
```

- As you can see, that we have opened the same file (persons.txt)again by using a set of streams.
- After that we have started for loop to iterate over arraylist as we did in searchPersonand deletePersonmethods.
- Inside forloop body, we have taken out PersonInfoobject and after type casting it we have assigned its reference to a PersonInfotype local variable p. This is achieved by the help of following line of code

```
p = (PersonInfo)persons.get(i);
```

- Next we build a string and insert commas between the PersonInfoattributes and assign the newly constructed string to string's local variable lineas shown in the following line of code.

```
line = p.name +","+ p.address +","+ p.phoneNum;
```

Note: Since, we haven't declare PersonInfo attributes private, therefore we are able to directly access them inside AddressBook.java.

- The next step is to write the line representing one PersonInfo object's information, to the file. This is done by using printlnmethod of PrintWriteras shown below

```
pw.println(line);
```

After writing line to the file, the printlnmethod will move the cursor/control to the next line. That's why each line is going to be written on separate line.

- The last step for saving information to the file is ordinary one - closing the streams but before that notice the code line that you have not seen/performed while loading persons records from file. That is

```
pw.flush( );
```

The above line immediately flushes data by writing any buffered output/data to file. This step is necessary to perform or otherwise you will most probably lose some data for the reason that PrintWriter is a Buffered Stream and they have their own internal memory/storage capacity for efficiency reasons. Buffered Streams do not send the data until their memory is full.

- Also we have written this code inside try-catch block.
- The last important step you have to perform is to call this method before exiting from the address book. The most appropriate place to call this method is under case4(exit scenario) in Test.java. So the case 4will now look like similar to the one given below:

```
……
case 4:
ab.savePersons();
System.exit(0);
……
```

Test.java

Compile & Execute

Now again after compiling all the classes, run the Test class. Initially we are assuming that out persons.txt file is empty, so our arraylist persons will be empty on the first start up of address book. Now add some records into it, perform search or delete operations. Exit from the address book by choosing option 4. Check out the persons.txt file. Don't get surprised by seeing that it contains all the person records in the format exactly we have seen above.

Next time you will run the address book; all the records will be available to you. Perform the search or delete operation to verify that. Finally you have done it!!!

8.6 References

- Example code, their explanations and corresponding figures for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

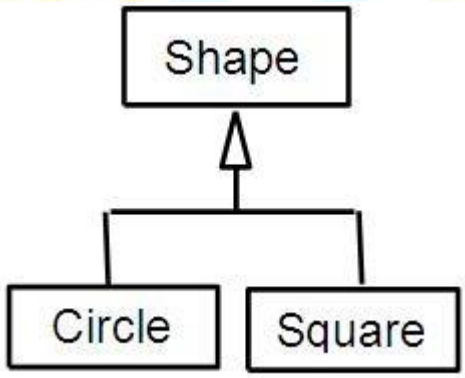
Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

© Copyright Virtual University of Pakistan

Lecture 9: Abstract Classes and Interfaces

9.1 Problem and Requirements

Before moving on to abstract classes, first examine the following class hierarchy shown below:



- Suppose that in order to exploit polymorphism, we specify that 2-D objects must be able to compute their area.
    - All 2-D classes must respond to area() message.
- How do we ensure that?
    - Define area method in class Shape
    - Force the subclasses of Shape to respond area() message
- Java's provides us two solutions to handle such problem
    - Abstract Classes
    - Interfaces

9.2 Abstract Classes

Abstract classes are used to define only part of an implementation. Because, information is not complete therefore an abstract class cannot be instantiate. However, like regular classes, they can also contain instance variables and methods that are fully implemented. The class that inherits from abstract class is responsible to provide details.

Any class with an abstract method (a method has no implementation similar to pure virtual function in C++) must be declared abstract, yet you can declare a class *abstract* that has no abstract method.

If subclass overrides all abstract methods of the superclass, than it becomes a concrete (a class whose object can be instantiate) class otherwise we have to declare it as abstract or we cannot compile it.

The most important aspect of abstract class is that reference of an abstract class can point to the object of concrete classes.

Code Example of Abstract Classes

The **Shape** class contains an abstract method calculateArea() with no definition.

```
public abstract class Shape{
public abstract void calculateArea();
}
```

Class **Circle** extends from abstract Shape class, therefore to become concrete class it must provides the definition of calculateArea() method.

```
public class Circle extends Shape {
private int x, y;
private int radius;

public Circle() {
x = 5;
y = 5;
radius = 10;
}

// providing definition of abstract method
public void calculateArea () {
double area = 3.14 * (radius * radius);
System.out.println("Area: " + area);
}
}//end of class
```

The Test class contains main method. Inside main, a reference s of abstract Shape class is created. This reference can point to Circle (subclass of abstract class Shape) class object as it is a concrete class. With the help of reference, method calculateArea() can be invoked of Circle class. This is all shown in the form of code below:

```
public class Test {
public static void main(String args[]){
```

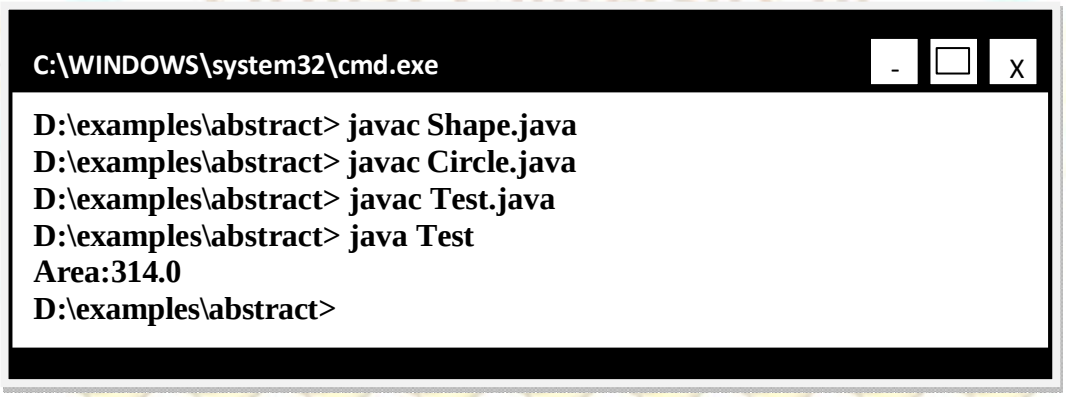
© Copyright Virtual University of Pakistan

```
//can only create references of A.C.

Shape s = null;

//Shape s1 = new Shape(); //cannot instantiate
//abstract class reference can point to concrete subclass
s = new Circle();
s.calculateArea();
}
}//end of class
```

The compilation and execution of the above program is shown below:



9.3 Interfaces

As we seen one possible java's solution to problem discussed in start of the tutorial. The second possible java's solution is Interfaces.

Interfaces are special java type which contains only a set of method prototypes, but does not provide the implementation for these prototypes. All the methods inside an interface are abstract by default thus an interface is tantamount to a pure abstract class - a class with zero implementation. Interface can also contains static final constants

9.3.1 Defining an Interface

Keyword interface is used instead of class as shown below:

```
public interface Speaker{
public void speak();
}
```

9.3.2 Implementing (using) Interface

Classes *implement* interfaces. Implementing an interface is like *signing a contract*. A class that implements an interface will have to provide the definition of all the methods that are present inside an interface. If the class does not provide definitions of all methods, the class would not compile. We have to declare it as an abstract class in order to get it compiled.

Relationship between a class and interface is equivalent to *"responds to"* while *"is a"* relationship exists in inheritance.

Code Example of Defining & Implementing an Interface The interface **Printable** contains print() method.

```
public interface Printable{
public void print();
}
```

Class **Student** is implementing the interface Printable. Note the use of keyword *implements* after the class name. Student class has to provide the definition of print method or we are unable to compile.

The code snippet of student class is given below:

```
public class Student implements Printable {
private String name;
private String address;
public String toString () {
return "name:"+name +" address:"+address;
}
//providing definition of interface's print method
public void print() {
System.out.println("Name:" +name+" address"+address);
}
}//end of class
```

9.4 Interface Characteristics

Similar to abstract class, interfaces imposes a design structure on any class that uses the

interface. Contrary to inheritance, a class can implement more than one interfaces. To do this separate the interface names with comma. This is java's way of multiple inheritance.

class Circle implements Drawable , Printable { ..... }

Objects of interfaces also cannot be instantiated.

Speaker s = new Speaker(); // not compile

However, a reference of interface can be created to point any of its implementation class. This is interface based polymorphism.

Code Example: Interface based polymorphism

Interface Speaker is implemented by three classes Politician, Coach and Lecturer. Code snippets of all these three classes are show below:

```
public class Politician implements Speaker{
public void speak(){
System.out.println("Politics Talks");
}
}
```

```
public class Coach implements Speaker{
public void speak(){
System.out.println("Sports Talks");
}
}
```

```
public class Lecturer implements Speaker{
public void speak(){
System.out.println("Web Design and Development Talks");
}
}
```

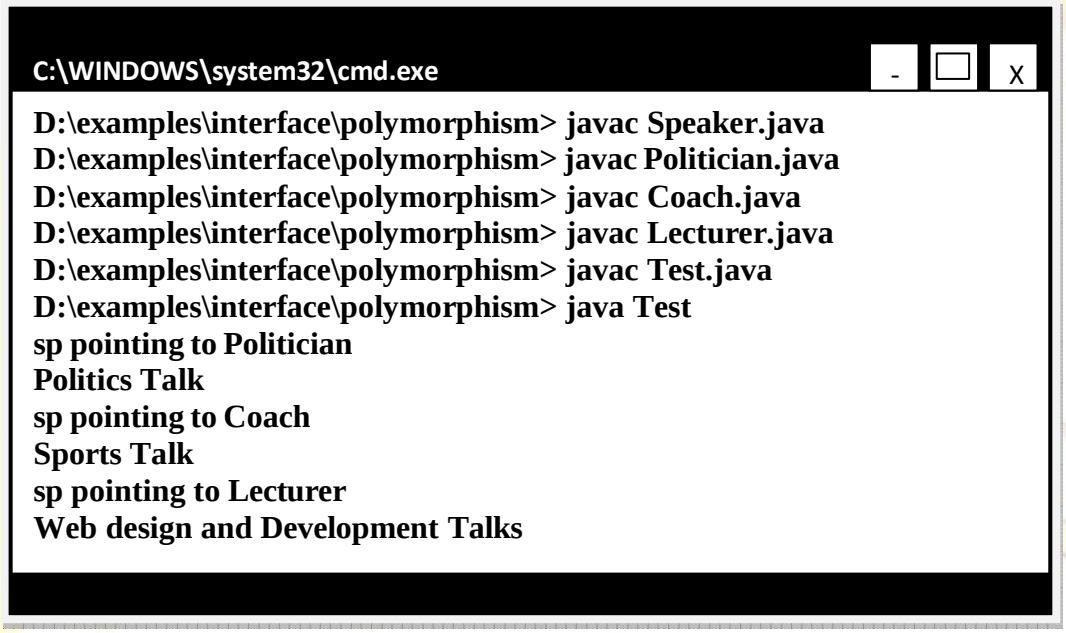
As usual, Test class contains main method. Inside main, a reference spis created of Speaker class. Later, this reference is used to point to the objects of Politician, Coach and Lecturer class. On calling speak method with the help of sp, will invoke the method of a class to which sp is pointing.

```
public class Test{
public static void main (String args[ ]) {
Speaker sp = null;
System.out.println("sp pointing to Politician");
sp = new Politician();
```

© Copyright Virtual University of Pakistan

```
sp.speak();
System.out.println("sp pointing to Coach");
sp = new Coach();
sp.speak();
System.out.println("sp pointing to Lecturer");
sp = new Lecturer();
sp.speak();
}
}
```

The compilation and execution of the above program is shown below:



9.5 References

- Example code, their explanations and corresponding figures for this handout are taken from the book JAVA A Lab Course by Umair Javed. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Lecture 10: Graphical User Interfaces

A graphical user interface is a visual interface to a program. GUIs are built from GUI components (buttons, menus, labels etc). A GUI component is an object with which the user interacts via the mouse or keyboard.

Together, the appearance and how user interacts with the program are known as the program look and feel.

10.1 Support for GUI in Java

The classes that are used to create GUI components are part of the "java.awt" or "javax.swing" package. Both these packages provide rich set of user interface components.

10.2 GUI classes vs. Non-GUI Support Classes

The classes present in the awt and swing packages can be classified into two broad categories. GUI classes & Non-GUI Support classes.

The GUI classes as the name indicates are visible and user can interact with them. Examples of these are JButton, JFrame & JRadioButton etc

The Non-GUI support classes provide services and perform necessary functions for GUI classes. They do not produce any visual output. Examples of these classes are Layout managers (discussed latter) & Event handling (see handout on it) classes etc.

10.3 java.awt package

AWT stands for "Abstract Windowing Toolkit" contains original GUI components that came with the first release of JDK. These components are tied directly to the local platform's (Windows, Linux, MAC etc) graphical user interface capabilities. Thus results in a java program executing on different java platforms (windows, linux, solaris etc) has a different appearance and sometimes even different user interaction on each platform.

AWT components are often called Heavy Weight Components (HWC) as they rely on the local platform's windowing system to determine their functionality and their look and feel. Every time you create an AWT component it creates a corresponding process on the operating system. As

© Copyright Virtual University of Pakistan

compared to this SWING components are managed through threads and are known as Light Weight Components.

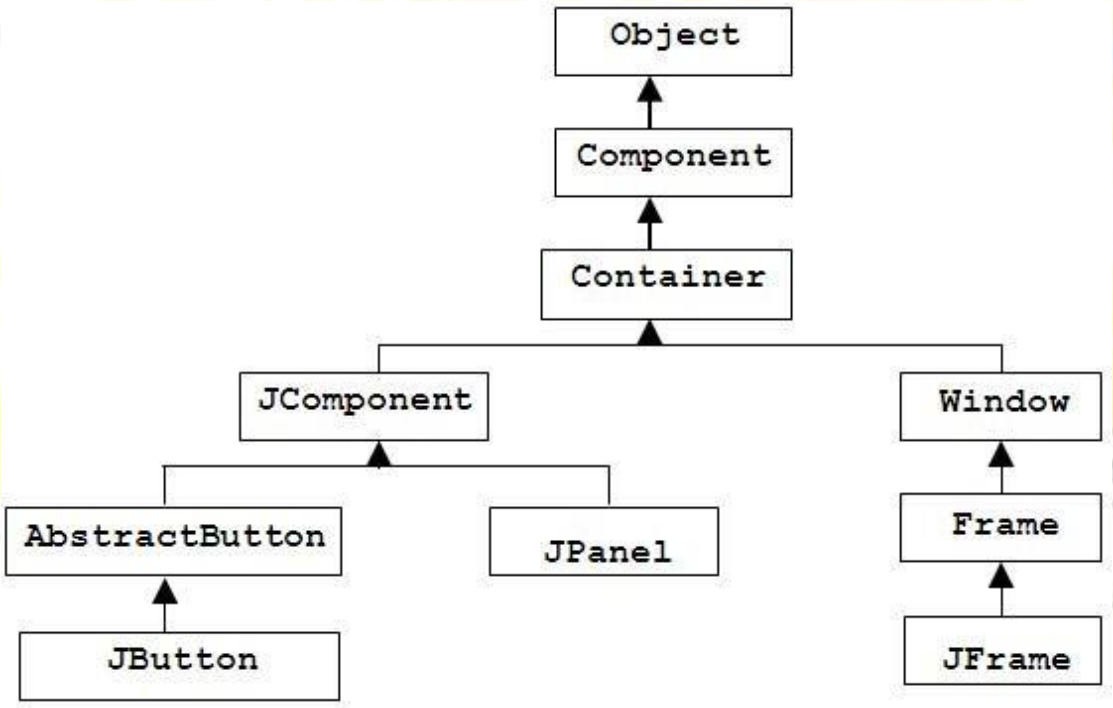
This package also provides the classes for robust event handling (see handout on it) and layout managers.

10.4 javax.swing package

These are the newest GUI components. Swing components are written, manipulated and displayed completely in java, therefore also called pure java components. The swing components allow the programmer to specify a uniform look and feel across all platforms.

Swing components are often referred to as Light Weight Components as they are completely written in java. Several swing components are still HWC e.g. JFrame etc.

10.5 A part of the Framework



10.6 GUI Creation Steps

10.6.1 import required packages

- import java.awt.* and/or javax.swing.* package.

10.6.2 Setup the top level containers

- A container is a collection of related components, which allows other components to be nested inside it. In application with JFrame, we attach components to the content pane - a container.
- Two important methods the container class has **add** and **setLayout.**
- The add method is used for adding components to the content pane while setLayout method is used to specify the layout manager.
- Container are classified into two broad categories that are Top Level Containers and General Purpose Containers
- Top level containers can contain (add) other containers as well as basic Components (buttons, labels etc) while general purpose containers are Typically used to collect basic components and are added to top level containers.
- General purpose containers cannot exist alone they must be added to top level containers
- Examples of top level container are JFrame, Dialog and Applet etc. Our application uses one of these.
- Examples of general purpose container are JPanel, Toolbar and ScrollPane etc.
- So, take a top level container and create its instance. Consider the following code of line if JFrame is selected as a top level container

JFrame *frame* = new JFrame();

10.6.3 Get the component area of the top level container

- Review the hierarchy given above, and observe that JFrame *is a* frame *is a* window. So, it can be interpreted as JFrame *is a* window.
- Every window has two areas. System Area & Component Area
- The programmer cannot add/remove components to the System Area.
- The Component Area often known as Client area is a workable place for the programmer. Components can be added/removed in this area.
- So, to add components, as you guessed right component area of the JFrame is required. It can be accomplished by the following code of line

  Container *con* = *frame*.getContentPane();
- *frame* is an instance of JFrame and by calling *getContentPane()* method on it, it returns the component area. This component area is of type container and that is why it is stored in a variable of a Container class. As already discussed, container allows other components to be added / removed.

© Copyright Virtual University of Pakistan

10.6.4 Apply layout to component area

- The layout (size & position etc. How they appear) of components in a container is usually governed by Layout Managers.
- The layout manager is responsible for deciding the layout policy and size of its components added to the container.
- Layout managers are represented in java as classes. (Layout Managers are going to be discussed in detail later in this handout)
- To set the layout, as already discussed use setLayout method and pass object of layout manager as an argument.

```
con.setLayout( new FlowLayout( ) );
```

- We passed an object of FlowLayout to the setLayout method here.
- We can also use the following lines of code instead of above.

```
FlowLayout layout = new FlowLayout();

con.setLayout(layout);
```

10.6.5 Create and Add components

- Create required components by calling their constructor.

```
JButton button = new JButton ( );
```

- After creating all components you are interested in, the next task is to add these components into the component area of your JFrame (i.e ContentPane, the reference to which is in variable con of type Container)
- Use *add* method of the Container to accomplish this and pass it the component to be added.

```
con.add(button);
```

10.6.6 Set size of frame and make it visible

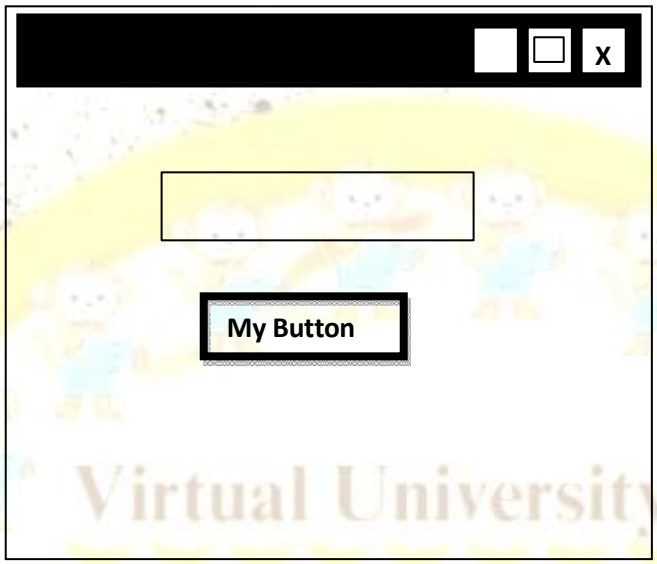
- A frame must be made visible via a call to setVisible(true) and its size defined via a call setSize(rows in pixel, columns in pixel) to be displayed on the screen.

```
frame.setSize(200,300) ;
frame.setVisible(true) ;
```

Note: By default, all JFrame's are invisible. To make visible frame visible we have passed *true* to the setVisible method.

```
frame.setVisible(false) ;
```

Example: Making a Simple GUI



The above figured GUI contains one text field and a button. Let's code it by following the six GUI creation steps we discussed.

Code for Simple GUI:

```
// File GUITest.java

//Step 1: import packages
import java.awt.*;
import javax.swing.*;

public class GUITest {
JFrame myFrame ;

//method used for setting layout of GUI
public void initGUI ( ) {

//Step 2: setup the top level container

myFrame = new JFrame();

//Step 3: Get the component area of top-level container
Container c = myFrame.getContentPane();

//Step 4: Apply layouts
c.setLayout( new FlowLayout( ) );
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

```
//Step 5: create & add components
JTextField tf = new JTextField(10);
JButton b1 = new JButton("My Button");

c.add(tf);
c.add(b1);

//Step 6: set size of frame and make it visible
myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
myFrame.setSize(200,150);
myFrame.setVisible(true);
} //end initGUI method

public GUITest () {// default constructor
initGUI ();
}

public static void main (String args[ ]) {
GUITest gui = new GUITest();
}
} // end of class
```

10.7 Important Points to Consider

- *main* method (from where program execution starts) is written in the same class. The main method can be in a separate class instead of writing in the same class its your choice.
- Inside main, an object of GUI test class is created that results in calling of constructor of the class and from the constructor, *initGUI* method is called that is responsible for setting up the GUI.
- The following line of code is used to exit the program when you close the window

```
myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

If you delete this line and run your program, the desired GUI would be displayed. However if you close the window by using (X) button on top left corner of your window, you'll notice that the control doesn't return back to command prompt. The reason for this is that the java process is still running. However if you put this line in your code, when you exit your prompt will return.

10.8 References:

- Sun java tutorial: http://java.sun.com/docs/books/tutorial/java
- Thinking in java by Bruce Eckle
- Beginning Java2 by Ivor Hortan
- GUI creation steps are taken from the book Java A Lab Course by Umair Javed

10.9 Graphical User Interfaces - 2

10.9.1 Layout Managers

Layout Managers are used to form the appearance of your GUI. They are concerned with the arrangement of components of GUI. A general question is "why we cannot place components at our desired location (may be using the x,y coordinate position?"

The answer is that you can create your GUI without using Layout Managers and you can also do VB style positioning of components at some x,y co-ordinate in Java, but that is generally not advisable if you desire to run the same program on different platforms

The appearance of the GUI also depends on the underlying platform and to keep that same the responsibility of arranging layout is given to the LayoutManagers so they can provide the same look and feel across different platforms

Commonly used layout managers are

- Flow Layout
- Grid Layout
- Border Layout
- Box Layout
- Card Layout
- GridBag Layout and so on

Let us discuss the top three in detail one by one with code examples. These top three will meet most of your basic needs

10.9.1.1 Flow Layout

- Position components on line by line basis. Each time a line is filled, a new line is started.
- The size of the line depends upon the size of your frame. If you stretch your frame while your program is running, your GUI will be disturbed.

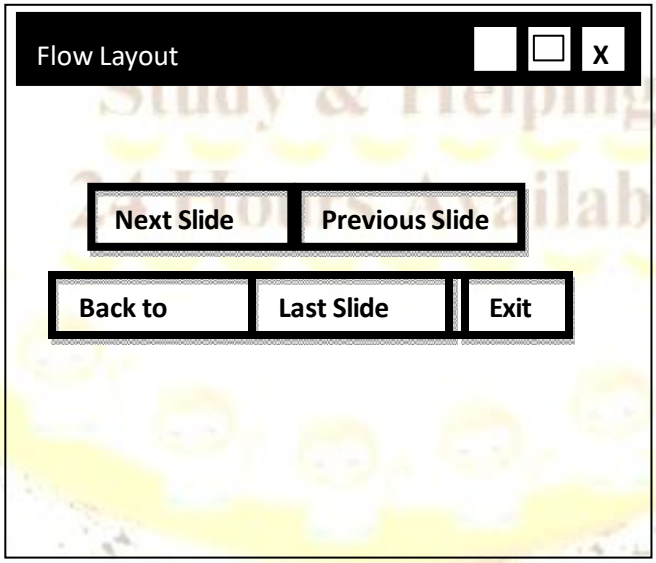
Example Code

```
// File FlowLayoutTest.
java import java.awt.*;
import javax.swing.*;
public class FlowLayoutTest {
JFrame myFrame ;
JButton b1, b2, b3, b4, b5;
//method used for setting layout of GUI
public void initGUI ( ) {
```

© Copyright Virtual University of Pakistan

```
myFrame = new JFrame("Flow Layout");
Container c = myFrame.getContentPane();
c.setLayout( new FlowLayout( ) );
b1 =    new JButton("Next Slide");
b2 =    new JButton("Previous Slide");
b3 =    new JButton("Back to Start");
b4 =    new JButton("Last Slide");
b5 =    new JButton("Exit");
c.add(b1);
c.add(b2);
c.add(b3);
c.add(b4);
c.add(b5);
myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
myFrame.setSize(300,150);
myFrame.setVisible(true);
} //end initGUI method
public FlowLayoutTest () {    // default constructor
initGUI ();
}
public static void main (String args[ ]) {
FlowLayoutTest flTest = new FlowLayoutTest();
}
} // end of class
```

Output



10.9.1.2 Grid Layout

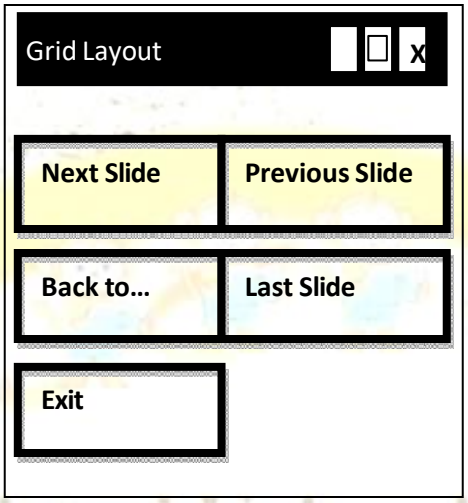
- Splits the panel/window into a grid(cells) with given number of rows and columns.
- Forces the size of each component to occupy the whole cell. Size of each component is same .
- Components are added row wise. When all the columns of the first row are get filled the components are then added to the next row.
- Only one component can be added into each cell.

Example Code

```
// File GridLayoutTest.java
 import java.awt.*;
import javax.swing.*;
public class GridLayoutTest {
JFrame myFrame ;
JButton b1, b2, b3, b4, b5;
//method used for setting layout of GUI
public void initGUI ( ) {
myFrame = new JFrame("Grid Layout");
Container c = myFrame.getContentPane();
                                    // rows , cols
c.setLayout( new GridLayout( 3   , 2 ) );
b1 = new JButton("Next Slide");
b2 = new JButton("Previous Slide");
b3 = new JButton("Back to Start");
b4 = new JButton("Last Slide");
b5 = new JButton("Exit");
c.add(b1);
c.add(b2);
c.add(b3);
c.add(b4);
c.add(b5);
myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
myFrame.setSize(300,150);
myFrame.setVisible(true);
} //end initGUI method
public GridLayoutTest () {     // default constructor
initGUI ();
}
public static void main (String args[ ]) {
GridLayoutTest glTest = new GridLayoutTest();
}
} // end of class
```

Virtual University of Pakistan
BANANA MONKEY
Study & Helping
24 Hours Available

Output

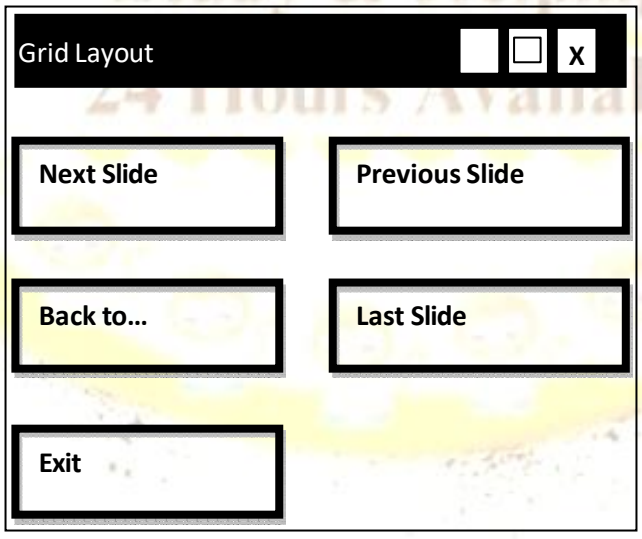


Modification

The grid layout also allows the spacing between cells. To achieve spacing between cells, modify the above program.
Pass additional parameters to the constructor of GridLayout, spaces between rows & spaces between columns as shown below

c.setLayout(new GridLayout(3 , 2,10 , 20));

The output is look similar to one given below.



© Copyright Virtual University of Pakistan

10.9.1.3 Border Layout

- Divides the area into five regions. North, South, East, West and Center
- Components are added to the specified region
- If any region not filled, the filled regions will occupy the space but the center region will still appear as background if it contains no component.
- Only one component can be added into each region.



Example Code:

```
// File BorderLayoutTest.
java import java.awt.*;
import javax.swing.*;
public class BorderLayoutTest {
JFrame myFrame ;
JButton b1, b2, b3, b4, b5;
//method used for setting layout of GUI
public void initGUI ( ) {
myFrame = new JFrame("Border Layout");
Container c = myFrame.getContentPane();
c.setLayout( new BorderLayout( ) );
b1 =    new JButton("Next Slide");
b2 =    new JButton("Previous Slide");
b3 =    new JButton("Back to Start");
b4 =    new JButton("Last Slide");
b5 =    new JButton("Exit");
c.add( b1 , BorderLayout.NORTH );
c.add( b2 , BorderLayout.SOUTH );
c.add( b3 , BorderLayout.EAST );
c.add( b4 , BorderLayout.WEST );
c.add( b5 , BorderLayout.CENTER);
myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
myFrame.setSize(300,150);
myFrame.setVisible(true);
} //end initGUI method
public BorderLayoutTest () { // default constructor
initGUI ();
```

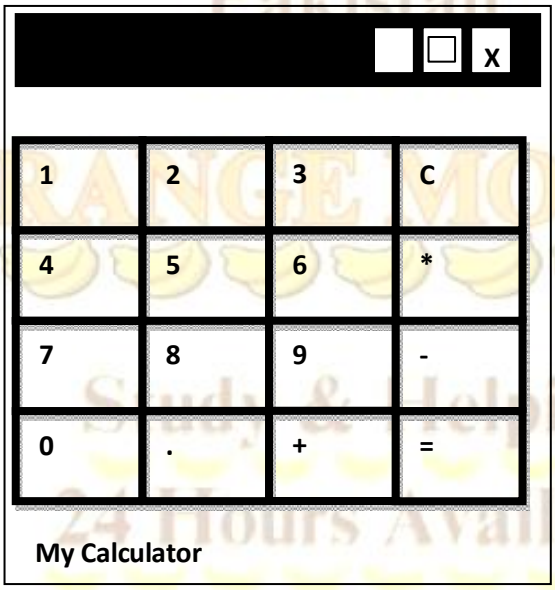
© Copyright Virtual University of Pakistan

```
}
public static void main (String args[ ]) {
BorderLayoutTest glTest = new BorderLayoutTest();
}
} // end of class
```

Points to Remember

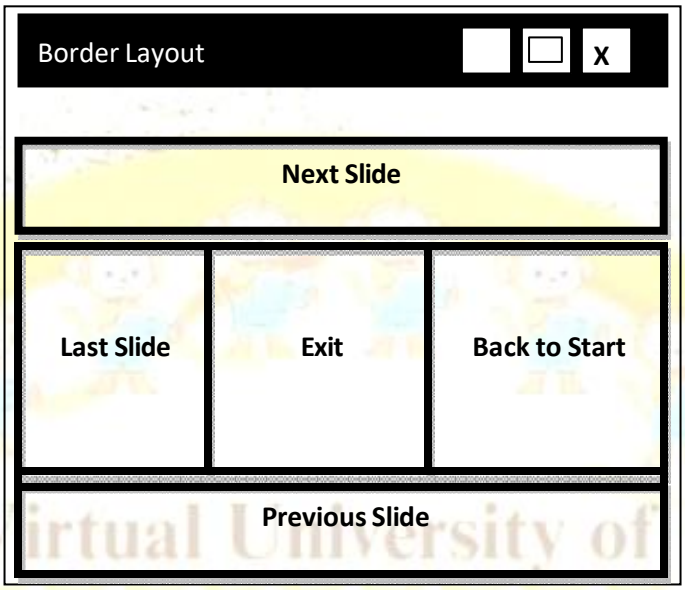
- Revisit the code of adding components, we specify the region in which we want to add component or otherwise they will not be visible.
- Consider the following segment of code: BorderLayout.NORTH, as you guessed correctly NORTH is a constant (final) defined in BorderLayout class public access modifier. Similarly the other ones are defined. Now you understand why so much emphasis has been made on following the naming conventions.

Output:



10.10 Making Complex GUIs

From the discussion above it seems that the basic Layout Managers may not help us in constructing complex GUIs, but generally a combination of these basic layouts can do the job. So let's try to create the calculator GUI given below



This GUI has 16 different buttons each of same size and text field on the top and a label 'my calculator' on the bottom.

So, how we can make this GUI? If Border Layout is selected, it has five regions (each region can have at most one component) but here we have more than five components to add. Lets try Grid Layout, but all the components in a Grid have same size and the text field at the top and label at the bottom has different size. Flow Layout cannot be selected because if we stretch our GUI it will destroy its shape.

Can we make this GUI? Yes, we can. Making of such GUI is a bit tricky business but General Purpose Containers are there to provide the solution.

10.10.1 JPanel

- It is general purpose container (can't exist alone, it has to be in some toplevel container) in which we can put in different components (JButton , JTextField etc even other JPanels)
- JPanel has its own layout that can be set while creating JPanel instance

```
JPanel myPanel = new JPanel ( new FlowLayout( ) );
```

- Add components by using *add* method like shown below.

```
myPanel.add (button);
```

- Must be added to a top level container (like JFrame etc) in order to be visible as they (general purpose containers) can't exist alone.

© Copyright Virtual University of Pakistan

10.10.2 Solution

To make the calculator GUI shown above, take JFrame (top level container) and set its layout to border. Then take JPanel (general purpose container) and set its layout to Grid with 4 rows and 4 columns.

Add buttons to JPanel as they all have equal size and JPanel layout has been set to GridLayout. Afterthat, add text field to the north region, label to the south region and panel to the center region of the JFrame's container. The east and west regions are left blank and the center region will be stretched to cover up these. So, that's how we can build our calculator GUI.

Code for Calculator GUI

```
// File CalculatorGUI.java
 import java.awt.*;
import javax.swing.*;
public class CalculatorGUI {
JFrame fCalc;
JButton b1, b2, b3, b4, b5, b6, b7, b8, b9, b0;
JButton bPlus, bMinus, bMul, bPoint, bEqual, bClear;
JPanel pButtons;
JTextField tfAnswer;
JLabel lMyCalc;
//method used for setting layout of GUI
public void initGUI ( ) {
fCalc = new JFrame();
b0 = new JButton("0");
b1 = new JButton("1");
b2 = new JButton("2");
b3 = new JButton("3");
b4 = new JButton("4");
b5 = new JButton("5");
b6 = new JButton("6");
b7 = new JButton("7");
b8 = new JButton("8");
b9 = new JButton("9");
bPlus  = new JButton("+");
bMinus  = new JButton("-");
bMul    = new JButton("*");
bPoint  = new JButton(".");
bEqual  = new JButton("=");
bClear  = new JButton("C");
tfAnswer = new JTextField();
lMyCalc = new JLabel("My Clacualator");
//creating panel object and setting its layout
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

```
pButtons = new JPanel (new GridLayout(4,4));
//adding components (buttons) to panel
pButtons.add(b1);
pButtons.add(b2);
pButtons.add(b3);
pButtons.add(bClear);
pButtons.add(b4);
pButtons.add(b5);
pButtons.add(b6);
pButtons.add(bMul);
pButtons.add(b7);
pButtons.add(b8);
pButtons.add(b9);
pButtons.add(bMinus);
pButtons.add(b0);
pButtons.add(bPoint);
pButtons.add(bPlus);
pButtons.add(bEqual);
// getting componenet area of JFrame
Container con = fCalc.getContentPane();
con.setLayout(new BorderLayout());
//adding components to container
con.add(tfAnswer, BorderLayout.NORTH);
con.add(lMyCalc, BorderLayout.SOUTH);
con.add(pButtons, BorderLayout.CENTER);
fCalc.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
fCalc.setSize(300, 300);
fCalc.setVisible(true);
} //end initGUI method
public CalculatorGUI () {       // default constructor
initGUI ();
}
public static void main (String args[ ]) {
CalculatorGUI calGUI = new CalculatorGUI ();
}
} // end of class
```

10.11 Reference:

- Sun java tutorial: http://java.sun.com/docs/books/tutorial/java
- Thinking in java by Bruce Eckle
- Beginning Java2 by Ivor Hortan
- Java A Lab Course by Umair Javed

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 11: Event Handling

One of the most important aspects of most non-trivial applications (especially UI type-apps) is the ability to respond to events that are generated by the various components of the application, both in response to user interactions and other system components such as client-server processing. In this handout we will look at how Java supports event generation and handling and how to create (and process) custom events.

GUIs generate events when the user interacts with GUI. For example,

- Clicking a button
- Moving the mouse
- Closing Window etc

Both AWT and swing components (not all) generate events

- java.awt.event.*;
- javax.swing.event.*;

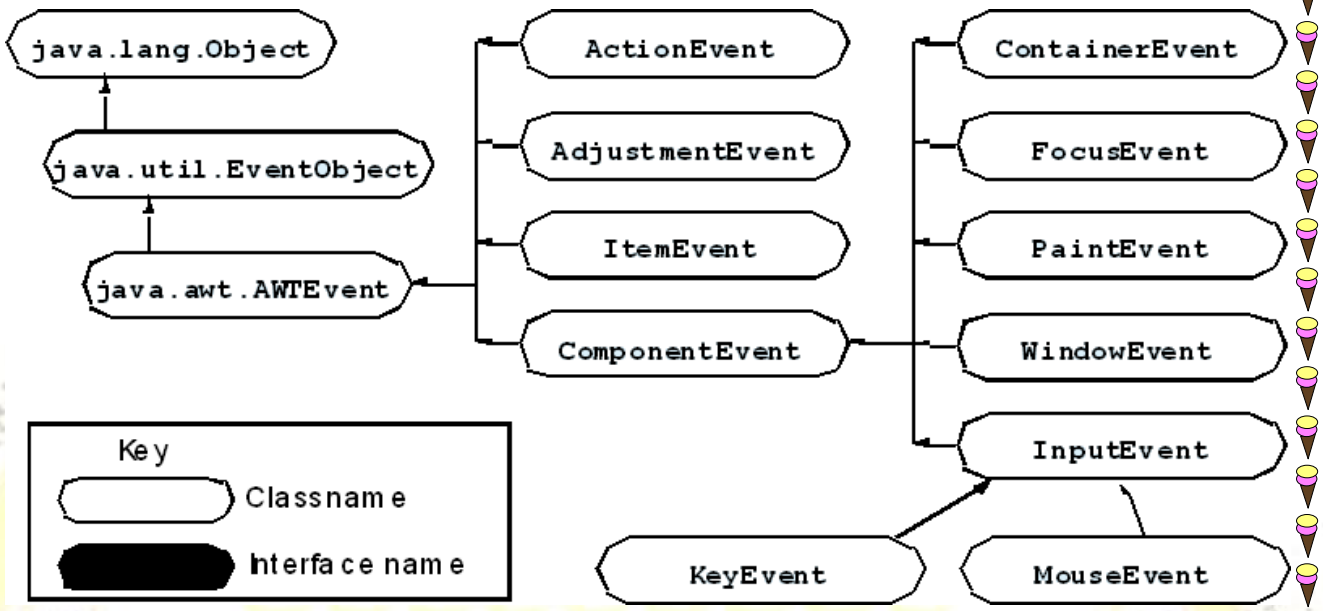
In java, events are represented by Objects

These objects tell us about event and its source. Examples are:

- ActionEvent (Clicking a button)
- WindowEvent (Doing something with window e.g. closing , minimizing)

Some event classes of java.awt.event are shown in diagram below

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available



11.1 Event Handling Model

In Java both AWT and Swing components use Event Delegation Model.

- In this model processing of an event is delegated to a particular object (handlers) in the program
- It's a Publish-Subscribe model. That is, event generating component publish an event and event handling components subscribe for that event. The publisher sends these events to subscribers. Similar to the way that you subscribe for newspaper and you get the newspaper at your home from the publisher.
- This model separates UI code from program logic, it means that we can create separate classes for UI components and event handlers and hence business/program logic is separated from GUI components.

11.2 Event Handling Steps

For a programmer the event Handling is a three step process in terms of code

- **Step 1:** Create components which can generate events (Event Generators)
- **Step 2:** Build component (objects) that can handle events (Event Handlers)
- **Step 3:** Register handlers with generators

© Copyright Virtual University of Pakistan

11.3 Event Handling Process

11.3.1 Step 1: Event Generators

The first step is that you create an event generator. You have already seen a lot of event generators like:

- Buttons
- Mouse
- Key
- Window etc

Most of GUI components can be created by calling their constructors. For example

JButton b1 = new JButton("Hello");

Now b1 can generate events

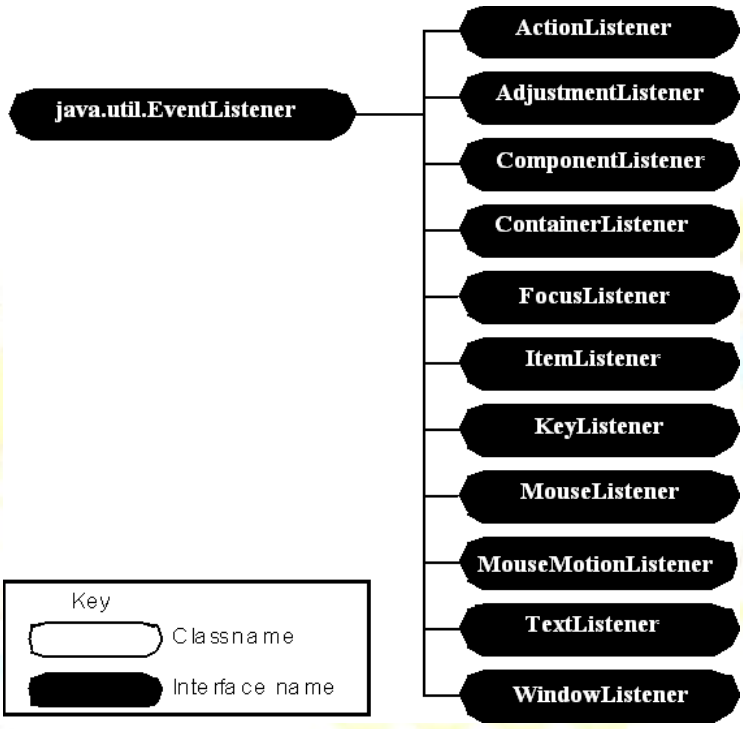
Note: We do not create Mouse/Keys etc as they are system components

11.3.2 Step 2: Event Handlers/ Event Listener

The second step is that you build components that can handle events

- First Technique - *By Implementing Listener Interfaces*
    - Java defines interfaces for every event type
    - If a class needs to handle an event. It needs to implement the corresponding listener interface
    - To handle "ActionEvent" a class needs to implement "ActionListener"
    - To handle "KeyEvent" a class needs to implement "KeyListener"
    - To handle "MouseEvent" a class needs to implement "MouseListener" and so on
    - Package java.awt.event contains different event Listener Interfaces which are shown in the following figure

© Copyright Virtual University of Pakistan



Some Example Listeners, the way they are defined in JDK by Sun

```
public interface ActionListener {
public void actionPerformed(ActionEvent e);
}
```

```
public interface ItemListener {
public void itemStateChanged(ItemEvent e);
}
```

```
public interface ComponentListener {
public void componentHidden(ComponentEvent e);
public void componentMoved(ComponentEvent e);
public void componentResized(ComponentEvent e);
public void componentShown(ComponentEvent e);
}
```

- By implementing an interface the class agrees to implement all the methods that are present in that interface. Implementing an interface is like *signing a contract*.
- Inside the method the class can do whatever it wants to do with that event
- Event Generator and Event Handler can be the same or different classes
- To handle events generated by Button. A class needs to implement ActionListener interface and thus needs to provide the definition of actionPerformed()method which is present in this interface.

```
public class Test implements ActionListener{
```

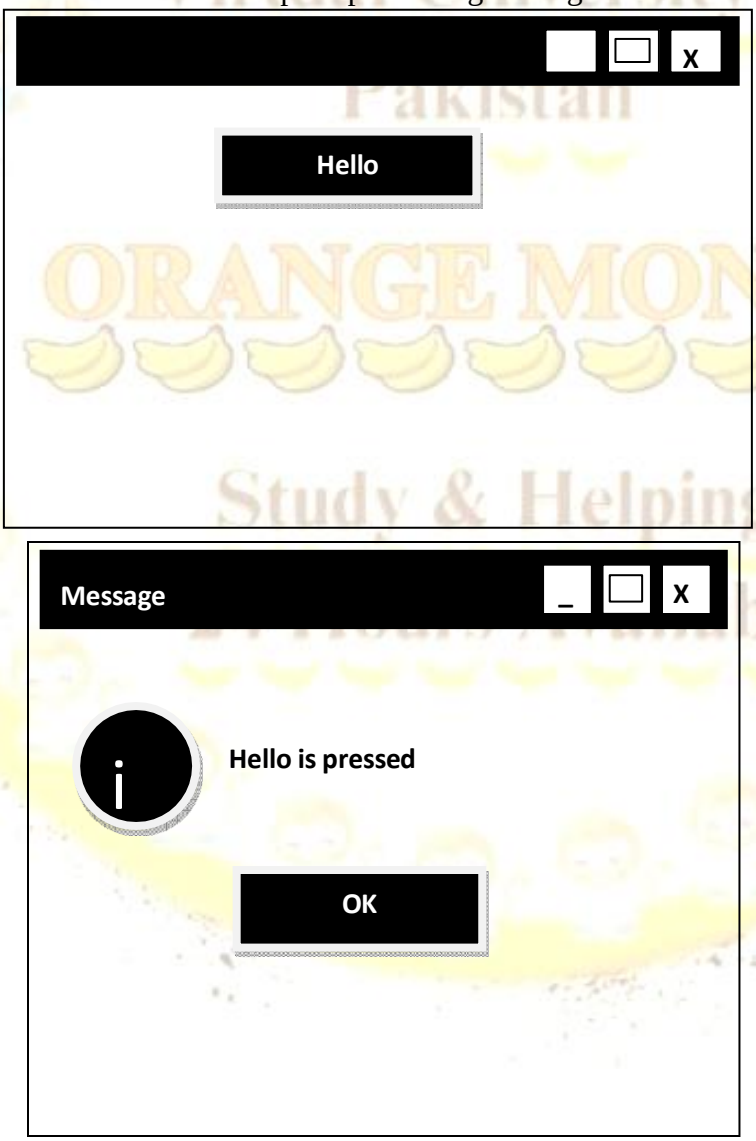
```
public void actionPerformed(ActionEvent ae) {
// do something
}
}
```

11.3.3 Step 3: Registering Handler with Generator

- The event generator is told about the object which can handle its events
- Event Generators have a method
  - addXXXListener(_reference to the object of Handler class_)
- For example, if b1 is JButtonthen
  - b1.addActionListener(this); // if listener and generator are same class

Event Handling Example

Clicking the "Hello" button will open up a message dialog shown below.



We will take the simplest approach of creating handler and generator in a single class. Button is our event generator and to handle that event our class needs to implement ActionListener Interface and to override its actionPerformedmethod and in last to do the registration.

```
1. import java.awt.*;
2. import javax.swing.*;
3. import java.awt.event.*;
/* Implementing the interface according to the type of the event, i.e. creating event handler (first part of step 2 of our process)
*/

4. public class ActionEventTest implements ActionListener{
5. JFrame frame;
6. JButton hello;
// setting layout components
7. public void initGUI ( ) {
8. frame = new JFrame();
9. Container cont =     frame.getContentPane();
10. cont.setLayout(new FlowLayout());
//Creating event generator step-1 of our process
11. hello = new JButton("Hello");
/* Registering event handler with event generator. Since event handler is in same object that contains button, we have used this to pass the reference.(step 3 of the process) */
12. hello.addActionListener(this);
13. cont.add(hello);
14. frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15. frame.setSize(150, 150);
16. frame.setVisible(true);
17. }
//constructor
18. public ActionEventTest ( ) {
19. initGUI();
20. }
/* Override actionPerformed method of ActionListener's interfacemethod of which will be called when event takes place (second part of step 2 of our process) */
21.       public void actionPerformed(ActionEvent event) {
22. JOptionPane.showMessageDialog(null,"Hello is pressed");
23. }
24.       public static void main(String args[]) {
25. ActionEventTest aeTest = new ActionEventTest();
26. }
27.} // end class
```

Virtual University of Pakistan
Study & Helping 24 Hours Available

11.4 How Event Handling Participants Interact Behind the Scenes?

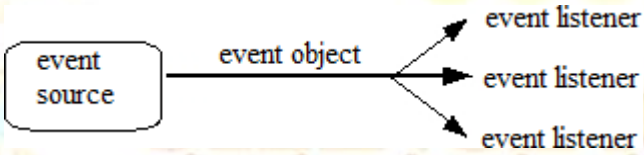
We have already seen that what a programmer needs to do handle events. Let's see what takes place behind the scenes, i.e How JVM handles event. Before doing that lets revisit different participants of Event Handling Process and briefly what they do.

11.4.1 Event Generator / Source

- Swing and awt components
- For example, JButton, JTextField, JFrame etc
- Generates an event object
- Registers listeners with itself

11.4.2 Event Object

- Encapsulate information about event that occurred and the source of that event
- For example, if you click a button, ActionEvent object is created



11.4.3 Event Listener/handler

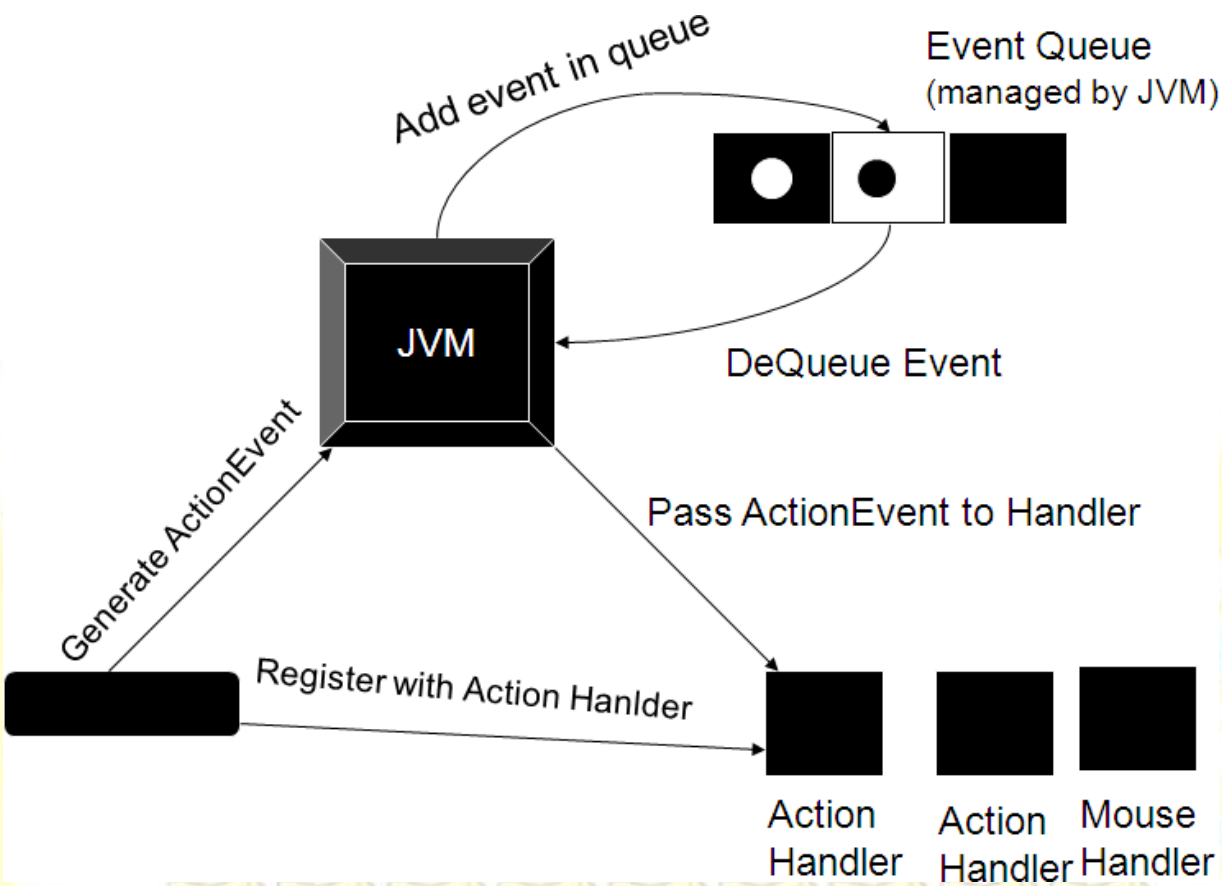
- Receives event objects when notified, then responds
- Each event source can have multiple listeners registered on it
- Conversely, a single listener can register with multiple event sources

11.4.4 JVM

- Receives an event whenever one is generated
- Looks for the listener/handler of that event
- If exist, delegate it for processing
- If not, discard it (event).

When button generates an ActionEvent it is sent to JVM which puts it in an event queue. After that when JVM find it appropriate it de-queue the event object and send it to all the listeners that are registered with that button. This is all what we shown in the pictorial form below:

© Copyright Virtual University of Pakistan

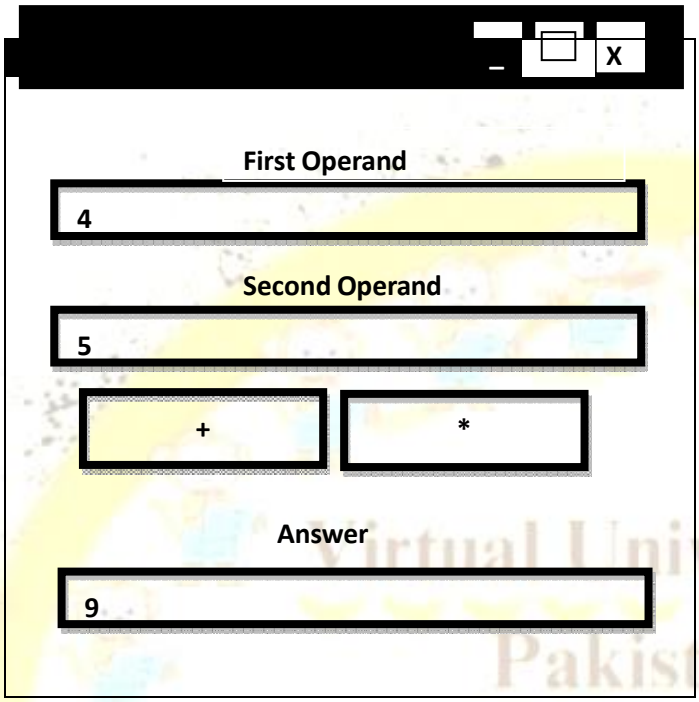


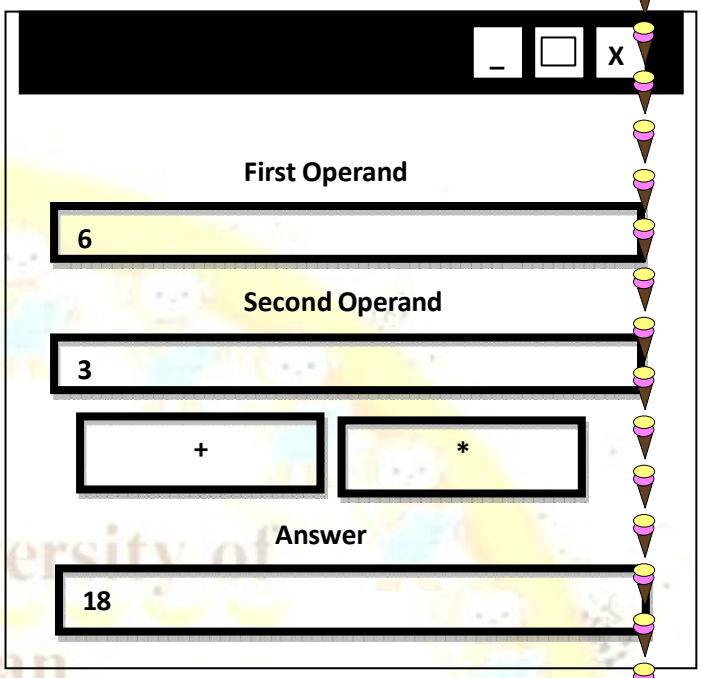
(Figure from JAVA A Lab Course)

Making Small Calculator

- User enters numbers in the provided fields
- On pressing "+" button, sum would be displayed in the answer field
- On pressing "*" button, product would be displayed in the answer field

Study & Helping
24 Hours Available





Example Code: Making Small Calculator

```
1. import java.awt.*;
2. import javax.swing.*;
3. import java.awt.event.*;
4. public class SmallCalcApp implements ActionListener{
5. JFrame frame;
6. JLabel firstOperand, secondOperand, answer;
7. JTextField op1, op2, ans;
8. JButton plus, mul;
9. // setting layout
10. public void initGUI ( ) {
11. frame = new JFrame();
12. firstOperand          = new JLabel("First Operand");
13. secondOperand = new JLabel("Second Operand");
14. answer       = new JLabel("Answer");
15. op1 = new JTextField (15);
16. op2 = new JTextField (15);
17. ans = new JTextField (15);
18. plus = new JButton("+");
19. plus.setPreferredSize(new Dimension(70,25));
20. mul = new JButton("*");
21. mul.setPreferredSize(new Dimension(70,25));
22. Container cont =   frame.getContentPane();
23. cont.setLayout(new FlowLayout());
24. cont.add(firstOperand);
25. cont.add(op1);
```

```
26. cont.add(secondOperand);
27. cont.add(op2);
28. cont.add(plus);
29. cont.add(mul);
30. cont.add(answer);
31. cont.add(ans);
32. plus.addActionListener(this);
33. mul.addActionListener(this);
34. frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
35. frame.setSize(200, 220);
36. frame.setVisible(true);
37. }
38. //constructor
39. public SmallCalcApp ( ) {
40. initGUI();
41. }
42. public void actionPerformed(ActionEvent event) {
43. String oper, result;
44. int num1, num2, res;
/* All the information regarding an event is contained inside the event object. Here we are calling the
getSource() method on the event object to figure out the button that has generated that event.      */
45. if (event.getSource() == plus) {
46. oper = op1.getText();
47. num1 = Integer.parseInt(oper);
48. oper = op2.getText();
49. num2 = Integer.parseInt (oper);
50. res = num1+num2;
51. result = res+"";
52. ans.setText(result);
53. }
54. else if (event.getSource() == mul) {
55. oper = op1.getText();
56. num1 = Integer.parseInt(oper);
57. oper = op2.getText();
58. num2 = Integer.parseInt (oper);
59. res = num1*num2;
60. result = res+"";
61. ans.setText(result);
62. }
63.       }
64. public static void main(String args[]) {
65. SmallCalcApp scApp = new SmallCalcApp();
66. }
67. }// end class
```

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

© Copyright Virtual University of Pakistan

Lecture 12: More Examples of Handling Events

12.1 Handling Mouse Event

Mouse events can be trapped for any GUI component that inherits from Component class. For example, JPanel, JFrame & JButton etc.

To handle Mouse events, two types of listener interfaces are available.

- MouseMotionListener
- MouseListener

The class that wants to handle mouse event needs to implement the corresponding interface and needs to provide the definition of all the methods in that interface.

12.1.1 MouseMotionListener interface

- Used for processing mouse motion events
- Mouse motion event is generated when mouse is moved or dragged

A MouseMotionListener interface isdefined in JDK as follows:

```
public interface MouseMotionListener {
public void mouseDragged (MouseEvent me);
public void mouseMoved (MouseEvent me);
}
```

12.1.2 MouseListener interface

- Used for processing “interesting” mouse events like when mouse is:
    - Pressed
    - Released
    - Clicked (pressed & released without moving the cursor)
    - Enter (mouse cursor enters the bounds of component)
    - Exit (mouse cursor leaves the bounds of component)

MouseListenerinterfaces are defined in JDK as follows:

```
public interface MouseListener {
public void mousePressed (MouseEvent me);
```

© Copyright Virtual University of Pakistan

```
public void mouseClicked (MouseEvent me);
public void mouseReleased (MouseEvent me);
public void mouseEntered (MouseEvent me);
public void mouseExited (MouseEvent me);
}
```

Example Code: Handling Mouse Events

Example to show Mouse Event Handling .Every time mouse is moved, the coordinates for a new place is shown in a label.

```
1. import java.awt.*;
2. import javax.swing.*;
3. import java.awt.event.*;
4. public class EventsEx implements MouseMotionListener{
5. JFrame frame;
6. JLabel coordinates;
7. // setting layout
8. public void initGUI ( ) {
9. // creating event generator
10. frame = new JFrame();
11. Container cont =frame.getContentPane();
12. cont.setLayout(new BorderLayout( ) );
13. coordinates = new JLabel ();
14. cont.add(coordinates, BorderLayout.NORTH);
15. // registring mouse event handler with generator
16. frame.addMouseMotionListener(this);
17. frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
18. frame.setSize(350, 350);
19. frame.setVisible(true);
20. } // end initGUI method
21. //default constructor
22.       public EventsEx ( ) {
23. initGUI();
24. }
// MouseMotionListener event hadler handling dragging
25.       public void mouseDragged (MouseEvent me) {
26. int x = me.getX();
27. int y = me.getY();
28. coordinates.setText("Dragged at [" + x + "," + y + "]");
29. }
// MouseMotionListener event handler handling motion
30.       public void mouseMoved (MouseEvent me) {
31. int x = me.getX();
```

© Copyright Virtual University of Pakistan

```
32. int y = me.getY();
33. coordinates.setText("Moved at [" + x + "," + y + "]");
34. }
35.       public static void main(String args[]) {
36. EventsEx ex = new EventsEx();
37. }
38. } // end class
```

Another Example: Handling Window Events

Task

We want to handle Window Exit event only

Why?

- When window is closed, control should return back to command prompt.
- But we have already achieved this functionality through following line of code

  frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

- But, what if we want to display some message (*Good Bye*) before exiting?



When user close the window, Message would be displayed



After pressing Ok button program will exit

How?

- To handle window events, we need to implement "WindowListner" interface.
- WindowListner" interface contains *7* methods We require only one i.e. windowClosing

- But, We have to provide definitions of all methods to make our class a concrete class
- WindowListener interface is defined in the JDK as follows

public interface **WindowListener** {
public void windowActivated(WindowEvent we); public void windowClosed(WindowEvent we); **public void windowClosing(WindowEvent we);** public void windowDeactivated(WindowEvent we); public void windowDeiconified(WindowEvent we); public void windowIconified(WindowEvent we); public void windowOpened(WindowEvent we);
}

- public void windowClosing(WindowEvent we)is our required method

Example Code: WindowExitHandler

This example code is modification of the last code example i.e. EventsEx.java

```
1. import java.awt.*;
2. import javax.swing.*;
3. import java.awt.event.*;
4. public class EventsEx implements MouseMotionListener , WindowListener {
5. JFrame frame;
6. JLabel coordinates;
// setting layout
7. public void initGUI ( ) {
// creating event generator
8. frame = new JFrame();
9. Container cont =     frame.getContentPane();
10. cont.setLayout(new BorderLayout( ) );
11. coordinates = new JLabel ();
12. cont.add(coordinates, BorderLayout.NORTH);
// registring mouse event handler with generator
13. frame.addMouseMotionListener(this);
// registering window handler with generator
14. frame.addWindowListener(this);
15. frame.setSize(350, 350);
16. frame.setVisible(true);
17. } // end initGUI method
//default constructor
18. public EventsEx ( ) {
```

© Copyright Virtual University of Pakistan

```
19. initGUI();
20. }

// MouseMotionListener event hadler handling dragging
21. public void mouseDragged (MouseEvent me) {
22. int x = me.getX();
23. int y = me.getY();
24. coordinates.setText("Dragged at [" + x + "," + y + "]");
25. }
// MouseMotionListener event handler handling motion
26. public void mouseMoved (MouseEvent me) {
27. int x = me.getX();
28. int y = me.getY();
29.
30. coordinates.setText("Moved at [" + x + "," + y + "]");
31. }
// window listener event handler
32. public void windowActivated (WindowEvent we) {      }
33. public void windowClosed (WindowEvent we) {         }
34. public void windowClosing (WindowEvent we) {
35. JOptionPane.showMessageDialog(null, “Good Bye”);
36. System.exit(0);
37. }
38. public void windowDeactivated (WindowEvent we) {    }
39. public void windowDeiconified (WindowEvent we) {    }
40. public void windowIconified (WindowEvent we) {      }
41. public void windowOpened (WindowEvent we) {         }
42. public static void main(String args[]) {
43. EventsEx ex = new EventsEx();
44. }
45. } // end class
```

Note: Coding exercises in working condition for this lecture are also available on **“Downloads”** section of LMS.

Lecture 13: Adapter Classes

Problem in Last Code Example

Problem

- We were interested in *windowClosing()* method only
- But have to provide definitions of all the methods, Why?
- Because a class implementing an interface has to provide definitions of all methods present in that interface.

Solution

- To avoid giving implementations of all methods of an interface when we are not using these methods we use Event *Adapter* classes

13.1 Adapter Classes

- For listener interfaces containing more than one event handling methods, jdk defines adapter classes. Examples are
  - For WindowListener < WindowAdapter
  - For MouseMotionListener < MouseMotionAdapter
  - and many more
- Adapter classes provide definitions for all the methods (empty bodies) of their corresponding Listener interface
- It means that WindowAdapter class implements WindowListener interface and provide the definition of all methods inside that Listener interface
- Consider the following example of MouseMotionAdapter and its corresponding MouseMotionListener interface

```
public interface MouseMotionListener {
public void mouseDragged (MouseEvent me);
public void mouseMoved (MouseEvent me); }
public class MouseMotionAdapter implements MouseMotionListener{
public void mouseDragged (MouseEvent me) {      }
public void mouseMoved (MouseEvent me) {      }
}
```

13.2 Available Adapter classes

© Copyright Virtual University of Pakistan

Listener	Adapter Class (If Any)	Registration Method
ActionListener		addActionListener
AdjustmentListener		addAdjustmentListener
ComponentListener	ComponentAdapter	addComponentListener
ContainerListener	ContainerAdapter	addContainerListener
FocusListener	FocusAdapter	addFocusListener
ItemListener		addItemListener
KeyListener	KeyAdapter	addKeyListener
MouseListener	MouseAdapter	addMouseListener
MouseMotionListener	MouseMotionAdapter	addMouseMotionListener
TextListener		addTextListener
WindowListener	WindowAdapter	addWindowListener

13.2.1 How to use Adapter Classes

- previously handler class need to implement interface

  public class EventsEx **implements MouseMotionListener**{...}
- Therefore it has to provide definitions of all the methods inside that interface now our handler class will inherit from adapter class

  public class EventsEx **extends MouseMotionAdapter**{...}
- Due to inheritance, all the methods of the adapter class will be available inside our handler class since adapter classes has already provided definitions with empty bodies.
- we do not have to provide implementations of all the methods again
- We only need to override our method of interest.

Example Code 13.1: Handling Window Events using Adapter Classes

Here we are modifying the window event code in the last example to show the use of WindowAdapter instead of WindowListener. Code related to MouseMotionListener is deleted to avoid cluttering of code.

```
1. import java.awt.*;
```

```
2. import javax.swing.*;
3. import java.awt.event.*;
4. public class EventsEx extends WindowAdapter {
5. JFrame frame;
6. JLabel coordinates;
// setting layout
7. public void initGUI ( ) {
// creating event generator
8. frame = new JFrame();
9. Container cont =     frame.getContentPane();
10. cont.setLayout(new BorderLayout( ) );
11. coordinates = new JLabel ();
12. cont.add(coordinates, BorderLayout.NORTH);
// registering window handler with generator
13. frame.addWindowListener(this);
14. frame.setSize(350, 350);
15. frame.setVisible(true);
16. } // end initGUI method
//default constructor
17. public EventsEx ( ) {
18. initGUI();
19. }
//          As you can see that we have only implemented
//          our required method
20. public void windowClosing (WindowEvent we) {
21. JOptionPane.showMessageDialog(null, "Good Bye");
22. System.exit(0);
23. }

24. public static void main(String args[]) {
25. EventsEx ex = new EventsEx();
26. }
27. } // end class
```

Problem in Last Code Example

- We have inherited from WindowAdapter
- What if we want to use MouseMotionAdpater as well? Or what if our class already inherited from some other class?

Problem

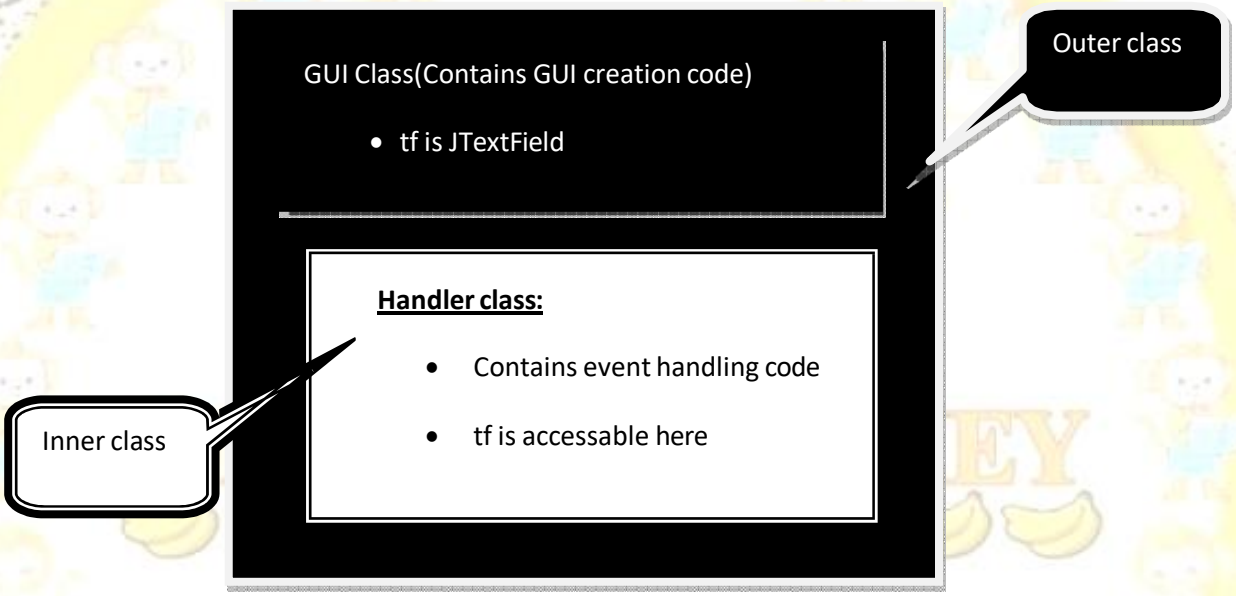
- Java allows single inheritance

Solution

- Use *Inner* classes

13.3 Inner Classes

- A class defined inside another class
- Inner class can access the instance variables and members of outer class
- It can have constructors, instance variables and methods, just like a regular class
- Generally used as a private utility class which does not need to be seen by others classes



Example Code13.2: Handling Window Event with Inner Class

Here we are modifying the window event code in the last example to show the use of WindowAdapter as an inner class.

```
1.      import java.awt.*;
2.      import javax.swing.*;
3.      import java.awt.event.*;
4.      public class EventEx {

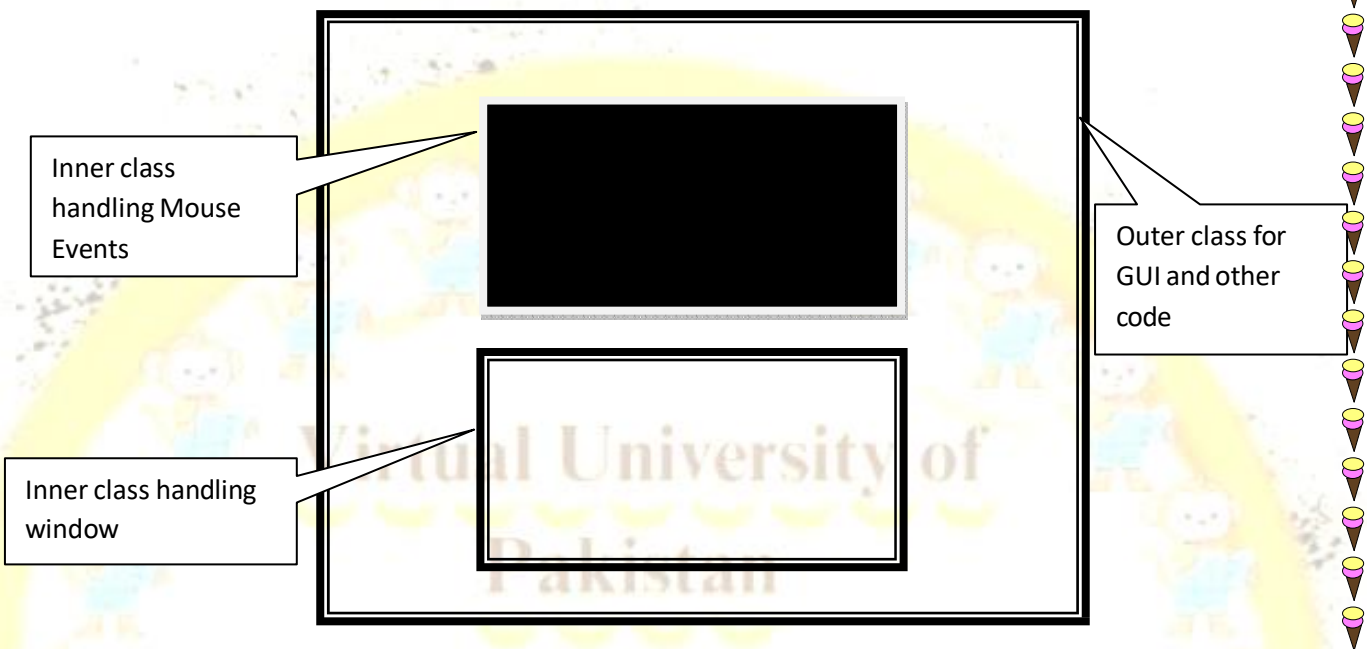
5. JFrame frame;
6. JLabel coordinates;
7. // setting layout
8. public void initGUI ( ) {
9. frame = new JFrame();
10. Container cont =frame.getContentPane();
11. cont.setLayout(new BorderLayout( ));
12. coordinates = new JLabel ();
```

```
13. cont.add(coordinates, BorderLayout.NORTH);
/* Creating an object of the class which is handling our
window events and registering it with generator      */
14. WindowHandler handler = new WindowHandler ();
15. frame.addWindowListener(handler);
16. frame.setSize(350, 350);
17. frame.setVisible(true);
18. } // end initGUI
//default constructor
19. public EventEx ( ) {
20. initGUI();
21. }
/* Inner class implementation of window adapter. Outer
class is free to inherit from any other class. */
22. private class WindowHandler extends WindowAdapter {
// Event Handler for WindowListener
23. public void windowClosing (WindowEvent we) {
24. JOptionPane.showMessageDialog(null, "Good Bye");
25. System.exit(0);
26. }
27. } // end of WindowHandler class
28. public static void main(String args[]) {
29. EventEx e = new EventEx();
30. }
31. } // end class
```

Example Code 13.3: Handling Window and Mouse Events with Inner Class

Here we are modifying the window event code of the last example to handle window and mouse events using inner classes. The diagram given below summarizes the approach.

© Copyright Virtual University of Pakistan




```
1.      import java.awt.*;
2.      import javax.swing.*;
3.      import java.awt.event.*;
4.      public class EventEx {
5. JFrame frame;
6. JLabel coordinates;
7. // setting layout
8. public void initGUI ( )
{
9. frame = new JFrame();
10. Container cont =frame.getContentPane();
11. cont.setLayout(new BorderLayout( ) );
12. coordinates = new JLabel ();
13. cont.add(coordinates, BorderLayout.NORTH);
/* Creating an object of the class which is handling our window events and registering it with
generator */
14. WindowHandler whandler = new WindowHandler ();
15. frame.addWindowListener(whandler);
/* Creating an object of the class which is handling our MouseMotion events & registering it with
generator */
16. MouseHandler mhandler = new MouseHandler ();
17. frame.addMouseMotionListener(mhandler);
18. frame.setSize(350, 350);
19. frame.setVisible(true);
```

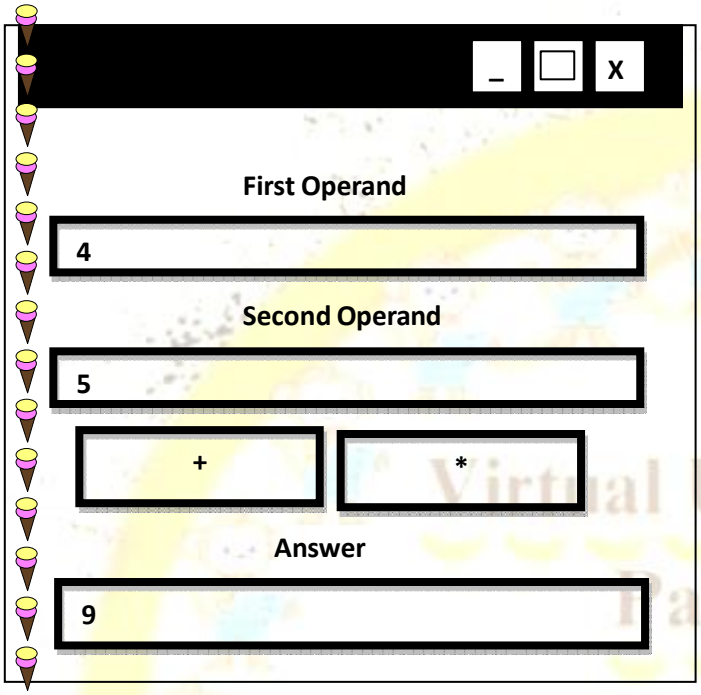
© Copyright Virtual University of Pakistan

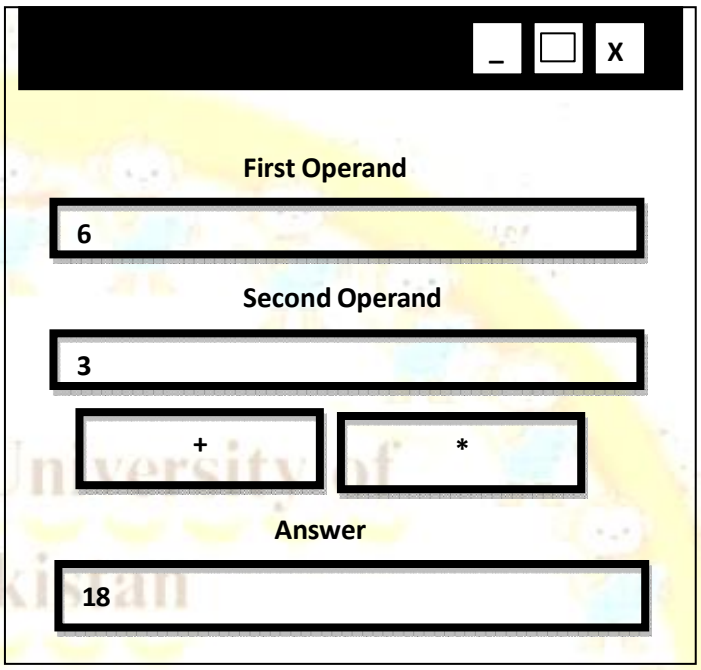
```
20. }
//default constructor
21. public EventEx ( ) {
22. initGUI();
23. }
/* Inner class implementation of WindowAdapter. Outer class
is free to inherit from any other class. */
24. private class WindowHandler extends WindowAdapter {
// Event Handler for WindowListener
25. public void windowClosing (WindowEvent we) {
26. JOptionPane.showMessageDialog(null, “Good Bye”);
27. System.exit(0);
28. }
29. } // end of WindowHandler
//Inner class implementation of MouseMotionAdapter
30. private class MouseHandler extends MouseMotionAdapter {
// Event Handler for mouse motion events
31. public void mouseMoved (MouseEvent me) {
32. int x = me.getX();
33. int y = me.getY();
34. coordinates.setText(“Moved at [" + x + "," + y + "]” );
35. }
36. } // end of MouseHandler
37. public static void main(String args[]) {
38. EventEx e = new EventEx();
39. }
40. } // end clas
```

Example Code: Making Small Calculator using Inner classes

- User enters numbers in the provided fields
- On pressing “+” button, sum would be displayed in the answer field
- On pressing “*” button, product would be displayed in the answer field

© Copyright Virtual University of Pakistan






```
1. import java.awt.*;
2. import javax.swing.*;
3. import java.awt.event.*;
4. public class SmallCalcApp{
5. JFrame frame;
6. JLabel firstOperand, secondOperand, answer;
7. JTextField op1, op2, ans;
8. JButton plus, mul;
9. // setting layout
10. public void initGUI ( ) {
11. frame = new JFrame();
12. firstOperand          = new JLabel("First Operand");
13. secondOperand = new JLabel("Second Operand");
14. answer       = new JLabel("Answer");
15. op1 = new JTextField (15);
16. op2 = new JTextField (15);
17. ans = new JTextField (15);
18. plus = new JButton("+");
19. plus.setPreferredSize(new Dimension(70,25));
20. mul = new JButton("*");
21. mul.setPreferredSize(new Dimension(70,25));
22. Container cont =   frame.getContentPane();
23. cont.setLayout(new FlowLayout());
```

© Copyright Virtual University of Pakistan

```
24. cont.add(firstOperand);
25. cont.add(op1);
26. cont.add(secondOperand);
27. cont.add(op2);
28. cont.add(plus);
29. cont.add(mul);
30. cont.add(answer);
31. cont.add(ans);
/* Creating an object of the class which is handling
button events & registering it with generators */
32. ButtonHandler bHandler = new ButtonHandler();
33. plus.addActionListener(bHandler);
34. mul.addActionListener(bHandler);
35. frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
36. frame.setSize(200, 220);
37. frame.setVisible(true);
38. }
39. //constructor
40. public SmallCalcApp ( ) {
41. initGUI();
42. }
//Inner class implementation of ActionListener
43. private class ButtonHandler implements ActionListener{
44. public void actionPerformed(ActionEvent event) {
45. String oper, result;
46. int num1, num2, res;
47. if (event.getSource() == plus) {
48. oper = op1.getText();
49. num1 = Integer.parseInt(oper);
50. oper = op2.getText();
51. num2 = Integer.parseInt (oper);
52. res = num1+num2;
53. result = res+"";
54. ans.setText(result);
55 }
56. else if (event.getSource() == mul) {
57. oper = op1.getText();
58. num1 = Integer.parseInt(oper);
59. oper = op2.getText();
60. num2 = Integer.parseInt (oper);
61. res = num1*num2;
62. result = res+"";
63. ans.setText(result);
64 }
65. } // end actionPerformed method
66. } // end inner class ButtonHandler
```

© Copyright Virtual University of Pakistan

```
67. public static void main(String args[]) {
68. SmallCalcApp scApp = new SmallCalcApp();
69. }
70. }// end class
```

13.4 Anonymous Inner Classes

- has no name
- same as inner class in capabilities
- much shorter
- Difficult to understand

13.5 Named vs. Anonymous Objects

13.5.1Named

- String s = "hello";
  System.out.println(s);
- "hello" has a named reference *s.*

13.5.2Anonymous

- System.out.println("hello");

We generally use anonymous object when there is just a onetime use of a particular object but in case of a repeated use we generally used named objects and use that named reference to use that objects again and again.

Example Code 13.4 Handling Window Event with Anonymous Inner Class

Here we are modifying the window event code of 13.3 to show the use of anonymous inner class.

```
28. import java.awt.*;
29. import javax.swing.*;
30. import java.awt.event.*;
31. public class EventsEx extends WindowAdapter {
32.  JFrame frame;
33.  JLabel coordinates;
// setting layout
34. public void initGUI ( ) {
// creating event generator
35.  frame = new JFrame();
36.  Container cont =frame.getContentPane();
37.  cont.setLayout(new BorderLayout( ) );
38.  coordinates = new JLabel ();
39.  cont.add(coordinates, BorderLayout.NORTH);
// registering event handler (anonymous inner class)
// with generator by using
40.  frame.addWindowListener (
41.  new WindowAdapter ( ) {
```

© Copyright Virtual University of Pakistan

```
42. public void windowClosing (WindowEvent we) {
43. JOptionPane.showMessageDialog(null, "Good Bye");
44. System.exit(0);
45. } // end window closing
46. } // end WindowAdapter
47. ); // end of addWindowListener
48. frame.setSize(350, 350);
49. frame.setVisible(true);
50. } // end initGUI method
//default constructor
51.     public EventsEx ( ) {
52. initGUI();
53. }
54.     public static void main(String args[]) {
55. EventsEx ex = new EventsEx();
56. }
57. } // end class
```

13.6 Summary of Approaches for Handling Events

- By implementing Interfaces
- By extending from Adapter classes

To implement the above two techniques we can use

- **Same class**
  - putting event handler & generator in one class
- **Separate class**
  - Outer class
    - Putting event handlers & generator in two different classes
  - Inner classes
  - Anonymous Inner classes

13.7 References

Java A Lab Course by Umair Javed

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 14: Java Database Connectivity

14.1 Introduction

Java Database Connectivity (JDBC) provides a standard library for accessing databases. The JDBC API contains number of interfaces and classes that are extensively helpful while communicating with a database.

14.2 The java.sql package

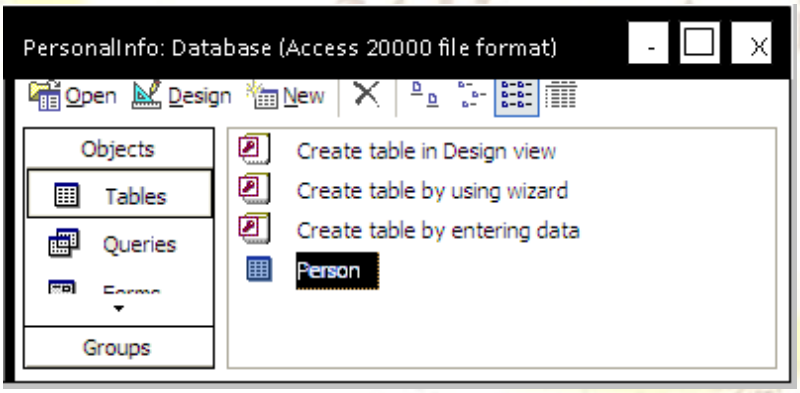
The java.sql package contains basic & most of the interfaces and classes. You automatically get this package when you download the J2SE™. You have to import this package whenever you want to interact with a relational database.

14.3 Connecting With Microsoft Access

In this handout, we will learn how to connect & communicate with Microsoft Access Database. We chooses Access because most of you are familiar with it and if not than it is very easy to learn.

14.3.1 Create Database

In start create a database "PersonInfo" using Microsoft Access. Create one table named "Person". The schema of the table is shown in the picture.



Person: Table

	Field Name	Data Type
🔑	id	AutoNumber
	name	Text
	address	Text
	phoneNum	Text

Add the following records into Person table as shown below:

Person: Table

	id	name	address	phoneNum
▶	1	ali	model town	9203256
	2	usman	gulberg	8219065
	3	raza	defence	5173946
*	0			

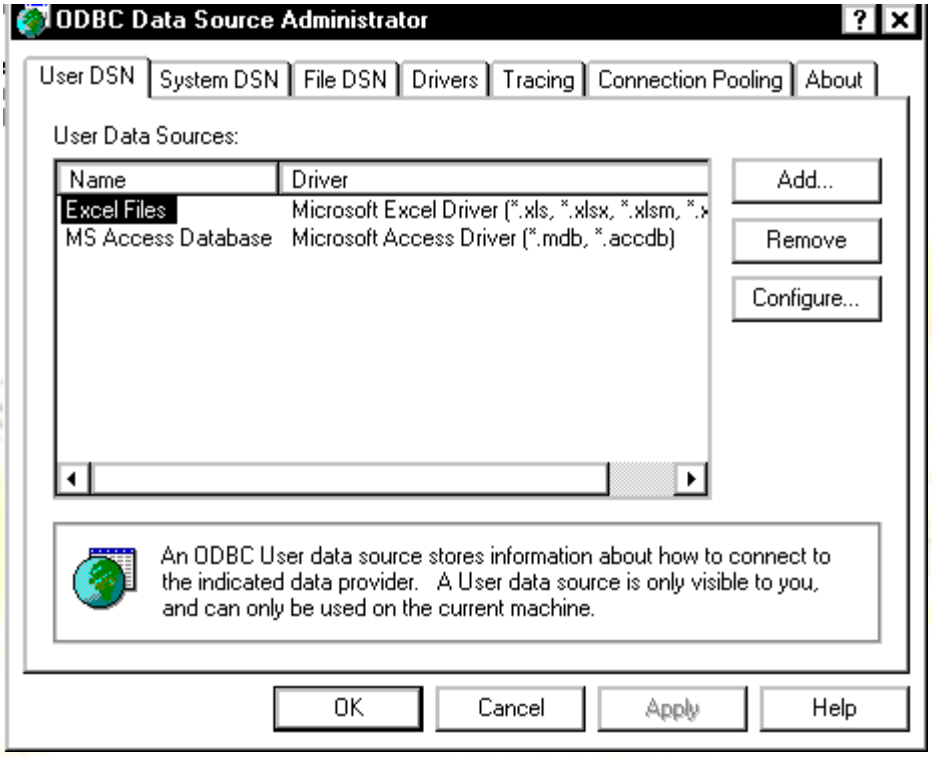
Record: 1 of 3

Save the data base in some folder. (Your database will be saved as an .mdb file)

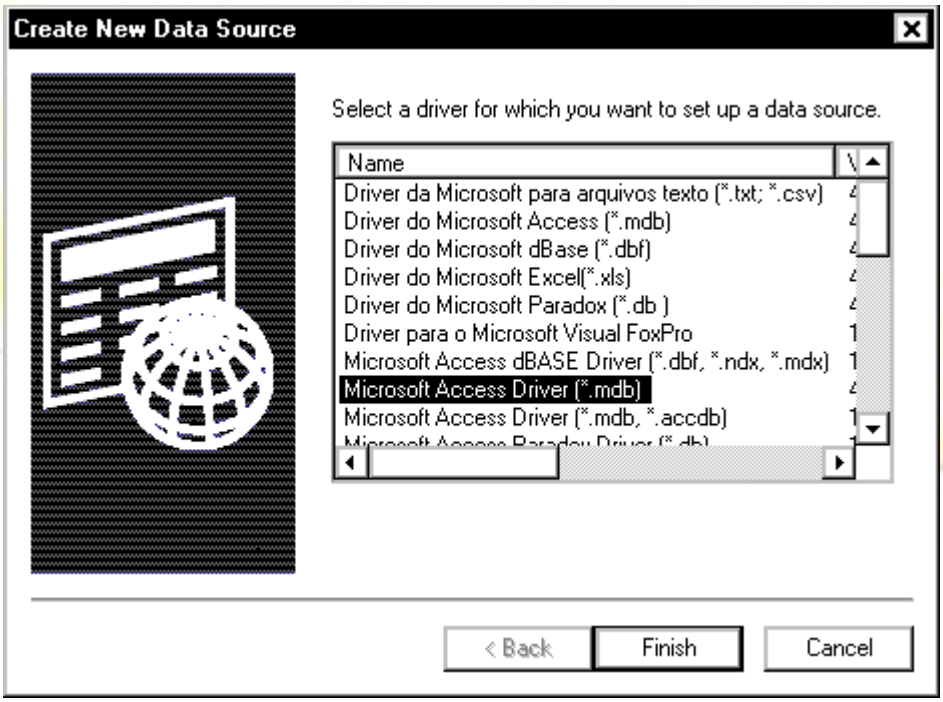
14.3.2 Setup System DSN

- After creating database, you have to setup a system Data Source Name (DSN). DSN is a name through which your system recognizes the underlying data source.
- Select Start -> Settings -> Control Panel -> Administrative Tools -> Data Sources (ODBC).
- The ODBC Data Source Administrator window would be opened as shown below: Select System DSN tab. (If you are unable to use System DSN tab due to security restrictions on your machine, you can use the User DSN tab)

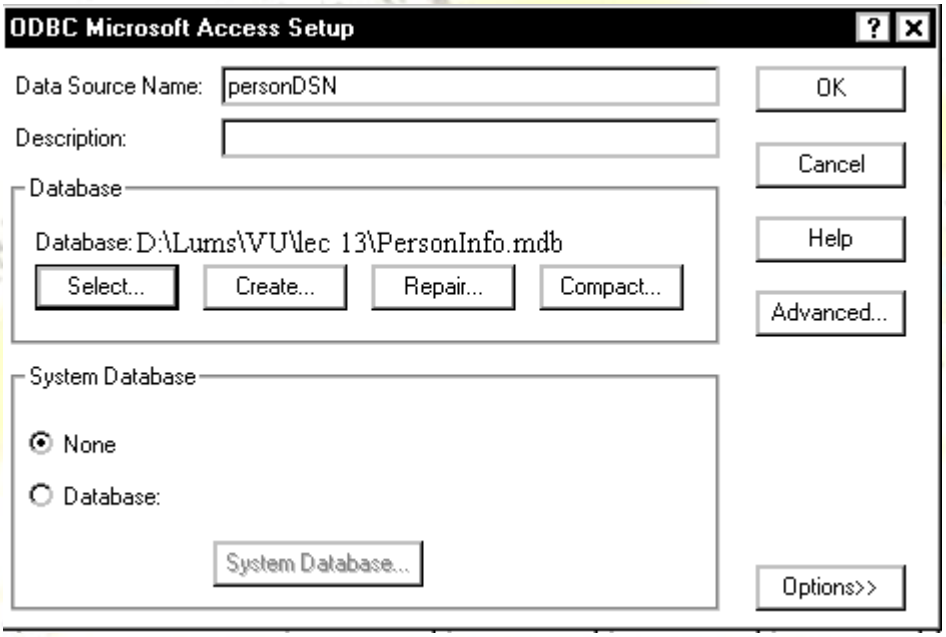
© Copyright Virtual University of Pakistan



- Press Add… button and choose Microsoft Access Driver (*.mdb) from Create New Data Source window and press Finish button as shown in diagram:



- After that, ODBC Microsoft Access Setup window would be opened as shown in following diagram:



- Enter the Data Source Name *personDSN* and select the database by pressing Select button. The browsing window would be opened, select the desired folder that contains the database (The database .mdb file you have created in the first step) Press Ok button.

14.4 Basic Steps in Using JDBC

There are eight (8) basic steps that must be followed in order to successfully communicate with a database. Let's take a detail overview of all these one by one.

14.4.1 Import Required Package

- Import the package java.sql.* that contains useful classes and interfaces to access & work with database.

14.4.2 Load Driver

```
import java.sql.*;
```

- Need to load suitable driver for underlying database.

- Different drivers & types for different databases are available.
- For MS Access, load following driver available with j2se.
- Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
- For Oracle, load the following driver. You have to download it explicitly.
- Class.forName("oracle.jdbc.driver.OracleDriver");

14.4.3 Define Connection URL

- To get a connection, we need to specify the URL of a database (Actually we need to specify the address of the database which is in the form of URL)
- As we are using Microsoft Access database and we have loaded a JDBC-ODBC driver. Using JDBC-ODBC driver requires a DSN which we have created earlier and named it personDSN. So the URL of the database will be

String conURL = "jdbc:odbc:personDSN";

14.4.4 Establish Connection With DataBase

- Use DriverManager to get the connection object.
- The URL of the database is passed to the getConnection method.
- **Connection con = DriverManager.getConnection(conURL);**
- If DataBase requires username & password, you can use the overloaded version of getConnection method as shown below:

```
String usr = "umair";
String pwd = "vu";
Connection con = null;
con = DriverManager.getConnection(conURL, usr, pwd);
```

14.4.5 Create Statement

- A Statement object is obtained from a Connection object.

Statement stmt = con.createStatement( );

- Once you have a statement, you can use it for various kinds of SQL queries.

14.4.6 Execute a Query

- The next step is to pass the SQL statements & to execute them.
- Two methods are generally used for executing SQL queries. These are:

© Copyright Virtual University of Pakistan

- **executeQuery(sql) method**
  - Used for SQL SELECT queries.
  - Returns the ResultSET object that contains the results of the query and can be used to access the query results.

```
String sql = "SELECT * from sometable";
ResultSet rs = stmt.executeQuery(sql);
```

- **executeUpdate(sql) method**
  - This method is used for executing an update statement like INSERT, UPDATE or DELETE
  - Returns an Integer value representing the number of rows updated

```
String sql = "INSERT INTO tablename " +
                "(columnNames) Values (values)" ;
int count = stmt.executeUpdate(sql);
```

14.4.7 Process Results of the Query

- The ResultSet provides various getXXX methods that takes a column index or name and returns the data
- The ResultSet maintains the data in the form tables (rows & columns)
- First row has index 1, not 0.
- The next method of ResultSet returns true or false depending upon whether the next row is available (exist) or not and moves the cursor
- Always remember to call next()method at-least once
- To retrieve the data of the column of the current row you need to use the various getters provided by the ResultSet.
- For example, the following code snippet will iterate over the whole ResultSetand illustrates the usage of getters methods

```
while ( rs.next() ){
String name = rs.getString("columnName");  //by using column name
String name = rs.getString(1); // or by using column index }
```

14.4.8 Close the Connection

- An opening connection is expensive, postpone this step if additional database operations are expected

```
con.close();
```

Example Code 14.1: Retrieving Data from ResultSet

The JdbcEx.java demonstrates the usage of all above explained steps. In this code example, we connect with the *PersonInfo* database, the one we have created earlier, and then execute the simple SQL SELECT query on *Person* table, and then process the query results.

```
// File JdbcEx.java
//Step 1: Import package
import java.sql.*;
public class JdbcEx {
public static void main (String args[ ]) {
try {
//Step 2: load driver
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
//Step 3: define the connection URL
String url = "jdbc:odbc:personDSN";
//Step 4: establish the connection
Connection con = DriverManager.getConnection(url);
//Step 5: create Statement
Statement st = con.createStatement();
//Step 6: preapare & execute the query
String sql = "SELECT * FROM Person";
ResultSet rs = st.executeQuery(sql);
//Step 7: process the results
while(rs.next()){
// The row name is "name" in database "PersonInfo,
// hence specified in the getString() method.
String name = rs.getString("name");
String add      = rs.getString("address");
String pNum = rs.getString("phoneNum");
System.out.println(name + " " + add + " " + pNum);
}
//Step 8: close the connection
con.close();
}catch(Exception sqlEx){
System.out.println(sqlEx);
}
} // end main
} // end class
```

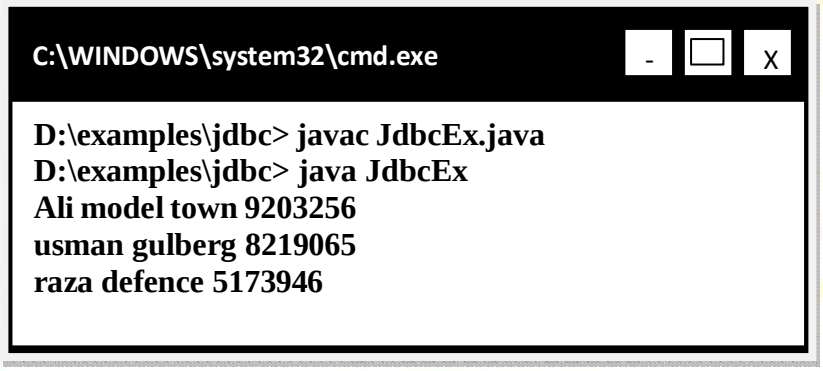
The important thing you must notice that we have put all code inside tryblock and then handle (in the above example, only printing the name of the exception raised) exception inside catch block.

© Copyright Virtual University of Pakistan

Why? Because we are dealing with an external resource (database). If you can recall all IO related operations involving external resources in java throw exceptions. These exceptions are checked exceptions and we must need to handle these exceptions.

Compile & Execute

Since the *Person* table contains only three records, so the following output would be produced on executing the above program.



14.5 References:

- Java - A Lab Course by Umair Javed
- Java tutorial by Sun: http://java.sun.com/docs/books/turorial
- Beginning Java2 by Ivor Hortan

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 15: More On JDBC

In the previous handout, we have discussed how to execute SQL statements. In this handout, we'll learn how to execute DML (insert, update, delete) statements as well some useful methods provided by the JDBC API.

Before jumping on to example, let's take a brief overview of executeUpdate() method that is used for executing DML statements.

15.1 Useful Statement Methods:

15.1.1 executeUpdate( )

- Used to execute for INSERT, UPDATE, or DELETE SQL statements.
- This method returns the number of rows that were affected in the database.
- Also supports DDL (Data Definition Language) statements CREATE TABLE, DROP TABLE, and ALERT TABLE etc.
- For example,

```
int num = stmt.executeUpdate("DELETE from Person WHERE id
= 2" );
```

Example Code 15.1 : Executing SQL DML Statements

This program will take two command line arguments that are used to update records in the database. executeUpdate( ) method will be used to achieve the purpose stated above.

```
// File JdbcDmlEx.java
//step 1:
import package import java.sql.*;
public class JdbcDmlEx {
public static void main (String args[ ]) {
try {
//Step 2: load driver
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

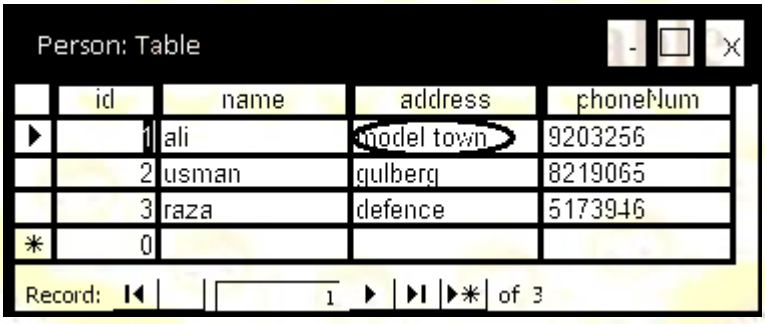
//Step 3: define the connection URL
```

```
String url = "jdbc:odbc:personDSN";
//Step 4: establish the connection
Connection con = DriverManager.getConnection(url);
//Step 5: create Statement
Statement st = con.createStatement();
// assigning first command line argument value
String addVar  = args[0];
// assigning second command line argument value

String nameVar = args[1];
// preparing query - nameVar & addVar strings are embedded
// into query within '" + string + "'
String sql =     "UPDATE Person SET address = '"+addVar+"'" +" WHERE
name = '"+nameVar+"' ";
// executing query
int num = st.executeUpdate(sql);
// Step 7: process the results of the query
// printing number of records affected
System.out.println(num + " records updated");
//Step 8: close the connection
con.close();
}catch(Exception sqlEx){
System.out.println(sqlEx);
} // end main
} // end class
```

Compile & Execute

The *Person* table is shown in the following diagram before execution of the program. We want to update first row i.e. address of the person ali.



The next diagram shows how we have executed our program. We passed it two arguments. The first one is the address (defence) and later one is the name (ali) of the person against whom we want to update the address value.

© Copyright Virtual University of Pakistan

```
C:\WINDOWS\system32\cmd.exe
D:\examples\jdbc> javac JdbcEx.java
D:\examples\jdbc> java JdbcEx defence ali
1 record updated
```

The Person table is shown in the following diagram after the execution of the program. Notice that address of the ali is now changed to defence.

Person: Table

	id	name	address	phoneNum
▶	1	ali	defence	9203256
	2	usman	gulberg	8219065
	3	raza	defence	5173946
*	0			

Record: 1 of 3

Note

When we execute DML statements (insert, update, delete) we have to commit it in the database explicitly to make the changes permanent or otherwise we can rollback the previously executed statements.

But in the above code, you have never seen such a statement. This is due to the fact that java will implicitly commit the changes. However, we can change this java behavior to manual commit. We will cover these in some later handout.

15.1.2 getMaxRows / setMaxRows(int)

- Used for determines the number of rows a ResultSetmay contain
- By default, the number of rows are unlimited (return value is 0), or by using setMaxRows(int), the number of rows can be specified.

15.1.3 getQueryTimeOut / setQueryTimeOut (int)

- Retrieves the number of seconds the driver will wait for a Statement object to execute.
- The current query time out limit in seconds, zero means there is no limit

© Copyright Virtual University of Pakistan

- If the limit is exceeded, a SQLException is thrown

15.2 Different Types of Statements

- As we have discussed in the previous handout that through Statement objects, SQL queries are sent to the databases.
- Three types of Statementobjects are available. These are:

15.2.1 Statement

- The Statementobjects are used for executing simple SQL statements.
- We have already seen its usage in the code examples.

15.2.2 PreparedStatement

- The PreparedStatement are used for executing precompiled SQL statements and passing in different parameters to it.
- We will talk about it in detail shortly.

15.2.3 CallableStatement

- These are used for executing stored procedures.
- We are not covering this topic; See the Java tutorial on it if you are interested in learning it.

15.2.4 Prepared Statements

- What if we want to execute same query multiple times by only changing parameters.
- PreparedStatement object differs from Statement object as that it is used to create a statement in standard form that is sent to database for compilation, before actually being used.
- Each time you use it, you simply replace some of the marked parameters (?) using some setter methods.
- We can create PreparedStatement object by using prepareStatement method of the connection class. The SQL query is passed to this method as an argument as shown below.

```
PreparedStatement pStmt = con.prepareStatement (
                                "UPDATE tableName SET columnName =
                                ? " + "WHERE columnName = ? " );
```

- Notices that we used marked parameters (?) in query. We will replace them later on

by using various setter methods.

- If we want to replace *first* **?** with Stringvalue, we use setStringmethod and to replace *second* **?** with int value, we use setInt method. This is shown in the following code snippet:

```
pStmt.setString (1 , stringValue);
pStmt.setInt (2 , intValue)
```

Note: The first market parameter has index 1.

- Next, we can call executeUpdate (for INSERT, UPDATE or DELETE queries) or executeQuery (for simple SELECT query) method**.**

```
pStmt.executeUpdate();
```

Modify Example Code: Executing SQL DML using Prepared

Statements

This example code is modification to the last example code (JdbcDmlEx.java).The modifications are highlighted as bold face.

```
// File JdbcDmlEx.java
//step 1: import package
import java.sql.*;
public class JdbcDmlEx {
public static void main (String args[ ]) {
try {

//Step 2: load driver
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

//Step 3: define the connection URL
String url = "jdbc:odbc:personDSN";

//Step 4: establish the connection
Connection con = DriverManager.getConnection(url, "", "");

// make query and place ? where values are to
//be inserted later
String sql =     "UPDATE Person SET address = ? " +
" WHERE name = ? ";
// creating statement using Connection object and passing
// sql statement as parameter
PreparedStatement pStmt = con.prepareStatement(sql);
// assigning first command line argument value
String addVar = args[0];
```

```
// assigning second command line argument value
String nameVar = args[1];

// setting first marked parameter (?) by using setString()
// method to address.
pStmt.setString(1 , addVar);

// setting second marked parameter(?) by using setString()
// method to name
pStmt.setString(2 , nameVar);

// suppose address is "defence" & name is "ali"
// by setting both marked parameters, the query will look
// like:
//        sql = "UPDATE Person SET address = "defence"
//        WHERE name = "ali" "


// executing update statemnt
int num = pStmt.executeUpdate();

// Step 7: process the results of the query
// printing number of records affected
System.out.println(num + " records updated");

//Step 8: close the connection
con.close();
}catch(Exception sqlEx){
System.out.println(sqlEx);
}
} // end main
} // end class
```

Compile & Execute

Execute this code in a similar way as we showed you in execution of the last program. Don't forget to pass the address & name values as the command line arguments.

15.3 References:

- Entire material for this handout is taken from the book **JAVA A Lab Course** by **Umair Javed**. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

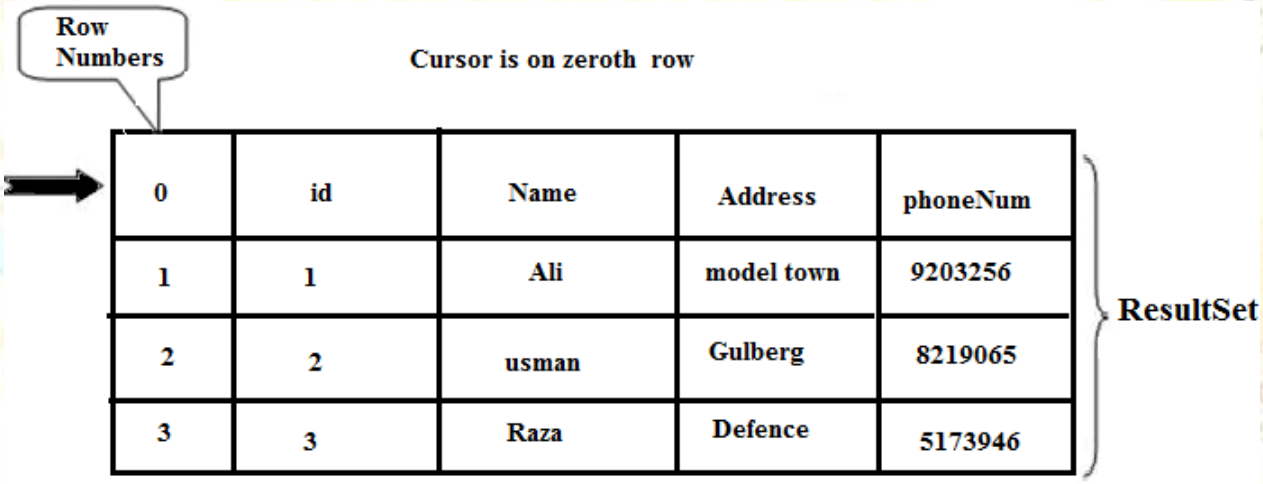
© Copyright Virtual University of Pakistan

Lecture 16: Result Set

This handout will familiarize you with another technique of inserting, updating & deleting rows. Before moving on, first we look at ResultSet.

16.1 ResultSet

- A ResultSet contains the results of the SQL query
  - Represented by a table with rows and columns
  - Maintains a cursor pointing to its current row of data.
  - Initially the cursor positioned before the row (0).
  - First row has index 1



16.1.1 Default ResultSet

- A default ResultSet object is not updatable and has a cursor that moves *forward only*.
- You can iterate over through it only *once* and only from the *first row to last row.*
- Until now, we have worked & used it in various examples.
- For a quick overview, here how we create a *default ResultSet* object. String sql = "SELECT * FROM Person"; PreparedStatement pStmt = con.prepareStatement(sql); **ResultSet rs** = pStmt.executeQuery( );

16.1.2 Useful ResultSet's Methods

Following methods are used often to work with *default* ResultSetobject. We already seen and used some of them in code examples.

16.1.2.1 next( )

- Attempts to move to the next row in the ResultSet, if available
- The next() method returns true or false depending upon whether the next row is available (exist) or not.
- Before retrieving any data from ResultSet, always remember to call next()at least once because initially cursor is positioned before first row.

16.1.2.2 getters

- To retrieve the data of the column of the current row you need to use the various getters provided by the ResultSet
- These getters return the value from the column by specifying column name or column index.
- For example, if the column name is "Name" and this column has index 3 in the ResultSet object, then we can retrieve the values by using one of the following methods:
- String name = rs.getString("Name");
- String name = rs.getString(3);
- These getter methods are also available for other types like getInt( ), getDouble( ) etc. Consult the Java API documentation for more references.

Note: Remember that first column has an index 1, NOT zero (0).

16.1.2.3 close( )

- Used to release the JDBC and database resources
- The ResultSet is implicitly closed when the associated Statement object executes a new query or closed by method call.

16.1.2.4 Updatable and/or Scrollable ResultSet

- It is possible to produce ResultSet objects that are scrollable and/or updatable (since JDK 1.2)
- With the help of such ResultSet, it is possible to move forward as well as backward with in ResultSet object.
- Another advantage is, rows can be inserted, updated or deleted by using updatable ResultSet object.

16.1.2.5 Creating Updatable & Scrollable ResultSet

The following code fragment, illustrates how to make a ResultSet object that is scrollable and updatable.

© Copyright Virtual University of Pakistan

String sql = “SELECT * FROM Person”; PreparedStatement pStmt = con.prepareStatement(sql, **ResultSet.TYPE_SCROLL_INSENSITIVE**, **ResultSet.CONCUR_UPDATABLE**);
ResultSet rs = pStmt.executeQuery( );

Two constants have been used of ResultSet class for producing a ResultSet rs that is scrollable, will not show changes made by others and will be updatable

16.1.2.6 previous( )

- Moves the cursor to the previous row in the ResultSet object, if available
- Returns true if cursor is on a valid row, false it is off the result set.
- Throws exception if result type is TYPE_FORWARD_ONLY.

Example Code 16.1: Use of previous ( ), next( ) & various getters methods

The ResultSetEx.javashows the use of previous, next and getters methods. We are using the same Persontable of *PersonInfo* database, the one we had created earlier in this example and later on.

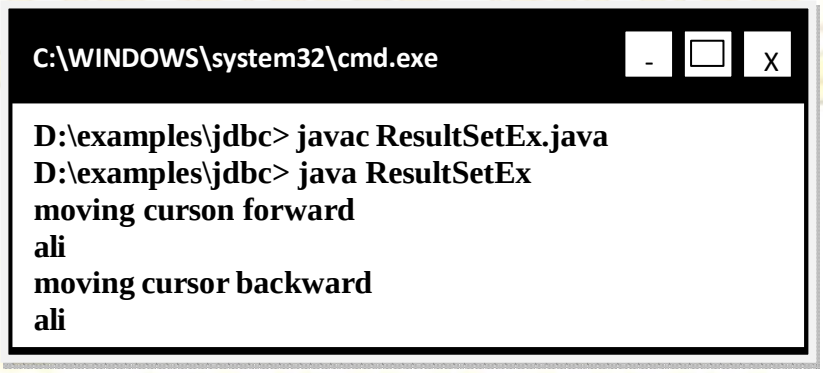
```
1. // File ResultSetEx.java
2. import java.sql.*;
3. public class ResultSetEx {
4. public static void main (String args[ ]) {
5. try {
6. //Step 2: load driver
7. Class.forName(“sun.jdbc.odbc.JdbcOdbcDriver”);
8. //Step 3: define the connection URL
9. String url = “jdbc:odbc:personDSN”;
10. //Step 4: establish the connection
11. Connection con = DriverManager.getConnection(url);
12.//Step 5: creating PrepareStatement by passing sql and
13.//ResultSet’s constants so that the ResultSet that will
14.//produce as a result of executing query will be
15.//scrollable & updatable
16.String sql = “SELECT * FROM Person”;
17.PreparedStatement pStmt = con.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);
20 //Step 6: execute the query
21 ResultSet rs = pStmt.executeQuery();
22 // moving cursor forward i.e. first row
23 rs.next( );
24 // printing column “name” value of current row (first)
```

© Copyright Virtual University of Pakistan

```
25 System.out.println("moving cursor forward");
26 String name = rs.getString("Name");
27 System.out.println(name);
28 // moving cursor forward i.e. on to second row
29 rs.next( );
   30 // moving cursor backward i.e to first row
31 rs.previous( );
32.// printing column "name" value of current row (first)
33.System.out.println("moving cursor forward");
34.name = rs.getString("Name");
35.System.out.println(name);
36.//Step 8: close the connection
37.con.close();
38.}catch(Exception sqlEx){
39.System.out.println(sqlEx);
40.}
41.} // end main
42.} // end class
```

Compile & Execute:

The sample output is given below:



16.1.2.7 absolute(int)

- Moves the cursor to the given row number in the ResultSet object.
- If given row number is positive, moves the cursor forward with respect to beginning of the result set.
- If the given row number is negative, the cursor moves to the absolute row position with respect to the end of the result set.
- For example, calling absolute(-1) positions the cursor on the last row; calling absolute(-2) moves the cursor to next-to-last row, and so on.
- Throws Exception if ResultSet type is TYPE_FORWARD_ONLY

© Copyright Virtual University of Pakistan

16.1.2.8 updaters (for primitives, String and Object)

- Used to update the column values in the current row or in insert row (discuss later)
- Do not update the underlying database
- Each update method is overloaded; one that takes column name while other takes column index. For example String updater are available as:

updateString(String columnName, String value)
updateString(String columnIndex, String value)

16.1.2.9 updateRow( )

- Updates the underlying database with new contents of the current row of this ResultSet object

Example Code 16.2: Updating values in existing rows

The following code example updates the *Name* column in the second row of the ResultSet object rs and then uses the method updateRow to update the *Person* table in database.

This code is the modification of the last one. Changes made are shown in bold face.

```
1. // File ResultSetEx.java
2. import java.sql.*;
3. public class ResultSetEx {
4. public static void main (String args[ ]) {
5. try {
6. //Step 2: load driver
7. Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
8. //Step 3: define the connection URL
9. String url = "jdbc:odbc:personDSN";
10.//Step 4: establish the connection
11.Connection con = DriverManager.getConnection(url);
12.//Step 5: create PrepareStatement by passing sql and
13.// ResultSet appropriate fields
14.String sql =      "SELECT * FROM Person";
15.PreparedStatement pStmt = con.prepareStatement(sql,
16.ResultSet.TYPE_SCROLL_INSENSITIVE,
17.ResultSet.CONCUR_UPDATABLE);
18.//Step 6: execute the query
19.ResultSet rs = pStmt.executeQuery();
20.// moving cursor to second row
21.rs.absolute(2);
```

```
22.// update address column of 2nd row in rs
23.rs.updateString("Address", "model town");
24.// update the row in database
25.rs.updateRow( );
26.//Step 8: close the connection
27.con.close();
28.}catch(Exception sqlEx){
29.System.out.println(sqlEx);
30.}
31.} // end main
32.} // end class
```

Compile & Execute

Given below are two states of *Person* table. Notice that address of 2nd row is updated.

Person table: Before execution

Person: Table

	id	name	address	phoneNum
	1	ali	model town	9203256
✎	2	usman	defence	8219065
	3	raza	defence	5173946
*	(Number)			

Record: 2 of 3

Person table: After execution

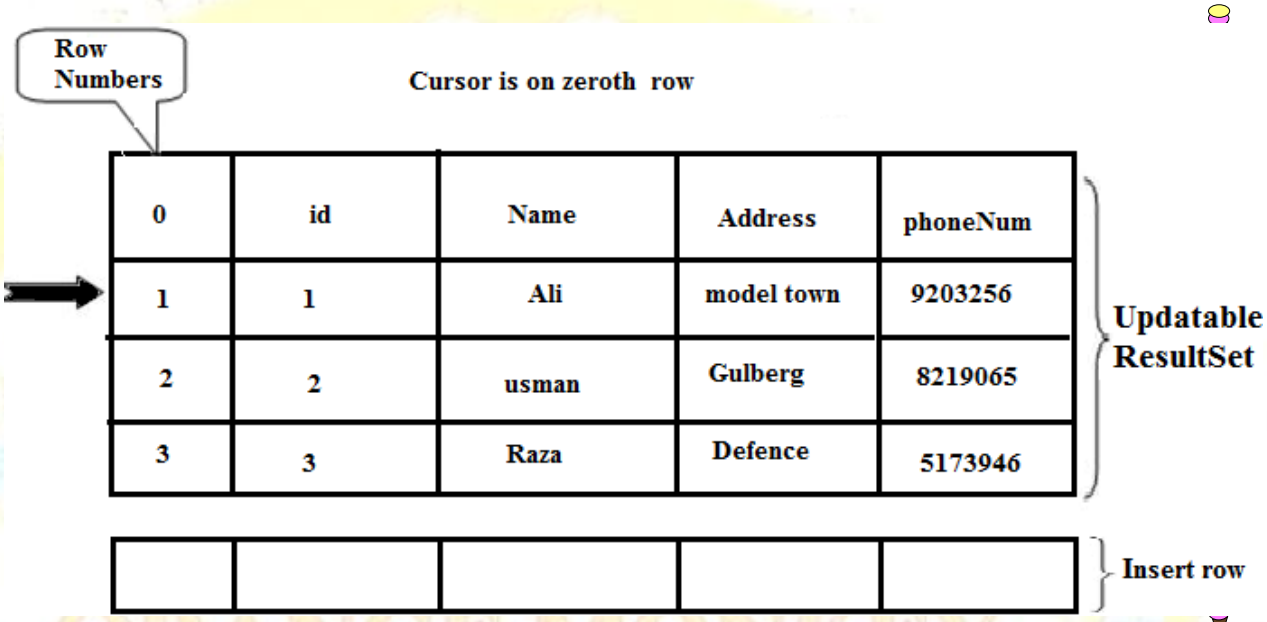
Person: Table

	id	name	address	phoneNum
	1	ali	model town	9203256
	2	usman	model town	8219065
▶	3	raza	defence	5173946
*	(Number)			

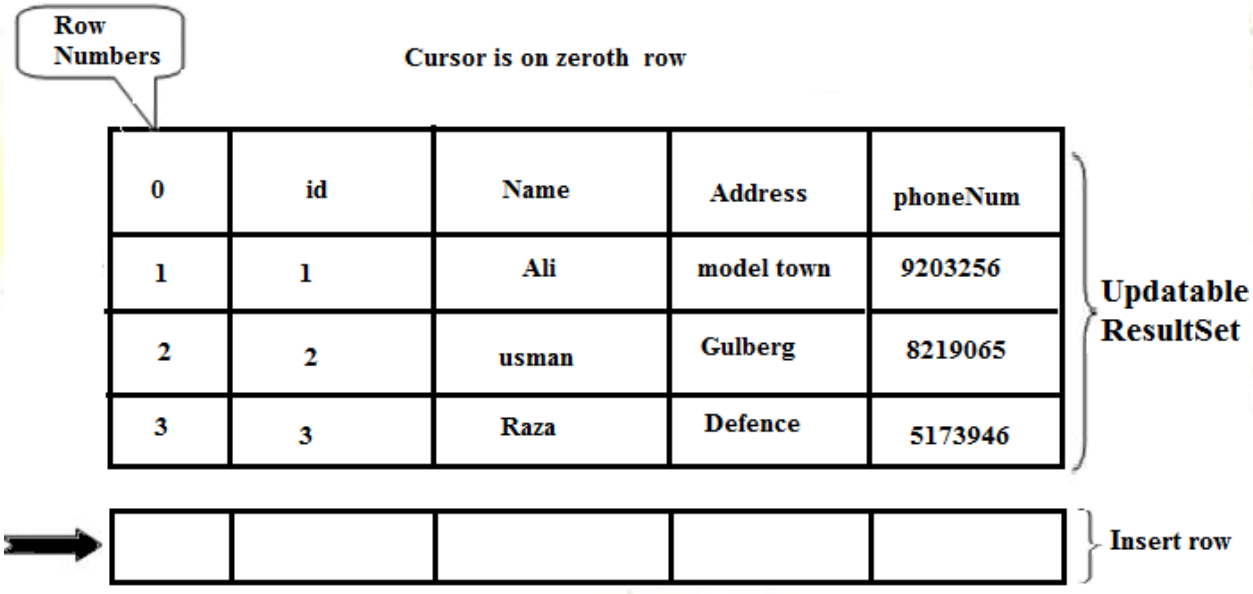
Record: 2 of 3

16.1.2.10 moveToInsertRow(int)

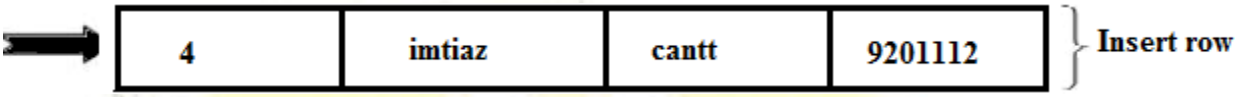
- An updatable resultset object has a special row associate with it i.e. insert row
- Insert row - a buffer, where a new row may be constructed by calling updater methods.
- Doesn't insert the row into a result set or into a database.
- For example, initially cursor is positioned on the first row as shown in the diagram:



- By calling moveToInsertRow( ), the cursor is moved to insert row as shown below:

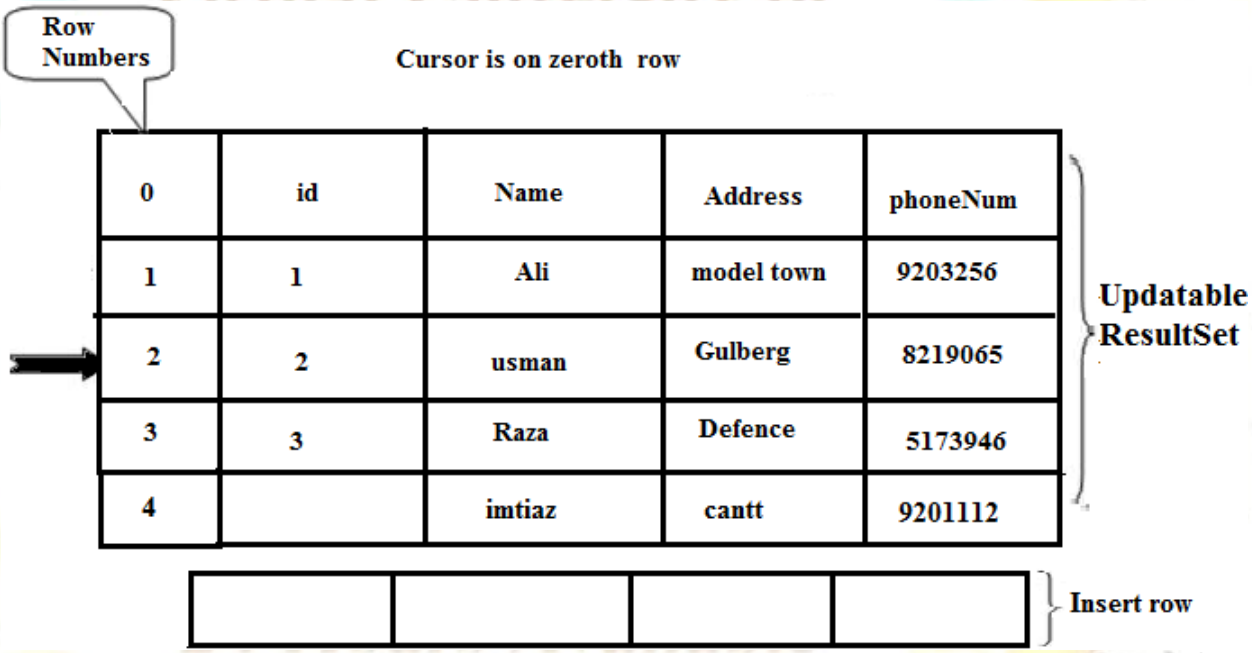


- Now, by calling various updaters, we can insert values into the columns of insert row as shown below.



16.1.2.11 insertRow( )

- Inserts the contents of the current row into this ResultSet object and into the database too.
- Moves the cursor back to the position where it was before calling moveToInsertRow()
- This is shown in the given below diagram



Note: The cursor must be on the insert row before calling this method or exception would be raised.

Example Code 16.3: Inserting new row

The following code example illustrates how to add/insert new row into the ResultSet as well into the database.

This code is the modification of the last one. Changes made are shown in bold face.

```
1. // File ResultSetEx.java
```

```
2. import java.sql.*;
3. public class ResultSetEx {
4. public static void main (String args[ ]) {
5. try {
6. //Step 2: load driver
7. Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

8. //Step 3: define the connection URL
9. String url = "jdbc:odbc:personDSN";

10.//Step 4: establish the connection
11.Connection con = DriverManager.getConnection(url);

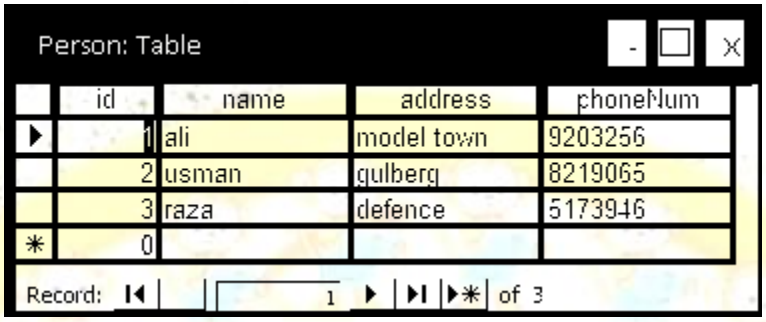
12.//Step 5: create PrepareStatement by passing sql and
13.// ResultSet appropriate fields
14.String sql = "SELECT * FROM Person";
15.PreparedStatement pStmt = con.prepareStatement(sql, ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);
18.//Step 6: execute the query

19.ResultSet rs = pStmt.executeQuery();
20.// moving cursor to insert row
21.rs.moveToInsertRow();
22.// updating values in insert row
23.rs.updateString( "Name"    ,        "imitiaz" );
24.rs.updateString( "Address" ,        "cantt" );
25.rs.updateString( "phoneNum" ,       "9201211" );
26.// inserting row in resultset & into database
27.rs.insertRow( );
28.//Step 8: close the connection
29.con.close();
30.}catch(Exception sqlEx){
31.System.out.println(sqlEx);
32.}
33.} // end main
34.} // end class
```

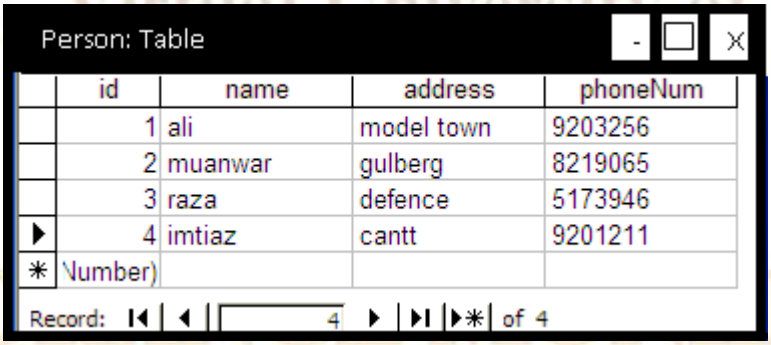
Compile & Execute

Given below are two states of *Person* table. Note that after executing program, a newly added row is present.

© Copyright Virtual University of Pakistan



Person table: Before execution



Person table: After execution

16.1.2.12 last( ) & first( )

- Moves the cursor to the last & first row of the ResultSet object respectively.
- Throws exception if the ResultSet is TYPE_FORWARD_ONLY

16.1.2.13 getRow( )

- Returns the current row number
- As mentioned earlier, the first row has index 1 and so on.

16.1.2.14 deleteRow( )

- Deletes the current row from this ResultSet object and from the underlying database.
- Throws exception if the cursor is on the insert row.

Example Code 16.4: Deleting existing row

The given below example code shows the usage of last( ), getRow( )and

deleteRow( )method.

This code is also the modification of the last one. Changes made are shown in bold face.

```
1. // File ResultSetEx.java
2. import java.sql.*;
3. public class ResultSetEx {
4. public static void main (String args[ ]) {
5.       try {

6.       //Step 2: load driver
7.       Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

8.       //Step 3: define the connection URL
9.       String url = "jdbc:odbc:personDSN";

10.      //Step 4: establish the connection
11.      Connection con = DriverManager.getConnection(url);

12.      //Step 5: create PrepareStatement by passing sql and

13.      //         ResultSet appropriate fields
14.      String sql =     "SELECT * FROM Person";

15.      PreparedStatement pStmt = con.prepareStatement(sql,

16.      ResultSet.TYPE_SCROLL_INSENSITIVE,
17.      ResultSet.CONCUR_UPDATABLE);

18.      //Step 6: execute the query
19.      ResultSet rs = pStmt.executeQuery();

20.      // moves to last row of the resultset
21.      rs.last();

22.      // retrieving the current row number
23.      int rNo = rs.getRow();

24.      System.out.println("current row number" + rNo);

25.      // delete current row from rs & db i.e. 4 because
26.      // previously we have called last() method
27.      rs.deleteRow( );
28.      //Step 8: close the connection
29.      con.close();
30.      }catch(Exception sqlEx){
31.      System.out.println(sqlEx);
```

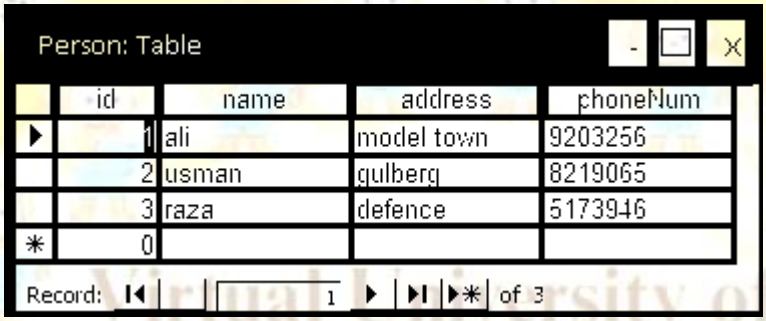
© Copyright Virtual University of Pakistan

```
32.     }

33.     } // end main
34.     } // end class
```

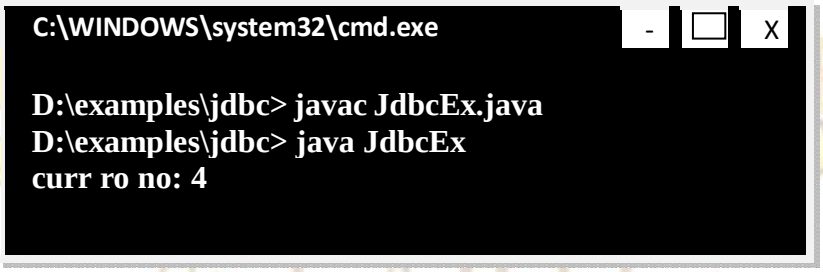
Compile & Execute

The first diagram shows the *Person* table before execution.



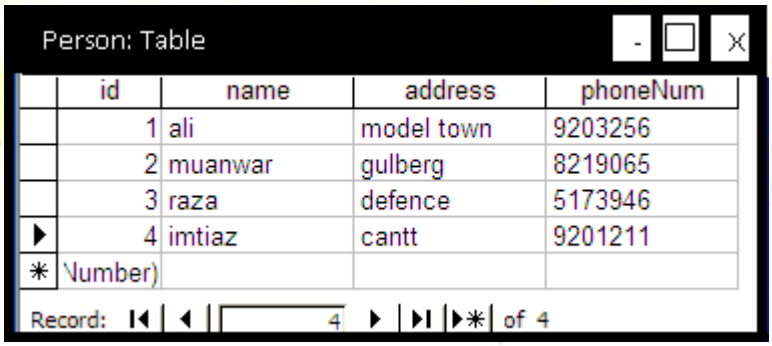
Person table: Before execution

Execution program from command prompt will result in displaying current row number on console. This can be confirmed from following diagram.



Executing Program from Command Prompt

After execution, the last row (4) is deleted from ResultSet as well as from database. The *Person* table is shown after execution



Person table: After execution

16.2 References:

- Entire material for this handout is taken from the book **JAVA A Lab Course** by **Umair Javed**. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 17: Meta Data

In simple terms, Meta Data is *data (information) about data*. The actual data has no meaning without existence of Meta data. To clarify this, let's look at an example. Given below are listed some numeric values

1000
2000
4000

What this information about? We cannot state accurately. These values might be representing some one's salaries, price, tax payable & utility bill etc. But if we specify Meta data about this data like shown below:

Salary
1000
2000
4000

Now, just casting a glance on these values, you can conclude that it's all about some ones salaries.

17.1 ResultSet Meta data

ResultSet Meta Data will help you in answering such questions

- How many columns are in the ResultSet?
- What is the name of given column?
- Are the column name case sensitive?
- What is the data type of a specific column?
- What is the maximum character size of a column?
- Can you search on a given column?

17.1.1 Creating ResultSetMetaData object

From a ResultSet (the return type of executeQuery()), derive a ResultSetMetaDataobject by calling getMetaData()method as shown in the given code snippet (here rsis a valid ResultSetobject):

```
ResultSetMetaData rsmd = rs.getMetaData();
```

Now, rsmd can be used to look up number, names & types of columns.

17.1.2 Useful ResultSetMetaData methods

17.1.2.1 getColumnCount ( )

- Returns the number of columns in the result set

17.1.2.2 getColumnDisplaySize (int)

- Returns the maximum width of the specified column in characters

17.1.2.3 getColumnName(int) / getColumnLabel (int)

- The getColumnName() method returns the database name of the column
- The getColumnLabel() method returns the suggested column label for printouts

17.1.2.4 getColumnType (int)

- Returns the SQL type for the column to compare against types in java.sql.Types

Example Code 17.1: Using ResultSetMetaData

The *MetaDataEx.java* will print the column names by using ResultSetMetaData object and column values on console. This is an excellent example of the scenario where we have no idea about the column names in advance.

Note: For this example code and for the coming ones, we are using the same database (*PersonInfo*) the one we created earlier and repeatedly used. Changes are shown in bold face

```
44. // File MetaDataEx.java
45. import java.sql.*;
46. public class MetaDataEx {
47. public static void main (String args[ ]) {
```

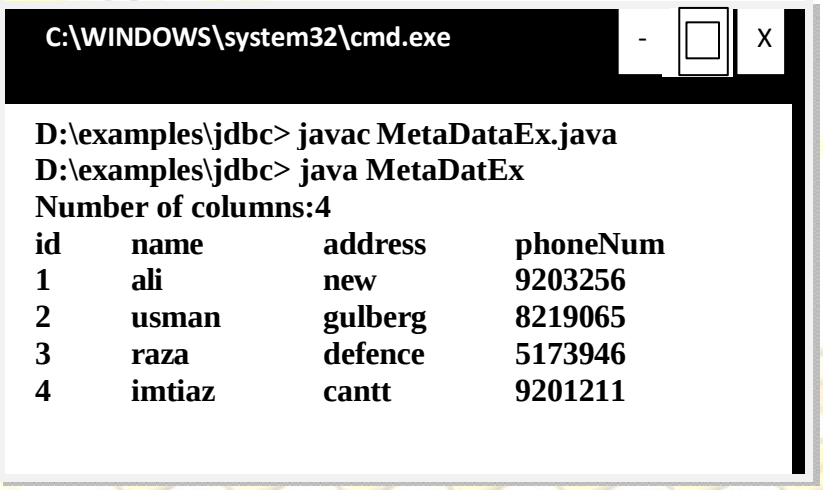
```
48. try {
49. //Step 2: load driver
50. Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
51. //Step 3: define the connection URL
52. String url = "jdbc:odbc:personDSN";
53. //Step 4: establish the connection
54. Connection con = null;
55. con = DriverManager.getConnection(url, "", "");
56. //Step 5: create PrepareStatement by passing sql and
57. //          ResultSet appropriate fields
58. String sql =      "SELECT * FROM Person";
59. PreparedStatement pStmt = con.prepareStatement(sql,
60. ResultSet.TYPE_SCROLL_INSENSITIVE,
61. ResultSet.CONCUR_UPDATABLE);
62. //Step 6: execute the query
63. ResultSet rs = pStmt.executeQuery();
64. // get ResultSetMetaData object from rs
65. ResultSetMetaData rsmd = rs.getMetaData( );
66. // printing no. of column contained by rs
67. int numColumns = rsmd.getColumnCount();
68. System.out.println("Number of Columns:" + numColumns);
69. // printing all column names by using for loop
70. String cName;
71. for(int i=1; i<= numColumns; i++) {
72. cName = rsmd.getColumnName(i);
73. System.out.println(cName);
74. System.out.println("\t");
75. }
76. // changing line or printing an empty string
77. System.out.println(" ");
78. // printing all values of ResultSet by iterating over it
79. String id, name, add, ph;
80. while( rs.next() )
81. {
82. id= rs.getString(1);
83. name = rs.getString(2);
84. add= rs.getString(3);
85. ph= rs.getString(4);
86. System.out.println(id);
87. System.out.println("\t");
88. System.out.println(name);
89. System.out.println("\t");
90. System.out.println(add);
91. System.out.println("\t");
92. System.out.println(ph);
93. System.out.println(" ");
```

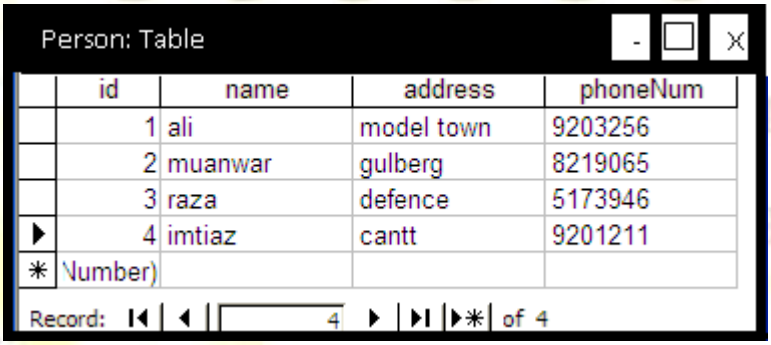
Virtual University of Pakistan ORANGE MONKEY Study & Helping 24 Hours Available

```
94. }
95. //Step 8: close the connection
96. con.close();
97. }catch(Exception sqlEx){
98. System.out.println(sqlEx); }
99. } // end main
100.} // end class
```

Compile & Execute:

The database contains the following values at the time of execution of this program. The database and the output are shown below:





7.2 DatabaseMetaData

Database Meta Data will help you in answering such questions

- What SQL types are supported by DBMS to create table?
- What is the name of a database product?
- What is the version number of this database product?
- What is the name of the JDBC driver that is used?
- Is the database in a read-only mode?

17.2.1 Creating DatabaseMetaData object

From a Connection object, a DatabaseMetaData object can be derived. The following code snippet demonstrates how to get DataBaseMetaDataobject.

```
Connectioncon= DriverManager.getConnection(url, usr, pwd);
DatabaseMetaData dbMetaData = con.getMeataData();
```

Now, you can use the dbMetaData to gain information about the database.

17.2.2 Useful ResultSetMetaData methods

17.2.2.1 getDatabaseProductName( )

- Returns the name of the database's product name

17.2.2.2 getDatabaseProductVersion( )

- Returns the version number of this database product

17.2.2.3 getDriverName( )

- Returns the name of the JDBC driver used to established the connection

17.2.2.4 isReadOnly( )

- Retrieves whether this database is in read-only mode
- Returns true if so, false otherwise

Example Code 17.2: using DatabaseMetaData

This code is modification of the example code 17.1. Changes made are shown in bold face.

```
102. // File MetaDataEx.java
103. import java.sql.*;
104. public class MetaDataEx {
105.         public static void main (String args[ ]) {
106.         try {
107.         //Step 2: load driver
```

```
108.        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
109.        //Step 3: define the connection URL
110.        String url = "jdbc:odbc:personDSN";
111.        //Step 4: establish the connection
112.        Connection con = null;
113.        con = DriverManager.getConnection(url, "", "");
114.        // getting DatabaseMetaDat object
115.        DatabaseMetaData dbMetaData = con.getMetaData();
116.        // printing database product name
117.        String pName = dbMetaData.getDatabaseProductName();
118.        System.out.println("DataBase: " + pName);
119.        // printing database product version
120.        String pVer = dbMetaData.getDatabaseProductVersion();
121.        System.out.println("Version: " + pVer);
122.        // printing driver name used to establish connection &
123.        // to retrieve data
124.        String dName = dbMetaData.getDriverName();
125.        System.out.println("Driver: " + dName);
126.        // printing whether database is read-only or not
127.        boolean rOnly = dbMetaData.isReadOnly();
128.        System.out.println("Read-Only: " + rOnly);
129.        // you can create & execute statements and can
130.        // process results over here if needed
131.        //Step 8: close the connection
132.        con.close();
133.        }catch(Exception sqlEx){
134.        System.out.println(sqlEx);
135.        }
136.        } // end main
137.} // end class
```

Compile & Execute

On executing the above program, the following output will produce:

```
C:\WINDOWS\system32\cmd.exe
D:\examples\jdbc> javac MetaDataEx.java
D:\examples\jdbc> java MetaDataEx
Database:  ACCESS
Version:    04.00.0000
Driver:     JDBC-ODBC Bridge   (odbcjt32.dll)
Read-only: false
```

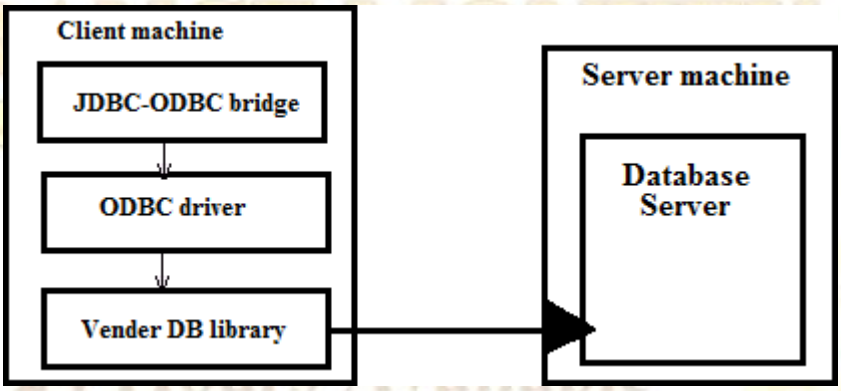
17.3 JDBC Driver Types

- JDBC Driver Types are divided into four types or levels.
- Each type defines a JDBC driver implementation with increasingly higher level of platform independence, performance, deployment and administration.
- The four types are:
  - Type - 1: JDBC - ODBC Bridge
  - Type 2: Native - API/partly Java driver
  - Type 3: Net - protocol/all-Java driver
  - Type 4: Native - protocol/all-Java driver

Now, let's look at each type in more detail

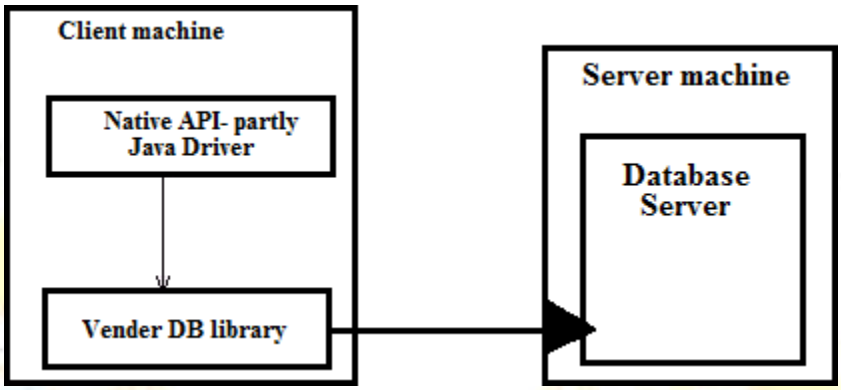
17.3.1 Type - 1: JDBC - ODBC Bridge

- Translates all JDBC calls into ODBC (Open Database Connectivity) calls and send them to the ODBC Driver
- Generally used for Microsoft database.
- Performance is degraded



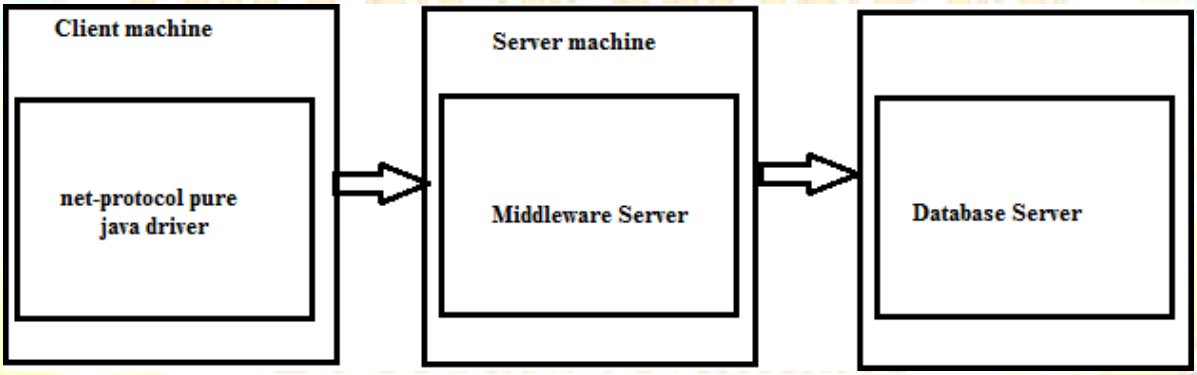
17.3.2 Type - 2: Native - API/partly Java driver

- Converts JDBC calls into database-specific calls such as SQL Server, Informix, Oracle or Sybase.
- Partly-Java drivers communicate with database-specific API (which may be in C/C++) using the Java Native Interface.
- Significantly better Performance than the JDBC-ODBC bridge.



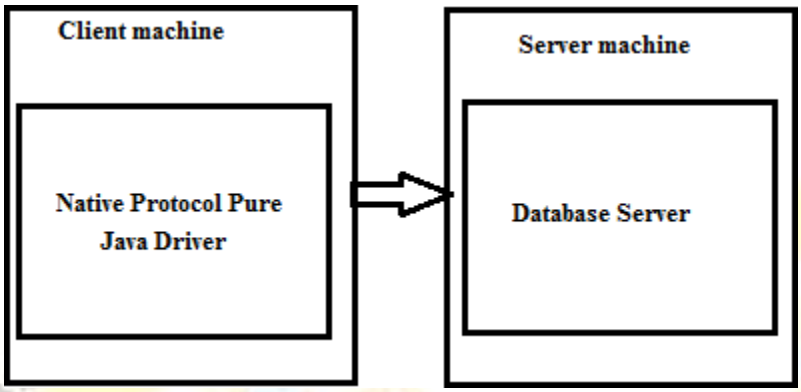
17.3.3 Type - 3: Net - protocol/all-Java driver4

- Follows a three-tiered approach whereby the JDBC database requests () are passed through the network to the middle-tier server
- Pure Java client to server drivers which send requests that are not database-specific to a server that translates them into a database-specific protocol.
- If the middle-tier server is written in java, it can use a type 1or type 2JDBC driver to do this



17.3.4 Type - 4: Native - protocol / all - java driver

- Converts JDBC calls into the vendor-specific DBMS protocol so that client application can communicate directly with the database server
- Completely implemented in Java to achieve platform independence and eliminate deployment issues.
- Performance is typically very good



17.4 Online Resources

- Sun's JDBC Site
  http://java.sun.com/products/jdbc/
- JDBC Tutorial
  http://java.sun.com/docs/books/tutorial/jdbc/
- List of available JDBC Drivers
  http://industry.java.sun.com/products/jdbc/drivers/
- RowSet Tutorial
  http://java.sun.com/developer/Books/JDBCTutorial/chapter5.html
- JDBC RowSets Implementation Tutorial

  http://java.sun.com/developer/onlineTraining/

  Database/**jdbcrowset**s.pdf

17.5 References:

- Java API documentation 5.0
- Java - A Lab Course by Umair Javed
- JDBC drivers in the wild
- http://www.javaworld.com/javaworld/jw-07-2000/jw-0707-jdbc_p.html

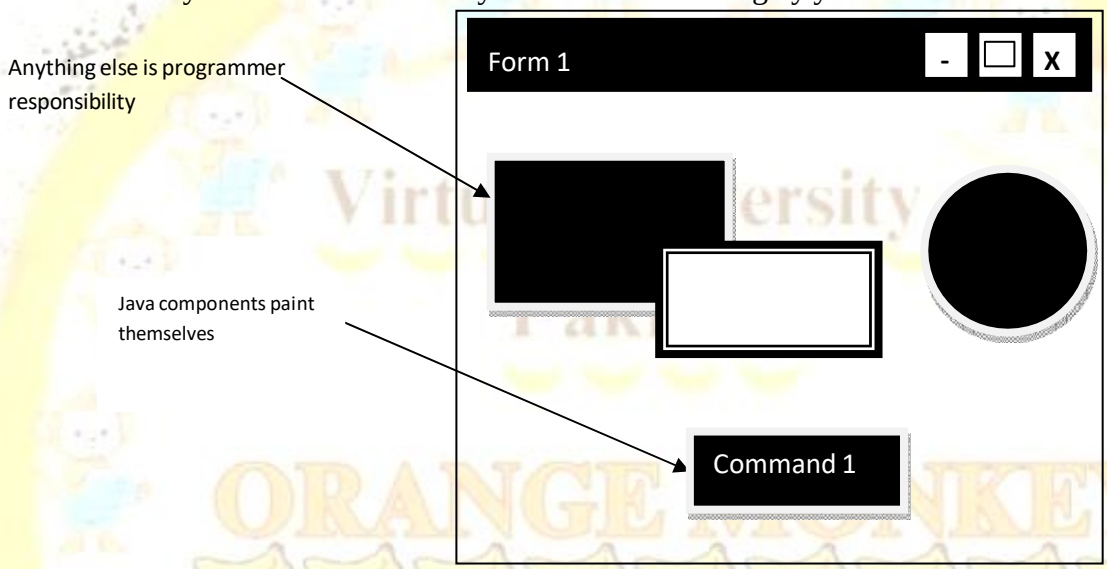
Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Lecture 18: Java Graphics

18.1 Painting

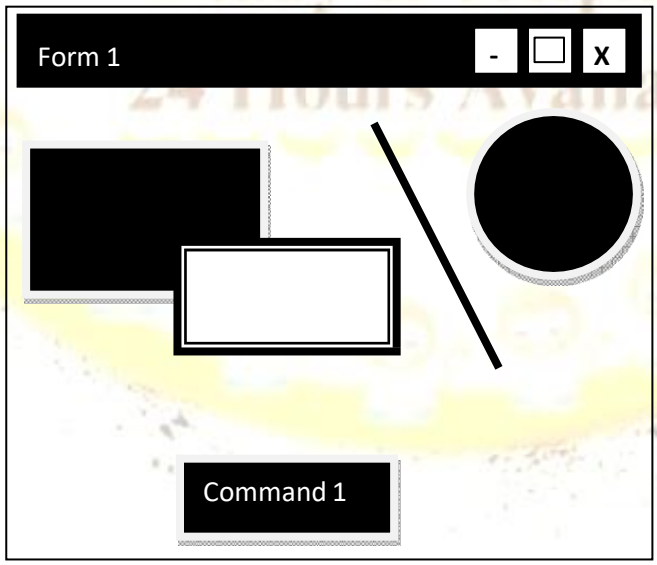
Window is like a painter's canvas. All window paints on the same surface. More importantly, windows don't remember what is under them. There is a need to repaint when portions are newly exposed.

Java components are also able to paint themselves. Most of time, painting is done automatically. However sometimes you need to do drawing by yourself.

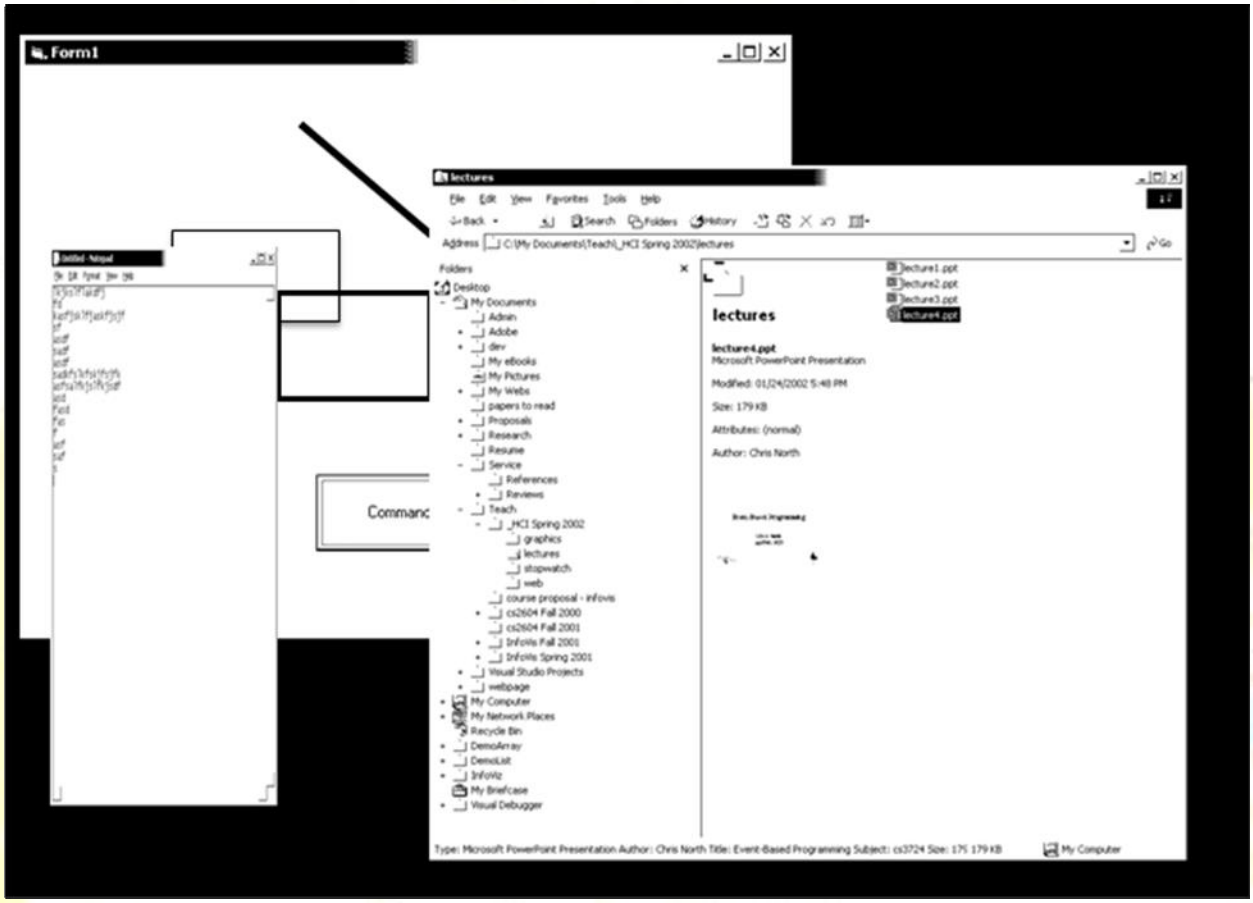


18.1.1 How painting works?

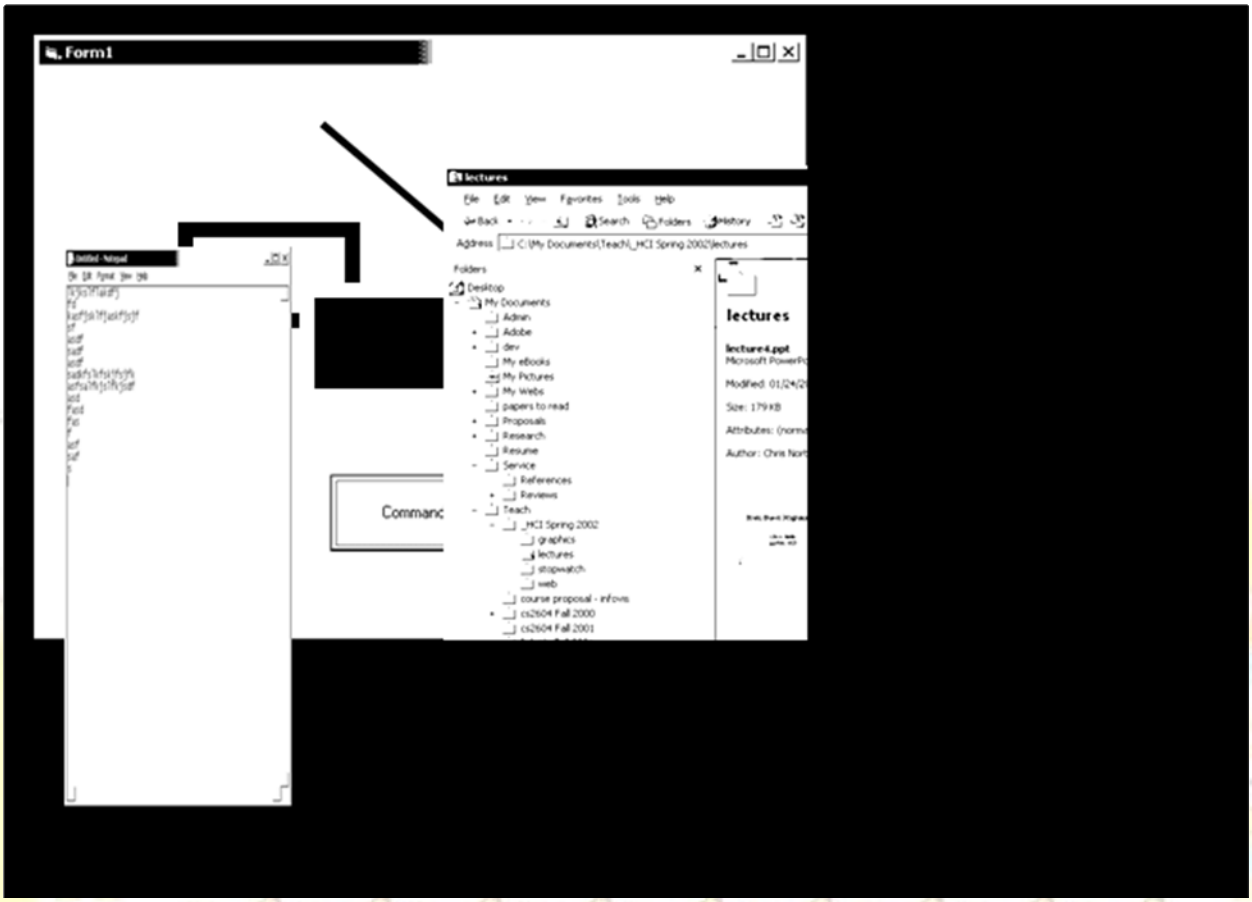
Let's take windows example. Consider the following diagram in which the blue area is representing the desktop. The one frame (myApp) is opened in front of desktop with some custom painting as shown below.



myApp consist of a JPanel. The JPanel contains a JButton. Two rectangles, a circle & a lines are also drawn on the JPanel.fter opening notepad and windows explorer window, diagram will look like this:

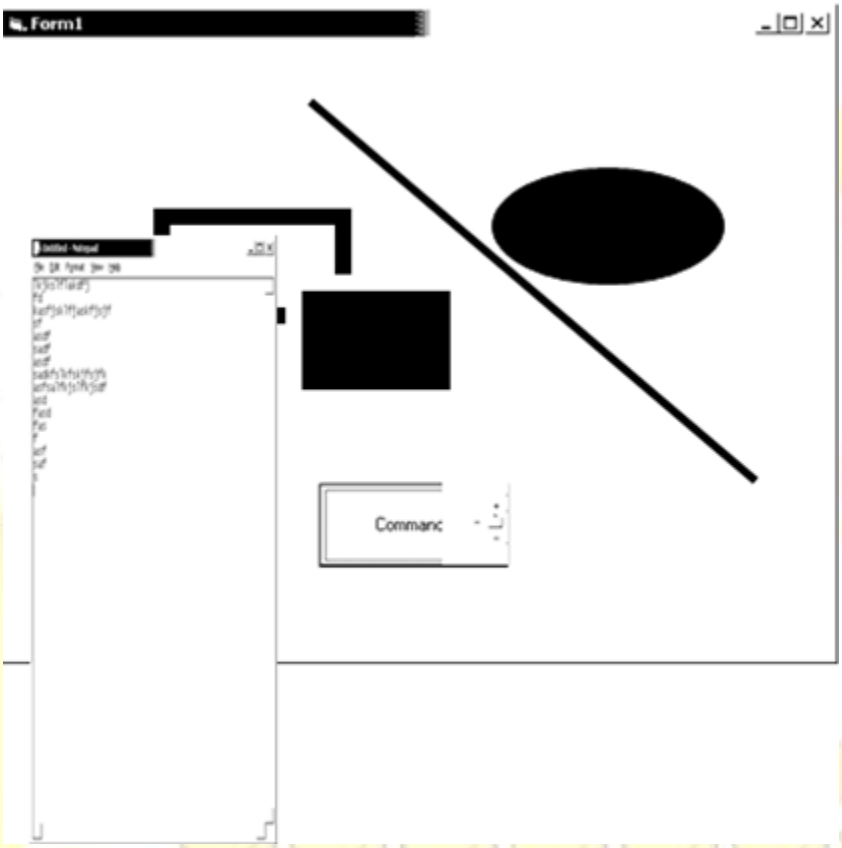


Lets shuts off the windows explorer, the repaint event is sent to desktop first and then to myApp. The figure shown below describes the situation after desktop repaint event get executed. Here you can clearly see that only desktop repaints itself and window explorer remaining part is still opened in front of myApp.

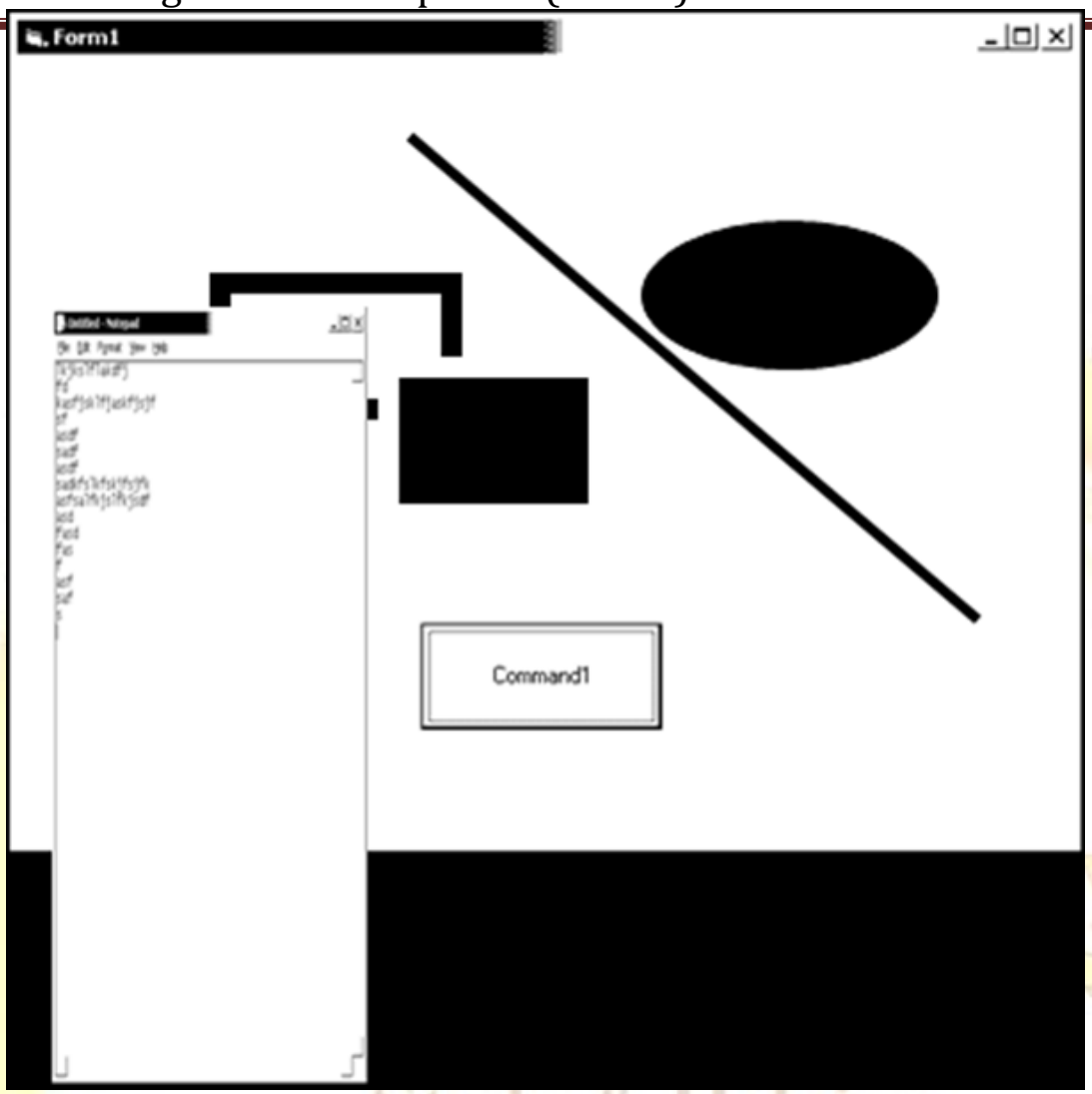


The following figure shows the situation when myApp's JPanel calls its repaint method. Notice that some portion of window explorer is still remains in front of JButton because yet not repaint event is sent to it.

Study & Preping
24 Hours Available



Next, JPanel forwards repaint event to JButton that causes the button to be displayed in its original form. This is all done automatically and we cannot feel this process cause of stunning speed of modern computers that performs all these steps in flash of eye.



18.1.2 Painting a Swing Component

Three methods are at the heart of painting a swing component like JPanel etc. For instance, paint()gets called when it's time to render -- then Swing further factors the paint() call into three separate methods, which are invoked in the following order:

protected void paintComponent(Graphics g) protected void
paintBorder(Graphics g) protected void
paintChildren(Graphics g)

Let’s look at these methods in order in which they get executed

18.1.2.1 paintComponent( )

- It is a main method for painting
- By default, it first paints the background
- After that, it performs custom painting (drawing circle, rectangles etc.)

18.1.2.2 paintBorder( )

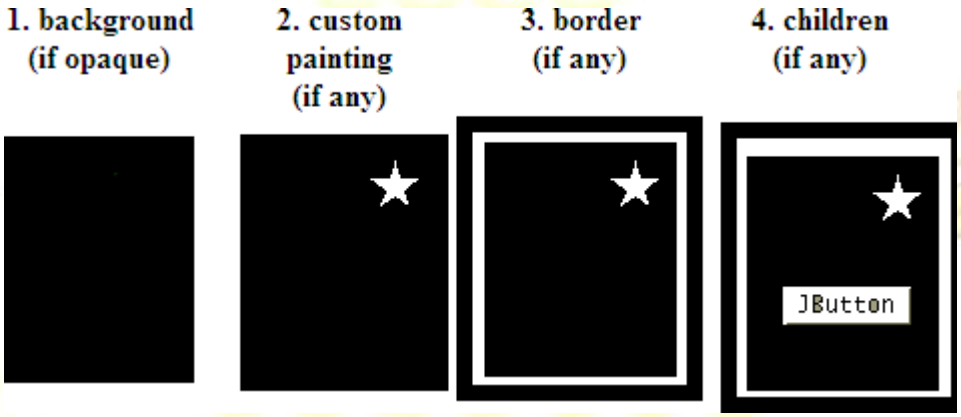
- Tells the components border (if any) to paint.
- It is suggested that you do not override or invoke this method

18.1.2.3 paintChildren( )

- Tells any components contained by this component to paint themselves
- It is suggested that you do not override or invoke this method too.

Example: Understanding methods calls

Consider the following figure



The figure above illustrates the order in which each component that inherits from JComponent paint itself.

Figure 1 to 2 -painting the background and performing custom painting is performed by the *paintComponent* method

In Figure 3 - *paintBorder* is get called
And finally in figure 4 - *paintChildre*n is called that causes the JButton to render itself.

Note: The important thing to note here is for JButton (since it is a JComponent), all these methods are also called in the same order.

Your Painting Strategy

- You must follow the three steps in order to perform painting.

Subclass JPanel

- class MyPanel *extends JPanel*
- Doing so MyPanel also becomes a JPanle due to inheritance

Override the paintComponent(Graphics g) method

- Inside method using graphics object, do whatever drawing you want to do

Install that JPanel inside a JFrame

- When frame becomes visible through the paintChildren() method your panel become visible
- To become visible your panel will call paintComponent() method which will do your custom drawing

Example Code 18.1:

Suppose we want to draw one circle & rectangle and a string "Hello World".




```
// importing required packages
import javax.swing.*;
import java.awt.*;
// extending class from JPanel
public class MyPanel extends JPanel {
// overriding paintComponent method
public void paintComponent(Graphics g){
// erasing behaviour - this will clear all the
```

© Copyright Virtual University of Pakistan

```
// previous painting
super.paintComponent(g);

// Down casting Graphics object to Graphics2D
Graphics2D g2 = (Graphics2D)g;
// drawing rectanle
g2.drawRect(20,20,20,20);
// changing the color to blue
g2.setColor(Color.blue);
// drawing filled oval with color i.e. blue
g2.fillOval(50,50,20,20);
// drawing string
g2.drawString("Hello World", 120, 50);
}// end paintComponent
} // end Test class
```

The Test class that contains the main method as well uses MyPanel (previously built) class is given below:

```
// importing required packages
import javax.swing.*;
import java.awt.*;
public class Test {
JFrame f;
// declaring Reference of MyPanel class
MyPanel p;
// parameter less constructor public Test(){
f = new JFrame();
Container c = f.getContentPane();
c.setLayout(new BorderLayout());
// instantiating reference
p = new MyPanel();
// adding MyPanel into container
c.add(p);
f.setSize(400,400);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
} // end constructor
// main method
```

© Copyright Virtual University of Pakistan

```
public static void main(String args[ ]){
Test t = new Test();
}
} // end Test class
```

Note: Here we have used only some methods (drawRect( ) & fillOval( ) etc.) of Graphics class. For a complete list, see the Java API documentation.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 19: How to Animate?

If we want to animate something like ball, moving from one place to another, we constantly need to call paintComponent( ) method and to draw the shape (ball etc.) at new place means at new coordinates.

Painting is managed by system, so calling paintComponent() directly is not recommended at all. Similarly calling paint( ) method is also not recommended. Why? Because such code may be invoked at times when it is not appropriate to paint -- for instance, before the component is visible or has access to a valid Graphicsobject.

Java gives us a solution in the form of *repaint( )* method. Whenever we need to repaint, we call this method that in fact makes a call to *paint( )* method at appropriate time.

19.1 Problem & Solution

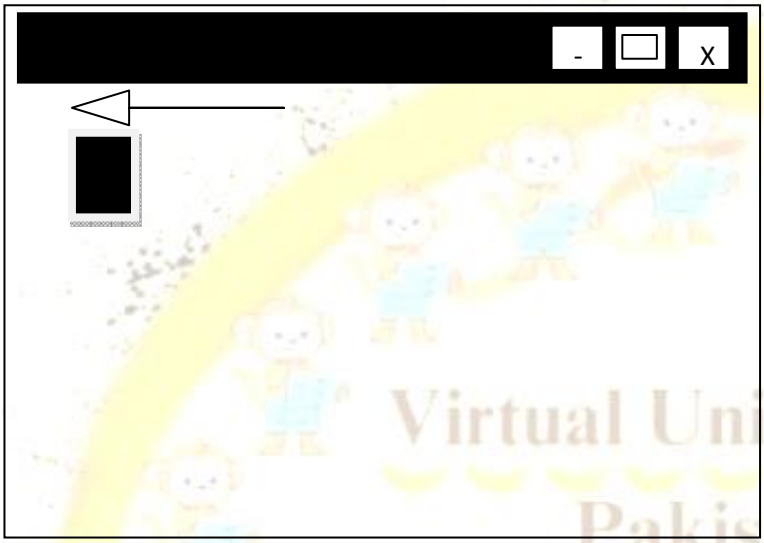
- What to do to move the shapes present in example code 18.1 (last example) when a mouse is dragged
- First time painting is what we already have done
- When a mouse is clicked find the co-ordinates of that place and paint Rectangle at that place by requesting, using repaint() call
- Here instead of Hard-coding the position of co-ordinates uses some variables. For example mx, my
    - In the last example code, we draw a rectangle by passing hard-coded values like 20

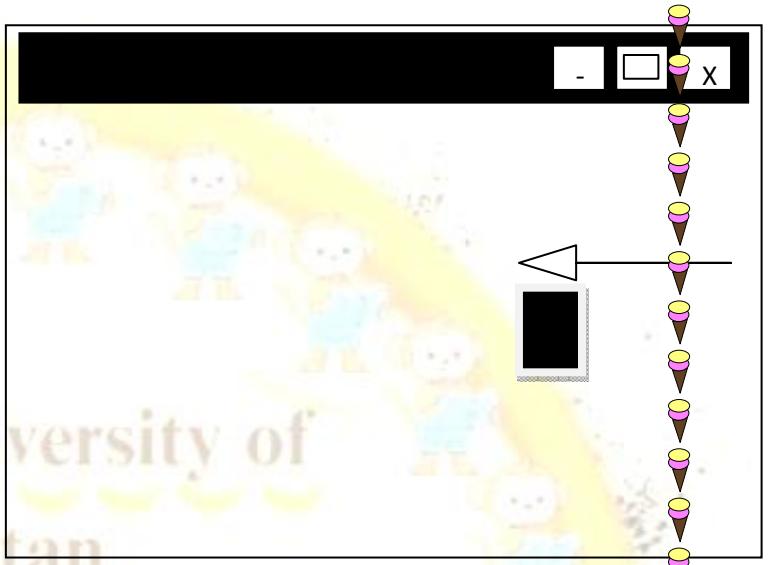
      g.drawRect(20,20,20,20);
    - Now, we'll use variables so that change in a variable value causes to display a rectangle at a new location

      g.drawRect(**mx**,**my**,20,20);
- Similarly, you have seen a tennis game (during lecture). Now, what to do code the paddle movement.
- In the coming up example. We are doing it using mouse, try it using mouse.

Example Code 19.1

The following outputs were produced when mouse is dragged from one location to anther





First we examine the MyPanel.java class that is drawing a filled rectangle.

```
import javax.swing.*;
import java.awt.*;
// extending class from JPanel
public class MyPanel extends JPanel {
// variables used to draw rectangles at different locations
int mX = 20;
int mY = 20;
// overriding paintComponent method
public void paintComponent(Graphics g){
// erasing behaviour - this will clear all the previous painting
super.paintComponent(g);
// Down casting Graphics object to Graphics2D
Graphics2D g2 = (Graphics2D)g;
// changing the color to blue
g2.setColor(Color.blue);
// drawing filled oval with color i.e. blue
// using instance variables
g2.fillRect(mX,mY,20,20);
}// end paintComponent
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

```
} // end MyPanel class
```

The Test class is given below. Additionally this class also contains the code for handling mouse events.

```
// importing required packages
 import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class Test {
JFrame f;
// declaring Reference of MyPanel class
MyPanel p;
// parameter less constructor
public Test(){
f = new JFrame();
Container c = f.getContentPane();
c.setLayout(new BorderLayout());
// instantiating reference
p = new MyPanel();
// adding MyPanel into container
c.add(p);
f.setSize(400,400);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// creating inner class object
Handler h = new Handler();
// registering MyPanel to handle events
p.addMouseMotionListener(h);
} // end constructor
// inner class used for handling events
public class Handler extends MouseMotionAdapter{
// capturing mouse dagged events
public void mouseDragged(MouseEvent me){
// getting the X-Position of mouse and assigning
// value to instance variable mX of MyPanel class
p.mX =me.getX();
// getting the Y-Position of mouse and assigning
// value to instance variable mX of MyPanel class
p.mY = me.getY();
// call to repaint causes rectangle to be drawn on
// new location
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

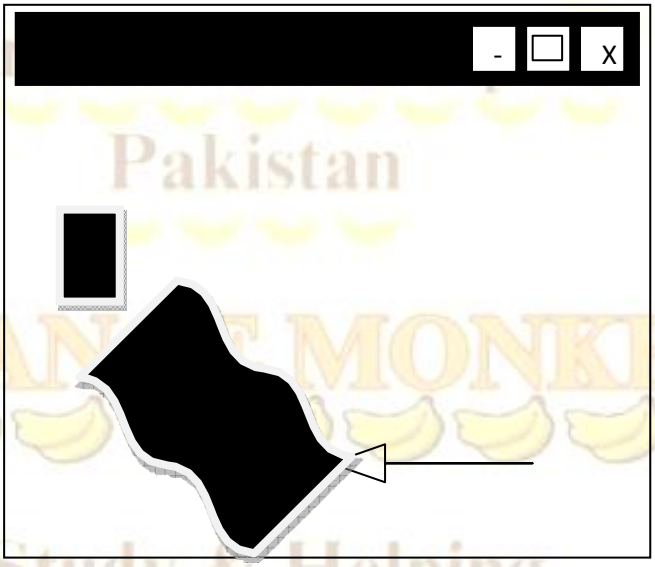
```
p.repaint() ;
} // end mouseDragged
} // end Handler class
// main method
public static void main(String args[ ]){
Test t = new Test();
}
} // end MyPanel class
```

On executing this program, when you drag mouse from one location to another, rectangle is also in sync with the movement of mouse. Notice that previously drawn rectangle is erased first.

If we exclude or comment out the following line from MyPanel class

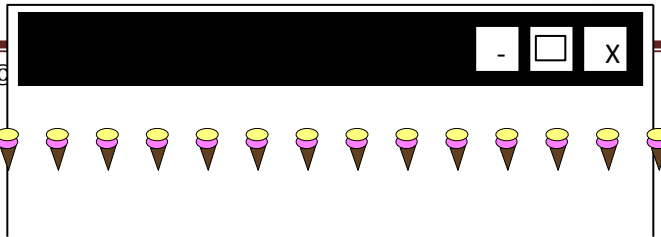
```
super.paintComponent(g);
```

Dragging a mouse will produce a similar kind of output shown next



Example Code 19.2: Ball Animation

The ball is continuously moving freely inside the corner of the frames. The sample outputs are shown below:





©

First we examine the MyPanel.java class that is drawing a filled oval.

```
import javax.swing.*;
import java.awt.*;
// extending class from JPanel
public class MyPanel extends JPanel {
// variables used to draw oval at different locations
int mX = 200;
int mY = 0;
// overriding paintComponent method
public void paintComponent(Graphics g){
// erasing behaviour - this will clear all the
// previous painting
super.paintComponent(g);
// Down casting Graphics object to Graphics2D
Graphics2D g2 = (Graphics2D)g;
// changing the color to blue
g2.setColor(Color.blue);
// drawing filled oval with blue color
// using instance variables
g2.fillOval(mX,mY,20,20);
}// end paintComponent
} // end MyPanel class
```

The Test class is given below. Additionally this class also contains the code for handling mouse events.

```
// importing required packages
 import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class AnimTest implements ActionListener {
```

© Copyright Virtual University of Pakistan

```
JFrame f;
MyPanel p;
// used to control the direction of ball int x, y;
public AnimTest(){
f = new JFrame();
Container c = f.getContentPane();
c.setLayout(new BorderLayout());
x = 5;
y = 3;
p = new MyPanel();
c.add(p);
f.setSize(400,400);
f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// creating a Timer class object, used for firing
// one or more action events after a specified delay
// Timer class constructor requires time in
// milliseconds and object of class that handles
// action events
Timer t = new Timer (5, this);
// starts the timer, causing it to start sending
// action events to listeners
t.start();
} // end constructor
// event handler method
public void actionPerformed(ActionEvent ae){
// if ball reached to maximum width of frame minus
// 40 since diameter of ball is 40 then change the
// X-direction of ball
if (f.getWidth()-40 == p.mX)
x = -5;
// if ball reached to maximum height of frame
// minus 40 then change the Y-direction of ball
if (f.getHeight()-40 == p.mY)
y = -3;
// if ball reached to min. of width of frame,
// change the X-direction of ball
if (p.mX == 0 )
x = 5;
// if ball reached to min. of height of frame,
// change the Y-direction of ball
if (p.mY == 0 )
y = 3;
// Assign x,y direction to MyPanel's mX & mY
p.mX += x;
```

© Copyright Virtual University of Pakistan

```
p.mY += y;
// call to repaint() method so that ball is drawn on
// new locations
p.repaint();
} // end actionPerformed() method
// main method
public static void main(String args[ ]){
AnimTest at = new AnimTest();
}
} // end AnimTest class
```

19.2 References

- Java, A Lab Course by Umair Javed
- Painting in AWT & Swing
  http://java.sun.com/products/jfc/tsc/articles/painting/index.html
- Performing Custom Painting
  http://java.sun.com/docs/books/tutorial/uiswing/14painting/index.html

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 20: Applets

20.1 Basic Definition

- A small program written in Java and included in a HTML page.
- It is independent of the operating system on which it runs
- An applet is a Panel that allows interaction with a Java program
- A applet is typically embedded in a Web page and can be run from a browser
- You need special HTML in the Web page to tell the browser about the applet
- For security reasons, applets run in a sandbox: they have no access to the client's file system

20.2 Applets Support

- Most modern browsers support Java 1.4 if they have the appropriate plugin
- Sun provides an application appletviewer to view applets without using browser.
- In general you should try to write applets that can be run with any browser

20.3 What an Applet is?

- You write an applet by extending the class Applet or JApplet
- Applet is just a class like any other; you can even use it in applications if you want
- When you write an applet, you are only writing part of a program
- The browser supplies the main method

20.4 The genealogy of Applet

The following figure shows the inheritance hierarchy of the JApplet class. This hierarchy determines much of what an applet can do and how, as you'll see on the next few pages.

```
java.lang.Object
   |
   +----java.awt.Component
            |
            +----java.awt.Container
                     |
                     +----java.awt.Panel
                              |
                              +----java.applet.Applet
                                       |
                                       +----javax.swing.JApplet
```

Example Code 20.1: Writing a Simple Applet

Below is the source code for an applet called HelloApplet. This displays a *"Hello World"* string. Note that no main method has been provided.

```
// File HelloApplet.java

//step 1: importing required packages
import java.awt.*;
import javax.swing.*;
// extending class from JApplet so that our class also becomes an
//applet
public class HelloApplet extends JApplet {

// overriding paint method
public void paint(Graphics g) {
// write code here u want to display & draw by using
// Graphics object
g.drawString("Hello World", 30 , 30);
}
} // end class
```

After defining the HelloApplet.java, the next step is to write .html file. Below is the source code of Test.html file. The Test.html contains the ordinary html code except one.

```
<html>
<head>
<title> Simple Applet </title> </head>

<body>

<!-- providing the class name of applet with width &
height
-->
<applet code="HelloApplet.class"
width=150 height=100>
</applet>
</body>
</html>
```

Compile & Execute

By simply double clicking on Test.html file, you can view the applet in your browser. However, you can also use the appletviewer java program for executing or running applets.

© Copyright Virtual University of Pakistan

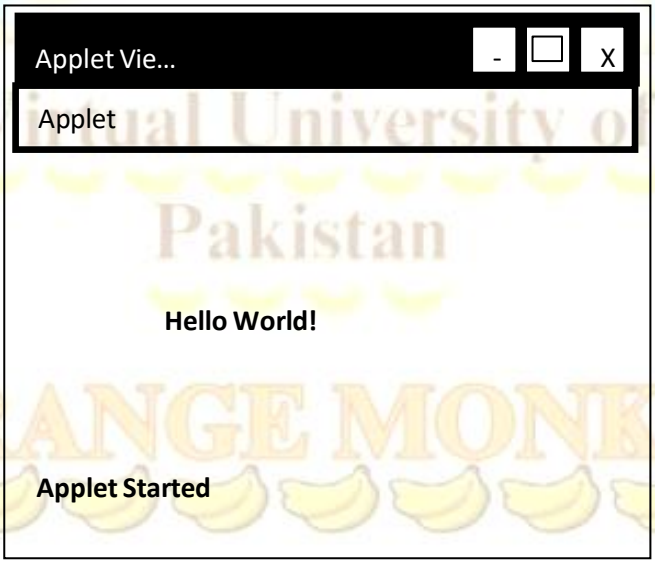
The applet viewer is invoked from the command line by the command

appletviewer *htmlfile*

where *htmlfile* is the name of the file that contains the html document. For our example, the command looks like this:

appletviewer *Test.html*

As a result, you will see the following output



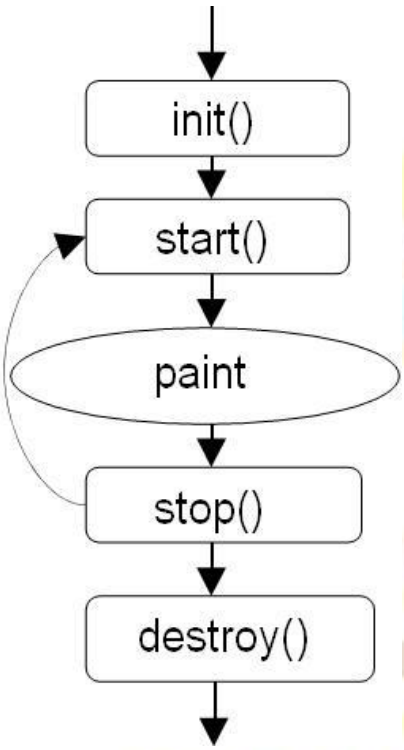
20.5 Applet Life Cycle Methods

When an applet is loaded, an instance of the applet's controlling class (an Applet subclass) is created. After that an applet passes through some stages or methods, each of them are build for specific purpose.

An applet can react to major events in the following ways:

- It can **initialize** itself.
- It can **start** running.
- It can **stop** running.
- It can perform a **final cleanup**, in preparation for being unloaded

The applet's life cycle methods are called in the specific order shown below. Not every applet needs to override every one of these methods.



Let's take a look on each method in detail and find out what they do:

20.5.1 init( )

- Is called only once.
- The purpose of init( ) is to *initialize* the applet each time it's loaded (or reloaded).
- You can think of it as a constructor

20.5.2 start( )

- To start the applet's execution
- For example, when the applet's loaded or when the user revisits a page that contains the applet
- start( ) is also called whenever the browser is maximized

20.5.3 paint( )

- paint( ) is called for the first time when the applet becomes visible
- Whenever applet needs to be repainted, paint( ) is called again
- Do all your painting in paint( ), or in a method that is called from paint( )

20.5.4 stop( )

- To stop the applet's execution, such as when the user leaves the applet's page or quits the browser.
- stop( ) is also called whenever the browser is minimized

20.5.5 destroy( )

- Is called only once.
- To perform a final cleanup in preparation for unloading

Example Code 20.2: Understanding Applet Life Cycle Methods

The following code example helps you in understanding the calling sequence of applet's life cycle methods. These methods are only displaying debugging statements on the console.

```
// File AppletDemo.java
//step 1: importing required packages
import java.awt.*;
import javax.swing.*;
// extending class from JApplet so that our class also becomes an
//applet
public class AppletDemo extends JApplet {
// overriding init method
public void init ( ) {
System.out.println("init() called");
}
// overriding start method
public void start ( ){
System.out.println("start() called");
}
// overriding paint method
public void paint(Graphics g){
System.out.println("paint() called");
}
// overriding stop method
public void stop(){
System.out.println("stop() called");
}
// overriding destroy method
public void destroy(){
System.out.println("destroy() called");
}
} // end class
```

© Copyright Virtual University of Pakistan

The *DemoTest.html* file is using this applet. The code snippet of it given below:

```
<html>
<head>
<title> Applet Life Cycle Methods </title> </head>
<body>
<!-- providing the class name of applet with width &
height
-->
<applet code="AppletDemo.class”
width=150 height=100>
</applet>
</body>
</html>
```

Compile & Execute

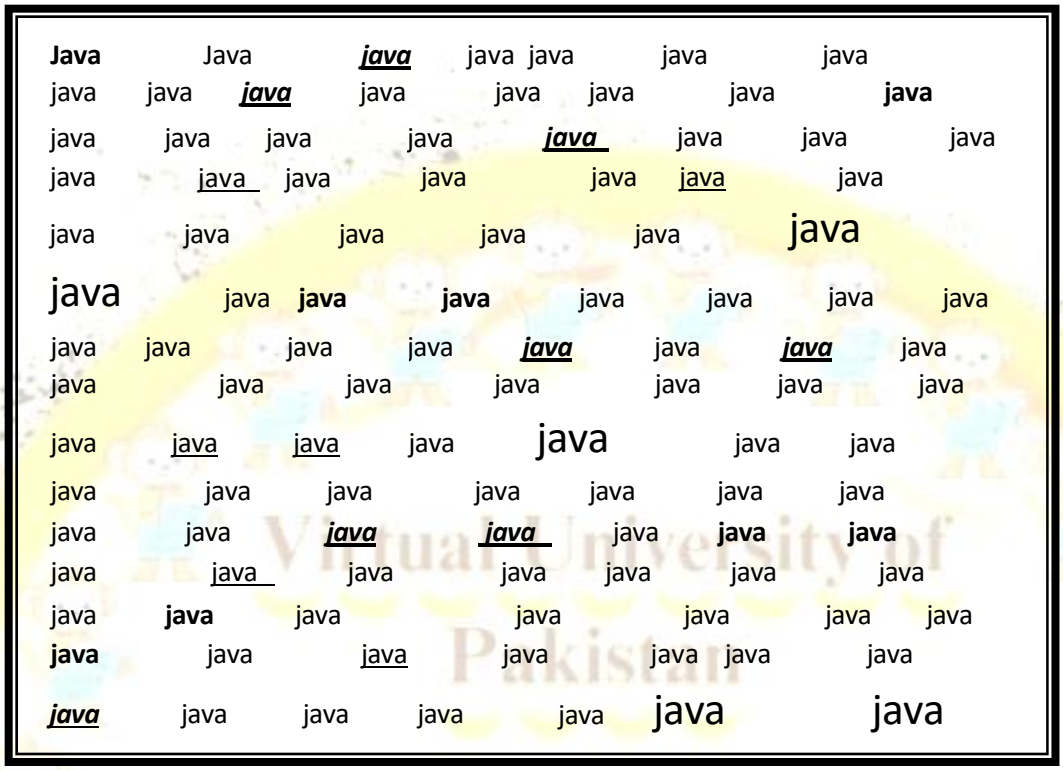
To understand the calling sequence of applet life cycle methods, you have to execute it by using appletviewer command. Do experiments like maximizing, minimizing the applet, bringing another window in front of applet and keep an eye on console output.

Example Code 20.3: Animated Java Word

Sample Output

The browser output of the program is given below:

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available


Java Java java java java java java
java java java java java java java java
java java java java java java java java
java java java java java java java
java java java java java java
java java java java java java java
java java java java java java java java
java java java java java java java
java java java java java java java

- **drawJava( )**

  As name indicates, this method will be used to write String "java" on random locations. The code is given below:

```
// method drawJava
public void drawJava(Graphics2D g2) {

// generate first number randomly. The panel width is 1000
int x = (int) (Math.random() * 1000);

// generate second number randomly. The panel height is 700
int y = (int) (Math.random() * 700);

// draw String on these randomly selected numbers
g2.drawString("java", x, y);
}
```

- **chooseColor( )**

  This method will choose color randomly out of 256 * 256 * 256 possible colors. The code snippet is given below:

```
// method chooseColor
public Color chooseColor() {
// choosing red color value randomly
 int r = (int) (Math.random() * 255);
// choosing green color value randomly
 int g = (int) (Math.random() * 255);
// choosing blue color value randomly
 int b = (int) (Math.random() * 255);
// constructing a color by providing R-G-B values
Color c = new Color(r, g, b);
// returning color
return c;
}
```

- **chooseFont( )**

  This method will choose a Font for text (java) to be displayed out of 4 available fonts. The code snippet is given below:

```
// method chooseFont
public Font chooseFont() {

// generating a random value that helps in choosing a font
```

© Copyright Virtual University of Pakistan

```
int fontChoice = (int) (Math.random() * 4) + 1;
// declaring font reference
Font f = null;

// using switch based logic for selecting font
switch (fontChoice) {
case 1:
f = new Font("Serif", Font.BOLD + Font.ITALIC, 20);
break;
case 2:
f = new Font("SansSerif", Font.PLAIN, 17);
break;
case 3:
f = new Font("Monospaced", Font.ITALIC, 23);
break;
case 4:
f = new Font("Dialog", Font.ITALIC, 30);
break;
} // end switch
// returns Font object
return f;

} //end chooseFont
```

- **paint( )**

  The last method to be discussed here is paint(). By overriding this method, we will print string "java" on 40 random locations. For every drawing, it selects random font out of 4 different fonts & random color out of 256 * 256 * 256 RGB colors.

  Let's see, how it happens:

```
// overriding method paint
public void paint(Graphics g) {
// incrementing clear counter variable.
clearCounter++;
// printing 40 "java" strings on different locations by
// selcting random font & color
for (int i = 1; i <= 40; i++) {
// choosing random color by calling chooseColor() method
Color c = chooseColor();
// setting color
g2.setColor(c);
// choosing random Font by calling chooseColor() method
```

© Copyright Virtual University of Pakistan

```
Font f = chooseFont();
g2.setFont(f);
// drawing string “java” by calling drawJava() method
drawJava(g2);
}
// end for loop
Graphics2D g2 = (Graphics2D) g;
// checking if paint is called 10 times then clears the
// screen and set counter again to zero
if (clearCounter == 10) {
g2.clearRect(0, 0, 1000, 700);
clearCounter = 0;
}
} // end paint method
```

Merging Pieces

By inserting all method inside *JavaAnim.java* class, the program will look like one given below. Notice that it contains methods discussed above with some extra code with which you are already familiar.

```
// File JavaAnim.java
//step 1: importing required packages import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class JavaAnim extends JApplet implements ActionListener {
// used to count how many times paint is called int clearCounter;
// declaring Timer reference
Timer t;
// overriding init method, used to initialize variables
public void init() {
setBackground(Color.black);
clearCounter = 0;
Timer t = new Timer(1000, this);
t.start();
}
// overriding paint method - discussed above
public void paint(Graphics g) {
clearCounter++;
Graphics2D g2 = (Graphics2D) g;
if (clearCounter == 10) {
g2.clearRect(0, 0, 1000, 700);
clearCounter = 0;
}
```

© Copyright Virtual University of Pakistan

```
for (int i = 1; i <= 40; i++) {
Color c = chooseColor();
g2.setColor(c);
Font f = chooseFont();
g2.setFont(f);
drawJava(g2);
}
}
// overriding actionPerformed()of ActionListener interface
// called by Timer object
public void actionPerformed(ActionEvent ae) {
repaint();
}
// chooseColor method - discussed above
public Color chooseColor() {
int r = (int) (Math.random() * 255);
int g = (int) (Math.random() * 255);
int b = (int) (Math.random() * 255);
Color c = new Color(r, g, b);
return c;
} // chooseFont method - discussed above
public Font chooseFont() {
int fontChoice = (int) (Math.random() * 4) + 1;
Font f = null;
switch (fontChoice) {
case 1:
f = new Font("Serif", Font.BOLD + Font.ITALIC, 20);
break;
case 2:
f = new Font("SansSerif", Font.PLAIN, 17);
break;
case 3:
f = new Font("Monospaced", Font.ITALIC, 23);
break;
case 4:
f = new Font("Dialog", Font.ITALIC, 30);
break;
}
return f;
}
// drawJava() method - discussed above
public void drawJava(Graphics2D g2) {
int x = (int) (Math.random() * 1000);
int y = (int) (Math.random() * 700);
g2.drawString("java", x, y);
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

```
}
} // end class
```

The *AnimTest.html* file is using this applet. The code snippet of it given below:

```
<html>
<head>
<title> Animated Java Word </title> </head>
<body>
<applet code="JavaAnim.class" width=1000 height=700> </applet>
</body>
</html>
```

Compile & Execute

You can execute it directly using browser or by using appletviewerapplication. For having fun, you can use "your name" instead of "java" and watch it in different colors.

20.6 References:

- Java, A Lab Course by Umair Javed
- Writing Applets
  - http://java.sun.com/docs/books/tutorial/**applet**/

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 21: Socket Programming

21.1 Basic Definition

- A socket is one endpoint of a two-way communication link between two programs running generally on a network.
- A socket is a bi-directional communication channel between hosts. A computer on a network often termed as host.

21.2 Socket Dynamics

- As you have already worked with files, you know that file is an abstraction of your hard drive. Similarly you can think of a socket as an abstraction of the network.
- Each end has input stream (to send data) and output stream (to receive data) wired up to the other host.
- You store and retrieve data through files from hard drive, without knowing the actual dynamics of the hard drive. Similarly you send and receive data to and from network through socket, without actually going into underlying mechanics.
- You read and write data from/to a file using streams. To read and write data to socket, you will also use streams.

21.3 What is Port?

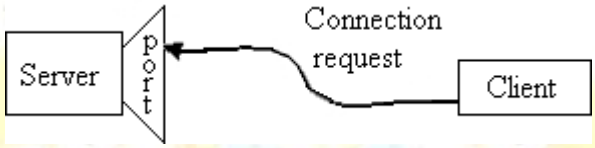
- It is a transport address to which processes can listen for connections request.
- There are different protocols available to communicate such as TCP and UDP. We will use TCP for programming in this handout
- There are 64k ports available for TCP sockets and 64k ports available for UDP, so at least theoretically we can open 128k simultaneous connections.
- There are well-known ports which are
    - below 1024
    - provides standard services
    - Some well-known ports are:
        - FTP works on port 21
        - HTTP works on port 80
        - TELNET works on port 23 etc.

21.4 How Client - Server Communicate

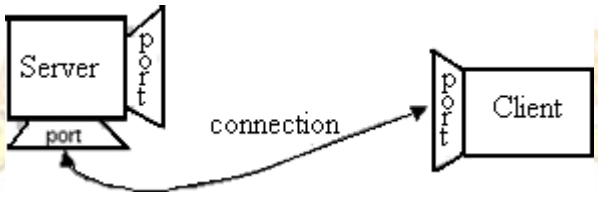
- Normally, a server runs on a specific computer and has a socket that is bound to a specific port number.
- The server just waits, listening to the socket for a client to make a connection request.

© Copyright Virtual University of Pakistan

- On the client side: The client knows the hostname of the machine on which the server is running and the port number to which the server is connected.



- As soon as client creates a socket that socket attempts to connect to the specified server.
- The server listens through a special kind of socket, which is named as server socket.
- The sole purpose of the server socket is to listen for incoming request; it is not used for communication.
- If everything goes well, the server accepts the connection. Upon acceptance, the server gets a new socket, a communication socket, bound to a different port number.
- The server needs a new socket (and consequently a different port number) so that it can continue to listen through the original server socket for connection requests while tending to the needs of the connected client. This scheme is helpful when two or more clients try to connect to a server simultaneously (a very common scenario).



- On the server side, if the connection is accepted, a socket is successfully created and the client can use the socket to communicate with the server.
- Note that the socket on the client side is not bound to the port number used to make contact with the server. Rather, the client is assigned a port number local to the machine on which the client is running.
- The client and server can now communicate by writing to or reading from their sockets.

21.5 Steps - To Make a Simple Client

To make a client, process can be split into 5 steps. These are:

21.5.1 Import required package

You have to import two packages

- java.net.*;

- java.io.*;

21.5.2 Connect / Open a Socket with Server

Create a client socket (communication socket)

```
Socket s        = new Socket("serverName", serverPort) ;
```

- **serverName:** Name or address of the server you wanted to connect such as http://www.google.com or 172.2.4.98 etc. For testing if you are running client and server on the same machine then you can specify "localhost" as the name of server
- **serverPort :** Port number you want to connect to

The scheme is very similar to our home address and then phone number.

21.5.3 Get I/O Streams of Socket

Get input & output streams connected to your socket

- **For reading data from socket**

As stated above, a socket has input stream attached to it.

```
InputStream is = s.getInputStream();
// now to convert byte oriented stream into character oriented buffered reader
// we use intermediary stream that helps in achieving above stated purpose
InputStreamReader isr= new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
```

- **For writing data to socket**

A socket has also output stream attached to it. Therefore,

```
OutputStream os = s.getOutputStream();
// now to convert byte oriented stream into character oriented print writer
// here we will not use any intermediary stream because PrintWriter constructor
// directly accepts an object of OutputStream
PrintWriter pw = new PrintWriter(os, true);
```

Here notice that true is also passed to so that output buffer will flush.

21.5.4 Send / Receive Message

Once you have the streams, sending or receiving messages isn't a big task. It's very much similar to

the way you did with files

- **To send messages**

```
pw.println("hello world");
```

- **To read messages**

```
String recMsg = br.readLine();
```

21.5.5 Close Socket

Don't forget to close the socket, when you finished your work

```
s.close();
```

21.6 Steps - To Make a Simple Server

To make a server, process can be split into 7 steps. Most of these are similar to steps used in making a client. These are:

21.6.1 Import required package

You need the similar set of packages you have used in making of client

- java.net.*;
- java.io.*;

21.6.2 Create a Server Socket

In order to create a server socket, you will need to specify port no eventually on which server will listen for client requests.

```
ServerSocket ss = new ServerSocket(serverPort) ;
```

- **serverPort:** port local to the server i.e. a free port on the server machine. This is the same port number that is given in the client socket constructor

21.6.3 Wait for Incoming Connections

The job of the server socket is to listen for the incoming connections. This listening part is done

through the accept method.

```
Socket s = ss.accept();
```

The server program blocks (stops) at the accept method and waits for the incoming client connection when a request for connection comes it opens a new communication socket (s) and use this socket to communicate with the client.

21.6.4 Get I/O Streams of Socket

Once you have the communication socket, getting I/O streams from communication socket is similar to the way did in making a client

- **For reading data from socket**

```
InputStream is = s.getInputStream();
InputStreamReader isr=new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
```

- **For writing data to socket**

```
OutputStream os = s.getOutputStream();
PrintWriter pw = new PrintWriter(os, true);
```

21.6.5 Send / Receive Message

Sending and receiving messages is very similar as discussed in making of client

- **To send messages:**

```
pw.println("hello world");
```

- **To read messages**

```
String recMsg = br.readLine();
```

21.6.6 Close Socket

```
s.close();
```

Example Code 21.1: Echo Server & Echo Client

The client will send its name to the server and server will append "hello" with the name send by the client. After that, server will send back the name with appended "hello".

EchoServer.java

Let's first see the code for the server

```
// step 1: importing required package
import java.net.*;
import java.io.*;
```

```
import javax.swing.*;
public class EchoServer{
public static void main(String args[]){
try {
//step 2: create a server socket
ServerSocket ss = new ServerSocket(2222);
System.out.println("Server started...");
/* Loop back to the accept method of the server socket and wait for a new connection request. So
server will continuously listen for requests
*/
while(true) {
// step 3: wait for incoming connection
Socket s = ss.accept();
System.out.println("connection request recieved");

// step 4: Get I/O streams
InputStream is = s.getInputStream();
InputStreamReader isr= new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);

OutputStream os = s.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);
// step 5: Send / Receive message
// reading name sent by client
String name = br.readLine();
// appending “hello” with the received name
String msg = "Hello " + name + " from Server";
// sending back to client
pw.println(msg);
// closing communication sockey
s.close();
} // end while
}catch(Exception ex){
System.out.println(ex);
}
}
} // end class
```

EchoClient.java

The code of the client is given below

```
// step 1: importing required package
import java.net.*;
import java.io.*;
import javax.swing.*;
public class EchoClient{
```

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

```
public static void main(String args[]){
try {
//step 2: create a communication socket

/* if your server will run on the same machine then you can pass “localhost” as server address.Notice
that port no is similar to one passed while creating server socket */
Socket s = new Socket(“localhost”, 2222);
// step 3: Get I/O streams
InputStream is = s.getInputStream();
InputStreamReader isr= new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
OutputStream os = s.getOutputStream();
PrintWriter pw = new PrintWriter(os,true);

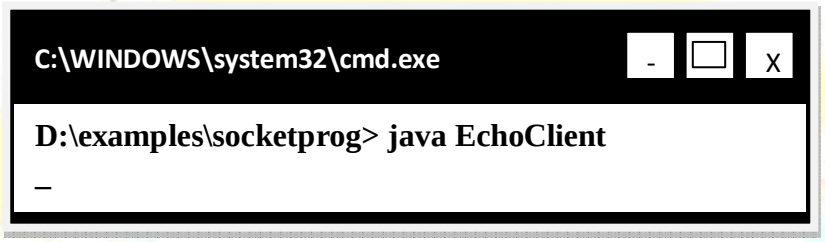
// step 4: Send / Receive message
// asking user to enter his/her name
String msg = JOptionPane.showInputDialog( "Enter your name");
// sending name to server
pw.println(msg);
// reading message (name appended with hello) from
// server
msg = br.readLine();
// displaying received message
JOptionPane.showMessageDialog(null , msg);
// closing communication socket
s.close();
}catch(Exception ex){
System.out.println(ex);
}
}
} // end class
```

Compile & Execute

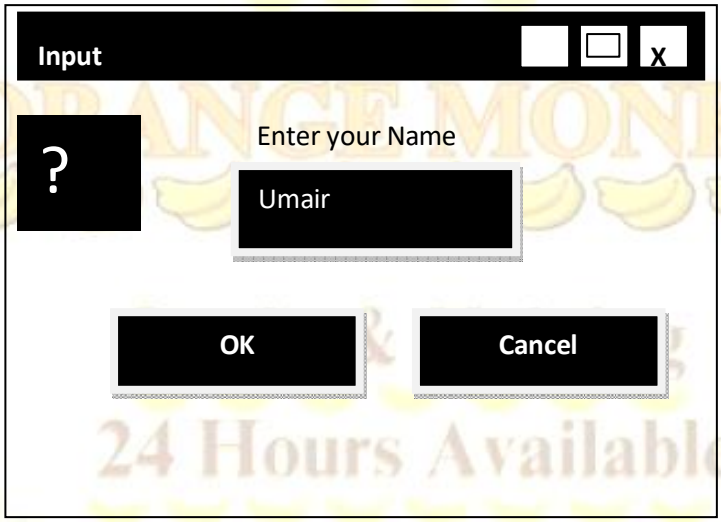
After compiling both files, run EchoServer.java first, from the command prompt window. You'll see a message of "server started" as shown in the figure below. Also notice that cursor is continuously blinking since server is waiting for client request



Now, open another command prompt window and run EchoClient.java from it. Look at EchoServer window; you'll see the message of "request received". Sooner, the EchoClient program will ask you to enter name in input dialog box. After entering name press ok button, with in no time, a message dialog box will pop up containing your name with appended "hello" from server. This whole process is illustrated below in pictorial form:







Sending name to server



Response from server

Notice that server is still running, you can run again EchoClient.java as many times untill server is running.

To have more fun, run the server on a different computer and client on a different. But before doing that find the IP of the computer machine on which your EchoServer will eventually run. Replace "localhost" with the new IP and start conversion over network

21.7 References

- Entire material for this handout is taken from the book **JAVA A Lab Course** by **Umair Javed**. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

Lecture 22:

22.1 Problem

22.1.1 What?

- You want to send an object to a stream.

22.1.2 Motivation

- A lot of code involves boring conversion from a file to memory
  - As you might recall that AddressBook program reads data from file and then parses it
- This is a common problem

22.1.3 Revisiting AddressBook

We read record from a text file named persons.txt. The person record was present in the file in the following format:

```
Ali,defence,9201211
Usman,gulberg,5173940
Salman,LUMS,5272670
```

persons.txt

The code that was used to construct Person objects after reading information from the file is given below. Here only the part of code is shown, for complete listing, see AddressBook code in your earlier handout.

```
……….
FileReader fr = new FileReader("persons.txt");
BufferedReader br = new BufferedReader(fr);
String line = br.readLine();

while ( line != null ) {
tokens = line.split(",");
name = tokens[0];
add = tokens[1];
ph    = tokens[2];
PersonInfo p =new PersonInfo(name, add, ph);
// you can add p into arraylist, if needed
```

© Copyright Virtual University of Pakistan

```
line = br.readLine();
}
```

As you have seen a lot of parsing code is required for converting a line into PersonInfo objects. Serialization mechanism eases developer's life by achieving all above in a very simple way.

22.2 Serialization in Java

- Java provides an extensive support for serialization
- Object knows how to read or write themselves to streams
- **Problem:**
    - As you know, objects get created on heap and have some values therefore Objects have some state in memory
    - You need to save and restore that state.
    - The good news is that java serialization takes care of it automatically

22.2.1 Serializable Interface

- By implementing this interface a class declares that it is willing to be read/written by automatic serialization machinery
- Found in java.io package
- Tagging interface - has no methods and serves only to identify the semantics of being serializable

22.2.2 Automatic Writing

- System knows how to recursively write out the state of an object to stream
- If an object has the reference of another object, the java serialization mechanism takes care of it and writes it too.

22.2.3 Automatic Reading

- System knows how to read the data from Stream and re-create object in memory
- The recreated object is of type "Object" therefore Down-casting is required to convert it into actual type.

© Copyright Virtual University of Pakistan

22.2.4 Serialization: How it works?

- To write an object of PersonInfo, ObejctOutputStream and its method writeObject( ) will be used

```
PersonInfo p = new PersonInfo( );
ObejctOutputStream out;
// writing PersonInfo's object p
out.writeObject(p);
```

- To read that object back, ObejctInputStream and its method readObject() will be used

```
ObejctInputStream in;
// reading PersonInfo's object. Remember type
casting // is required
PersonInfo obj = (PersonInfo)in.readObject( );
```

Example Code 22.1: Reading / Writing PersonInfo objects

We want to send PersonInfo object to stream. You have already seen this class number of times before. Here it will also implement serializable interface.

```
PersonInfo.java
import javax.swing.*;
import java.io.* ;
class PersonInfo implements Serializable{
String name;
String address;
 String phoneNum;
//parameterized constructor
public PersonInfo(String n, String a, String p) {
name = n;
address = a;
 phoneNum = p;
}
//method for displaying person record on GUI
public void printPersonInfo( ) {
 JOptionPane.showMessageDialog(null ,"name: " + name + "address:" +address + "phone no:" +
phoneNum);
}
} // end class
```

WriteEx.java

The following class will serialize PersonInfo object to a file

```
import java.io.*;
public class WriteEx{
public static void main(String args[ ]){
PersonInfo pWrite = new PersonInfo("ali", "defence", "9201211");
try {
// attaching FileOutput stream with “ali.dat”
FileOutputStream fos = new FileOutputStream("ali.dat");
// attaching ObjectOutput stream over FileOutput stream
ObjectOutputStream out = new ObjectOutputStream(fos);
//serialization
// writing object to ‘ali.dat’
out.writeObject(pWrite);
// closing streams out.close();
fos.close();
} catch (Exception ex){ System.out.println(ex);
}
}
}      // end class
```

ReadEx.java

The following class will read serialized object of PersonInfo from file i.e “ali.dat”

```
import java.io.*;
public class ReadEx{
public static void main(String args[ ]){
try {
// attaching FileInput stream with “ali.dat”
FileInputStream fis = new FileInputStream("ali.dat");
// attaching FileInput stream over ObjectInput stream
ObjectInputStream in = new ObjectInputStream(fis);
//de-serialization
// reading object from ‘ali.dat’
PersonInfo pRead = (PersonInfo)in.readObject( );
// calling printPersonInfo method to confirm that
// object contains same set of values before
// serializatoion
pRead.printPersonInfo();
// closing streams
 in.close();
fis.close();
} catch (Exception ex){
System.out.println(ex);
}
}//end main function
} // end class
```

© Copyright Virtual University of Pakistan

Compile & Execute

After compilation, first run the WriteEx.java file and visit the "ali.dat" file. Then run ReadEx.javafrom different command or same command prompt.

22.3 Object Serialization & Network

- You can read / write to a network using sockets.
- All you need to do is attach your stream with socket rather than file.
- The class version should be same on both sides (client & network) of the network .

Example Code 22.2: Sending/Reading Objects to/from Network

We are going to use same PersonInfo class listed in example code 22.1. An object of PersonInfo class will be sent by client on network using sockets and then be read by server from network.

Sending Objects over Network

The following class ClientWriteNetEx.javawill send an object on network

```
import java.net.*;
import java.io.*;
import javax.swing.*;
public class ClientWriteNetEx{
public static void main(String args[]){
try {
PersonInfo p = new PersonInfo("ali", "defence", "9201211");
// create a communication socket
Socket s = new Socket("localhost", 2222);
// Get I/O streams
OutputStream is = s.getOutputStream();
// attaching ObjectOutput stream over Input stream
ObjectOutputStream oos= new ObjectOutputStream(is);
// writing object to network
oos.writeObject(p);
// closing communication socket
s.close();
}catch(Exception ex){
System.out.println(ex);
 }
```

© Copyright Virtual University of Pakistan

```
}
} // end class
```

Reading Objects over Network

The following class ServerReadNetEx.javawill read an object of PersonInfo sent by client.

```
import java.net.*;
import java.io.*;
import javax.swing.*;
public class ServerReadNetEx{
public static void main(String args[]){
try {
// create a server socket
ServerSocket ss = new ServerSocket(2222);
System.out.println("Server started...");
/* Loop back to the accept method of the server socket and wait for a new connection request. So
server will continuously listen for requests
*/
while(true) {
// wait for incoming connection
Socket s = ss.accept();
System.out.println("connection request recieved");
// Get I/O streams
InputStream is = s.getInputStream();
// attaching ObjectOutput stream over Input stream
ObjectInputStream ois = new ObjectInputStream(is);
// read PersonInfo object from network
PersonInfo p = (PersonInfo)ois.readObject( );
p.printPersonInfo();
// closing communication socket
s.close();
} // end while
}catch(Exception ex){
System.out.println(ex); }
}
} // end class
```

Compile & Execute

After compiling both files, run ServerReadNetEx.javafirst, from the command prompt window. Open another command prompt window and run ClientWriteNetEx.java from it. The ClientWriteNetEx.java will send an Object of PersonInfo to ServerReadNetEx.java that displays that object values in dialog box after reading it from network.

© Copyright Virtual University of Pakistan

22.4 Preventing Serialization

- Often there is no need to serialize sockets, streams & DB connections etc because they do not represent the state of object, rather connections to external resources
- To do so, **transient** keyword is used to mark a field that should not be serialized
- So we can mark them as,
  - **transient** Socket s;
  - **transient** OutputStream os;
  - **transient** Connection con;
- Transient fields are returned as null on reading

Example Code 22. 3: transient

Assume that we do not want to serialize phoneNumattribute of PersonInfo class, this can be done as shown below

PersonInfo.java

```
import javax.swing.*;
import java.io.*
class PersonInfo implements Serializable{
String name;
String address;
transient String phoneNum;
public PresonInfo(String n, String a, String p) {
name = n;
address = a;
phoneNm = p;
}
public void printPersonInfo( ) {
JOptionPane.showMessageDialog(null , "name: " + name + "address:" +address + "phone no:" +
phoneNum);
}
} // end class
```

22.5 References

Entire material for this handout is taken from the book **JAVA A Lab Course** by **Umair Javed**. This material is available just for the use of VU students of the course Web Design and Development and not for any other commercial purpose without the consent of author.

Note: Coding exercises in working condition for this lecture are also available on **"Downloads"** section of LMS.

Virtual University of Pakistan
ORANGE MONKEY
Study & Helping
24 Hours Available

MIDTERM EXAMINATION
Spring 2010
CS506- Web Design and Development (Session - 3)

Question No: 1 (Marks: 1) - Please choose one
If we remove static from main then program _____
►Program cannot compile
►Program will compile and work properly
►Run time errors
►**Exception throws in main** Click here for Detail

Question No: 2 (Marks: 1) - Please choose one
If we remove String array from main then program_____
►Program cannot compile
►Program will compile and work properly
►**Run time errors** Click here for Detail
►Exception throws in main

Question No: 3 (Marks: 1) - Please choose one
Java developed in _____
►1990
►**1995 (Page 12)**
►1997
►2000

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

MIDTERM EXAMINATION
Fall 2008
CS506- Web Design and Development (Session - 2)

Question No: 1 (Marks: 1) - Please choose one
Which of the following methods are invoked by the AWT to support paint and repaint operations?

- ► **paint( )** Click here for Detail
- ► repaint( )
- ► draw( )
- ► redraw( )

Question No: 2 (Marks: 1) - Please choose one
When defining a method you must include a _____ to declare any exception that might be thrown but is not caught in the method.

- ► try block
- ► finally block
- ► catch block
- ► **throws-clause** Click here for Detail

Question No: 3 (Marks: 1) - Please choose one
Aside from the Scanner class, which of the following classes can be used for text-file file input?

- ► ObjectInputStream
- ► **BufferedReader** Click here for detail
- ► StringTokenizer
- ► None of these

Question No: 4 (Marks: 1) - Please choose one
Which of the following statements is true regarding Vectors with no specified base type?

- ► A base type is needed, because Java needs to know how to allocate memory.
- ► **No base type is needed, because Java will use the Object class as a base type.** Click here for detail
- ► If a base type is not specified the code will not compile.
- ► No base type is needed because Vectors default to storing String objects.

Question No: 5 (Marks: 1) - Please choose one
A serializable class must implement the method(s) _____.

- ► readObject and writeObject
- ► Either readObject or writeObject, or both, depending upon the desired behavior
- ► **No need to implement any method** Click here for Detail
- ► None of the given option

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question No: 6 (Marks: 1) - Please choose one
You can ---------to a network using sockets.

- ► Read
- ► Write
- ► **Read/Write (Page 222)**
- ► None of these

Question No: 7 (Marks: 1) - Please choose one
Socket is a ----------communication channel between hosts.

- ► Uni-directional
- ► **Bi-directional (Page 211)**
- ► Multi-directional
- ► None of these

Question No: 8 (Marks: 1) - Please choose one
Which of the following are passed as an argument to the paint ( ) method?

- ► A Canvas object
- ► **A Graphics object** Click here for detail
- ► An Image object
- ► A paint object

Question No: 9 (Marks: 1) - Please choose one
The code below draws a line. What color is the line?
g.setColor(Color.red.green.yellow.red);
g.drawLine(0, 0, 100,100);

- ► Red
- ► Green
- ► Yellow
- ► **Black**

Question No: 10 (Marks: 1) - Please choose one
To create DataBaseMetaData object we write following line of code

- ► **DataBaseMetaData db = con.getMeataData(); (Page 177)**
- ► ResultSetMetaData rsmd = rs.getMetaData();
- ► ResultSetMetaData rsmd = rs.setMetaData();
- ► DataBaseMetaData db = con.setMeataData();

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question No: 11 (Marks: 1) - Please choose one
Event source can have --------- listeners registered on it.

- ► Single
- ► Double
- ► Triple
- ► **Multiple (Page 125)**

Question No: 12 (Marks: 1) - Please choose one
An abstract class ---------- instantiated.

- ► **Can't be (Page 96)**
- ► Can be
- ► Must
- ► None of these

Question No: 13 (Marks: 1) - Please choose one
A _____ defines the way and method of communication between two parties.

- ► Compiler
- ► **Protocol (Page 242)**
- ► Tool
- ► Technique

Question No: 14 (Marks: 1) - Please choose one
What if the static modifier is removed from the signature of the main method?

- ► The program does not compile
- ► The program compiles but does not run
- ► The program compiles and runs properly
- ► **The program throws an exception on compile time** Click here for Detail

Question No: 15 (Marks: 1) - Please choose one
What if the main method is declared as private?

- ► The program does not compile…
- ► **The program compiles but does not run** Click here for Detail
- ► The program compiles and runs properly
- ► **The program throws an exception on compile time**

Question No: 16 (Marks: 1) - Please choose one
A member variable or method prefixed by the **protected** access modifier can be accessed:

- ► Within the same class
- ► Within the sub-class
- ► Within the same package
- ► **All of the above** Click here for Detail

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

MIDTERM EXAMINATION
Spring 2006
CS506- Web Design and Development

Question No: 1 (Marks: 1) - Please choose one
In Java a final class must be sub-classed before it.
► True
► **False** Click here for Detail

Question No: 2 (Marks: 1) - Please choose one
A top level class without any modifier is accessible to
► any class
► **any class within the same package** Click here for Detail
► any class within the same file
► any subclass of this class

Question No: 3 (Marks: 1) - Please choose one
A top level class may have only the following access modifier.
► Package
► Private
► Protected
► **Public** Click here for Detail

Question No: 4 (Marks: 1) - Please choose one
In Java an abstract class cannot be sub-classed
► True
► **False** Click here For Detail

Question No: 5 (Marks: 1) - Please choose one
Given a one dimensional array arr, what is the correct way of getting the number of elements in arr
► **arr.length** Click here for Detail
► arr.length – 1
► arr.size – 1
► arr.length()

Question No: 6 (Marks: 1) - Please choose one
When recursive method is called to solve a problem, the method actually is capable of solving only the simpler case(s), or base case(s).
► **True** Click here For Detail
► False

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question No: 7 (Marks: 1) - Please choose one
Which of these are valid declarations for the main method?
► public void main();
► **public static void main(String args[]); (Page 26)**
► static public void main(String);
► public static int main(String args[]);

Question No: 8 (Marks: 1) - Please choose one
Map interface is derived from the Collection interface.
► True
► **False** Click here For Detail

Question No: 9 (Marks: 1) - Please choose one
In Java, which of these classes implement the LayoutManager interface?
► RowLayout
► ColumnLayout
► **GridBagLayout** Click here for Detail
► FlowLayoutManager

Question No: 10 (Marks: 1) - Please choose one
BorderLayout is the default layout manager for a JFrame's content pane
► **True** Click here For Detail
► False

MIDTERM EXAMINATION
Spring 2006
CS506- Web Design and Development (Session 1)

Question No: 1 (Marks: 1) - Please choose one
You want a class to have access to members of another class in the same package. Which is the most restrictive access that accomplishes this objective?
► public
► private
► protected
► transient
► **default access** Click here For Detail

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question No: 2 (Marks: 1) - Please choose one
In Java garbage collection is done by_____

► **JVM (Java Virtual Machine) (Page 47)**
► Programmer
► Both JVM (Java Virtual Machine)and Programmer
► OS (Operating System)
► None of given

Question No: 3 (Marks: 1) - Please choose one
Which of following can be thrown using the throw statement?

► Error
► Throwable
► **Exception (Page 72)**
► RuntimeException
► All of Given

Question No: 4 (Marks: 1) - Please choose one
_____is/are the JSP Implicit Object(s).

► session
► application
► config
► **All of Given** Click here For Detail
► None of Given

Question No: 5 (Marks: 1) - Please choose one
JavaBeans are?

► A special Java class file
► Servlets
► Applets
► **A Special form of JSP (Page 367)**
► None of Given

Question No: 6 (Marks: 1) - Please choose one
An expression tag contains a scripting language expression that is evaluated.

► **True** Click here For Detail
► False

Question No: 7 (Marks: 1) - Please choose one
A final class can't be extended.

► **True** Click here For Detail
► **False**

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question No: 8 (Marks: 1) - Please choose one
ServletConfig defines a set of methods that a servlet uses to communicate with its servlet container.
► True
► **False** Click here For Detail

Question No: 9 (Marks: 1) - Please choose one
A method can not be made final until whole class is made final.
► True
► False

Question No: 10 (Marks: 1) - Please choose one
Servlet session and JSP session have different abilities.
► True
► **False** Click here For Detail

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

CS506- Web Design and Development
Some other MCQ's & quizzes

1 - What if the main method is declared as private?
►The program does not compile
►**The program compiles but does not run** Click here for Detail **rep**
►The program compiles and runs properly
►The program throws an exception on compile time

2 - Java program code is compiled into form called
►Machine code
►native Code
►**Byte Code** **(Page 16)**
►Source Code

3 - A top level class without any modifier is accessible to
►any class
►**any class within the same package** Click here For Detail
►any class within the same file
►any subclass of this class

4 - Overloading is _____ whereas overriding is _____
► Run time binding, compile time binding
► Late binding, compile time binding
► **Compile time binding, run time binding**
► Run time binding, late time binding

5 - Static methods only access ___
► instance variable
► instance method
► **static variables and methods** Click here For Detail
► both static and instance members

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

6 - Which of the following is modifier is provided by default if no access is written explicitly?

► public
► Private
► Protected
► **Default** Click here For Detail

7 - A top level class may have only the following access modifier.

► Package
► Private
► Protected
► **Public** Click here for Detail **rep**

8 - Which of the following is used for inheritance in java?

►implements
►extends
►**: (colon)** Click here For Detail
► inherit

9 - A collection can store.....

► **Homogenous objects**
► Heterogeneous objects
► Objects as well as primitive values
► At most 100 objects

10 - Which of the following syntax is used to attach an input stream to console?

► **FileReader fr = new FileReader(' input.txt ') ; (Page 77)**
► FileReader fr = new FileReader(FileDescriptor.in);
► FileReader fr = new FileReader(FileDescriptor);
► FileReader fr = new FileReader(console);

11 - Window frame and dialog use _____ as their default layout.

► **Border layout** Click here For Detail
► Flow layout
► GridBag layout
► Grid layout

12 - There are _____ types of exceptions in Java

► **2 (Page 72)**
► 3
► 4
► 5

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

13 - Which of the following belongs to a category of checked exception?

► Null Pointer exception
► **IOException** Click here For Detail
► Array index out of bounds
► NumberFormatException

14 - Which of the following exception belongs to a category of un-checked exception?

► IOException
► **Null Pointer exception** Click here For Detail
► AWTException
► ClassNotFoundException

15 - All the exceptions and errors in java are inherited from _____ class.

► Exception
► Error
► **Throwable** Click here For Detail
► IOException

16 - Exceptions must be handled while handling files otherwise it may lead to _____

► Logical error
► Syntax error
► **Run-time error** **(Page 70)**
► No error

17 - Which of the following package needs to be import while handling files?

► java.util
► **java.io** Click here For Detail
► java.awt
► javax.swing

18 - When defining a method you must include a/an ..... to declare any exception that might be thrown but is not caught in the method:

► try block
► finally block
► catch block
► **throws clause** Click here For Detail

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

19 - The classes which contain the word _____ are byte oriented streams.

► Reader
► Writer
► **Stream (Page 87)**
► Byte

20 - Which of the following stream read/write data in the form of bytes?

► FileReader
► FileWriter
► PrintWriter
► **FileInputStream (Page 87)**

21 - An instance of abstract class cannot be created.
► **TRUE** Click here For Detail
► FALSE

22 – An abstract class ----------instantiated.

► **Cannot be** Click here For Detail
► Can be
► Must
► None of these

23 - The relationship between class and interface is called _____
► **' Is a ' relationship (Page 98)**
► ' Has a ' relationship
► ' Responds to ' relationship
► None of the given options

24 - Which of the following is called ' pure abstract class ' ?
► Concrete class
► Wrapper class
► **Interface (Page 98)**
► Abstract class with no abstract method

25 - Which of the following is called ' pure abstract class ' ?
► Concrete class
► Wrapper class
► **Interface (Page 98) rep**
► Abstract class with no abstract method

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

26 - Which of the following is true about abstract class?

► An abstract class must have all methods declared as abstract methods.
► A class must have at least one abstract method to be an abstract class.
► **A class without any abstract method can be declared as abstract class.** Click here for detail
► An instance of abstract class can be created.

27 - Which of the following stream is a filter stream?

► FileWriter
► FileReader
► **BufferedReader (Page 91)**
► All of given options

28 - JPanel and Applet use _____ as their default layout.

► **Flow layout** Click here for detail
► Border layout
► Grid layout
► GridBag layout

29 - Which of the following is true about AWT and SWING components?

► **AWT components create a process whereas SWING component creates a thread.**
► AWT components create a thread whereas SWING component creates a process.
► Both AWT and SWING component creates a process.
► Both AWT and SWING component creates a thread.

30 - BorderLayout is the default layout manager for a JFrame content pane

► **TRUE** Click here For Detail
► FALSE

31 - Which of the following is a general purpose container?

► JFrame
► Dialog
► JPanel Click Here For Detail
► JApplet

32 - Border layout divides the area into _____ regions

► 3
► 4
► **5 (Page 113)**
► 6

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

33 - Which of the following function will be used to register event handler with events generator (button)?

► addAction()
► **addActionListener() (Page 123)**
► addListener()
► registerListener()

34 - Event source can have --------- listeners registered on it.

► Single
► Double
► Triple
► **Multiple (Page 125)**

35 - If a class needs to handle events generated by button then which of the following interface a class needs to implement?

► ComponentListener
► KeyListener
► MouseListener
► **ActionListener (Page 122)**

36 - Which of the following function is declared in MouseMotionListener interface?

► public void mousePressed (MouseEvent me);
► **public void mouseDragged (MouseEvent me); (Page 130)**
► public void mouseClicked (MouseEvent me);
► public void mouseEntered (MouseEvent me);

37 - Window Listener interface contains _____ methods.
► Four
► Six
► **Seven (Page 133)**
► Eight

38 - Mouse events can be trapped for _____ GUI component.
► JPanel
► JFrame
► JButton
► **All of given (Page 130)**

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

39 - Adapter classes have been defined for listener interfaces except _____ interface.

► MouseListener
► KeyListener
► WindowListener
► **ActionListener** Click here For Detail

40 - Which of the following package needs to import while interacting with relational database?

► java.io
► **java.sql (Page 149)**
► javax.swing
► java.awt

41 - DSN stands for _____

► Data System Name
► Domain system Name
► **Data Source Name** Click here For Detail
► Database System Name

42 - Which of the following method is used to execute SELECT SQL statements?

► executeUpdate(sql);
► ExecuteUpdate(sql);
► **executeQuery(sql); (Page 154)**
► ExecuteQuery(sql);

43 - CREATE, ALTER, DROP are _____ SQL statements.

► DML
► **DDL (Page 156)**
► DCL
► None of given

44 - Which of the following method is used to execute INSERT, UPDATE and Delete SQL statements?

► executeQuery(sql);
► ExecuteQuery(sql);
► **executeUpdate(sql); (Page 153)**
► ExecuteUpdate(sql);

45 - Which of the following statement object is used to execute stored procedures?

► Statement
► PreparedStatement
► **CallableStatement (Page 159)**
► None of given options

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

46 - HahMap takes key as a/an _____

► **Object (Page 62)**
► Array
► Primitive value
► Function

47 - To create DataBaseMetaData object we write following line of code
► **DataBaseMetaData db = con.getMeataData(); (Page 177)**
► ResultSetMetaData rsmd = rs.getMetaData();
► ResultSetMetaData rsmd = rs.setMetaData();
► DataBaseMetaData db = con.setMeataData();

48 - A .....defines the way and method of communication between two parties

► Compiler
► **Protocol (Page 242)**
► Tool
► Technique

49 - Which of the following are passed as an argument to the paint () method:

► A Canvas object-
► **A Graphics object-** Click here for detail
► An Image object-
► A paint object-

50 - The code below draws a line. What color is the line?
g.setColor(Color.red.green.yellow.red);g.drawLine(0, 0, 100,100);
► Red
► Green
► Yellow
► **Black**

51 - Which of the following methods are invoked by the AWT to support paint and repaint operations?

► **paint( )** Click here For Detail
► repaint( )
► draw( )
► redraw( )

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

52 - If a super class method is protected then overriding method _____

► must be protected
► **must be public** Click here For Detail
► must be private
► may be either protected or public

53 - Socket is a ----------communication channel between hosts.

► Uni-directional
► **Bi-directional (Page 211)**
► Multi-directional
► None of these

54 - Aside from the Scanner class, which of the following class/es can be used for text file file input

► ObjectInputStream
► **BufferedReader** Click here for detail
► StringTokenizer
► None of these

55 - You can ---------to a network using sockets.

► Read
► Write
► **Read/Write (Page 222)**
► None of these

56 - A serializable class must implement the method(s).....

► readObject and writeObject
► Either readObject or writeObject, or both, depending upon the desired behavior
► **No need to implement any method:** Click here For Detail
► None of the given option

CS506- Web Design and Development
Some other MCQ 's & quizzes

Question # 1 of 10
Which of the following feature provide facility for multiple inheritances:-

► Adapter Classes
► Wrapper Classes
► **Interface** Click here For Detail
► Collection

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question # 2 of 10
The relationship between class and interface is called _____

► **'Is a' relationship (Page 98) rep**
► 'Has a' relationship
► '**Responds to' relationship**
► None of the given options

Question # 3 of 10
Which of the following function will be used to register event handler with events generator (button)?

► addAction()
► addListener()
► **addActionListener() (Page 123) rep**
► Register eventhandeler()

Question # 4 of 10
The collections API are defined in _____ package?

► Java.io
► Java.util
► **Java.sql (Page 180)**
► Java.awt

Question # 5 of 10
Which of the following is true about abstract class?

► An abstract class must have all methods declared as abstract methods.
► **A class must have at least one abstract method to be an abstract class.**
► **A class without any abstract method can be declared as abstract class**. Click here for detail **rep**
► An instance of abstract class can be created.

Question # 6 of 10
Exceptions must be handled while handling files otherwise it may lead to____?

► Logical error
► Syntax error
► No error
► **Run time error (Page 70) rep**

Question # 7 of 10
Which of the following stream is a filter stream?

► **BufferedReader (Page 91) rep**
► FileWriter
► FileReader
► All of given options

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari

Question # 8 of 10
CREATE, ALTER, DROP are _____ SQL statements.

- DML
- **DDL (Page 156) rep**
- DCL
- None of above

Question # 9 of 10
Which of the following statement object is used to execute stored procedures?

- Statement
- PreparedStatement
- **CallableStatement (Page 159) rep**
- None of given options

Question # 10 of 10
Based on functionality, the streams can be categorized as_____

- Byte oriented streams and Node Streams
- Filter stream and Character oriented stream
- **Node stream and Filter stream** Click here for detail
- Byte oriented stream and Character oriented stream

Muhammad Moaaz Siddiq MCS (3rd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies
(IEMS) Samundari